

PSOPT User Manual

Victor Becerra
Email: vmbecerra@vmb1.com
www: <http://www.psopt.net>
October 2, 2026

Copyright © 2026 Victor Becerra



Disclaimer

This software is provided “as is” and is distributed free of charge. It comes with no warranties of any kind. See the license terms for more details. The author does hope, however, that users will find this software useful for research and other purposes.

Contents

1	Introduction to PSOPT	9
1.1	About PSOPT	9
1.1.1	Reasons to use PSOPT	10
1.1.2	About the author	11
1.1.3	Contacting the author	11
1.1.4	How you can help	11
1.2	Rolling Release	12
1.3	Other documents in this set	12
1.4	GitHub repository and home page	12
1.5	General problem formulation	13
1.6	Overview of the discretisation methods employed by PSOPT	14
1.7	Pseudospectral approximations	16
1.7.1	Interpolation and the Lagrange polynomial	16
1.7.2	Polynomial expansions	16
1.7.3	Legendre polynomials and numerical quadrature	17
1.7.4	Interpolation and Legendre polynomials	19
1.7.5	Approximate differentiation	20
1.7.6	Approximating a continuous function using Chebyshev polynomials	22
1.7.7	Differentiation with Chebyshev polynomials	25
1.7.8	Numerical quadrature at the Chebyshev-Gauss-Lobatto nodes	25
1.7.9	Differentiation with reduced round-off errors	26
1.8	Legendre and Chebyshev pseudospectral discretisations	26
1.8.1	Costate estimates	31
1.8.2	Discretizing a multiphase problem	32
1.9	Radau and Gauss pseudospectral discretisations	32
1.9.1	Legendre–Gauss–Radau (LGR) collocation	33
1.9.2	Legendre–Gauss (LG) collocation and the terminal-state variable	33
1.9.3	Costate estimates for the Radau and Gauss methods	33
1.10	Local discretisations	34
1.10.1	Trapezoidal method	35
1.10.2	Hermite-Simpson method	35
1.10.3	Costate estimates with local discretisations	36
1.11	Parameter estimation problems	36

1.11.1	Single-phase case	36
1.11.2	Multi-phase case	37
1.11.3	Statistical measures on parameter estimates	38
1.11.4	Remarks on parameter estimation	40
1.12	Limitations and known issues	41
2	Mesh refinement	44
2.1	Evaluating the discretisation error	44
2.2	Manual mesh refinement	45
2.3	Automatic <i>hp</i> -adaptive mesh refinement with pseudospectral grids	45
2.4	Automatic mesh refinement with local collocation	48
3	Integrated residual transcription	51
3.1	Motivation: the integrated residual	51
3.2	Local representation and residual quadrature	51
3.3	Equality path constraints in the residual	52
3.4	Least-squares and regularised formulations	53
3.5	The residual box (robust) formulation	54
3.6	The Direct Alternating Integrated Residual (DAIR) algorithm	54
3.7	Costate recovery by adjoint post-processing	55
3.8	The Nie–Kerrigan flexible-order representation	55
3.9	The flexible mesh	56
3.10	Automatic refinement of an element mesh	58
3.11	Behaviour in practice	59
4	Multiple shooting	62
4.1	Overview of multiple shooting	62
4.2	The segment integrator	63
4.3	The control parameterisation	66
4.4	Flexible segment boundaries	67
4.5	Path constraints	69
4.6	Semi-explicit index-1 DAEs	70
4.7	Costates	72
4.8	Example: multiple shooting on a problem with a closed form	72
4.9	Automatic segment refinement	74
4.10	Choosing the step count automatically	76
4.11	What this transcription does not do	77
5	Choosing a suitable discretisation approach for a given optimal control problem	78
5.1	Pseudospectral or local discretisation?	78
5.2	Choosing among the pseudospectral methods	78
5.3	Choosing among the local methods	79
5.4	When to use mesh refinement	79

5.5	When to use the integrated-residual transcription	79
5.6	When to use multiple shooting	80
5.7	Summary guidance	80
6	Solution diagnostics	82
6.1	Defect rows that hold no dynamics	83
7	Defining optimal control and estimation problems for PSOPT	85
7.1	Interface data structures	85
7.2	Required functions	85
7.2.1	<code>endpoint_cost</code> function	85
7.2.2	<code>integrand_cost</code> function	87
7.2.3	<code>dae</code> function	87
7.2.4	<code>events</code> function	89
7.2.5	<code>linkages</code> function	90
7.2.6	Using the power of Eigen from within the user functions	91
7.2.7	Main function	92
7.3	The built-in sparse SQP solver	116
7.3.1	What it is	116
7.3.2	When to use it, and when not to	117
7.3.3	Choosing a quadratic programming backend	118
7.3.4	Building and using it	119
7.4	Error handling	120
7.4.1	The <code>psopt()</code> contract	120
7.4.2	Accessing a solution after a failed solve	120
7.4.3	Default behaviour: <code>fail-fast</code>	121
7.4.4	Rationale	121
7.4.5	Configuring the policy: <code>algorithm.on_error</code>	121
7.4.6	Output utilities and file output	121
7.5	Specifying a parameter estimation problem	122
7.6	Automatic scaling	123
7.7	Differentiation	123
7.7.1	First derivatives: numerical gradient and Jacobian	124
7.7.2	Second derivatives: the numerical Hessian	124
7.8	Generation of initial guesses	125
7.8.1	<code>L^AT_EX</code> code generation	126
7.9	Using the integrated residual transcription	126
7.10	Implementing multi-segment problems	129
7.11	Other auxiliary functions available to the user	131
7.11.1	<code>cross</code> function	131
7.11.2	<code>dot</code> function	131
7.11.3	<code>get_delayed_state</code> function	131
7.11.4	<code>get_delayed_control</code> function	132
7.11.5	The initial history of a problem with delays	133

7.11.6	get_interpolated_state function	134
7.11.7	get_interpolated_control function	134
7.11.8	get_control_derivative function	135
7.11.9	get_state_derivative function	135
7.11.10	get_initial_states function	136
7.11.11	get_final_states function	136
7.11.12	get_initial_controls function	137
7.11.13	get_final_controls function	137
7.11.14	get_initial_time function	137
7.11.15	get_final_time function	138
7.11.16	auto_link function	138
7.11.17	auto_link_2 function	138
7.11.18	auto_phase_guess function	139
7.11.19	set_guess_from_solution function	139
7.11.20	us76 function	140
7.11.21	Branching on optimisation variables, and the conditional helpers	141
7.11.22	linear_interpolation function	142
7.11.23	smooth_linear_interpolation function	143
7.11.24	spline_interpolation function	143
7.11.25	bilinear_interpolation function	144
7.11.26	smooth_bilinear_interpolation function	144
7.11.27	spline_2d_interpolation function	145
7.11.28	zoh_interpolation function	146
7.11.29	smooth_heaviside function	146
7.11.30	smooth_sign function	146
7.11.31	smooth_fabs function	147
7.11.32	integrate function	147
7.11.33	product_ad functions	148
7.11.34	sum_ad function	149
7.11.35	subtract_ad function	149
7.11.36	inverse_ad function	149
7.11.37	resample_trajectory function	149
7.11.38	linspace function	150
7.11.39	zeros function	150
7.11.40	ones function	150
7.11.41	eye function	151
7.11.42	GaussianRandom function	151
7.11.43	Elementwise mathematical functions on MatrixXd objects	151
7.11.44	Save_to_json_file function	152
7.12	Pre-defined constants	153
7.13	Standard output	154
7.13.1	The constraint-coverage check	155
7.14	Implementing your own problem	155

7.14.1	Building the user code	155
8	Integer (discrete-valued) controls	156
8.1	Method	156
8.1.1	Relaxation by outer convexification	156
8.1.2	Several integer controls: the product relaxation	157
8.1.3	Sum-up rounding	157
8.2	Declaring an integer control	158
8.2.1	Conventions	159
8.2.2	Initial guess	159
8.3	What PSOPT does internally	159
8.4	Reconstructing the integer controls	160
8.5	Worked example: the F-8 aircraft	161
8.6	Worked example: two independent integer controls	162
8.7	Limitations	162
9	Integer (discrete-valued) static parameters	164
9.1	Method	164
9.1.1	Relation to integer controls	165
9.2	Declaring an integer parameter	165
9.2.1	Conventions	165
9.3	Solving with the dedicated driver	166
9.4	Reconstructing the selected value	167
9.5	Worked example: a discrete acceleration	167
9.6	Limitations	168
10	Robust optimal control	169
10.1	The scenario transcription	169
10.1.1	What a robust control is, and what it is not	170
10.1.2	The scenario replacement	170
10.1.3	Generating the scenarios	171
10.2	Uncertainty description	172
10.3	Risk measures	173
10.3.1	About the tail measure	174
10.4	Ancillary feedback	175
10.4.1	About the gain	176
10.5	How many scenarios will fit	178
10.6	More than one phase	179
10.7	Verification	180
10.7.1	What control the verifier integrates	181
10.7.2	What a certificate is	182
10.8	The C++ driver	183
10.8.1	The uncertainty and the building blocks	183
10.8.2	The design, the oracle and the driver	184

10.8.3	The nominal model, and what the driver builds from it	187
10.8.4	Worked example	191
10.9	The Python driver	192
10.9.1	Phases	194
10.9.2	Examples	194
10.10	The same problem through both drivers	195
10.10.1	A two-phase problem, where the two do agree	196
10.10.2	What the objective is, and is not	197
10.11	Limitations	199
11	The Python interface	201
11.1	Overview and when to use it	201
11.2	How it works	201
11.3	Building and installing	202
11.4	Defining and solving a problem	203
11.5	Multiphase problems and linkages	204
11.6	Parameter estimation	205
11.7	Algorithm options and transcriptions	205
11.8	Integer controls and integer parameters	206
11.9	Robust optimal control	207
11.10	Examples and validation	208

1. Introduction to PSOPT

1.1 About PSOPT

PSOPT is an open source optimal control package written in C++ that uses direct collocation methods. These methods solve optimal control problems by approximating the time-dependent variables using global or local polynomials. This allows to discretize the differential equations and continuous constraints over a grid of nodes, and to compute any integrals associated with the problem using well known quadrature formulas. Non-linear programming then is used to find local optimal solutions. PSOPT has also been extended to use the integrated residuals transcription, a method that is particularly useful for problems that have singular arcs, stiff dynamics or non-smooth solutions. PSOPT is able to deal with problems with the following characteristics:

- Single or multiphase problems
- Continuous time nonlinear dynamics
- General endpoint constraints
- Nonlinear path constraints (equalities or inequalities) on states and/or control variables
- Integral constraints
- Interior point constraints
- Bounds on controls and state variables
- General cost function with Lagrange and Mayer terms.
- Free or fixed initial and final conditions
- Linear or nonlinear linkages between phases
- Fixed or free initial time
- Fixed or free final time
- Optimisation of static parameters

- Parameter estimation problems with sampled measurements
- Differential equations with delayed variables.

The implementation has the following features:

- Automatic scaling
- Automatic first and second derivatives using the CppAD library
- Numerical differentiation by using sparse finite differences
- Automatic mesh refinement
- Automatic identification of the Jacobian and Hessian sparsity.
- DAE formulation, so that differential and algebraic constraints can be implemented in the same C++ function.

PSOPT has interfaces to the following NLP solvers:

- IPOPT: an open source C++ implementation of an interior point method for large scale problems. See <https://projects.coin-or.org/Ipopt> for further details. This is the default, and the solver used for most of the examples distributed with PSOPT.
- PSOPT's own sparse sequential quadratic programming solver, described in Section 7.3. It is optional at build time, is not the default, and requires a sparse quadratic programming backend to be built alongside it.

1.1.1 Reasons to use PSOPT

These are some reasons why users may wish to use PSOPT:

- Users who for any reason do not have access to commercial optimal control solvers and wish to employ a free open source package for optimal control which does not need a proprietary software environment to run.
- Users who need to link an optimal control solver from stand alone applications written in C++ or other programming languages.
- Users who want to do research with the software, for instance by implementing their own problems, or by customising the code.

PSOPT does not require a commercial software environment to run on, or to be compiled. PSOPT is fully compatible with the `gcc` compiler, and has been developed under Linux, a free operating system. Note also that the default NLP solver (IPOPT) requires a sparse linear solver from a range of options, some of which are available at no cost. The author has personally used free linear solver `MUMPS`.

1.1.2 About the author

Victor Becerra obtained his first degree in Electrical Engineering in 1990, and worked for two years in power systems analysis and control for a power generation and transmission company. He obtained his PhD for his work on the development of nonlinear optimal control methods from City University, London, in 1994. Between 1994 and 1999 he was a Research Fellow at the Control Engineering Research Centre at City University, London. Between 2000 and 2015 he was an academic at the School of Systems Engineering, University of Reading, UK, where he became a Professor of Automatic Control in 2012. Between 2011 and 2012, he was seconded at the Ford Motor Company in Dunton, Essex, with funding by the Royal Academy of Engineering, where he developed methods for the calibration of gasoline engine oil temperature dynamic models. In 2015, he took the position of Professor of Power Systems Engineering at the University of Portsmouth, UK. He is a Senior Member of the IEEE, a Senior Member of the AIAA, and a Fellow of the Institute of Engineering and Technology. During his career, he has received research funding from the EPSRC, the Royal Academy of Engineering, the European Union, the Knowledge Transfer Partnership programme, Innovate UK and UK industry. He has published over 150 research papers and two books. His web site is:

<https://researchportal.port.ac.uk/en/persons/victor-becerra>

1.1.3 Contacting the author

The author is open to discussing with users potential research collaboration leading to publications, academic exchanges, or joint projects. He can be contacted directly at his email address v.m.becerra@ieee.org.

1.1.4 How you can help

You may help improve PSOPT in a number of ways.

- Sending bug reports (bug tracking system: <https://github.com/PSOPT/psopt/issues/>)
- Sending corrections to the documentation, please use the above link.
- Discussing with the author ways to improve the computational aspects or capabilities of the software.
- Sending to the author proposed modifications to the source code, for consideration to be included in a future release of PSOPT, usually through pull requests in GitHub.
- Sending source code with new examples which may be included (with due acknowledgement) in future releases of PSOPT.
- Porting the software to new architectures.

- If you have had a good experience with PSOPT, tell your students or colleagues about it.
- Quoting the use of PSOPT in your scientific publications. The recommended reference for PSOPT is:
 - Becerra, V.M. (2010). "Solving complex optimal control problems at no cost with PSOPT". *Proc. IEEE Multi-conference on Systems and Control*, Yokohama, Japan, September 7-10, 2010, pp. 1391-1396

and the following is the recommended form to cite this document:

- Becerra, V.M. (2020). *PSOPT Optimal Control Solver User Manual. Release 5*. Available: <https://github.com/PSOPT/psopt/blob/master/doc/>
- Developing interfaces to other NLP solvers.

1.2 Rolling Release

From March 2025, PSOPT will feature a rolling release mode. Rolling release is a concept in software development of frequently delivering updates to applications. This is in contrast to a standard or point release development model which uses software versions which replace the previous version. Users can download the latest source code from the GitHub repository. The documentation will also be updated on a rolling release basis.

1.3 Other documents in this set

The PSOPT documentation is now made of this document, which introduces the software, its methods and its usage. A second document provides details and results from a number of examples. Finally, the GitHub page provides the latest installation instructions.

1.4 GitHub repository and home page

The downloadable PSOPT distribution is held in its GitHub page:

<https://github.com/PSOPT/psopt>

A web page is maintained to provide general information about PSOPT:

<http://www.psopt.net>

1.5 General problem formulation

PSOPT solves the following general optimal control problem with N_p phases:

Problem $\mathcal{P}_1$

Find the control trajectories, $u^{(i)}(t), t \in [t_0^{(i)}, t_f^{(i)}]$, state trajectories $x^{(i)}(t), t \in [t_0^{(i)}, t_f^{(i)}]$, static parameters $p^{(i)}$, and times $t_0^{(i)}, t_f^{(i)}, i = 1, \dots, N_p$, to minimise the following performance index:

$$J = \sum_{i=1}^{N_p} \left[\varphi^{(i)}[x^{(i)}(t_f^{(i)}), p^{(i)}, t_f^{(i)}] + \int_{t_0^{(i)}}^{t_f^{(i)}} L^{(i)}[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t] dt \right]$$

subject to the differential constraints:

$$\dot{x}^{(i)}(t) = f^{(i)}[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t], \quad t \in [t_0^{(i)}, t_f^{(i)}],$$

the path constraints

$$h_L^{(i)} \leq h^{(i)}[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t] \leq h_U^{(i)}, \quad t \in [t_0^{(i)}, t_f^{(i)}],$$

the event constraints:

$$e_L^{(i)} \leq e^{(i)}[x^{(i)}(t_0^{(i)}), u^{(i)}(t_0^{(i)}), x^{(i)}(t_f^{(i)}), u^{(i)}(t_f^{(i)}), p^{(i)}, t_0^{(i)}, t_f^{(i)}] \leq e_U^{(i)},$$

the phase linkage constraints:

$$\begin{aligned} \Psi_l &\leq \Psi[x^{(1)}(t_0^{(1)}), u^{(1)}(t_0^{(1)}), \\ &\quad x^{(1)}(t_f^{(1)}), u^{(1)}(t_f^{(1)}), p^{(1)}, t_0^{(1)}, t_f^{(1)}, \\ &\quad x^{(2)}(t_0^{(2)}), u^{(2)}(t_0^{(2)}) \\ &\quad , x^{(2)}(t_f^{(2)}), u^{(2)}(t_f^{(2)}), p^{(2)}, t_0^{(2)}, t_f^{(2)}, \\ &\quad \vdots \\ &\quad x^{(N_p)}(t_0^{(N_p)}), u^{(N_p)}(t_0^{(N_p)}), \\ &\quad x^{(N_p)}(t_f^{(N_p)}), u^{(N_p)}(t_f^{(N_p)}), p^{(N_p)}, t_0^{(N_p)}, t_f^{(N_p)}] \leq \Psi_u \end{aligned}$$

the bound constraints:

$$\begin{aligned} u_L^{(i)} &\leq u^i(t) \leq u_U^{(i)}, \quad t \in [t_0^{(i)}, t_f^{(i)}], \\ x_L^{(i)} &\leq x^i(t) \leq x_U^{(i)}, \quad t \in [t_0^{(i)}, t_f^{(i)}], \\ p_L^{(i)} &\leq p^{(i)} \leq p_U^{(i)}, \\ \underline{t}_0^{(i)} &\leq t_0^{(i)} \leq \bar{t}_0^{(i)}, \end{aligned}$$

$$\underline{t}_f^{(i)} \leq t_f^{(i)} \leq \bar{t}_f^{(i)},$$

and the following constraints:

$$t_f^{(i)} - t_0^{(i)} \geq 0,$$

where $i = 1, \dots, N_p$, and

$$\begin{aligned} u^{(i)} &: [t_0^{(i)}, t_f^{(i)}] \rightarrow \mathcal{R}^{n_u} \\ x^{(i)} &: [t_0^{(i)}, t_f^{(i)}] \rightarrow \mathcal{R}^{n_x} \\ p^{(i)} &\in \mathcal{R}^{n_p} \\ \varphi^{(i)} &: \mathcal{R}^{n_x} \times \mathcal{R}^{n_x} \times \mathcal{R}^{n_p} \times \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R} \\ L^{(i)} &: \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times [t_0^{(i)}, t_f^{(i)}] \rightarrow \mathcal{R} \\ f^{(i)} &: \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times [t_0^{(i)}, t_f^{(i)}] \rightarrow \mathcal{R}^{n_x} \\ h^{(i)} &: \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times [t_0^{(i)}, t_f^{(i)}] \rightarrow \mathcal{R}^{n_h} \\ e^{(i)} &: \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R}^{n_e} \\ \Psi &: U_\Psi \rightarrow \mathcal{R}^{n_\Psi} \end{aligned} \tag{1.1}$$

where U_Ψ is the domain of function Ψ .

A multiphase problem like $\mathcal{P}_1$ is defined and discussed in the book by Betts [5].

1.6 Overview of the discretisation methods employed by PSOPT

PSOPT uses pseudospectral, local (trapezoidal and Hermite-Simpson) and integrated residual approaches to discretise and solve optimal control problems. Pseudospectral methods were originally developed for the solution of partial differential equations and have become a widely applied computational tool in fluid dynamics [12, 11]. Moreover, over the last 15 years or so, pseudospectral techniques have emerged as important computational methods for solving optimal control problems [17, 18, 20, 40, 27]. While finite difference methods approximate the derivatives of a function using local information, pseudospectral methods are, in contrast, global in the sense that they use information over samples of the whole domain of the function to approximate its derivatives at selected points. Using these methods, the state and control functions are approximated as a weighted sum of smooth basis functions, which are often chosen to be Legendre or Chebyshev polynomials in the interval $[-1, 1]$, and collocation of the differential-algebraic equations is performed at orthogonal collocation points, which are selected to yield interpolation of high accuracy. One of the main appeals of pseudospectral methods is their exponential (or spectral) rate of convergence, which is faster than any polynomial rate. Another advantage is that with relatively coarse grids it is possible to achieve good accuracy [46]. In cases where global collocation is unsuitable (for example, when the solution

exhibits discontinuities), multi-domain pseudospectral techniques have been proposed, where the problem is divided into a number of subintervals and global collocation is performed along each subinterval [11].

Pseudospectral methods directly discretize the original optimal control problem to formulate a nonlinear programming problem, which is then solved numerically using a sparse nonlinear programming solver to find approximate local optimal solutions. Approximation theory and practice shows that pseudospectral methods are well suited for approximating smooth functions, integrations, and differentiations [10, 46], all of which are relevant to optimal control problems. For differentiation, the derivatives of the state functions at the discretisation nodes are easily computed by multiplying a constant differentiation matrix by a matrix with the state values at the nodes. Thus, the differential equations of the optimal control problem are approximated by a set of algebraic equations. The integration in the cost functional of an optimal control problem is approximated by well known Gauss quadrature rules, consisting of a weighted sum of the function values at the discretisation nodes. Moreover, as is the case with other direct methods for optimal control, it is easy to represent state and control dependent constraints.

The Legendre pseudospectral method for optimal control problems was originally proposed by Elnagar and co-workers in 1995 [17]. Since then, authors such as Ross, Fahroo and co-workers have analysed, extended and applied the method. For instance, convergence analysis is presented in [28], while an extension of the method to multi-phase problems is given in [40]. An application that has received publicity is the use of the Legendre pseudospectral method for generating real time trajectories for a NASA spacecraft manoeuvre [27]. The Chebyshev pseudospectral method for optimal control problems was originally proposed in 1988 [47]. Fahroo and Ross proposed an alternative method for trajectory optimisation using Chebyshev polynomials [20]. The Gauss pseudospectral method, in which the dynamics are collocated at the interior Legendre–Gauss points, was developed by Benson and co-workers [3, 4], who also established a costate estimation procedure relating the multipliers of the discretised problem to the continuous costates. The Radau pseudospectral method collocates at the Legendre–Gauss–Radau points, one of which lies at an endpoint of the interval; convergence rates for Radau collocation were established by Kameswaran and Biegler [26], and the method was developed for both finite- and infinite-horizon problems, together with the associated costate estimation, by Garg, Rao and co-workers [22]. A unified framework relating the Gauss, Radau and Lobatto (Legendre–Gauss–Lobatto) pseudospectral schemes was subsequently presented in [21].

Some details on approximating continuous functions using Legendre and Chebyshev polynomials are given below. Interested readers are referred to [10] for further details.

PSOPT implements several global pseudospectral discretisations: the Legendre (LGL) and Chebyshev (CGL) methods described in this chapter, together with the Legendre–Gauss–Radau (LGR) and Legendre–Gauss (LG) methods of Section 1.9. PSOPT is not restricted to pseudospectral methods: it also provides the local discretisations of Section 1.10, including the trapezoidal and Hermite–Simpson collocation methods. In ad-

dition to these collocation methods, PSOPT offers an integrated residual transcription, described in Chapter 3, which measures (and optionally bounds) the dynamic error between the nodes and is particularly useful for problems with singular arcs, stiff dynamics or non-smooth solutions.

1.7 Pseudospectral approximations

1.7.1 Interpolation and the Lagrange polynomial

It is a well known fact in numerical analysis [8] that if $\tau_0, \tau_1, \dots, \tau_N$ are $N + 1$ distinct numbers and f is a function whose values are given at those numbers, then a unique polynomial $P(\tau)$ of degree at most N exists with

$$f(\tau_k) = P(\tau_k), \text{ for } k = 0, 1, \dots, N$$

This polynomial is given by:

$$P(\tau) = \sum_{k=0}^N f(\tau_k) \mathcal{L}_k(\tau)$$

where

$$\mathcal{L}_k(\tau) = \prod_{i=0, i \neq k}^N \frac{\tau - \tau_i}{\tau_k - \tau_i} \quad (1.2)$$

$P(\tau)$ is known as the Lagrange interpolating polynomial and $\mathcal{L}_k(\tau)$ are known as Lagrange basis polynomials.

1.7.2 Polynomial expansions

Assume that $\{p_k\}_{k=0,1,\dots}$ is a system of algebraic polynomials, with degree of $p_k = k$, that are mutually orthogonal over the interval $[-1, 1]$ with respect to a weight function w :

$$\int_{-1}^1 p_k(\tau) p_m(\tau) w(\tau) d\tau = 0, \text{ for } m \neq k$$

Define $L_w^2[-1, 1]$ as the space of functions where the norm:

$$\|v\|_w = \left(\int_{-1}^1 |v(\tau)|^2 w(\tau) d\tau \right)^{1/2}$$

is finite. A function $f \in L_w^2[-1, 1]$ in terms of the system $\{p_k\}$ can be represented as a series expansion:

$$f(\tau) = \sum_{k=0}^{\infty} \hat{f}_k p_k(\tau)$$

where the coefficients of the expansion are given by:

$$\hat{f}_k = \frac{1}{\|p_k\|^2} \int_{-1}^1 f(\tau) p_k(\tau) w(\tau) d\tau \quad (1.3)$$

The truncated expansion of f for a given N is:

$$\mathcal{P}_N f(\tau) = \sum_{k=0}^N \hat{f}_k p_k(\tau)$$

This type of expansion is at the heart of spectral and pseudospectral methods.

1.7.3 Legendre polynomials and numerical quadrature

A particular class of orthogonal polynomials are the Legendre polynomials, which are the eigenfunctions of a singular Sturm-Liouville problem [10]. Let $L_N(\tau)$ denote the Legendre polynomial of order N , which may be generated from:

$$L_N(\tau) = \frac{1}{2^N N!} \frac{d^N}{d\tau^N} (\tau^2 - 1)^N$$

Legendre polynomials are orthogonal over $[-1, 1]$ with the weight function $w = 1$. Examples of Legendre polynomials are:

$$\begin{aligned} L_0(\tau) &= 1 \\ L_1(\tau) &= \tau \\ L_2(\tau) &= \frac{1}{2}(3\tau^2 - 1) \\ L_3(\tau) &= \frac{1}{2}(5\tau^3 - 3\tau) \end{aligned}$$

Figure 1.1 illustrates the Legendre polynomials $L_N(\tau)$ for $N = 0, 1, 2, 4, 5, 10$.

Let τ_k , $k = 0, \dots, N$ be the Lagrange-Gauss-Lobatto (LGL) nodes, which are defined as $\tau_0 = -1$, $\tau_N = 1$, and τ_k , being the roots of $\dot{L}_N(\tau)$ in the interval $[-1, 1]$ for $k = 1, 2, \dots, N-1$. There are no explicit formulas to compute the roots of $\dot{L}_N(\tau)$, but they can be computed using known numerical algorithms. For example, for $N = 20$, the LGL nodes τ_k , $k = 0, \dots, 20$ are shown in Figure 1.2.

Note that if $h(\tau)$ is a polynomial of degree $\leq 2N-1$, its integral over $\tau \in [-1, 1]$ can be exactly computed as follows:

$$\int_{-1}^1 h(\tau) d\tau = \sum_{k=0}^N h(\tau_k) w_k \quad (1.4)$$

where τ_k , $k = 0, \dots, N$ are the LGL nodes and the weights w_k are given by:

$$w_k = \frac{2}{N(N+1)} \frac{1}{[L_N(\tau_k)]^2}, \quad k = 0, \dots, N. \quad (1.5)$$

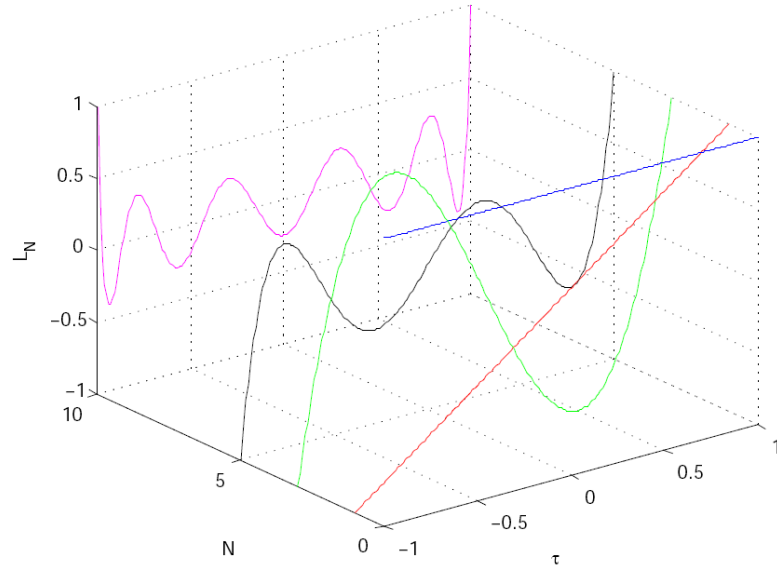


Figure 1.1: Illustration of the Legendre polynomials $L_N(\tau)$ for $N = 0, 1, 2, 4, 5, 10$.

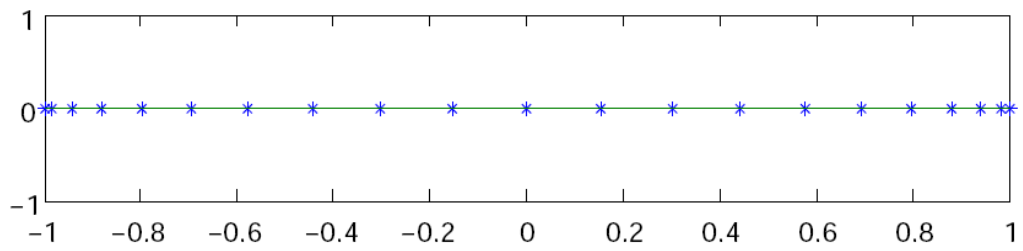


Figure 1.2: Illustration of the Legendre Gauss Lobatto (LGL) nodes for $N = 20$.

If $L(\tau)$ is a general smooth function, then for a suitable N , its integral over $\tau \in [-1, 1]$ can be approximated as follows:

$$\int_{-1}^1 L(\tau) d\tau \approx \sum_{k=0}^N L(\tau_k) w_k \quad (1.6)$$

The LGL nodes are selected to yield highly accurate numerical integrals. For example, consider the definite integral

$$\int_{-1}^1 e^t \cos(t) dt$$

The exact value of this integral to 7 decimal places is 1.9334214. For $N = 3$ we have $\tau = [-1, -0.4472, 0.4472, 1]$, $w = [0.1667, 0.8333, 0.8333, 0.1667]$, hence

$$\int_{-1}^1 e^t \cos(t) dt \approx w^T h(\tau) = 1.9335$$

so that the error is $\mathcal{O}(10^{-5})$. On the other hand, if $N = 5$, then the approximate value is 1.9334215, so that the error is $\mathcal{O}(10^{-7})$.

1.7.4 Interpolation and Legendre polynomials

The Legendre-Gauss-Lobatto quadrature motivates the following expression to approximate the weights of the expansion (1.3):

$$\hat{f}_k \approx \tilde{f}_k = \frac{1}{\gamma_k} \sum_{j=0}^N f(\tau_j) L_k(\tau_j) w_j$$

where

$$\gamma_k = \sum_{j=0}^N L_k^2(\tau_j) w_j$$

It is simple to prove (see [24]) that with these weights, function $f : [-1, 1] \rightarrow \mathfrak{R}$ can be interpolated over the LGL nodes as a discrete expansion using Legendre polynomials:

$$I_N f(\tau) = \sum_{k=0}^N \tilde{f}_k L_k(\tau) \quad (1.7)$$

such that

$$I_N f(\tau_j) = f(\tau_j) \quad (1.8)$$

Because $I_N f(\tau)$ is an interpolant of $f(\tau)$ at the LGL nodes, and since the interpolating polynomial is unique, we may express $I_N f(\tau)$ as a Lagrange interpolating polynomial:

$$I_N f(\tau) = \sum_{k=0}^N f(\tau_k) \mathcal{L}_k(\tau) \quad (1.9)$$

so that the expressions (1.7) and (1.9) are mathematically equivalent. Expression (1.9) is computationally advantageous since, as discussed below, it allows to express the approximate values of the derivatives of the function f at the nodes as a matrix multiplication. It is possible to write the Lagrange basis polynomials $\mathcal{L}_k(\tau)$ as follows [24]:

$$\mathcal{L}_k(\tau) = \frac{1}{N(N+1)L_N(\tau_k)} \frac{(\tau^2 - 1)\dot{L}_N(\tau)}{\tau - \tau_k}$$

The use of polynomial interpolation to approximate a function using the LGL points is known in the literature as the *Legendre pseudospectral approximation method*. Denote $f^N(\tau) = I_N f(\tau)$. Then, we have:

$$f(\tau) \approx f^N(\tau) = \sum_{k=0}^N f(\tau_k) \mathcal{L}_k(\tau) \quad (1.10)$$

It should be noted that $\mathcal{L}_k(\tau_j) = 1$ if $k = j$ and $\mathcal{L}_k(\tau_j) = 0$, if $k \neq j$, so that:

$$f^N(\tau_k) = f(\tau_k) \quad (1.11)$$

Regarding the accuracy and error estimates of the Legendre pseudospectral approximation, it is well known that for smooth functions $f(\tau)$, the rate of convergence of $f^N(\tau)$ to $f(\tau)$ at the collocation points is faster than any power of $1/N$. The convergence of the pseudospectral approximations used by PSOPT has been analysed by Canuto *et al* [10].

Figure 1.3 shows the degree N interpolation of the function $f(\tau) = 1/(1+\tau+15\tau^2)$ in $(N+1)$ equispaced and LGL points for $N = 20$. With increasing N , the errors increase exponentially in the equispaced case (this is known as the Runge phenomenon) whereas in the LGL case they decrease exponentially.

1.7.5 Approximate differentiation

The derivatives of $f^N(\tau)$ in terms of $f(\tau)$ at the LGL points τ_k can be obtained by differentiating Eqn. (1.10). The result can be expressed as a matrix multiplication, such that:

$$\dot{f}(\tau_k) \approx \dot{f}^N(\tau_k) = \sum_{i=0}^N D_{ki} f(\tau_i)$$

where

$$D_{ki} = \begin{cases} -\frac{L_N(\tau_k)}{L_N(\tau_i)} \frac{1}{\tau_k - \tau_i} & \text{if } k \neq i \\ -N(N+1)/4 & \text{if } k = i = 0 \\ N(N+1)/4 & \text{if } k = i = N \\ 0 & \text{otherwise} \end{cases} \quad (1.12)$$

which is known as the differentiation matrix.

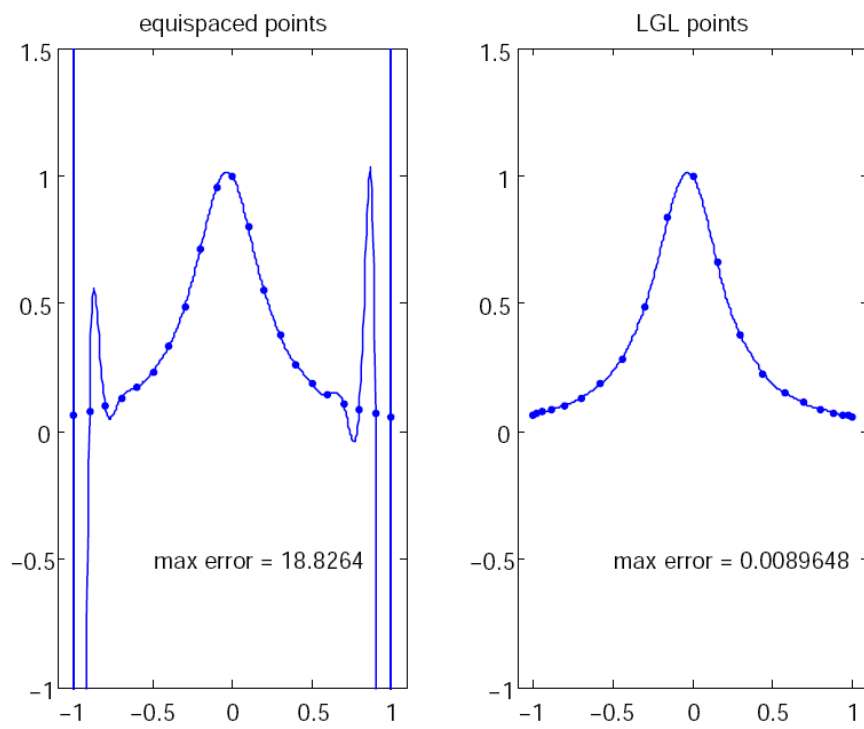


Figure 1.3: Illustration of polynomial interpolation over equispaced and LGL nodes

Two conventions in (1.12) need to be clear, because the signs depend on them and a reader who assumes otherwise will not reproduce the matrix below. PSOPT stores the LGL nodes in *descending* order, so that $\tau_0 = +1$ and $\tau_N = -1$; and the operator carries an overall minus sign relative to the form usually printed in the spectral-methods literature, so that $D\tau = -\mathbf{1}$ and not $+\mathbf{1}$. That sign is cancelled by the $(t_f - t_0)/2$ factor carried on the right-hand side of the differential defects in (1.32), so the defects themselves are the usual ones. The two must be read together: adopting the literature's differentiation matrix without also changing that factor, or the reverse, gives dynamics integrated backwards in time.

For example, this is the Legendre differentiation matrix for $N = 5$.

$$D = \begin{bmatrix} -7.5000 & 10.1414 & -4.0362 & 2.2447 & -1.3499 & 0.5000 \\ -1.7864 & 0 & 2.5234 & -1.1528 & 0.6535 & -0.2378 \\ 0.4850 & -1.7213 & 0 & 1.7530 & -0.7864 & 0.2697 \\ -0.2697 & 0.7864 & -1.7530 & 0 & 1.7213 & -0.4850 \\ 0.2378 & -0.6535 & 1.1528 & -2.5234 & 0 & 1.7864 \\ -0.5000 & 1.3499 & -2.2447 & 4.0362 & -10.1414 & 7.5000 \end{bmatrix}$$

Figure 1.4 shows the Legendre differentiation of $f(t) = \sin(5t^2)$ for $N = 20$ and $N = 30$. Note the vertical scales in the error curves. Figure 1.5 shows the maximum error in the Legendre differentiation of $f(t) = \sin(5t^2)$ as a function of N . Notice that the error initially decreases very rapidly until such high precision is achieved (accuracy in the order of 10^{-12}) that round off errors due to the finite precision of the computer prevent any further reductions. This phenomenon is known as *spectral accuracy*.

1.7.6 Approximating a continuous function using Chebyshev polynomials

PSOPT also has facilities for pseudospectral function approximation using Chebyshev polynomials. Let $T_N(\tau)$ denote the Chebyshev polynomial of order N , which may be generated from:

$$T_N(\tau) = \cos(N \cos^{-1}(\tau)) \quad (1.13)$$

Let τ_k , $k = 0, \dots, N$ be the Chebyshev-Gauss-Lobatto (CGL) nodes in the interval $[-1, 1]$, which are defined as $\tau_k = -\cos(\pi k/N)$ for $k = 0, 1, \dots, N$.

Given any real-valued function $f(\tau) : [-1, 1] \rightarrow \mathfrak{R}$, it can be approximated by the Chebyshev pseudospectral method:

$$f(\tau) \approx f^N(\tau) = \sum_{k=0}^N f(\tau_k) \varphi_k(\tau) \quad (1.14)$$

where the Lagrange interpolating polynomial $\varphi_k(\tau)$ is defined by:

$$\varphi_k(\tau) = \frac{(-1)^{k+1}}{N^2 \bar{c}_k} \frac{(1 - \tau^2) \dot{T}_N(\tau)}{\tau - \tau_k} \quad (1.15)$$

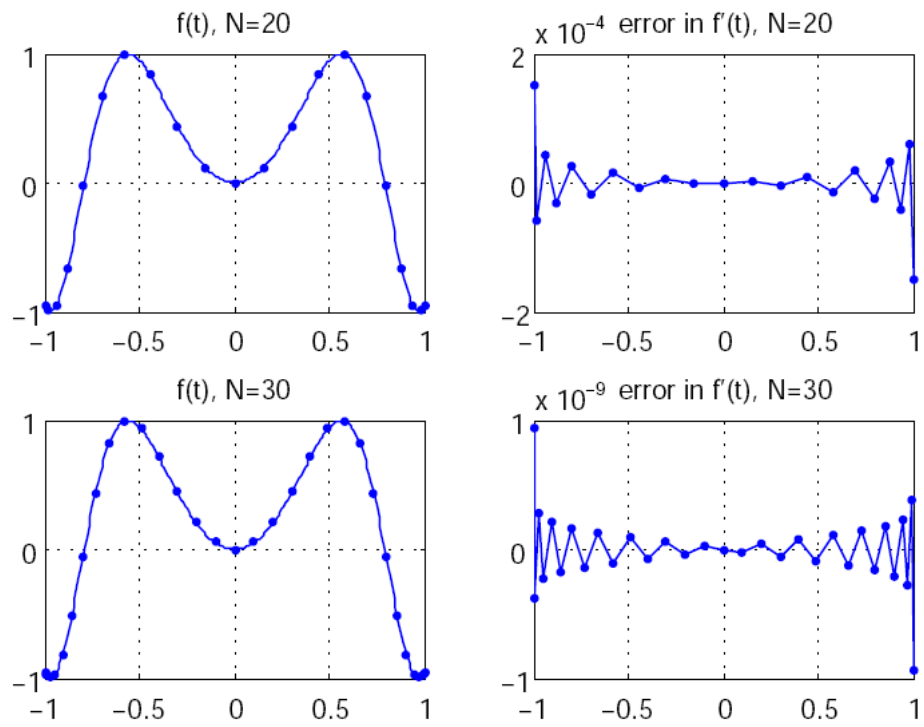


Figure 1.4: Legendre differentiation of $f(t) = \sin(5t^2)$ for $N = 20$ and $N = 30$.

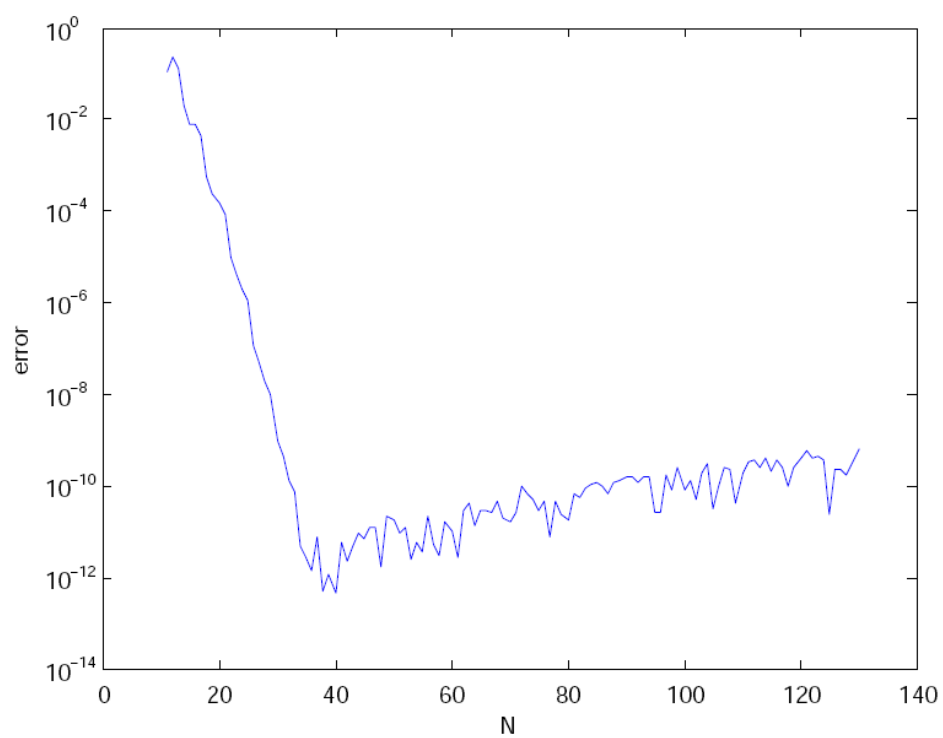


Figure 1.5: Maximum error in the Legendre differentiation of $f(t) = \sin(5t^2)$ as a function of N .

where

$$\bar{c}_k = \begin{cases} 2 & k = 0, N \\ 1 & 1 \leq k \leq N-1 \end{cases} \quad (1.16)$$

It should be noted that $\varphi_k(\tau_j) = 1$ if $k = j$ and $\varphi_k(\tau_j) = 0$, if $k \neq j$, so that:

$$f^N(\tau_k) = f(\tau_k) \quad (1.17)$$

1.7.7 Differentiation with Chebyshev polynomials

The derivatives of $f^N(\tau)$ in terms of $f(\tau)$ at the CGL points τ_k can be obtained by differentiating (1.14). The result can be expressed as a matrix multiplication, such that:

$$\dot{f}(\tau_k) \approx \dot{F}^N(\tau_k) = \sum_{i=0}^N D_{ki} f(\tau_i) \quad (1.18)$$

where

$$D_{ki} = \begin{cases} \frac{\bar{c}_k}{2\bar{c}_i} \frac{(-1)^{k+i}}{\sin[(k+i)\pi/2N] \sin[(k-i)\pi/2N]} & \text{if } k \neq i \\ \frac{2 \sin^2 \tau_k}{2 \sin^2[k\pi/N]} & \text{if } i \leq k = i \leq N-1 \\ -\frac{2N^2+1}{6} & \text{if } k = i = 0 \\ \frac{2N^2+1}{6} & \text{if } k = i = N \end{cases} \quad (1.19)$$

which is known as the differentiation matrix.

1.7.8 Numerical quadrature at the Chebyshev-Gauss-Lobatto nodes

Integrals of the running cost and other smooth functions, sampled at specific points, are calculated using the following approach. In PSOPT, values at Chebyshev-Gauss-Lobatto (CGL) nodes are evaluated using Clenshaw-Curtis quadrature [46]. For a general function $L(\tau)$ and a suitable N ,

$$\int_{-1}^1 L(\tau) d\tau \approx \sum_{k=0}^N w_k L(\tau_k), \quad (1.20)$$

where τ_k , $k = 0, \dots, N$ are the CGL nodes and the weights w_k are obtained from the discrete cosine transform of the sequence $2/(1-j^2)$, taken over the even indices j . (The explicit form and a short implementation are given in [46]). The weights are symmetric and strictly positive. This rule is exact for polynomials of degree $\leq N$, and it converges spectrally, meaning it converges faster than any fixed algebraic order, for smooth integrands. Because the rule integrates constant functions exactly, the weights add up to the length of the interval.

$$\sum_{k=0}^N w_k = 2. \quad (1.21)$$

Unlike the classical Gauss—Lobatto rule, which is associated with the Chebyshev weight, where $g(\tau) = 1/\sqrt{1-\tau^2}$, equation (1.20) uses a unit weight and so it applies directly to the integrand $L(\tau)$. Earlier versions of PSOPT used the weighted rule, which required dividing the integrand by g before summing. Since $1/g(\tau) = \sqrt{1-\tau^2}$ vanishes with infinite slope at $\tau = \pm 1$, the resulting product is not smooth at the endpoints and so the discretized cost only converged at an algebraic rate. Clenshaw—Curtis quadrature removes this weight, which restores spectral accuracy for the cost and makes the cost quadrature consistent with the other global pseudospectral methods. The same weights w_k are also used in the covector (costate) mapping for the Chebyshev method (Section 1.8.1).

1.7.9 Differentiation with reduced round-off errors

The following differentiation matrix, which offers reduced round-off errors [10], is employed optionally by PSOPT. It can be used both with Legendre and Chebyshev points.

$$D_{jl} = \begin{cases} -\frac{\delta_l}{\delta_j} \frac{(-1)^{j+l}}{\tau_j - \tau_l} & j \neq l \\ \sum_{i=0, i \neq j}^N \frac{\delta_i}{\delta_j} \frac{(-1)^{i+j}}{\tau_j - \tau_i} & j = l \end{cases} \quad (1.22)$$

1.8 Legendre and Chebyshev pseudospectral discretisations

To illustrate the pseudospectral discretisations employed in PSOPT, consider the following single phase continuous optimal control problem:

Problem $\mathcal{P}_2$

Find the control trajectories, $u(t), t \in [t_0, t_f]$, state trajectories $x(t), t \in [t_0, t_f]$, static parameters p , and times t_0, t_f , to minimise the following performance index:

$$J = \varphi[x(t_0), x(t_f), p, t_0, t_f] + \int_{t_0}^{t_f} L[x(t), u(t), p, t] dt$$

subject to the differential constraints:

$$\dot{x}(t) = f[x(t), u(t), p, t], \quad t \in [t_0, t_f],$$

the path constraints

$$h_L \leq h[x(t), u(t), p, t] \leq h_U, \quad t \in [t_0, t_f]$$

the event constraints:

$$e_L \leq e[x(t_0), u(t_0), x(t_f), u(t_f), p, t_0, t_f] \leq e_U,$$

the bound constraints on states and controls:

$$u_L \leq u(t) \leq u_U, \quad t \in [t_0, t_f],$$

$$x_L \leq x(t) \leq x_U, \quad t \in [t_0, t_f],$$

and the constraints:

$$\underline{t}_0 \leq t_0 \leq \bar{t}_0,$$

$$\underline{t}_f \leq t_f \leq \bar{t}_f,$$

$$t_f - t_0 \geq 0,$$

where

$$\begin{aligned} u &: [t_0, t_f] \rightarrow \mathcal{R}^{n_u} \\ x &: [t_0, t_f] \rightarrow \mathcal{R}^{n_x} \\ p &\in \mathcal{R}^{n_p} \\ \varphi &: \mathcal{R}^{n_x} \times \mathcal{R}^{n_x} \times \mathcal{R}^{n_p} \times \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R} \\ L &: \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times [t_0, t_f] \rightarrow \mathcal{R} \\ f &: \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times [t_0, t_f] \rightarrow \mathcal{R}^{n_x} \\ h &: \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times [t_0, t_f] \rightarrow \mathcal{R}^{n_h} \\ e &: \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R}^{n_e} \end{aligned} \tag{1.23}$$

By introducing the transformation:

$$\tau \leftarrow \frac{2}{t_f - t_0} t - \frac{t_f + t_0}{t_f - t_0},$$

it is possible to write problem $\mathcal{P}_3$ using a new independent variable τ in the interval $[-1, 1]$, as follows:

Problem $\mathcal{P}_3$

Find the control trajectories, $u(\tau), \tau \in [-1, 1]$, state trajectories $x(\tau), \tau \in [-1, 1]$, and times t_0, t_f , to minimise the following performance index:

$$J = \varphi[x(-1), x(1), p, t_0, t_f] + \frac{t_f - t_0}{2} \int_{-1}^1 L[x(\tau), u(\tau), p, \tau] d\tau$$

subject to the differential constraints:

$$\dot{x}(\tau) = \frac{t_f - t_0}{2} f[x(\tau), u(\tau), p, \tau], \quad \tau \in [-1, 1],$$

the path constraints

$$h_L \leq h[x(\tau), u(\tau), p, \tau] \leq h_U, \quad \tau \in [-1, 1]$$

the event constraints:

$$e_L \leq e[x(-1), u(-1), x(1), u(1), p, t_0, t_f] \leq e_U,$$

the bound constraints on controls and states:

$$u_L \leq u(\tau) \leq u_U, \tau \in [-1, 1],$$

$$x_L \leq x(\tau) \leq x_U, \tau \in [-1, 1],$$

and the constraints:

$$\underline{t}_0 \leq t_0 \leq \bar{t}_0,$$

$$\underline{t}_f \leq t_f \leq \bar{t}_f,$$

$$t_f - t_0 \geq 0,$$

The description below refers to the Legendre pseudospectral approximation method. The procedure employed with the Chebyshev approximation method is very similar. In the Legendre pseudospectral approximation of problem $\mathcal{P}_3$, the state $x(\tau)$, $\tau \in [-1, 1]$ is approximated by the N -order Lagrange polynomial $x^N(\tau)$ based on interpolation at the Legendre-Gauss-Lobatto (LGL) quadrature nodes, so that:

$$x(\tau) \approx x^N(\tau) = \sum_{k=0}^N x(\tau_k) \phi_k(\tau) \quad (1.24)$$

Moreover, the control $u(\tau)$, $\tau \in [-1, 1]$ is similarly approximated using an interpolating polynomial:

$$u(\tau) \approx u^N(\tau) = \sum_{k=0}^N u(\tau_k) \phi_k(\tau) \quad (1.25)$$

Note that, from (1.11), $x^N(\tau_k) = x(\tau_k)$ and $u^N(\tau_k) = u(\tau_k)$. The derivative of the state vector is approximated as follows:

$$\dot{x}(\tau_k) \approx \dot{x}^N(\tau_k) = \sum_{i=0}^N D_{ki} x^N(\tau_i), i = 0, \dots, N \quad (1.26)$$

where D is the $(N+1) \times (N+1)$ the differentiation matrix given by (1.12).

Define the following $n_u \times (N+1)$ matrix to store the trajectories of the controls at the LGL nodes:

$$U^N = \begin{bmatrix} u_1(\tau_0) & u_1(\tau_1) & \dots & u_1(\tau_N) \\ u_2(\tau_0) & u_2(\tau_1) & \dots & u_2(\tau_N) \\ \vdots & \vdots & \ddots & \vdots \\ u_{n_u}(\tau_0) & u_{n_u}(\tau_1) & \dots & u_{n_u}(\tau_N) \end{bmatrix} \quad (1.27)$$

Define the following $n_x \times (N+1)$ matrices to store, respectively, the trajectories of the states and their derivatives at the LGL nodes:

$$X^N = \begin{bmatrix} x_1(\tau_0) & x_1(\tau_1) & \dots & x_1(\tau_N) \\ x_2(\tau_0) & x_2(\tau_1) & \dots & x_2(\tau_N) \\ \vdots & \vdots & \ddots & \vdots \\ x_{n_x}(\tau_0) & x_{n_x}(\tau_1) & \dots & x_{n_x}(\tau_N) \end{bmatrix} \quad (1.28)$$

and

$$\dot{X}^N = \begin{bmatrix} \dot{x}_1(\tau_0) & \dot{x}_1(\tau_1) & \dots & \dot{x}_1(\tau_N) \\ \dot{x}_2(\tau_0) & \dot{x}_2(\tau_1) & \dots & \dot{x}_2(\tau_N) \\ \vdots & \vdots & \ddots & \vdots \\ \dot{x}_{n_x}(\tau_0) & \dot{x}_{n_x}(\tau_1) & \dots & \dot{x}_{n_x}(\tau_N) \end{bmatrix} \quad (1.29)$$

From (1.26), X^N and $\dot{X}^N$ are related as follows:

$$\dot{X}^N = X^N D^T \quad (1.30)$$

Now, form the following $n_x \times (N+1)$ matrix with the right hand side of the differential constraints evaluated at the LGL nodes:

$$F^N = \frac{t_f - t_0}{2} \begin{bmatrix} f_1(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & f_1(x^N(\tau_N), u^N(\tau_N), p, \tau_N) \\ f_2(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & f_2(x^N(\tau_N), u^N(\tau_N), p, \tau_N) \\ \vdots & \ddots & \vdots \\ f_{n_x}(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & f_{n_x}(x^N(\tau_N), u^N(\tau_N), p, \tau_N) \end{bmatrix} \quad (1.31)$$

Now, define the differential defects at the collocation points as the $n_x \times (N+1)$ matrix:

$$\zeta^N = \dot{X}^N - F^N = X^N D^T - F^N \quad (1.32)$$

Define the matrix of path constraint function values evaluated at the LGL nodes:

$$H^N = \begin{bmatrix} h_1(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & h_1(x^N(\tau_N), u^N(\tau_N), p, \tau_N) \\ h_2(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & h_2(x^N(\tau_N), u^N(\tau_N), p, \tau_N) \\ \vdots & \ddots & \vdots \\ h_{n_h}(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & h_{n_h}(x^N(\tau_N), u^N(\tau_N), p, \tau_N) \end{bmatrix} \quad (1.33)$$

The objective function of $\mathcal{P}_3$ is approximated as follows:

$$\begin{aligned} J &= \varphi[x(-1), x(1), p, t_0, t_f] + \frac{t_f - t_0}{2} \int_{-1}^1 L[x(\tau), u(\tau), p, \tau] d\tau \\ &\approx \varphi[x^N(-1), x^N(1), p, t_0, t_f] + \frac{t_f - t_0}{2} \sum_{k=0}^N L[x^N(\tau_k), u^N(\tau_k), p, \tau_k] w_k \end{aligned} \quad (1.34)$$

where the weights w_k are defined in (1.5).

We are now ready to express problem $\mathcal{P}_3$ as a nonlinear programming problem, as follows.

Problem $\mathcal{P}_4$

$$\min_y F(y) \quad (1.35)$$

subject to:

$$\begin{aligned} G_l &\leq G(y) \leq G_u \\ y_l &\leq y \leq y_u \end{aligned} \quad (1.36)$$

The decision vector y , which has dimension $n_y = (n_u(N+1) + n_x(N+1) + n_p + 2)$, is constructed as follows:

$$y = \begin{bmatrix} \text{vec}(U^N) \\ \text{vec}(X^N) \\ p \\ t_0 \\ t_f \end{bmatrix} \quad (1.37)$$

The objective function is:

$$F(y) = \varphi[x^N(-1), x^N(1), p, t_0, t_f] + \frac{t_f - t_0}{2} \sum_{k=0}^N L[x^N(\tau_k), u^N(\tau_k), p, \tau_k] w_k \quad (1.38)$$

while the constraint function $G(y)$, which is of dimension $n_g = n_x(N+1) + n_h(N+1) + n_e + 1$, is given by:

$$G(y) = \begin{bmatrix} \text{vec}(\zeta^N) \\ \text{vec}(H^N) \\ e[x^N(-1), u^N(-1), x^N(1), u^N(1), p, t_0, t_f] \\ t_f - t_0 \end{bmatrix}, \quad (1.39)$$

The constraint bounds are given by:

$$G_l = \begin{bmatrix} \mathbf{0}_{n_x(N+1)} \\ \text{stack}(h_L, N+1) \\ e_L \\ (t_0 - \bar{t}_f) \end{bmatrix}, \quad G_u = \begin{bmatrix} \mathbf{0}_{n_x(N+1)} \\ \text{stack}(h_U, N+1) \\ e_U \\ 0 \end{bmatrix}, \quad (1.40)$$

and the bounds on the decision vector are given by:

$$y_l = \begin{bmatrix} \text{stack}(u_l, N+1) \\ \text{stack}(x_L, N+1) \\ p_L \\ \underline{t}_0 \\ \underline{t}_f \end{bmatrix}, \quad y_u = \begin{bmatrix} \text{stack}(u_U, N+1) \\ \text{stack}(x_U, N+1) \\ p_U \\ \bar{t}_0 \\ \bar{t}_f \end{bmatrix}, \quad (1.41)$$

where $\text{vec}(A)$ forms a nm -column vector by vertically stacking the columns of the $n \times m$ matrix A , and $\text{stack}(x, n)$ creates a mn -column vector by stacking n copies of column m -vector x .

1.8.1 Costate estimates

The costate estimates for the Legendre and Chebyshev methods are described below; the corresponding estimates for the Radau and Gauss methods are given in Section 1.9.

Legendre approximation method

PSOPT implements the following approximation for the costates $\lambda(\tau) \in \mathbb{R}^{n_x}, \tau \in [-1, 1]$ associated with $\mathcal{P}_3$ [19]:

$$\lambda(\tau) \approx \lambda^N(\tau) = \sum_{k=0}^N \lambda(\tau_k) \phi_k(\tau), \tau \in [-1, 1] \quad (1.42)$$

The costate values at the LGL nodes are given by:

$$\lambda(\tau_k) = \frac{\tilde{\lambda}_k}{w_k}, \quad k = 0, \dots, N \quad (1.43)$$

where w_k are the weights given by (1.5), and $\tilde{\lambda}_k \in \mathbb{R}^{n_x}, k = 0, \dots, N$ are the KKTs multiplier associated with the collocation constraints $\text{vec}(\zeta^N) = 0$. The KKT multipliers can normally be obtained from the NLP solver, which allows PSOPT to return estimates of the costate trajectories at the LGL nodes.

It is known from the literature [19] that the costate estimates in the Legendre discretisation method sometimes oscillate around the true values. To mitigate this, the estimates are smoothed by taking a weighted average for the estimates at k using the costate estimates at $k - 1, k$ and $k + 1$ obtained from (1.42).

Chebyshev approximation method

When the cost quadrature uses the Clenshaw–Curtis weights of Section 1.7.8, the covector (costate) mapping for the Chebyshev method takes the same form as for the Legendre method:

$$\lambda(\tau_k) = \frac{\tilde{\lambda}_k}{w_k}, \quad k = 0, \dots, N \quad (1.44)$$

where w_k are the Clenshaw–Curtis weights (equation (1.20)) and $\tilde{\lambda}_k \in \mathbb{R}^{n_x}, k = 0, \dots, N$ are the KKT multipliers associated with the collocation constraints $\text{vec}(\zeta^N) = 0$. Because these weights are strictly positive at every CGL node, including the endpoints $\tau_0 = -1$ and $\tau_N = 1$, equation (1.44) is well defined for all $k = 0, \dots, N$ and, unlike the earlier mapping that carried a $1/\sqrt{1 - \tau_k^2}$ factor [37], requires no endpoint extrapolation. With this mapping the smoothing applied to the Legendre estimates (Section 1.8.1) is not needed for the Chebyshev method.

1.8.2 Discretizing a multiphase problem

It now becomes straightforward to describe the discretisation used by PSOPT in the case of $\mathcal{P}_1$, a problem with multiple phases, to form a nonlinear programming problem like $\mathcal{P}_4$. The decision variables of the NLP associated with $\mathcal{P}_1$ are given by:

$$y = \begin{bmatrix} \text{vec}(U^{N,(1)}) \\ \text{vec}(X^{N,(1)}) \\ p^{(1)} \\ t_0^{(1)} \\ t_f^{(1)} \\ \vdots \\ \text{vec}(U^{N,(N_p)}) \\ \text{vec}(X^{N,(N_p)}) \\ p^{(N_p)} \\ t_0^{(N_p)} \\ t_f^{(N_p)} \end{bmatrix} \quad (1.45)$$

where N_p is the number of phases in the problem, and the superindex in parenthesis indicates the phase to which the variables belong. The constraint function $G(y)$ is given by:

$$G(y) = \begin{bmatrix} \text{vec}(\zeta^{N,(1)}) \\ \text{vec}(H^{N,(1)}) \\ e[x^{N,(1)}(-1), u^{N,(1)}(-1), x^{N,(1)}(1), u^{N,(1)}(1), p^{(1)}, t_0^{(1)}, t_f^{(1)}] \\ t_f^{(1)} - t_0^{(1)} \\ \vdots \\ \text{vec}(\zeta^{N,(N_p)}) \\ \text{vec}(H^{N,(N_p)}) \\ e[x^{N,(N_p)}(-1), u^{N,(N_p)}(-1), x^{N,(N_p)}(1), u^{N,(N_p)}(1), p^{(N_p)}, t_0^{(N_p)}, t_f^{(N_p)}] \\ t_f^{(N_p)} - t_0^{(N_p)} \\ \Psi \end{bmatrix}, \quad (1.46)$$

where Ψ corresponds to the linkage constraints associated with the problem, evaluated at y .

Based on the problem information, it is straightforward (but not shown here) to form the bounds on the decision variables y_l , y_u and the bounds on the constraints function G_l , G_u to complete the definition of the NLP problem associated with $\mathcal{P}_1$.

1.9 Radau and Gauss pseudospectral discretisations

Besides the Legendre (LGL) and Chebyshev (CGL) methods mentioned earlier, PSOPT also offers two more global pseudospectral discretisations. You can select them by set-

ting `algorithm.collocation_method` to "Radau" or "Gauss". All three Legendre families use the roots of the Legendre polynomial L_N and its derivative, but they differ in which endpoints of $[-1, 1]$ they include as collocation points: LGL points include both endpoints, Legendre-Gauss-Radau (LGR) points include only one, and Legendre-Gauss (LG) points include neither [21]. Although this may seem like a small difference, it has a significant effect on the accuracy of costate estimates. This is the main reason for providing these options. Detailed definitions of the node sets and a unified explanation of the three methods are provided by Garg et al. [21, 22].

1.9.1 Legendre–Gauss–Radau (LGR) collocation

The LGR points include one endpoint of $[-1, 1]$ along with N interior points. The other endpoint is not collocated. The state and control are interpolated by Lagrange polynomials, and the dynamics are enforced at the LGR points using a rectangular, full-rank differentiation matrix [21]. The rest of the discretisation follows the same approach as the Legendre method described earlier. Specifically, the differential defects $\zeta = XD^\top - F$ are set as equality constraints, and the Lagrange term of the cost is calculated using the Gauss-Radau quadrature weights. The Radau scheme is asymmetric in time, which makes it well-suited for problems similar to initial-value problems and for the individual phases of a multi-phase formulation.

1.9.2 Legendre–Gauss (LG) collocation and the terminal-state variable

The LG points are the N interior roots of the Legendre polynomial L_N ; neither $\tau = -1$ nor $\tau = +1$ is collocated. Because the final state is then not attached to any collocation node, the Gauss scheme introduces the terminal state $x_f = x(t_f)$ as an explicit decision variable, related to the initial state through a Gauss quadrature of the dynamics,

$$x_f = x(t_0) + \frac{t_f - t_0}{2} \sum_{k=1}^N w_k f[x(\tau_k), u(\tau_k), p, \tau_k], \quad (1.47)$$

which is appended to the nonlinear program as an additional vector equality constraint, while the dynamics are collocated at the N interior LG points. This construction, due to Benson [3] and placed in a unified framework by Garg et al. [21], is what gives the Gauss method its characteristically accurate costates.

1.9.3 Costate estimates for the Radau and Gauss methods

For both schemes the discrete costates are recovered from the Karush–Kuhn–Tucker multipliers $\tilde{\lambda}_k$ of the differential defect constraints, scaled by the quadrature weights,

$$\lambda(\tau_k) = \frac{\tilde{\lambda}_k}{w_k}, \quad (1.48)$$

in close analogy with the Legendre case of Section 1.8.1. The essential difference is that the LGR and LG differentiation operators are full rank, so the discrete adjoint system

is an exact transcription of the continuous one; the resulting costates do not exhibit the boundary oscillations of the LGL method, and the Gauss method in particular yields the most accurate costates of the pseudospectral families [21, 22]. A unified derivation of these costate mappings is given in [21].

For the Radau scheme (1.48) is the whole of the mapping. For the Gauss scheme it is only the interior part of it, and taken alone it is not the costate: because the Gauss dynamics are linked to the terminal state through the quadrature constraint (1.47) as opposed to by collocating at the endpoint, (1.48) recovers each interior costate *relative to* the costate at the right end of its interval. PSOPT therefore completes the Gauss mapping in three further steps, following the Darby–Garg–Rao covector mapping generalised to a multi-interval mesh:

- the multiplier $\lambda_{f,j}$ of interval j 's quadrature constraint is added back to that interval's interior costates, undoing the constant shift;
- each interface breakpoint carries $\lambda_{f,j}$, which is the costate at the right end of interval j and the left end of interval $j + 1$, and the terminal costate is $\lambda_{f,K}$ for the last interval K ;
- the initial costate at $\tau = -1$, where the Gauss scheme does not collocate and so has no multiplier of its own, is obtained by Lagrange extrapolation from the corrected interior costates of the first interval. The extrapolation is local to that interval: over the clustered nodes of a multi-interval mesh a global extrapolation would be ill-conditioned.

With a single interval this reduces to the recovery given by Benson.

1.10 Local discretisations

Direct collocation methods that use local information to approximate the functions associated with an optimal control problem are well established [5]. These include for instance the trapezoidal and Hermite-Simpson methods. A further, non-collocation alternative (the integrated residual transcription, which controls the dynamic error between the nodes instead of only at them) is described in Chapter 3. Sometimes, it may be convenient for users to compare the performance and solutions obtained by means of the pseudospectral methods implemented in PSOPT, with local discretisation methods. Also, if a given problem cannot be solved by means of a pseudospectral discretisation, the user has the option to try the local discretisations implemented in PSOPT. The main impact of using a local discretisation method as opposed to a pseudospectral discretisation method, is that the resulting Jacobian and Hessian matrices needed by the NLP solver are more sparse with local methods, which facilitates the NLP solution. This becomes more noticeable as the number of grid points increases. The disadvantage of using a local method is that the spectral accuracy in the discretisation of the differential constraints offered by pseudospectral methods is lost. Moreover, the accuracy of Gauss

type integration employed in pseudospectral methods is also lost if pseudospectral grids are not used.

Note also that local mesh refinement methods are well established. These methods concentrate more grid points in areas of greater activity in the function, which helps improve the local accuracy of the solution. The trapezoidal method has an accuracy of $\mathcal{O}(h^2)$, while the Hermite-Simpson method has an accuracy of $\mathcal{O}(h^4)$, where h is the local interval between grid points. Both the trapezoidal and Hermite-Simpson discretisation methods are widely used in computational optimal control, and have solve many challenging problems [5]. When the user selects the trapezoidal or Hermite-Simpson discretisations, and if the initial grid points are not provided, the grid is started with equal spacing between grid points. In these two cases any integrals associated with the problem are computed using the trapezoidal and Simpson quadrature method, respectively.

The local discretisations implemented in PSOPT are described below. For simplicity, the phase index has been omitted and reference is made to single phase problems. However, the methods can also be used with multi-phase problems.

1.10.1 Trapezoidal method

With the trapezoidal method [5], the defect constraints are computed as follows:

$$\zeta(\tau_k) = x(\tau_{k+1}) - x(\tau_k) - \frac{h_k}{2}(f_k + f_{k+1}), \quad (1.49)$$

where $\zeta(\tau_k) \in \mathbb{R}^{n_x}$ is the vector of differential defect constraints at node τ_k , $k = 0, \dots, N-1$, $h_k = \tau_{k+1} - \tau_k$, $f_k = f[(\tau_k), u(\tau_k), p, \tau_k]$, $f_{k+1} = f[x(\tau_{k+1}), u(\tau_{k+1}), p, \tau_{k+1}]$. This gives rise to $n_x N$ differential defect constraints. In this case, the decision vector for single phase problems is given by equation (1.37), so that it is the same as the one used in the Legendre and Chebyshev methods.

1.10.2 Hermite-Simpson method

With the Hermite-Simpson method [5], the defect constraints are computed as follows:

$$\zeta(\tau_k) = x(\tau_{k+1}) - x(\tau_k) - \frac{h_k}{6}(f_k + 4\bar{f}_{k+1} + f_{k+1}), \quad (1.50)$$

where

$$\begin{aligned} \bar{f}_{k+1} &= f[\bar{x}_{k+1}, \bar{u}_{k+1}, p, \tau_k + \frac{h_k}{2}] \\ \bar{x}_{k+1} &= \frac{1}{2}(x(\tau_k) + x(\tau_{k+1})) + \frac{h_k}{8}(f_k - f_{k+1}) \end{aligned}$$

where $\zeta(\tau_k) \in \mathbb{R}^{n_x}$ is the vector of differential defect constraints at node τ_k , $k = 0, \dots, N-1$, $h_k = \tau_{k+1} - \tau_k$, $f_k = f[(\tau_k), u(\tau_k), p, \tau_k]$, $f_{k+1} = f[x(\tau_{k+1}), u(\tau_{k+1}), p, \tau_{k+1}]$, and $\bar{u}_{k+1} = \bar{u}(\tau_{k+1})$ is a vector of midpoint controls (which are also decision variables).

This gives rise to $n_x N$ differential defect constraints. In this case, the decision vector for single phase problems is given by

$$y = \begin{bmatrix} \text{vec}(U^N) \\ \text{vec}(X^N) \\ p \\ \text{vec}(\bar{U}^N) \\ t_0 \\ t_f \end{bmatrix} \quad (1.51)$$

with

$$\bar{U}^N = \begin{bmatrix} \bar{u}_1(\tau_1) & \bar{u}_1(\tau_2) & \dots & \bar{u}_1(\tau_N) \\ \bar{u}_2(\tau_1) & \bar{u}_2(\tau_2) & \dots & \bar{u}_2(\tau_N) \\ \vdots & \vdots & \ddots & \vdots \\ \bar{u}_{n_u}(\tau_1) & \bar{u}_{n_u}(\tau_2) & \dots & \bar{u}_{n_u}(\tau_N) \end{bmatrix} \quad (1.52)$$

so that this decision vector is different from the one used in the Legendre and Chebyshev methods as it includes the midpoint controls.

1.10.3 Costate estimates with local discretisations

In the case of the trapezoidal and Hermite-Simpson discretisations, the costates at the discretisation nodes are approximated according to the following equation:

$$\lambda(t_{k+\frac{1}{2}}) \approx \frac{\tilde{\lambda}_k}{2h_k}, \quad k = 0, \dots, N-1$$

where $\lambda(t_{k+\frac{1}{2}})$ is the costate estimate at the midpoint in the interval between t_k and t_{k+1} , $\tilde{\lambda}_k \in \mathbb{R}^{n_x}$ is the vector of Lagrange multipliers obtained from the nonlinear programming solver corresponding to the differential defect constraints, and $h_k = t_{k+1} - t_k$.

1.11 Parameter estimation problems

A parameter estimation problem arises when it is required to find values for parameters associated with a model of a system based on observations from the actual system. These are also called *inverse problems*. The approach used in the PSOPT implementation uses the same techniques used for solving optimal control problems, with a special objective function used to measure the accuracy of the model for given parameter values.

1.11.1 Single-phase case

For the sake of simplicity consider first a single phase problem defined over $t_0 \leq t \leq t_f$ with the dynamics given by a set of ODEs:

$$\dot{x} = f[x(t), u(t), p, t]$$

the path constraints

$$h_L \leq h[x(t), u(t), p, t] \leq h_U$$

the event constraints

$$e_L \leq e[x(t_0), u(t_0), x(t_f), u(t_f), p, t_0, t_f] \leq e_U$$

Consider the following model of the observations (or measurements) taken from the system:

$$y(\theta) = g[x(\theta), u(\theta), p, \theta]$$

where $g : \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times \mathcal{R} \rightarrow \mathcal{R}^{n_o}$ is the observations function, and $y(\theta) \in \mathcal{R}^{n_o}$ is the estimated observation at sampling instant θ . Assume that $\{\tilde{y}\}_{k=1}^{n_s}$ is a sequence of n_s observations corresponding to the sampling instants $\{\theta_k\}_{k=1}^{n_s}$.

The objective is to choose the parameter vector $p \in \mathcal{R}^{n_p}$ to minimise the cost function:

$$J = \sum_{k=1}^{n_s} \sum_{j=1}^{n_o} r_{j,k}^2$$

where the residual $r_{j,k} \in \mathcal{R}$ is given by:

$$r_{j,k} = w_{j,k}[g_j[x(\theta_k), u(\theta_k), p, \theta_k] - \tilde{y}_{j,k}]$$

where $w_{j,k} \in \mathcal{R}$, $j = 1, \dots, n_o$, $k = 1, \dots, n_s$ is a positive residual weight, g_j is the j -th element of the vector observations function g , and $\tilde{y}_{j,k}$ is the j -th element of the actual observation vector at time instant θ_k .

Note that in the parameter estimation case the times t_0 and t_f are assumed to be fixed. The sampling instants need not coincide with the collocation points, but they must obey the relationship:

$$t_0 \leq \theta_k \leq t_f, \quad k = 1, \dots, n_s$$

1.11.2 Multi-phase case

In the case of a problem with N_p phases, let $t_0^{(i)} \leq t \leq t_f^{(i)}$ be the intervals for each phase, with the dynamics given by a set of ODEs:

$$\dot{x}^{(i)} = f^{(i)}[x(t)^{(i)}, u^{(i)}(t), p^{(i)}, t]$$

the path constraints

$$h_L^{(i)} \leq h^{(i)}[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t] \leq h_U^{(i)}$$

the event constraints

$$e_L^{(i)} \leq e^{(i)}[x^{(i)}(t_0), u^{(i)}(t_0), x^{(i)}(t_f), u^{(i)}(t_f), p^{(i)}, t_0^{(i)}, t_f^{(i)}] \leq e_U^{(i)}$$

Consider the following model of the observations (or measurements) taken from the system for each phase:

$$y^{(i)}(\theta_k^{(i)}) = g^{(i)}[x^{(i)}(\theta_k^{(i)}), u^{(i)}(\theta_k^{(i)}), p^{(i)}, \theta_k^{(i)}]$$

where $g^{(i)} : \mathcal{R}^{n_x^{(i)}} \times \mathcal{R}^{n_u^{(i)}} \times \mathcal{R}^{n_p^{(i)}} \times \mathcal{R} \rightarrow \mathcal{R}^{n_o^{(i)}}$ is the observations function for each phase, and $y^{(i)}(\theta_k^{(i)}) \in \mathcal{R}^{n_o^{(i)}}$ is the estimated observation at sampling instant $\theta_k^{(i)}$. Assume that $\{\tilde{y}^{(i)}\}_{k=1}^{n_s^{(i)}}$ is a sequence of $n_s^{(i)}$ observations corresponding to the sampling instants $\{\theta_k^{(i)}\}_{k=1}^{n_s^{(i)}}$.

The objective is to choose the set of parameter vectors $p^{(i)} \in \mathcal{R}^{n_p^{(i)}}$, $i \in [1, N_p]$ to minimise the cost function:

$$J = \sum_{i=1}^{N_p} \sum_{k=1}^{n_s^{(i)}} \sum_{j=1}^{n_o^{(i)}} \left[r_{j,k}^{(i)} \right]^2$$

where the residual $r_{j,k}^{(i)} \in \mathcal{R}$ is given by:

$$r_{j,k}^{(i)} = w_{j,k}^{(i)} [g_j^{(i)}[x(\theta_k^{(i)}), u^{(i)}(\theta_k^{(i)}), p^{(i)}, \theta_k^{(i)}] - \tilde{y}_j^{(i)}]$$

where $w_{j,k}^{(i)} \in \mathcal{R}$ is a positive residual weight, $g_j^{(i)}$ is the j -th element of the vector observations function $g^{(i)}$, and $\tilde{y}_j^{(i)}$ is the j -th element of the actual observation vector at time instant $\theta_k^{(i)}$

Note that in the parameter estimation case the times $t_0^{(i)}$ and $t_f^{(i)}$ are assumed to be fixed. The sampling instants need not coincide with the collocation points, but they must obey the relationship:

$$t_0^{(i)} \leq \theta_k^{(i)} \leq t_f^{(i)}, \quad k = 1, \dots, n_s^{(i)}$$

1.11.3 Statistical measures on parameter estimates

PSOPT computes a residual matrix $[r_{j,k}^{(i)}]$ for each phase i and, for the final value of the estimated parameters in all phases $\hat{p} \in \mathcal{R}^{n_p}$, the covariance matrix of those estimates and 95% confidence intervals on them. All of it requires

```
algorithm.parameter_statistics = "yes";
```

which is off by default because the calculation costs two Jacobians and a QR decomposition of a matrix the size of the discretised problem.

The covariance. Stack the residuals of every phase into a vector $r(z) \in \mathcal{R}^{N_s}$, where

$$N_s = \sum_{i=1}^{N_p} n_s^{(i)} n_o^{(i)} \tag{1.53}$$

is the total number of *scalar* observations and z is the full vector of decision variables. Note that N_s counts individual measurements and not sampling instants: two observed variables sampled at 21 instants contribute $N_s = 42$.

Because the discretised states are decision variables tied to the parameters by the collocation defects, this is a *constrained* least-squares problem and the covariance cannot be obtained from the residual Jacobian alone. Let $J_r = \partial r / \partial z$, let J_c be the Jacobian of the equality constraints *together with every active inequality and every active simple bound*, and let the columns of Z be an orthonormal basis for the null space of J_c , taken from a rank-revealing QR decomposition of $J_c^\top$. Then [29, 6]

$$C_z = \hat{\sigma}^2 Z \left(Z^\top J_r^\top J_r Z \right)^{-1} Z^\top, \quad (1.54)$$

and the covariance $C \in \mathcal{R}^{n_p \times n_p}$ of the parameter estimates is the submatrix of C_z belonging to the static parameters. Two parts of this deserve emphasis, because both have been got wrong in PSOPT itself and neither error is visible in the result: the *active bounds* belong in J_c , since a variable pinned at a bound – in an estimation problem this always includes the fixed t_0 and t_f – is not free to vary and must not contribute a direction to Z ; and (1.54) is the *inverse* of the reduced Gauss–Newton matrix, which is an information matrix, so reporting the latter gives roughly the reciprocal of the covariance.

The variance factor. The scalar $\hat{\sigma}^2$ accounts for the scale of the measurement noise. If the weights are $w_{j,k} = 1/\sigma_{j,k}$, so that the residuals have unit variance, then $\hat{\sigma}^2 = 1$ is correct. When the noise level is not known in advance, which is the usual case, PSOPT estimates it from the residuals,

$$\hat{\sigma}^2 = \frac{J^*}{N_s - n_f}, \quad (1.55)$$

where J^* is the optimal objective and $n_f = n_z - n_c$ is the number of degrees of freedom of the discretised problem: the number of quantities actually being fitted, which is the number of parameters only when nothing else is free. Estimating the factor is also the right thing to do when the weights are already $1/\sigma$, since it then comes out near unity of its own accord. Omitting it is a common source of intervals wrong by orders of magnitude in either direction.

The Jacobians above are taken with respect to the *scaled* decision variables, so C_z is a covariance in scaled units; PSOPT converts it to physical units before it is reported, dividing entry (i, j) by $s_i s_j$.

The confidence intervals. Given C , the limits about $\hat{p}_i$ follow in the usual way [31]:

$$\delta_i = \pm t_{N_s - n_f}^{1 - (\alpha/2)} \sqrt{C_{ii}} \quad (1.56)$$

where t is the inverse two-tailed cumulative t-distribution with confidence level α and $N_s - n_f$ degrees of freedom – again n_f , not the number of parameters.

What is reported. With `algorithm.print_level` above zero the solution summary file carries the covariance matrix and its rank, the correlation matrix with entries $C_{ij}/\sqrt{C_{ii}C_{jj}}$, the values $\hat{\sigma}$, N_s , n_f and J^* , the confidence limits, and the residual of every observed variable at every sampling instant. The correlation matrix should be read before the diagonal: it is what says whether two parameters are separately identifiable, and a pair correlated at -0.99 is a warning that the data determine one combination of them far better than either alone.

The same quantities are available programmatically, whatever the print level, on the solution object:

<code>solution.parameter_statistics_ok</code>	<code>// false if it could not be computed</code>
<code>solution.parameter_covariance</code>	<code>// n_p x n_p, physical units</code>
<code>solution.parameter_confidence_low</code>	<code>// 95 per cent limits</code>
<code>solution.parameter_confidence_high</code>	
<code>solution.sigma_hat</code>	<code>// estimated residual standard deviation</code>
<code>solution.parameter_dof</code>	<code>// n_f</code>
<code>solution.n_observations</code>	<code>// N_s</code>
<code>solution.observation_residuals</code>	<code>// one entry per scalar observation</code>

A caveat on the discretisation. The tangent space Z belongs to the discretised constraint set, not the continuous problem, and these two can be different. A global pseudospectral discretisation collocates the dynamics at every node while the boundary conditions are imposed on those same node values, so it carries one more independent condition per phase than the continuous problem. As a result, the null space is one dimension shorter, and the covariance matrix becomes rank deficient. The degrees of freedom reported can be fewer than the number of parameters being estimated, and the correlation between two parameters can be exactly ± 1 . The confidence limits themselves are barely affected, which is why this is important to know. PSOPT prints a warning when it happens. A local discretisation ("`trapezoidal`" or "`Hermite-Simpson`") does not have the defect.

The residual matrix, the covariance matrix, and the confidence intervals can be used to refine the parameter estimation problem. If one parameter has a large confidence interval, so the data provide little information about it, its value may be fixed in a later run. This often improves the estimates of the remaining parameters and reduces the chance of having an over-parameterised model. See [44] pages 210–211 for more details.

However, notice that the statistical analysis is based on a linearisation of the model. So, its validity depends on how good that linearisation is and on the curvature of the functions being linearised. Care must be taken when interpreting the results [44].

1.11.4 Remarks on parameter estimation

- In contrast to continuous optimal control problems, the kind of parameter estimation problem considered involves the evaluation of the objective at a finite number of sampling points. Internally, the values of the state and controls are interpolated

over the collocation points to find estimated values at the sampling points. The type of interpolation employed depends on the collocation method specified by the user.

- Note that the sampling instants do not have to be sorted in ascending or descending order. Because of this, it is possible to accommodate problems with non-simultaneous observations of different variables by stacking the measured data and sampling instants for the different variables.
- It is possible to use an alternative objective function where the residuals are weighted with the covariance of the measurements simply by multiplying the observations function and the measurements vectors by the square root of the covariance matrix; see [5] for more details.
- When defining parameter estimation problems, the user needs to ensure that the underlying nonlinear programming problem has sufficient degrees of freedom. This is particularly important as it is common for parameter estimation problems not to involve any control variables. The number of degrees of freedom is the difference between the number of decision variables and the total number of equality and active inequality constraints. For example, in the case of a single phase problem having n_x differential states, n_u control variables (or algebraic states), n_h equality path constraints, and n_e equality event constraints, the number of relevant constraints is given by:

$$n_c = n_x(N + 1) + n_h(N + 1) + n_e + 1$$

where N is the degree of the polynomial approximation (in the case of a pseudospectral discretisation). The number of decision variables is given by:

$$n_y = n_u(N + 1) + n_x(N + 1) + n_p + 2$$

The difference is:

$$n_y - n_c = (n_u - n_h)(N + 1) + n_p + 1 - n_e$$

For the problem to be solvable it is important that $n_y - n_c \geq 0$, ideally $n_y - n_c \geq 1$. The total numbers of constraints and decision variables are always reported in the terminal window when a PSOPT problem is run. It should be noted that the nonlinear programming solver (IPOPT) may modify the numbers by eliminating redundant constraints or decision variables. These modifications are also visible when a problem is run.

1.12 Limitations and known issues

1. The discretisation techniques used by PSOPT give approximate solutions for the state and control trajectories. The software is intended to be used for problems

where the control variables are continuous (within a phase) and the state variables have continuous derivatives (within a phase). If within a phase the solution to the optimal control problem is of a different nature, the results may be incorrect or the optimization algorithm may fail to converge. Furthermore, PSOPT may not be suitable for solving problems involving differential-algebraic equations with index greater than one. Some of these issues can be avoided by reformulating the problem to have several phases. A control that is discontinuous at an isolated, unknown switching time is also addressed by the flexible mesh of Section 3.9, which makes the element boundaries decision variables so that the optimisation can isolate the switch; a multi-phase formulation remains the better tool when the switching structure is known in advance.

2. The solution obtained by PSOPT corresponds to a local minimum of the discretized optimization problem. If the problem is suspected to have several local minima, then it may be worth trying various initial guesses.
3. The automatic scaling procedures work well for all the examples provided. However, note that the scaling of variables depends on the user provided bounds. If these bounds are not adequate for the problem, then the resulting scaling may be poor and this may lead to incorrect results or convergence problems. In some cases, users may need to provide the scaling factors manually to obtain satisfactory results.
4. The automatic mesh refinement procedures require an initial guess for the number of nodes in the global case (the number and/or initial distribution of nodes in the local case). If this initial guess is not adequate (e.g. the grid is too coarse or too dense), the mesh refinement procedure may fail to converge. In some cases, the user may need to manually tune some of the parameters of the mesh refinement procedure to achieve satisfactory results.
5. The efficiency with which the optimal control problem is solved depends in a good deal on the correct formulation of the problem. Unsuitable formulations may lead to trouble in finding a solution. Moreover, if the constraints are such that the problem is infeasible or if for any other reason the solution does not exist, then the nonlinear programming algorithm will fail.
6. The user supplied functions which define the cost function, DAE's, event and linkage constraints, are all assumed to be continuous and to have continuous first and second derivatives. Non-differentiable functions may cause convergence problems to the optimization algorithm. Moreover, it is known that discontinuities in the second derivatives may also cause convergence problems.
7. Only single phase problems are supported if the dynamics involve delays in the states or controls.

8. Note that the constraints associated with the problem are only enforced at the discretisation nodes, not in the interval between the nodes. The integrated residual transcription of Chapter 3 mitigates this by measuring, and optionally bounding, the dynamic residual between the nodes.
9. When the problem requires a large number of nodes (say over 200) the nonlinear programming algorithm may have problems to converge if global collocation is being used. This may be due to numerical difficulties within the nonlinear programming solver as the Jacobian (and Hessian) matrices may not be sufficiently sparse. This occurs because the pseudospectral differentiation matrices are dense. When faced with this problem the user may wish to try the local collocation options available within PSOPT, or to split the problem into multiple segments to increase the sparsity of the derivatives. Note that the sparsity of the Jacobian and Hessian matrices is problem dependent.
10. The co-state approximations resulting from the Legendre pseudospectral method are not as accurate as those obtained by means of the Gauss or Radau pseudospectral methods now available in PSOPT (Section 1.9); the Gauss method in particular yields the most accurate costates [21]. Moreover, the co-state approximations obtained by PSOPT using the Chebyshev pseudospectral methods are less accurate than those of the Gauss or Radau methods, although they are now in the same band as the Legendre ones: both Lobatto schemes collocate their endpoints and both therefore carry the alternating covector mode that Fahroo and Ross describe, and the same node-aware filter is applied to both. On a minimum-energy double integrator, where the discretisation contributes nothing and the whole of the error is the covector mapping, filtering improves the Chebyshev costates by about a factor of ten at every node count. Also the co-state approximation used in the case of local discretisations (trapezoidal, Hermite-Simpson) converges at a lower rate (is less accurate) than the states or the controls.

2. Mesh refinement

It is possible to refine the mesh of the discretisation nodes to improve the accuracy of the solutions. With PSOPT, this can be done manually or automatically. To be able to refine a discretisation mesh, it is necessary to have some measure of the discretisation error. This is discussed in the next section.

2.1 Evaluating the discretisation error

PSOPT evaluates the discretisation error using a method adopted from [5]. Define the error in the differential equation as a function of time:

$$\epsilon(t) = \dot{\tilde{x}}(t) - f[\tilde{x}(t), \tilde{u}(t), p, t]$$

where $\tilde{x}$ is an interpolated value of the state vector given the grid point values of the state vector, $\dot{\tilde{x}}$ is an estimate of the derivative of the state vector given the state vector interpolant, and $\tilde{u}$ is an interpolated value of the control vector given the grid points values of the control vector. The type of interpolation used depends on the collocation method employed. For Legendre and Chebyshev methods, the interpolation done by the Lagrange interpolant. For Trapezoidal and Hermite-Simpson methods, cubic spline interpolation is used. The absolute local error corresponding to state i on a particular interval $t \in [t_k, t_{k+1}]$, is defined as follows:

$$\eta_{i,k} = \int_{t_k}^{t_{k+1}} |\epsilon_i(t)| dt$$

where the integral is computed using the composite Simpson method. The default number of integration steps for each interval is 10, but this can be changed by means of the input parameter `algorithm.nsteps_error_integration`. The relative local error is defined as:

$$\epsilon_k = \max_i \frac{\eta_{i,k}}{w_i + 1}$$

where

$$w_i = \max_{k=1}^N [|\tilde{x}_{i,k}|, |\dot{\tilde{x}}_{i,k}|]$$

After each PSOPT run, the sequence ϵ_k for each phase is available through the solution structure as follows:

```
epsilon = solution.get_relative_local_error_in_phase(iphase)
```

where `epsilon` is a `MatrixXd` object. The error sequence can be analysed by the user to assess the quality of the discretisation. This information may be useful to aid the mesh refinement process.

Additionally, the maximum value of the sequence ϵ_k for each phase is printed in the solution summary at the end of an execution.

2.2 Manual mesh refinement

Manual mesh refinement, which is the default option, is performed by interpolating a previous solution based on n_1 nodes, into a new mesh based on n_2 nodes, where $n_2 > n_1$, and using the interpolated solution as an initial guess for a new optimization. If global collocation is being used, PSOPT employs Lagrange polynomials to perform the interpolation associated with mesh refinement. If local collocation is being used, PSOPT employs cubic splines to perform the interpolation. The variables which are interpolated include the controls, states and Lagrange multipliers associated with the differential defect constraints, which are related to the co-states. The other decision variables (start and final times, and static parameters) do not need to be interpolated.

To perform mesh refinement, the user must supply the desired sequence of grid points (or nodes) for each phase through the parameter:

```
problem.phases(iphase).nodes
```

For example, to try the sequence 40, 50 and 80 nodes in phase `iphase`, then the following command specifies that:

```
RowVectorXi my_vector << 40, 50, 80;
problem.phases(iphase).nodes = my_vector;
```

If the user wishes to try only a single grid size (with no mesh refinement), this is specified by providing a single value as follows:

```
problem.phases(iphase).nodes << INTEGER;
```

In problems with more than one phase, the length of the node sequence to be tried needs to be the same in each phase, but the actual grid sizes need not be the same between phases.

2.3 Automatic *hp*-adaptive mesh refinement with pseudospectral grids

When a global pseudospectral method (Legendre, Chebyshev, Radau or Gauss) is selected and `algorithm.mesh_refinement` is set to "automatic", PSOPT refines the mesh

using an *hp*-adaptive (or *ph*) strategy in the spirit of [16, 36, 30]. Instead of raising the order of a single global block, the time domain of each phase is partitioned into K subintervals (segments), and on segment j the state is approximated by a local polynomial of degree n_j . The refinement adjusts simultaneously the number of segments K , the breakpoints that separate them, and the per-segment degrees n_j , so as to reach a user-specified accuracy with an economical distribution of collocation points. The total number of collocation points in a phase is $N_{\text{eff}} = \sum_{j=1}^K n_j$.

Per-segment error and smoothness. After each solve PSOPT computes the maximum relative discretisation error ϵ_j on every segment j (Section 2.1). Segments whose error is already below the tolerance ϵ_{max} (`ode_tolerance`) are left unchanged. For each remaining segment the smoothness of the local solution is assessed from the decay rate of its Legendre coefficients: writing the local state as $x(\tau) \approx \sum_{\ell=0}^{n_j} a_\ell P_\ell(\tau)$, a rapid (geometric) decay of $|a_\ell|$ indicates a smooth solution, whereas a slow decay flags a likely nonsmooth feature, such as a kink or a near-discontinuity, within the segment [30].

***p*- versus *h*-refinement.** The smoothness assessment selects the action taken on each flagged segment:

- *p-refinement* (smooth segment): increase the local polynomial degree $n_j \leftarrow \min(n_j + \Delta n_j, N_{\text{max}})$. When the solution is smooth, the spectral convergence of the pseudospectral approximation makes raising the degree the most efficient way to reduce the error.
- *h-refinement* (nonsmooth segment, or n_j already at N_{max}): split the segment in two, each half seeded at degree $\max(N_{\text{min}}, \min(N_{\text{max}}, \lfloor n_j/2 \rfloor + 1))$. The seed is therefore taken from the degree of the segment being split: splitting a segment of degree n_j produces two halves of roughly half that degree, so the split is approximately node-preserving and the mesh does not lose the resolution already paid for. With the default band a segment at $N_{\text{max}} = 12$ splits into two halves of degree 7, not 3. Placing breakpoints around the nonsmooth feature confines the low-order region and restores fast convergence on the smooth parts.

The degrees are constrained to the band $N_{\text{min}} \leq n_j \leq N_{\text{max}}$ (`mr_min_order` and `mr_max_order`), where N_{max} is the threshold at which an interval switches from *p*- to *h*-refinement. The per-iteration growth of N_{eff} is limited to a factor `mr_max_growth_factor` of its current value, so that the mesh grows gradually and the worst-case problem size is bounded a priori.

Common driver for the four methods. All four global methods share this driver. The Legendre (LGL) and Chebyshev (CGL) methods collocate at both segment endpoints and the Radau (LGR) method at the left endpoint, so adjacent segments share a breakpoint; the Gauss (LG) method collocates at the strictly interior points, and the breakpoints enter as additional non-collocated interface variables (Section 1.9). These

structural differences are handled internally, and the user interface and the refinement controls are identical across the four methods.

The user specifies an initial number of nodes for each phase,

```
problem.phases(iphase).nodes << INTEGER;
```

which seeds the first iteration as a single segment, together with (optionally) the controls listed below. The algorithm proceeds as follows.

1. Set the iteration index $m = 1$ and start from the seed mesh.
2. Solve the nonlinear programming problem on the current mesh and compute the per-segment errors $\epsilon_j^{(i,m)}$ for every phase i .
3. If $\max_j \epsilon_j^{(i,m)} \leq \epsilon_{\max}$ for all phases, terminate.
4. If $m \geq m_{\max}$ (`mr_max_iterations`), terminate.
5. For each phase build the next mesh: leave the converged segments unchanged and apply p - or h -refinement to each remaining segment according to its smoothness assessment, subject to the degree band $[N_{\min}, N_{\max}]$ and the per-iteration growth limit.
6. Interpolate the previous solution onto the new mesh to warm-start the next solve, set $m \leftarrow m + 1$, and return to step 2.

The parameters that control the procedure, with their default values, are as follows.

Maximum relative discretisation error, $\epsilon_{\max}$:

```
algorithm.ode_tolerance = 1.e-3;
```

Maximum number of mesh refinement iterations, $m_{\max}$:

```
algorithm.mr_max_iterations = 7;
```

Minimum per-segment polynomial degree, $N_{\min}$:

```
algorithm.mr_min_order = 3;
```

Maximum per-segment polynomial degree, $N_{\max}$:

```
algorithm.mr_max_order = 12;
```

Maximum per-iteration growth factor of N_{eff} :

```
algorithm.mr_max_growth_factor = 0.4;
```

The reader is referred to [16, 36, 30] for the details and convergence properties of hp -adaptive pseudospectral mesh refinement.

2.4 Automatic mesh refinement with local collocation

If a local collocation method (trapezoidal, Hermite-Simpson) is being used, and the user parameter `algorithm.mesh_refinement` is set to "automatic", then, mesh refinement is carried out as described below. PSOPT will compute the discretisation error $\epsilon^{(i,m)}$ for every phase i at every mesh refinement iteration m , as described in Section 2.1. The method is the local mesh refinement algorithm of Betts, and is described in narrative form below; readers who need the algorithm stated in full, together with the derivation of its error-prediction and order-reduction formulae, should consult the book by Betts [5], which is the authoritative source. The order p of the discretisation is 2 for the trapezoidal method and 4 for the Hermite-Simpson method, and the two are tied together: raising p from 2 to 4 is what switches PSOPT from the trapezoidal to the Hermite-Simpson discretisation.

The user specifies an initial number of nodes for each phase, as follows:

```
problem.phases(iphase).nodes << INTEGER.
```

The user may also specify values for the following parameters which control the mesh refinement procedure. The default values are those shown:

Maximum discretisation error, $\epsilon_{\max}$:

```
algorithm.ode_tolerance = 1.e-3;
```

Minimum increment factor, κ :

```
algorithm.mr_kappa = 0.1;
```

Maximum growth factor, ρ :

```
algorithm.mr_max_growth_factor = 0.4;
```

Maximum number of mesh refinement iterations, $m_{\max}$:

```
algorithm.mr_max_iterations = 7;
```

Maximum nodes to add within a single interval, M_1 :

```
algorithm.mr_M1 = 5;
```

How the algorithm works

The refinement process depends on the discretisation error for each interval. As described in Section 2.1, this process works as an outer loop of NLP solutions. In each iteration, PSOPT solves the nonlinear programming problem on the current mesh. It then computes the error for every interval in each phase and stops if the largest error in each phase is already below $\epsilon_{\max}$, or if $m_{\max}$ iterations have been completed. If not, it creates a new mesh. The method for building the new mesh is based on three main ideas.

Choosing between a better method and a better mesh. A large error can mean either that the discretisation is not accurate enough or that the nodes are in the wrong places, and the two call for different remedies. PSOPT distinguishes between them by comparing the largest interval error in a phase with the average interval error in that phase. If the error is spread fairly evenly over the mesh, no redistribution of nodes will help much, and the limitation is the order of the method: PSOPT therefore raises p from 2 to 4, which switches the phase from the trapezoidal to the Hermite-Simpson discretisation. If instead the error is concentrated in a few intervals, the mesh is at fault, the order is left alone, and nodes are added where the error is.

Predicting what a node will achieve. Adding a node to an interval reduces The rate at which the error decreases depends on the method's order, but the formal order p is only reached when the solution is smooth. If an interval contains a corner, a switch, or a discontinuity, the actual order is lower, and refining these intervals gives much less improvement than the formal order would suggest. Therefore, PSOPT estimates the order reduction on each interval by comparing the error on that interval before and after the last subdivision, and then uses the reduced order to predict what will happen if another node is added. This approach prevents the algorithm from adding too many nodes to an interval where further refinement does not help.

Placing the nodes one at a time. New nodes are not all distributed at once. PSOPT repeatedly finds the interval with the largest *predicted* error in the phase, adds a single node to it, and then updates that interval's predicted error using the estimated order before looking for the next candidate. Since the prediction is updated after each addition, the new nodes are spread across the intervals that need them most. This prevents all the new nodes from being placed in just the worst interval. The placement loop for a phase ends when any of the following holds: enough nodes have been added for this iteration, the quota being $M' = \min(M_1, \kappa N_i) + 1$ where N_i is the current number of nodes in the phase; the predicted error everywhere in the phase has fallen below the tolerance; the growth limit of $\rho(N_i - 1)$ added nodes has been reached; or M_1 nodes have already gone into a single interval. After that, the mesh is rebuilt, and the outer loop starts again.

What the parameters do. In terms of the description above, $\epsilon_{\max}$ (`ode_tolerance`) is the accuracy being asked for and the outer-loop stopping test; $m_{\max}$ (`mr_max_iterations`) caps the number of NLP solves; ρ (`mr_max_growth_factor`) caps how fast a phase may grow in one iteration, and so bounds the cost of the next solution; M_1 (`mr_M1`) caps how many nodes may be concentrated in a single interval, which prevents one troublesome interval from consuming the whole budget; and κ (`mr_kappa`) sets both the per-iteration node quota through M' and the margin by which the predicted error must beat the tolerance before the placement loop is allowed to stop early. Loosening ρ and M_1 makes each iteration more aggressive and usually reduces the number of iterations at the cost of a larger NLP; tightening them does the opposite.

The mesh refinement history (the discretisation method, number of nodes, problem size, function-evaluation counts and maximum relative error at each iteration) is reported in the mesh statistics table written by PSOPT, so the behaviour of the algorithm on a given problem can be inspected directly.

3. Integrated residual transcription

The collocation methods of the previous chapter enforce the dynamics only at a finite set of nodes (see the limitations noted in Section 1.12). For problems with singular arcs, stiff dynamics or near-discontinuous solutions, this can lead to constraint violation between the nodes, oscillatory controls and unreliable convergence. PSOPT therefore also provides an *integrated residual* transcription, following the work of Nie, Kerrigan and co-workers [34, 33, 32, 35]. It is selected by setting `algorithm.transcription_method` to "integrated-residual"; the default is "collocation".

3.1 Motivation: the integrated residual

Let the dynamic residual of a candidate trajectory be

$$r(t) = \dot{\tilde{x}}(t) - f[\tilde{x}(t), \tilde{u}(t), p, t], \quad (3.1)$$

where $\tilde{x}$ and $\tilde{u}$ are the interpolated state and control. Collocation drives r to zero at the nodes only. The integrated-residual method instead measures the dynamics through the integral of the squared residual over each phase,

$$R = \int_{t_0}^{t_f} \|r(t)\|^2 dt, \quad (3.2)$$

which is a genuine measure of how well the trajectory satisfies the differential equations *everywhere*, and not merely at the nodes [34]. Working with R turns the dynamic constraints into a single smooth scalar that can be minimised, penalised or bounded, giving the formulations described below.

3.2 Local representation and residual quadrature

By default, within each mesh interval $[t_k, t_{k+1}]$ the state is represented by a cubic Hermite polynomial built from the node values x_k, x_{k+1} and the node dynamics f_k, f_{k+1} , and the control by a quadratic through u_k , a mid-interval value and u_{k+1} . The residual (3.1) is then a known function of the decision variables on the interval, and the interval contribution to R is evaluated by Gauss–Legendre quadrature at m points,

$$\int_{t_k}^{t_{k+1}} \|r(t)\|^2 dt \approx \frac{h_k}{2} \sum_{q=1}^m \omega_q \|r(\xi_q)\|^2, \quad (3.3)$$

where $h_k = t_{k+1} - t_k$ and ξ_q, ω_q are the quadrature nodes and weights. The number of residual-quadrature points is set by `algorithm.ir_residual_nodes`; it is counted per mesh interval under the default cubic-Hermite representation, and per *element*, a block of d intervals, under the flexible-order representation of Section 3.8. Sampling the residual away from the mesh nodes is precisely what allows the method to “see” the inter-node dynamic error.

The count is important, because under the residual box of Section 3.8 the box constrains the residual only where the rule samples it, and a rule with too few points leaves the residual free to oscillate through the gaps. Measured on `examples/dae_i3` at local order $d = 4$ with a box of 10^{-6} , the maximum relative local error of the returned trajectory is 9.7×10^{-2} with $m = 4$ points per element, 7.9×10^{-4} with $m = 5$, and 1.5×10^{-8} , the box itself, from $m = 6$ upward. PSOPT therefore requires $m \geq d + 2$ whenever the flexible-order representation is in use, and rejects smaller values.

3.3 Equality path constraints in the residual

Many problems carry algebraic equations alongside the differential ones. A semi-explicit differential-algebraic system

$$\dot{x} = f[x(t), u(t), p, t], \quad 0 = g[x(t), u(t), p, t], \quad (3.4)$$

is declared in PSOPT by returning g through the `path` array of the `dae` function and giving it equal lower and upper bounds (Chapter 7). Such an equation is as much a part of the dynamics as $\dot{x} = f$, and the integrated-residual transcription treats it as such: the residual vector (3.1) is extended to

$$r(t) = \begin{bmatrix} \dot{\tilde{x}}(t) - f[\tilde{x}(t), \tilde{u}(t), p, t] \\ \varpi(g[\tilde{x}(t), \tilde{u}(t), p, t] - c) \end{bmatrix}, \quad (3.5)$$

where c is the common bound value of the folded constraint (usually zero) and $\varpi \geq 0$ is a weight set by `algorithm.ir_path_weight` (default 1). Everything downstream, namely the least-squares objective (3.7), the residual box (3.9) and the DAIR alternation of Section 3.6, applies unchanged to the extended residual, so the algebraic equations are measured and controlled *between* the nodes and not merely at them. This is what allows a high-index DAE to be solved in the form in which it was written, without index reduction: no differentiation of the algebraic equation, no introduction of the derived velocity- and acceleration-level constraints, and no drift stabilisation.

Which constraints are folded. Only path constraints declared as equalities enter the residual. The test is exact equality of the declared bounds, so a constraint given a deliberate tolerance band remains an ordinary pointwise path constraint. Inequality path constraints are never folded: an inequality has no residual that must be driven to zero, and penalising its value would push the solution away from feasible points in the interior of the admissible set. Inequalities therefore remain pointwise constraints under either setting of the option below.

Folded constraints leave the pointwise set. A folded equality is imposed through the residual *alone*: its rows in the discretised constraint vector are freed. This keeps the transcription faithful to the formulation of Neuenhofen and Kerrigan [32], in which the algebraic equations appear only inside the integrated residual, and it avoids imposing two different discretisations of the same equation (exact satisfaction at the nodes on top of a bounded residual everywhere), which over-determines the discretised problem and, on a coarse mesh or at low local order, leaves the two requirements competing.

Effect on the size of the nonlinear program. With n_x states and n_a folded algebraic equations, each residual group contributes $m(n_x + n_a)$ box rows instead of mn_x , where m is `ir_residual_nodes` and the number of groups is the number of mesh intervals for the cubic-Hermite representation, or the number of elements N/d for the flexible-order representation of Section 3.8.

Scaling. The algebraic residual rows enter with unit scaling unless user scaling is in force and supplies `scale.path`; the differential rows are scaled as before. Where the algebraic equations are badly scaled relative to the differential ones (a length constraint in metres against velocities in metres per second, for example), `ir_path_weight` is the knob that restores the balance, since a single box tolerance δ must serve every component of r .

Example: an index-3 DAE solved without index reduction. The example `examples/dae_i3` is a pendulum written in Cartesian coordinates [44]: positions x_1, x_2 and velocities x_3, x_4 obey $\dot{x}_1 = x_3$, $\dot{x}_2 = x_4$, $\dot{x}_3 = \lambda x_1$, $\dot{x}_4 = \lambda x_2$, with the multiplier λ carried as an algebraic variable and the position-level constraint

$$g = L^2 - x_1^2 - x_2^2 = 0 \quad (3.6)$$

closing the system, which is therefore of index three. The pendulum length L is a static parameter estimated from measurements of x_1 and x_2 . Solved with the flexible-order representation at $d = 5$ on 61 nodes with a residual box $\delta = 10^{-4}$, folding (3.6) into the residual recovers $L = 1.000004$ against a true value of 1, and holds the algebraic equation to 2.50×10^{-5} at the nodes and 2.79×10^{-5} on a dense grid through the element interpolants; that is, the constraint is satisfied *between* the nodes to the same order as at them, although it is nowhere imposed pointwise. The identical run with `ir_include_path = "none"`, in which the algebraic equation is discarded by the residual and imposed pointwise at the nodes only, fails to converge.

3.4 Least-squares and regularised formulations

The simplest use of the integrated residual is the *least-squares* transcription, in which the objective is the total residual itself,

$$\min_y R(y), \quad (3.7)$$

selected by `algorithm.ir_objective = "residual"`. This finds the most dynamically feasible trajectory for the given mesh, and is a robust way to obtain a feasible initial guess, but it ignores the user cost J .

To trade feasibility against optimality in a single solution, PSOPT can instead minimise a *regularised* objective, in which a weighted integrated residual is added to the user cost while the collocation defects are retained,

$$\min_y J(y) + \rho R(y), \quad (3.8)$$

where the penalty weight ρ is set by `algorithm.ir_regularization`. This integrated-residual regularisation damps the spurious control oscillations associated with singular arcs [34, 32]; it is currently available with the Hermite–Simpson collocation method.

3.5 The residual box (robust) formulation

A more flexible way to control the dynamic error is to bound it directly. In the *residual box* formulation the user cost is minimised subject to a box constraint on every residual component at every quadrature point,

$$\min_y J(y) \quad \text{subject to} \quad |r_j(\xi_q)| \leq \delta \quad \forall j, q, \quad (3.9)$$

selected by setting `algorithm.ir_objective = "cost"` together with a non-negative tolerance `algorithm.ir_residual_bound =  $\delta$` . The box makes the dynamics an explicit, bounded constraint, so the solution accuracy is governed by the single parameter δ : as δ is reduced, the trajectory is forced closer to dynamic feasibility, at the cost of a tighter nonlinear program.

3.6 The Direct Alternating Integrated Residual (DAIR) algorithm

Choosing δ by hand, and converging the cost minimisation directly from a poor initial guess, can both be difficult. The Direct Alternating Integrated Residual (DAIR) algorithm of Nie and Kerrigan [34] removes both difficulties by alternating two sub-problems at each mesh:

1. a *feasibility* step that minimises the integrated residual R (the least-squares problem of Section 3.4), producing a dynamically feasible primal;
2. an *optimality* step that minimises the user cost J subject to the residual box of Section 3.5, warm-started from the feasibility solution.

The box tolerance is not chosen by the user but tied to the mesh scale. For a local representation of order p the residual behaves like the p th power of the node spacing, so

PSOPT uses the horizon-normalised schedule

$$\delta_i = K \left(\frac{h_i}{T_i} \right)^p = \frac{K}{N_i^p}, \quad (3.10)$$

where h_i is the node spacing in phase i , T_i its time span, $N_i = T_i/h_i$ the number of intervals, p the order of the local representation ($p = 2$ for the default cubic-Hermite representation), and K is set by `algorithm.ir_dair_delta_factor` (default 1). Normalising the spacing by the phase duration is what makes the schedule scale invariant. The absolute spacing h_i grows with the horizon, so the unnormalised choice $\delta_i = K h_i^p$ would let the tolerance grow as T_i^p and become non-binding on a long horizon; on the Fuller problem of Section 3.11 ($T = 300$) the unnormalised box exceeded 10^2 and the optimality step collapsed to a degenerate solution. With the normalisation the tolerance depends only on the interval count N_i , so a single K of order unity serves problems whose time scales differ by orders of magnitude. As the mesh-refinement loop refines the grid, N_i grows and hence δ_i falls, and the cost converges monotonically to the optimum without the continuation difficulties of a fixed penalty weight. Each phase receives its own tolerance δ_i . DAIR is enabled by setting `algorithm.ir_dair = true`.

3.7 Costate recovery by adjoint post-processing

When the dynamics are imposed by the residual box instead of by defect equalities, the box multipliers are sensitivities to δ and do not provide costate estimates. PSOPT therefore recovers the costates by adjoint post-processing: along the computed primal trajectory it integrates the adjoint equation

$$\dot{\lambda} = - \left(\frac{\partial L}{\partial x} + \left(\frac{\partial f}{\partial x} \right)^\top \lambda \right) \quad (3.11)$$

backwards in time from the transversality condition

$$\lambda(t_f) = \frac{\partial \varphi}{\partial x_f} + \sum_e \nu_e \frac{\partial e_e}{\partial x_f}, \quad (3.12)$$

where the ν_e are the multipliers of any terminal event constraints. This single expression covers the free-terminal, Mayer and fixed-terminal cases without special handling. The required Jacobians are formed from the user functions by finite differences. The recovered costates are returned in the solution structure in the same way as for the collocation methods.

3.8 The Nie–Kerrigan flexible-order representation

The default cubic-Hermite representation is second order. For smooth problems a higher-order local representation gives much greater accuracy for the same number of nodes. PSOPT provides the flexible-order representation of Nie and Kerrigan [34, 33], enabled

by setting `algorithm.ir_local_order` to an integer $d \geq 2$ (the default 0 retains the cubic-Hermite representation).

The mesh nodes are grouped into elements of $d + 1$ local Legendre–Gauss–Lobatto (LGL) sub-nodes; on each element the state and control are degree- d Lagrange polynomials through those sub-nodes. With $M = N/d$ elements, the number of intervals N per phase must be a multiple of d . Refining the mesh (h -refinement) and raising the order d (p -refinement) can both be used to drive the residual down; for smooth solutions the convergence in d is spectral. The residual is integrated per element, and the Lagrange term of the cost is evaluated with the element LGL (Lobatto) quadrature, which is exact to degree $2d - 1$ and so matched to the order of the representation. The number of residual-quadrature points set by `algorithm.ir_residual_nodes` is counted per *element* under this representation, and per mesh interval under the default cubic-Hermite one.

The state is continuous at the element boundaries, the shared boundary value being a single decision variable belonging to both elements. The *control* is not: each element carries its own control value at its left-hand end, so the control may jump from one element to the next. This is deliberate, and the reason needs stating because the opposite choice looks harmless. A state must be C^0 because it is a state; a control need not be. Sharing the control at the boundary makes it continuous, and the residual box drives $\dot{x} - f$ towards zero at every sample point on an element, which for a chain of integrators annihilates one polynomial degree per link: at $d = 2$ it leaves the control constant on each element, and a continuous piecewise-constant function is a single constant. The solve then converges, reports success, and returns the best *chained-constant* control instead of the right one. On the minimum-energy double integrator, whose answer is $J^* = 6$ with $u^* = 6 - 12t$, sharing gave 7.14, 7.51, 7.74 and 7.87 at 21 to 161 nodes, heading for the 8 that a two-level step costs, so that refining the mesh lengthened the chain instead of helping. Element-local controls are therefore the default, through `algorithm.ir_element_local_controls`. The one situation in which the option should be set `false` is when what PSOPT is carrying as a control is not a control: the algebraic variable of a DAE is a function of the state and is genuinely continuous, and giving it a jump at every element boundary is an approximation error.

The flexible-order representation composes with both the residual box and DAIR. When used with DAIR, the box schedule (3.10) is made order-aware, $\delta_i = K (h_i/T_i)^d$, so that the tolerance tracks the achievable residual of the degree- d representation. As in the cubic-Hermite case, the costates are recovered by the adjoint post-processing of Section 3.7.

3.9 The flexible mesh

A fixed mesh has one limitation that should be stated plainly, because no tolerance will remove it. If the optimal control is discontinuous and the switching time falls in the *interior* of an element, the element’s polynomial cannot be that discontinuity, and the continuous basis produces a local, Gibbs-like error around the switch which raising the

degree d does not remove. The solve converges, it converges to the wrong answer, and tightening the residual box does not move it, because the residual being driven down is the residual of a problem the mesh cannot state.

The remedy of Nita et al. [35] is a *flexible mesh*, in which the element boundary locations are themselves decision variables, so that the optimisation places a boundary at the switching time instead of refining around it. PSOPT provides it through

```
algorithm.ir_flexible_mesh = true;
```

and it applies to both local representations. The Nie–Kerrigan element of degree d spans d intervals and shares its end nodes with its neighbours, so consecutive elements start d apart; the default cubic-Hermite element *is* one interval carrying one cubic, so a node position and an element boundary are the same thing there and a flexible mesh makes every interval width a variable. That is a larger decision vector for a given node count and a lower order per element, but it is the same facility and it moves a node onto a switch by the same mechanism.

The parameterisation. The element *widths* h_e on the normalised interval $[-1, 1]$ are taken as the decision variables, subject to one linear equality per phase,

$$\sum_{e=1}^M h_e = 2, \quad h_e \in [h_{\min}, 2], \quad (3.13)$$

instead of the boundary locations themselves, which would need a chain of $M-2$ ordering inequalities. One equality and a set of simple bounds is a far better conditioned object for an interior-point method than an ordering chain, and the boundaries follow as $a_e = -1 + \sum_{i < e} h_i$, with node $ed + r$ of the phase at $a_e + \ell_r h_e$, where ℓ_r are the reference element’s local LGL abscissae on $[0, 1]$. Those local coordinates are affine invariant, so the reference basis is built once and does not depend on where the boundaries are; only the affine map from the reference element to the physical one does. Since the physical node times were already differentiable expressions in t_0 and t_f , making them depend on the widths as well changes the size of the decision vector and very little else. The Jacobian sparsity follows from the automatic-differentiation tape as before, although a width variable couples to its element’s whole node block, so a problem close to the density limit of `algorithm.jac_sparsity_ratio` may cross it.

The lower bound $h_{\min}$ is a modelling choice, because the optimiser will drive an element towards it: a width free to reach zero gives a singular local problem, so the floor is imposed as a bound and not as a penalty. It is set as a fraction of the uniform width $2/M$ through `algorithm.ir_min_element_fraction`, whose default is 0.05.

What the flexible mesh achieves. The example `examples/flexmesh` described in in Section 3.11 illustrates the use and benefits of a flexible mesh. This is a minimum-time problem whose single switch falls at $t_f/3$, where no uniform element boundary sits. The fixed mesh is wrong by eight parts in a thousand at every residual box from 10^{-4}

to 10^{-7} ; the flexible mesh, on the same nodes, the same element degree and the same box, is three orders of magnitude better.

What the flexible mesh does not do The flexible mesh helps a solution with *finitely* many switches and enough elements to isolate each of them. It does not help a solution that chatters, as the measurements on Fuller’s problem in Section 3.11 show: infinitely many, geometrically shrinking switches cannot be bracketed by finitely many boundaries, so the boundaries that move to catch the resolvable ones must stretch the elements elsewhere, where the local error is then larger. That case also illustrates a subtlety in reading the result. A residual box bounds the residual of whatever discretisation the solver chose, so making the mesh part of that choice weakens what the bound certifies; the cost may therefore fall slightly under a flexible mesh, and a lower cost under a looser certificate is not a better answer.

Composing with automatic mesh refinement. The flexible mesh and the element basis can both be combined with `mesh_refinement = "automatic"`, through a refinement written in the currency this transcription uses; Section 3.10 describes it. The Betts refinement of Section 2.4 cannot serve, because it inserts equally spaced nodes while the element basis is built for the reference LGL abscissae, so after such a refinement the residual would be a polynomial evaluated off its own nodes and the divisibility rule would stop holding.

3.10 Automatic refinement of an element mesh

Automatic mesh refinement for this transcription refines *elements* as opposed to inserting nodes. An element is either kept or split into k equal sub-elements, each of which is a proper element carrying its own abscissae, so the interval count moves in multiples of the element stride by construction and the divisibility rule of Section 3.8 cannot be broken. It runs after each solve, from the local error estimate of that solve.

When the mesh is fixed, the estimator decides both how many elements and where: the elements whose error exceeds `algorithm.ode_tolerance` are split, and the cut is graded by how far it exceeds it. On the minimum-time problem of Section 3.11, with the control bounds altered to $u \in [-1, 1.5]$ so that the switch falls at $0.4t_f$ where no sequence of equal splits can land by luck, refinement takes the relative error in t_f from 1.5×10^{-2} on nine nodes to 2.6×10^{-5} on thirty-seven.

When the mesh is *flexible*, the two mechanisms are given deliberately disjoint jobs, and the reason deserves setting out, because the obvious arrangement is the wrong one. Splitting by error with a flexible mesh makes the answer worse: once the optimisation has isolated a discontinuity inside a thin element, that element’s local error stays large however thin it is, the error being the size of the jump and not the resolution, so an estimator built for smooth solutions flags it at every iteration, the refinement splits it at every iteration, and the mesh starves the rest of the trajectory to feed a point that had already been dealt with. Measured on the $u \in [-1, 2]$ variant, that arrangement

takes the flexible mesh’s own 9.1×10^{-8} to 1.9×10^{-6} , and the cubic-Hermite form to 6.2×10^{-5} with thirty tiny elements packed around the switch.

So with a flexible mesh the estimator is asked only *how many* elements the phase needs, which is the question it can answer; the new elements are seeded by splitting the widest, where resolution is cheap and where the seed contains no degenerate element; and where the boundaries finally go is the flexible mesh’s business, since it re-places every one of them in the next solve in any case. Paired with DAIR, so that the residual box tightens as the mesh grows, the combination gives the most accurate result of any measured on that problem, 4.1×10^{-8} against 1.6×10^{-7} for the flexible mesh alone and 1.5×10^{-2} for the coarse fixed mesh.

Two cautions. Refining the mesh while leaving the residual box fixed does not reliably improve the answer, because the box and not the mesh is then what limits it, which is the lesson of Section 3.11; pair it with `ir_dair`. And the cubic-Hermite representation gains little: at order two the DAIR schedule $\delta = K(h/T)^2$ asks for 6×10^{-4} on a forty-interval mesh, so the answer stops improving because that is what was asked for and not because the refinement failed.

The legacy cubic-Hermite form on a *fixed* mesh continues to use the Betts refinement of Section 2.4, unchanged.

3.11 Behaviour in practice

Three examples distributed with PSOPT illustrate the behaviour of the transcription and the practical lessons that shaped it.

A smooth singular problem (examples/x1). The analytic problem of Neuenhofen and Kerrigan [32],

$$\min \int_0^{\pi/2} (y^2 + \cos(t)u) dt \quad \text{s.t.} \quad \dot{y} = \tfrac{1}{2}y^2 + u, \quad y(0) = 0, \quad -1.5 \leq u \leq 1, \quad (3.14)$$

has a known optimal cost J^* and a smooth solution on which ordinary Hermite–Simpson collocation nonetheless rings. It demonstrates the value of the flexible-order representation directly: with the order-aware box, raising the local order d drives the cost error $|J - J^*|$ down at the expected p -rate. On eight elements the example reports

local order d	nodes	$ J - J^* $
2	17	3.91×10^{-1}
3	25	1.14×10^{-2}
4	33	2.36×10^{-4}
5	41	4.19×10^{-6}

a reduction of roughly two orders of magnitude for each unit of d . The order is capped at five here because the order-aware box h^d must stay above the NLP tolerance of 10^{-7} ; asking for a tighter box than the solver can certify does not produce a better answer. It

also exposes a subtlety that motivates the higher-order representation. Tightening the box on the *low*-order cubic-Hermite representation does not improve the cost; it makes it *worse*, because a second-order representation cannot be simultaneously optimal and dynamically accurate to a tight tolerance. Only raising the local order lets the trajectory satisfy a tight box while remaining optimal, so the order of the local representation must be matched to the accuracy demanded of the box.

A chattering problem (examples/fuller). Fuller’s problem [35], in state form $x_1 = p$, $x_2 = \dot{p}$,

$$\min \int_0^T x_1^2 dt \quad \text{s.t.} \quad \dot{x}_1 = x_2, \quad \dot{x}_2 = u, \quad |u| \leq 0.01, \quad (3.15)$$

with $x_1(0) = x_1(T) = x_2(T) = 0$, $x_2(0) = 1$ and $T = 300$, has an optimal control that chatters: it switches infinitely often, the switches accumulating near $t \approx 280$. Collocation captures a few switches but leaves an uncontrolled inter-node residual of order 10^{-2} . The residual box instead turns the computed cost into a *certified lower bound* on the true optimum that rises towards it as δ is tightened: on a fixed mesh, reducing δ from 10^{-3} to 10^{-6} raises the computed cost monotonically towards the optimal value, tracing an accuracy–cost Pareto front that ordinary collocation cannot provide.

The Fuller example records two practical points. First, on a strongly non-smooth problem the integrated-residual optimality step should be warm-started from a feasibility (or collocation) solution; a cold start from a poor guess can converge to a degenerate low-cost, high-residual point. This is exactly the feasibility-then-optimality alternation that DAIR automates. Second, on a fixed mesh raising the local order helps a discontinuous solution only up to a point: beyond order two the switch discontinuities induce Gibbs-type oscillations that a higher-degree polynomial does not remove.

It is natural to expect the flexible mesh of Section 3.9 to be the remedy for that second point, and on this problem it is not. Turned on at $\delta = 10^{-4}$ on the same 81 nodes, the maximum relative local error *rises*: from 3.3×10^{-6} to 4.4×10^{-3} at $d = 2$, and from 4.3×10^{-6} to 4.8×10^{-5} at $d = 4$. The reason is the chattering itself. The switches are infinite in number and geometrically shrinking, so no finite set of element boundaries brackets them, and the boundaries that move to isolate the few resolvable switches must stretch the elements elsewhere. The computed cost falls slightly at the same time, from 2.6847×10^5 to 2.6833×10^5 at $d = 4$, which should be read carefully: the box bounds the residual of whatever discretisation the solver chose, so making the mesh itself a decision weakens what the bound certifies, and a lower cost under a looser certificate is not a better answer. The flexible mesh is for a solution with finitely many switches and enough elements to isolate each one, which is the next example.

A switching time inside an element (examples/flexmesh). The minimum-time problem

$$\min t_f \quad \text{s.t.} \quad \dot{x}_1 = x_2, \quad \dot{x}_2 = u, \quad u(t) \in [-1, 2], \quad (3.16)$$

with (x_1, x_2) driven from $(0, 0)$ to $(1, 0)$, has a bang-bang solution with a single switch. The asymmetry of the control bounds is deliberate: symmetric bounds would place the

switch at exactly $t_f/2$, which a uniform partition into an even number of elements already has a boundary at, so a fixed mesh would pass the test for a reason that has nothing to do with the mesh being good. With $u \in [-1, 2]$ the exact answer is $t_f^* = \sqrt{3} = 1.7320508$ with the switch at $t_f^*/3$, and $x_1(t) = t^2$ on the first arc. On 17 nodes at local order $d = 4$ (four elements of degree four, none of whose uniform boundaries lies at one third) minimising the cost subject to a residual box gives

residual box δ	fixed mesh	flexible mesh
10^{-4}	7.88×10^{-3}	4.28×10^{-6}
10^{-5}	7.90×10^{-3}	1.42×10^{-6}
10^{-6}	7.91×10^{-3}	1.00×10^{-5}
10^{-7}	7.91×10^{-3}	8.22×10^{-6}

as the relative error in t_f . The fixed-mesh column is the signature that distinguishes the two ways a transcription can be inaccurate: a discretisation that has not yet been solved accurately enough improves when the tolerance is tightened, and one that cannot state the problem does not.

What the optimiser does with the freedom is instructive, and is not quite what the description of the method suggests. The example prints the solved element boundaries, which at $\delta = 10^{-6}$ are at $t = 0, 0.560, 0.582, 1.175$ and 1.732 . No boundary sits *on* the switch at 0.577 . The switch is instead *bracketed* by a thin element about one per cent of the horizon wide, which achieves the same end by a different route: the discontinuity is isolated inside an element small enough for its misrepresentation not to matter, and the two elements on either side are then smooth, where a degree-four polynomial represents a parabola exactly. This is also why $h_{\min}$ is a real modelling choice: the optimiser drives an element towards it.

4. Multiple shooting

4.1 Overview of multiple shooting

The collocation methods of Chapter 1 and the integrated-residual transcription of Chapter 3 both make the whole trajectory out of decision variables: the state at every node is a variable, and constraints relate them. Multiple shooting does not. The time interval of a phase is divided into *segments*; the decision variables are the state at the start of each segment and the controls, and the trajectory *inside* a segment is produced by integrating the dynamics from that start state. What ties the segments together is the matching condition

$$x_{k+1} - \varphi(x_k, u_k, p, h_k) = 0, \quad k = 0, \dots, M-1, \quad (4.1)$$

where φ is the segment integrator and h_k the segment's duration. The transcription is selected with

```
algorithm.transcription_method = "multiple-shooting";
```

Three properties follow from (4.1) and should be kept in mind before the options. The dynamics are satisfied *exactly* inside a segment, to whatever accuracy the integrator has, as opposed to approximately at a set of nodes, so there is no inter-node residual of the kind Chapter 3 exists to measure. The constraint set is block-bidiagonal, row block k touching only $x_k, x_{k+1}, u_k, u_{k+1}$ and the phase's two times, which is the whole computational argument for multiple shooting over single shooting and which PSOPT detects from the automatic-differentiation tape. And the state between segment boundaries is not a decision variable, which is what makes path constraints the delicate part (Section 4.5).

A shooting phase reuses the collocation layout exactly. The phase's interval count is the number of *segments*; its `nodes` entry is the number of segment boundaries; the stored node states are the segment-start states, with the phase's final state last; and the rows that carry the collocation defects carry (4.1) instead. Nothing about the decision vector, the bounds, the initial guess, the hot start between meshes, the reported times or the plotting is different, and multiple phases with linkages work exactly as they do elsewhere. The two optional blocks a shooting phase can add are also blocks the collocation layout already had: the midpoint controls of the quadratic parameterisation (Section 4.3) occupy the slot Hermite–Simpson's midpoint controls occupy, and the segment widths of a flexible partition (Section 4.4) occupy the slot the integrated residual's element widths occupy, immediately before t_0 and t_f .

4.2 The segment integrator

The dynamics are propagated by an explicit Runge–Kutta scheme at a fixed step, recorded onto the same automatic-differentiation tape as the rest of the problem. The scheme and the number of steps across a segment are set by

```
algorithm.ms_integrator      = "RK4"; // "RK8", "TRBDF2" or "ESDIRK3"  
algorithm.ms_steps_per_segment = 10; // the default
```

The first two are explicit and the last two implicit. The explicit schemes are described first; the implicit ones, which exist for stiff dynamics, are described at the end of this section.

That choice is not a convenience, and the alternative that suggests itself is a trap worth naming. A shooting method needs the derivative of a segment’s end state with respect to its start state, its control, the parameters and its duration, which are derivatives *through* an integrator. There are two sound ways to get them: tape a fixed-step scheme, which is what is done here, or propagate sensitivities with the same step sequence and the same factorisations as the nominal trajectory, which is the internal numerical differentiation of Bock [6]. What must not be done is to difference an adaptive integrator: doing so differentiates its step controller along with its dynamics, and the result is noise. A fixed-step scheme has no controller to differentiate, so the taped derivative is the exact derivative of the discrete map the transcription actually uses, which is the derivative the NLP wants and not the derivative of the differential equation.

The running cost is accumulated through the same Runge–Kutta stages instead of using a quadrature on the segment ends. That is state augmentation without the extra variable: $\dot{J} = L$, so the same scheme applied to the augmented system integrates the cost to the same order as the trajectory. A trapezoidal rule on the two ends would be second order against the trajectory’s fourth or eighth, and the objective the NLP minimised would then not be the objective that was written down.

Choosing the scheme

“RK4” is the classical four-stage fourth-order method. “RK8” is Cooper and Verner’s eleven-stage *eighth*-order formula [14], whose coefficients lie in $\mathbb{Q}(\sqrt{21})$ and which uses the fewest stages any explicit method of its order can.

What the choice brings is accuracy *per right-hand-side evaluation*, and that is the quantity that matters here, as opposed to accuracy per step. The tape a shooting transcription builds is stages $\times$ steps $\times$ segments long, and both the memory it occupies and the time to evaluate the constraints are linear in it. RK8 costs 2.75 times the stages per step and converges four orders faster, so it reaches a given accuracy in far fewer evaluations. On the oscillator of Section 4.8 at ten segments, measuring the scheme’s own error against that problem’s closed-form segment map:

RK4			RK8		
steps	evaluations	error	steps	evaluations	error
4	160	1.476×10^{-3}	1	110	7.003×10^{-4}
8	320	9.486×10^{-5}	2	220	4.001×10^{-6}
16	640	5.986×10^{-6}	3	330	1.663×10^{-7}
32	1280	3.755×10^{-7}	4	440	1.705×10^{-8}
64	2560	2.351×10^{-8}			

The measured orders are 15.6, 15.8, 15.9 against 2^4 and 175, 235, 249 against 2^8 . RK8 is more accurate at 440 evaluations than RK4 is at 2560.

What a higher-order scheme does *not* do is improve the answer once the integrator has stopped being the binding error, which on most problems it is: under a piecewise-constant control the cost is capped at $O(h^2)$ by the control parameterisation (Section 4.3) and the two schemes then agree to five figures. The scheme is worth raising where the control parameterisation is already good: with a quadratic control at two steps per segment the relative cost error is 1.76×10^{-4} under RK4 and 4.84×10^{-7} under RK8, and RK8 reaches in two steps the floor RK4 needs ten to reach. Raise the order of the integrator to stop the dynamics being the limit; raise the order of the control to move the limit itself.

The discretisation error reported for a shooting phase is a different quantity from the one reported for a collocation phase, and is computed differently. Because (4.1) holds to the NLP's tolerance there is no defect between the nodes: the only difference between the computed trajectory and the true one is what the fixed-step scheme lost inside a segment. Each segment is therefore run again at half the step and the two ends compared (Richardson's estimate). The factor deserves stating, because the obvious one is wrong. Writing Ch^p for the leading error of a scheme of order p , the run at h and the run at $h/2$ differ by

$$x_{h/2} - x_h = Ch^p (2^{-p} - 1), \quad (4.2)$$

so the error of the run *that was solved* is $|x_{h/2} - x_h|/(1 - 2^{-p})$. Dividing by $2^p - 1$ instead gives the error of the half-step run, which is a trajectory nobody computed and nobody has, and which is smaller by exactly 2^p . PSOPT reports the former. Measured against the true relative local error on a problem whose segment map is available in closed form, the reported estimate is 1.00 for both schemes and every step count.

Implicit schemes for stiff dynamics

An explicit scheme does not become inaccurate on a stiff problem, it becomes unbounded, and that is the distinction the two implicit schemes exist for. RK4's stability region reaches $|\lambda h| = 2.78$, so on $\dot{x}_1 = -1000x_1 + u$, $\dot{x}_2 = x_1$ with ten segments it needs at least 36 steps per segment before the propagation is bounded at all, and below that it does not return a poor answer, it returns a meaningless one:

scheme	steps/segment	J
RK4	8	0.755
RK4	16	1.77×10^9
RK4	36	500984.04
RK4	72	500996.98
TR-BDF2	1	500996.52
ESDIRK3	1	500996.71

“TRBDF2” is the scheme of Bank and co-authors [2], a trapezoidal step to γ followed by a BDF2 step to the end, which as a Runge–Kutta method is three stages of order two. “ESDIRK3” is four stages of order three. Both are *ESDIRK*: an explicit first stage, a single repeated diagonal γ , stiff accuracy, and L-stability.

Each of those four properties does a job. The explicit first stage adds no cost and raises the stage order. *Stiff accuracy*, meaning that the last stage is the step itself, is what puts the algebraic constraint of a DAE (Section 4.6) at the *step end*, which is where the matching conditions live; an A-stable scheme that is not stiffly accurate leaves the algebraic components undamped and satisfies the constraint at no point the transcription uses. *L-stability* damps the stiff components, as opposed to merely bounding them. And the *single repeated diagonal* is what makes the scheme tapeable at all: one factorisation serves every stage of a step.

That last point is the whole implementation. For stage i the unknown is $Y = (X, Z)$, the state and the algebraic variables together, satisfying

$$X - S_i - h\gamma f(X, Z, u, t_i) = 0, \quad g(X, Z, u, t_i) = 0, \quad (4.3)$$

where S_i collects what the earlier stages have already fixed. The algebraic block is folded into the *same* system, because on a stiff DAE the two parts are coupled through the stiffness. The matrix $W = I - h\gamma \partial f / \partial y$ is formed by finite differences once per *step*, factorised once, and reused by every stage, and the stage systems are then solved by a fixed, unrolled number of modified-Newton iterations:

```
algorithm.ms_implicit_iterations = 4;    // the default
```

Nothing in that iteration is differentiated in order to build it, so the nested automatic differentiation an implicit scheme is usually said to require does not arise; the finite differences, the factorisation and a fixed iteration count are ordinary arithmetic and the tape records them as such. This is also why an ESDIRK is chosen here instead of a fully implicit scheme such as Radau IIA: the stages are solved one at a time, so the tape is a sequence of blocks of the same shape the explicit loop already has, where Radau IIA couples $s(n_s + n_z)$ unknowns at once.

The default of four iterations is derived. The stage solve is warm-started from the previous stage, $O(h)$ away, and modified Newton with a Jacobian formed at the step’s own start contracts by $O(h)$ an iteration, so m iterations leave $O(h^{m+1})$. Measured on a *nonlinear* stiff problem, the relative change from eight iterations is 6.4×10^{-2} , 2.0×10^{-4} , 1.8×10^{-6} , 3.1×10^{-9} , 5.7×10^{-11} for $m = 1, \dots, 5$: four saturate and a fifth

is margin. The problem has to be nonlinear for that measurement to mean anything, because modified Newton solves a *linear* stage system exactly in one iteration and a linear problem makes every count look sufficient.

Three cautions, all of them measured.

First, an implicit step costs several times an explicit one, so on a *non-stiff* problem these are the wrong choice: lower order than RK8 and dearer than RK4.

Second, the classical order of an implicit scheme can only be measured on a non-stiff problem, which is a trap. In the stiff limit an L-stable scheme resolves the quasi-steady state almost exactly and the measured rate is not the tableau's order at all: on the stiff problem above, TR-BDF2, a *second*-order scheme, is measured as order eight. At $\lambda = 1$ the error falls by 4.02, 4.05, 4.20 for TR-BDF2 and 7.89, 7.96, 8.08 for ESDIRK3 for each doubling of the step count, which are 2^2 and 2^3 .

Third, in the stiff limit a Runge–Kutta method converges below its classical order, an effect known as order reduction and a property of the method class, so the reported discretisation error, which scales the Richardson difference of (4.2) by that order, is optimistic there. Measured on an index-1 DAE: 3.00 at $\lambda = 0$, 2.90 at $\lambda = 20$ and 2.39 at $\lambda = 500$, the last still climbing towards 3 as the step falls. Both tables here satisfy the stage-order condition $\sum_j a_{ij}c_j = c_i^2/2$, which puts a floor of two under that reduction; it was imposed when they were derived.

4.3 The control parameterisation

The control across a segment is held, ramped, or carried as a parabola:

```
// "constant" (default), "linear" to ramp across the segment,
// or "quadratic" for the parabola through node, midpoint and node
algorithm.ms_control_parameterisation = "quadratic";
```

Writing u_k for the control at the start of segment k and $\bar{u}_k$ for a control variable at its midpoint, and $s \in [0, 1]$ for the local coordinate across the segment, the three forms are

$$\begin{aligned} \text{constant: } u(s) &= u_k, \\ \text{linear: } u(s) &= (1-s)u_k + s u_{k+1}, \\ \text{quadratic: } u(s) &= (2s-1)(s-1)u_k + 4s(1-s)\bar{u}_k + s(2s-1)u_{k+1}, \end{aligned} \tag{4.4}$$

the last being the same quadratic that Hermite–Simpson carries, and $\bar{u}_k$ occupying the same slot in the decision vector that Hermite–Simpson's midpoint control does.

This is not a detail; it is what decides the accuracy of the answer. The two error sources of a shooting transcription are separable, and only one of them binds. The dynamics are integrated to whatever `ms_steps_per_segment` achieves, at a cost linear in CPU and none at all in decision variables: on a representative problem, driving the integrator error down to 7.8×10^{-17} leaves the relative cost error at 5.5×10^{-6} , where it stays. What is left is the control approximation, so *the order of the control parameterisation is the order of the method*. A control error of $O(h^p)$ gives a cost error

of $O(h^{2p})$, the cost being stationary and quadratic at the optimum, and the three forms of (4.4) are $p = 1, 2, 3$. Section 4.8 measures all three and recovers the ratios 4, 16 and 64 for each halving of the segment width.

Two things follow that a user should know before choosing.

First, the higher forms are exact when the optimal control lies in them. Section 4.8 makes this point twice: with a problem whose u^* is linear, where the linear form attains the continuous optimum at five segments; and with the minimum-energy triple integrator, whose u^* is exactly a parabola, where the quadratic form returns the continuous optimum to fourteen digits at five segments and the linear form stops at a relative error of 1.6×10^{-3} however fine the integrator is made.

Second, **the parabola can leave the control bounds**. A quadratic through three values inside the box need not stay inside it, since it overshoots by a quarter of the second difference, and the sharpest second difference a solution can present is a jump. Since putting a segment boundary exactly on a jump is what `ms_flexible_segments` is for, the two features collide precisely where each is doing its job: on the minimum-time problem of Section 4.4, with $u \in [-1, 2]$ and the boundaries free, the reported control reaches 2.375, nineteen per cent above its own upper bound, and that is the control the segment integrator is handed. `ms_path_samples` does not help, since it samples the *path* constraints and this is a variable bound. PSOPT prints a note when the combination is set. The rule is to use “constant” where the optimal control has corners or rides its bounds, and the higher forms where it is smooth and interior.

The constant form remains the default for that reason and for one more: it is the only one of the three that is discontinuous across a segment boundary, and a problem with a discontinuous optimal control wants exactly that. Under the constant form the last of the $M + 1$ control slots belongs to no segment; PSOPT pins it to the control of the segment that ends there. Under the linear and quadratic forms every slot is read by some segment and there is nothing to pin.

Under the quadratic form the midpoint controls are part of the answer, and the usual accessor `get_controls_in_phase` does not report them: it holds the $M + 1$ values at the segment boundaries, as it does for every other transcription. The complete control history, node and midpoint values interleaved into one increasing sequence, comes from `get_hs_controls_in_phase`, with `get_hs_time_in_phase` beside it. That accessor keeps its Hermite–Simpson name, and what it reports is the complete control history of any phase whose control is the parabola through node, midpoint and node. Reading the node values alone under this parameterisation gives two thirds of the control variables and none of the curvature.

4.4 Flexible segment boundaries

A uniform partition is a guess about where the solution needs resolution, and on a problem whose optimal control switches it is usually the wrong guess. The remedy is the one the integrated-residual transcription uses for its mesh (Section 3.9), applied here to the segments:

```

algorithm.ms_flexible_segments    = true;
algorithm.ms_min_segment_fraction = 0.8;    // the default

```

The segment *widths* on the normalised interval $[-1, 1]$ become decision variables, with one equality per phase requiring them to sum to 2 and a floor imposed as a simple bound. The parameterisation is by widths and not by boundary positions, so that the ordering of the boundaries is a set of bounds.

What flexible segment boundaries achieve Take the minimum-time double integrator with the asymmetric bounds $u \in [-1, 2]$, so that the single switch falls at $t_f/3$ with t_f free and $t_f^* = \sqrt{3}$. A uniform partition has a boundary on the switch exactly when the segment count is divisible by three, and nothing else about the problem changes between $M = 9$ and $M = 10$:

segments M	uniform partition	flexible boundaries
5	2.062×10^{-2}	8.056×10^{-9}
6	7.224×10^{-9} *	8.010×10^{-9}
7	1.036×10^{-2}	7.964×10^{-9}
9	7.363×10^{-9} *	7.871×10^{-9}
10	5.038×10^{-3}	7.825×10^{-9}
20	1.252×10^{-3}	7.363×10^{-9}

(relative error in t_f ; * the segment count is divisible by three, so a boundary lands on the switch by arithmetic). The flexible column does not depend on M at all, which is the whole object of it: at five segments it is both faster than the uniform partition at forty and six orders of magnitude more accurate.

Why the floor is necessary A moving partition is a *free-knot* approximation problem, and those are degenerate by construction: moving a boundary and adjusting the controls either side of it compensates to second order, so the objective is flat along a manifold and there is no isolated solution for a Newton-type method to converge to. A tight floor removes most of that manifold, and the effect is large: on the problem above at ten segments, a floor of 0.99 converges in 75 iterations, 0.9 in 235, and 0.5 not at all. The default of 0.8 is therefore a modelling statement instead of a guard. PSOPT also imposes a ceiling symmetric with the floor in the logarithm, so that a width is confined to $[\phi h, h/\phi]$ about the uniform width h as opposed to being free to grow to the whole phase; without it the box on a twenty-segment problem would span a factor of four hundred for a variable whose answer is within a few per cent of uniform.

When not to use flexible segment boundaries Do not turn this on for a problem whose optimal control is smooth. There is then no corner for a boundary to sit on, the degeneracy above is complete, and the solution does not converge at any floor, measured, at some two hundred times the CPU of the fixed partition and a worse answer. This facility is for solutions with corners. Note also that it composes badly with the quadratic

control parameterisation, for the reason given in Section 4.3: the boundary lands on the jump, and a parabola through the values either side of a jump leaves the control bounds.

Path constraints deserve a second thought here, because a boundary that has moved is not where the user put it. With `ms_path_samples` at its default of zero the path constraints are imposed only at boundaries that the optimisation is now free to move, and PSOPT prints a note saying so.

4.5 Path constraints

A path constraint is imposed at the segment boundaries, and *optionally* at a number of points inside each segment:

```
algorithm.ms_path_samples = 2;    // default 0, i.e. boundaries only
```

The samples are placed at integrator step boundaries, so that the state at a sample is one the integrator actually produced; this requires `ms_steps_per_segment` to exceed `ms_path_samples`.

The default is correct but weak, and the way it fails should be understood before it is relied on, because it fails in the direction that resembles success. Between two segment boundaries there is an integrator and nothing at all constraining what it does, so a constraint imposed only at the boundaries can be violated freely in between, and the optimiser, given a constraint that is cheaper to satisfy than the one that was written down, returns a cost *below* the true optimum. On Bryson and Denham’s problem, whose optimum is $J^* = 4$ and where no feasible trajectory can cost less, the boundary-only form returns 3.9934 on ten segments. Sampling inside the segments puts the answer back above J^* , which is where a restricted control parameterisation has to leave it. The numbers are in Section 4.8.

There is one more thing to know about a path constraint under the *constant* parameterisation, and it is handled for the user. The terminal control slot belongs to no segment there and is pinned to the control of the segment that ends at it (Section 4.3). A path component that depends on the controls *alone* is therefore evaluated at the terminal node on exactly the arguments its own row at the node before uses, and the two rows are dependent through the pin: one redundant equality, and multipliers that are no longer unique. Measured on Bryson’s maximum-range problem with $\|u\|^2 = 1$ imposed as a path equality and a held control, the solve ends in **Restoration_Failed** at fourteen, sixteen, eighteen, twenty, twenty-two and twenty-four segments, which are the counts measured, while six to twelve succeed. The rank diagnostic of Chapter 6 reports the problem deficient by one beyond the rows that are empty by construction, and it does so at every count, including the ones that succeed, so the redundancy alone does not explain where the threshold falls.

PSOPT detects such a component from the sparsity of the assembled constraint Jacobian and leaves its terminal row *unimposed*. Nothing is given up: the constraint still holds at the terminal node, imposed by the row before it at arguments the pin makes identical. The detection is a whitelist: the terminal row’s nonzero columns must lie

entirely in the pinned terminal-node controls or in the static parameters, so a state column, either time, an element width, a midpoint control or an algebraic control all leave the row in place. It has to be a whitelist, because freeing the terminal row of a *state*-dependent component would be a constraint silently not imposed at the final time. With the redundant row gone, all ten even segment counts from six to twenty-four solve, and the cost descends monotonically towards the collocation answer: the failures were concealing a convergence. The same detection is skipped entirely under numerical derivatives, a numerically detected sparsity pattern being able to miss a dependence that vanishes where it was probed.

4.6 Semi-explicit index-1 DAEs

A system written

$$\dot{x} = f(x, z, u, p, t), \quad 0 = g(x, z, u, p, t), \quad (4.5)$$

with $\partial g / \partial z$ nonsingular, is a semi-explicit differential-algebraic system of index one. PSOPT carries the algebraic variable z as a control, there being no other place for it, and the natural way to write (4.5) is therefore to put $g = 0$ into the path constraints. Under this transcription that is not enough, and the reason is the one Section 4.5 gives: a path constraint is imposed where the trajectory is a decision variable, which here is the segment boundaries, and between them z is *held* while x moves under it. The algebraic relation then drifts at first order in the segment width. Measured on $\dot{x}_1 = x_2$, $\dot{x}_2 = u - z$, $0 = z^3 + z - x_1$ (index one, and with no closed form for z in terms of x_1 , so that it cannot be eliminated by hand) the worst $|g|$ is 1.67×10^{-1} at ten segments and the cost is wrong in its first significant figure.

The remedy is to say so:

```
problem.phases(iphase).nalgebraic = 1;    // default 0
```

The convention is a count, like every other size in that structure: the *last* **nalgebraic** of the phase's controls are the algebraic variables, and the *first* **nalgebraic** of its path constraints are the equations that determine them, which must be equalities. A problem already written the obvious way therefore becomes a genuine DAE by setting one number, with the user's own **dae** function unchanged.

What changes is what the segment integrator does between the boundaries. With an explicit scheme the propagation becomes *half-explicit*: at each Runge–Kutta stage the differential part is explicit, so the stage state is known before anything is solved, and the only unknowns left are the n_z algebraic variables, determined by n_z equations. With one of the implicit schemes of Section 4.2 the algebraic block is folded into the stage system (4.3) instead. Either way the algebraic relation holds wherever z is defined at all, by construction.

The half-explicit scheme keeps the order of the underlying table, and that is a theorem and not a hope: index one means $g = 0$ defines $z = G(x, u, p, t)$ locally, so an explicit Runge–Kutta applied to the *reduced* ordinary system $\dot{x} = f(x, G(\cdot), u, p, t)$ produces exactly the stage sequence a half-explicit method produces. They are the same

computation, not an approximation of one another. Measured on the problem above at ten segments, against the same formulation at sixty RK8 steps:

scheme	steps/segment	$ J - J_{\text{fine}} $	ratio
RK4	2	6.647×10^{-7}	
RK4	4	4.254×10^{-8}	15.6
RK4	8	2.690×10^{-9}	15.8
RK4	16	1.691×10^{-10}	15.9
RK8	2	1.6×10^{-14}	(at the floor)

and the worst $|g|$ along the returned trajectory is 2.2×10^{-16} against the 1.67×10^{-1} of the path-constraint formulation.

The stage equations are solved by a fixed, unrolled Broyden iteration:

```
algorithm.ms_algebraic_iterations = 4;    // the default
```

Broyden's method builds its slope from function values alone, so nothing inside the iteration has to be differentiated and no automatic differentiation is nested; and the count is fixed, because a loop whose length depended on the values could not be taped and, which is the deeper objection, would make the constraint function non-smooth in the decision variables. The default of four is derived. The solve is warm-started from the previous stage, whose algebraic variables differ by $O(h)$, and the secant recurrence $e_{k+1} \sim e_k e_{k-1}$ then gives exponents 1, 2, 3, 5, 8, so m iterations support a scheme of order 2, 3, 5, 8 for $m = 1, 2, 3, 4$. Measured inside PSOPT at four RK8 steps per segment, the cost error is 1.7×10^{-5} , 6.5×10^{-8} , 2.5×10^{-12} , 8.9×10^{-15} for those four counts, and a fifth iteration adds nothing. The count does not have to grow with n_z : at one, two and three coupled algebraic components alike, four iterations give the full eighth order.

Two things are worth knowing about the layout. An algebraic component is never pinned at the terminal node: the terminal control slot belongs to no segment under a held control and is ordinarily pinned to its neighbour, but an algebraic variable is already determined there by its own path row, evaluated at a state that *is* a decision variable, and a pin would be a second equation for a variable that has one. And the diagnostics of Chapter 6 confirm it: on the problem above the declared formulation is rank-deficient by exactly the structurally empty rows, where the path-constraint formulation of the same problem is deficient by one beyond them.

Index reduction, which also works

The alternative needs no declaration at all. Differentiating the algebraic relation along the trajectory gives

$$\frac{\partial g}{\partial z} \dot{z} = -\frac{\partial g}{\partial x} f - \frac{\partial g}{\partial t}, \quad (4.6)$$

which for an index-1 system determines $\dot{z}$. Carrying z as an extra *state* with that derivative, and imposing $g = 0$ once, as an event at t_0 , turns the algebraic relation from a constraint into an invariant of an ordinary differential equation, and the residual of an

invariant is then the integrator's error, which is what this transcription is good at. On the same problem, with z carried as a state, $|g|$ at the states PSOPT returns is

	2 steps/segment		20 steps/segment	
	RK4	RK8	RK4	RK8
$\max g $ at 20 segments	4.6×10^{-8}	1.8×10^{-15}	4.6×10^{-12}	3.3×10^{-15}

and the cost agrees to nine figures across the four. The classical objection to index reduction is drift; here the drift is an integrator's error and is bought off for almost nothing. What it asks of the user is that they can write (4.6) by hand or by computer algebra. The declaration asks nothing, so it is the better default; index reduction remains useful when the reduced form is wanted for its own sake, and the two agree to ten figures on the problem above, which is a check on both.

4.7 Costates

The costates are recovered by the adjoint post-processing of Section 3.7: once the primal is available, the adjoint equation is integrated backwards from the transversality condition along the converged trajectory. They are returned in the solution structure in the usual way.

They are *not* the multipliers of the matching conditions. Those multipliers are a discrete adjoint and it is tempting to read them as costates, but the covector mapping of Section 1.8.1 is written for collocation defects: it has a quadrature weight to divide by and a differentiation matrix to transpose, and a matching condition has neither. Applied to them it returns a quantity of the right shape and the wrong size, +96 on the worked example below where the answer is -12 , which is the least useful kind of wrong. Against the closed-form costates of that problem the adjoint post-processing gives 2.3×10^{-10} .

4.8 Example: multiple shooting on a problem with a closed form

Consider the minimum-energy double integrator

$$\min \frac{1}{2} \int_0^1 u^2 dt \quad \text{s.t.} \quad \dot{x}_1 = x_2, \quad \dot{x}_2 = u, \quad (4.7)$$

with (x_1, x_2) driven from $(0, 0)$ to $(1, 0)$, whose solution is $u^*(t) = 6 - 12t$ with $J^* = 6$ and costates $\lambda_1 = -12$, $\lambda_2 = 12t - 6$.

The verification that matters here is not that the transcription returns something near 6. With the control held constant on M equal segments, the problem it actually poses is least-norm in an M -dimensional space, and projecting its two linear constraints onto that space gives the optimum of the *discrete* problem in closed form,

$$J^*(M) = \frac{6M^2}{M^2 - 1}. \quad (4.8)$$

PSOPT returns 6.2500000000, 6.0606060606, 6.0150375940 and 6.0037523452 at $M = 5, 10, 20, 40$, equal to (4.8) to fourteen digits at every segment count, every step count and either scheme. A transcription returning something close to 6 instead would be solving a different problem and getting a plausible answer, which is the failure mode worth being able to tell apart, because it looks like success.

The control parameterisation then decides everything about the rate of convergence. Since u^* is linear, the linear parameterisation is exact:

segments M	constant control	linear control
5	6.2500000000	6.0000000000
10	6.0606060606	6.0000000000
20	6.0150375940	6.0000000000

The same statement holds one order higher, and the problem that makes it is the minimum-energy *triple* integrator, $\dot{x}_1 = x_2$, $\dot{x}_2 = x_3$, $\dot{x}_3 = u$, driven from the origin to $(1, 0, 0)$ on $[0, 1]$. Its costates satisfy $\dot{\lambda}_1 = 0$, $\dot{\lambda}_2 = -\lambda_1$, $\dot{\lambda}_3 = -\lambda_2$ with $u = -\lambda_3$, so u^* is a parabola; equivalently, the minimiser of $\int u^2$ subject to three linear functionals of u lies in the span of their Riesz representers, which are 1, $1 - t$ and $(1 - t)^2/2$ and no higher. Here $u^*(t) = 60 - 360t + 360t^2$ and $J^* = 360$ exactly. At five segments, refining the segment integrator:

steps per segment	J , linear control	J , quadratic control
10	360.5769230603	359.9999136000
40	360.5769230603	359.9999996625
320	360.5769230603	360.0000000000

The error of the ramp does not move, because it belongs to the parameterisation; the error of the parabola falls to fourteen digits, because it belonged to the integrator all along. This is the cleanest statement of the separation described in Section 4.3.

Where the optimal control lies in none of the three, which is the usual case, what is left is the order. On the oscillator $\ddot{x} = -100x + u$ driven from the origin to $(1, 0)$ on $[0, 1]$, whose optimal control is a sinusoid, with the segment integrator held fine enough that only the parameterisation is being measured, the relative error in the cost and its ratio after each doubling of M are

M	constant		linear		quadratic	
10	9.796×10^{-2}		1.729×10^{-3}		2.366×10^{-5}	
20	2.328×10^{-2}	4.2	9.218×10^{-5}	18.8	4.824×10^{-7}	49.0
40	5.748×10^{-3}	4.0	5.511×10^{-6}	16.7	8.334×10^{-9}	57.9
80	1.433×10^{-3}	4.0	3.404×10^{-7}	16.2	1.351×10^{-10}	61.7

against the 4, 16 and 64 that $p = 1, 2, 3$ predict. At equal numbers of decision variables the quadratic form is some eight hundred times more accurate in the cost than Hermite–Simpson collocation on this problem at twenty segments, and some three thousand times at forty.

For the path constraints, take Bryson and Denham's problem: the same dynamics with $x_1 \leq 1/9$ and $x_1(0) = 0$, $x_2(0) = 1$, $x_1(1) = 0$, $x_2(1) = -1$, for which $J^* = 4$. No feasible trajectory costs less than that, so a smaller number is a measurement of how much the constraint has leaked:

segments M	boundaries only	2 interior samples	4 interior samples
10	3.9933815	4.0022954	4.0024330
20	3.9981567	4.0002524	4.0003177

Finally the costates of (4.7), against their closed form, on twenty segments: Hermite–Simpson collocation gives $\max |\lambda_1 + 12| = 3.7 \times 10^{-11}$, multiple shooting with a linear control 2.3×10^{-10} , and multiple shooting with a constant control 3.0×10^{-2} . The last is not worse machinery: the constant-control solution really is one and a half per cent away from the continuous optimum, so its costates are too, and the transcription is being consistent with itself.

The code for this example is in `examples/multiple_shooting` of the PSOPT distribution.

4.9 Automatic segment refinement

```
algorithm.mesh_refinement      = "automatic";
algorithm.ms_refine_tolerance = 1.0e-3;    // the default
```

This was refused until the refinement could be written in the currency the transcription is stated in, and the reason is the one that governs everything else in this chapter. On a collocation mesh more nodes means a better approximation of the dynamics, so one estimator and one tolerance serve. Here they do not. The dynamics are integrated to whatever `ms_steps_per_segment` and `ms_integrator` buy, however many segments there are, so refining segments to drive the reported discretisation error down spends decision variables on an error a step count achieves for none. What the segment count controls is the resolution of the *control parameterisation* (Section 4.3) and the coverage of the *path constraints* (Section 4.5), and the indicator measures those two and takes the worse. `ms_refine_tolerance` is the tolerance on it.

`ode_tolerance` is not that tolerance and is not ignored either: it still bounds the reported discretisation error, which on a shooting mesh is the integrator's. When that error is above it PSOPT reports the fact and names the option that fixes it, which is the step count or the scheme and not the segment count.

What automatic segment refinement achieves Minimum time with $u \in [-1, 2]$ again, from uniform partitions whose segment count is not divisible by three, so that no boundary sits on the switch:

initial M	uniform error in t_f	refined M	refined error	gap to the switch
7	1.036×10^{-2}	12	7.637×10^{-9}	1.1×10^{-16}
10	5.038×10^{-3}	14	7.511×10^{-9}	0
13	2.972×10^{-3}	31	6.616×10^{-9}	1.1×10^{-5}

The last column is how far the nearest segment boundary ended up from the switch. The answer stops depending on where the partition started, which is what Section 4.4 buys by a different route, and this route needs no floor to stay well posed, the partition never becoming a decision variable. On a smooth problem it does the ordinary thing: the oscillator of Section 4.8 with a held control goes from a relative cost error of 5.0×10^{-1} at five segments to 8.3×10^{-3} at thirty, which is better than a uniform partition of thirty gives.

The indicator The natural estimate of a representation’s error is its departure from a reconstruction of the same data carrying more degrees. Written naively that flags a corner of the optimal control sitting exactly *on* a segment boundary, where the representation is not in error at all, a piecewise form being free to have a corner at a boundary; and it flags it just as loudly however fine the mesh becomes, because the jump does not shrink with the mesh. It is the same trap that makes error-directed refinement fight the flexible mesh in Section 3.9.

The departure is therefore formed twice, from a window extended to the left of the segment and one extended to the right, and the *smaller* is taken. A corner on a boundary spoils exactly one of the two windows; a corner inside a segment spoils both; on a smooth arc the two agree. Started from twelve segments, which already resolve the switch, the refinement leaves the mesh at twelve.

The path constraints Between two boundaries the state is not a decision variable, so a constraint imposed only at boundaries leaks, and the optimiser returns a cost *below* the true optimum. Nothing in the NLP reports that: every constraint the NLP was given is satisfied. The indicator propagates each segment again after the solve and asks the user’s own path function what happened inside it. On Bryson and Denham’s problem, where $J^* = 4$ and no feasible trajectory costs less:

	M	J	
fixed, no interior samples	10	3.9933815	leaking by 6.6×10^{-3}
automatic, no interior samples	18	3.9999982	leaking by 1.8×10^{-6}
automatic, two samples	22	4.0000006	above J^* , where it belongs

Refinement narrows the leak because narrower segments leave less room between the points where the constraint is imposed; `ms_path_samples` closes it because it adds points. The two are complementary and neither replaces the other.

How much it grows. `mr_max_growth_factor` limits the growth per iteration and `mr_max_iterations` the number of iterations, exactly as they do for the local-collocation refinement of Section 2.4. The final segment count is path-dependent, the refinement making a discrete choice each iteration from an estimate computed on the previous solve, so it should be read as an outcome and not as a specification.

4.10 Choosing the step count automatically

```
algorithm.mesh_refinement      = "automatic";
algorithm.ms_adaptive_steps    = true;      // default false
algorithm.ms_max_steps_per_segment = 200;    // the default
```

This is a different question from the one Section 4.9 answers, and it has its own driver for the reason that governs the whole chapter: the segment count controls the control parameterisation and the path constraints' coverage, and the step count controls the integrator. Neither can fix the other's error. With `ms_adaptive_steps` on, `ms_steps_per_segment` stops being a number to guess and becomes a starting point, and `ode_tolerance` becomes the thing that is met, per *segment*, from the error the previous solve reported, by the usual step-size control for a scheme of order p .

The adaptivity is *between* solves, and that is not a limitation of the implementation. A step count varying with the decision variables would make the constraint function non-smooth in them: a step-acceptance test flipping as the iterate moves changes the discrete map that (4.1) is written on, so the matching condition jumps by the difference between two discretisations and the NLP is given derivatives that do not describe its own residual. This is Bock's argument again [6], one level further out than Section 4.2 uses it, and it is why the option requires `mesh_refinement = "automatic"`: there has to be another solve for a new step count to be used in. Within a solution the step sequence is frozen.

Doing it per segment instead of per phase is what matters here, and the comparison that shows it is not the one against a uniform count. On a problem whose integrator error is *localised* ($\dot{x}_2 = u + 40 \exp(-\frac{1}{2}((t - \frac{1}{2})/0.02)^2)$, perfectly smooth but with a scale of 0.02, so the error lives in two or three of twenty segments), at `ode_tolerance = 10-8`:

step count	stage evals/solve	reported error
uniform, set by hand	1600	1.485×10^{-9}
the best one per-phase count could do	1200	$\approx 5 \times 10^{-9}$
per segment, adapted (2 to 15)	348	4.913×10^{-9}
per segment, adapted, RK8	264	4.913×10^{-9}

A factor of 4.6 against the uniform count and 3.4 against the best a per-phase count could manage; and the second of those is the one that justifies the design, because a per-phase count is set by the worst segment and on a problem of this shape gains nothing at all over a uniform one.

PSOPT prints the table’s smallest, largest and total entry after each adaptation. A *flat* table is the useful negative result: it says the problem did not need this and a uniform count would have served. Reaching `ms_max_steps_per_segment` should be read as a diagnosis and not as a limit. An explicit scheme that cannot resolve a segment in two hundred steps is usually meeting stiffness, and the schemes for that are in Section 4.2.

4.11 What this transcription does not do

Differential-algebraic systems of index above one are not solved. The declaration of Section 4.6 rests on $\partial g/\partial z$ being nonsingular, which is the index-1 condition itself, and the stage solution has nothing to solve for without it. A higher-index system has to be reduced first, which means differentiating the algebraic relation more than once and then stabilising the reduction, since the invariants a repeated differentiation produces are not all enforced by what is left. Both the reduction and the stabilisation are the user’s to do; PSOPT does neither. The integrated-residual transcription of Chapter 3 takes a DAE in the form in which it was written and is the right tool when the reduction is not practical.

The reported discretisation error is optimistic on a stiff problem. It scales the Richardson difference of (4.2) by the scheme’s classical order, and in the stiff limit a Runge–Kutta method converges below that order. Measured on an index-1 DAE with the third-order implicit scheme: an observed order of 3.00 at $\lambda = 0$, 2.90 at $\lambda = 20$ and 2.39 at $\lambda = 500$, the last still climbing towards three as the step falls. The estimate is therefore a guide where stiffness is present, and the reliable check there is the one Section 4.8 uses throughout: take the control the transcription returned, integrate the true dynamics finely, and see where the trajectory lands.

The solution diagnostics of Chapter 6 do run with this transcription. The costates they read are recovered by the adjoint post-processing of Section 4.7, and the rank and conditioning of the constraint Jacobian never depended on the transcription at all. One thing to know when reading that report here: this transcription reuses the collocation layout, whose defect block is one node longer than the number of matching conditions, so n_s rows per phase are empty by construction and count towards any rank deficiency reported. PSOPT counts them separately and says how much of the deficiency they are, so that a report of “deficient by n_s ” can be read as full rank among the constraints that are actually written. Those rows are also counted by IPOPT against the variables, which costs nothing on an ordinary problem and is decisive on one whose state has been replicated; `algorithm.free_padded_defect_rows` and Section 6.1 deal with that.

5. Choosing a suitable discretisation approach for a given optimal control problem

PSOPT provides several families of discretisation: the global pseudospectral methods (the Legendre and Chebyshev methods, which collocate at both endpoints, and the Radau and Gauss methods of Section 1.9); the local methods (trapezoidal and Hermite–Simpson, Section 1.10); optional automatic mesh refinement (the *hp*-adaptive driver for the pseudospectral methods, Section 2.3, or the Betts refinement for the local methods, Section 2.4); and the integrated-residual transcription of Chapter 3. There are no universal rules, but some kinds of problem are solved more easily and accurately by particular approaches. The most relevant factors are the smoothness of the solution, the presence of discontinuities or active constraint arcs, the accuracy required, the problem size, and whether costate estimates or a certified measure of dynamic feasibility are needed. This chapter offers practical guidance; the methods themselves are described in the earlier chapters.

5.1 Pseudospectral or local discretisation?

The pseudospectral methods approximate the trajectory on each mesh segment by a single high-degree polynomial and converge *spectrally* for smooth solutions: the error decreases faster than any fixed power of the number of nodes, so a smooth problem can be solved to very high accuracy with comparatively few nodes. The cost is a dense Jacobian block per segment, whose size grows with the polynomial degree. The local methods place a low-order rule (second-order trapezoidal or fourth-order Hermite–Simpson) on each subinterval; they converge only algebraically, but their banded, sparse Jacobian scales well to large meshes and they are robust when the solution is nonsmooth. As a rule of thumb, a smooth solution for which high accuracy is wanted favours a pseudospectral method, whereas a nonsmooth solution (bang–bang control, kinks, or many active path or state constraints) or one that simply needs a large number of nodes favours a local method, or an *hp*-adaptive pseudospectral mesh, which combines the two behaviours.

5.2 Choosing among the pseudospectral methods

For a smooth single-phase problem the Legendre and Chebyshev methods are good general-purpose choices; both collocate the endpoints and return costate estimates at

every node (Section 1.8.1), and the Chebyshev cost integral is evaluated with the spectrally accurate Clenshaw–Curtis quadrature of Section 1.7.8. The Radau method collocates a single endpoint and underpins most of the convergence and costate theory used by the *hp*-adaptive driver, which makes it a robust default when automatic refinement is enabled. The Gauss method collocates the strictly interior points and treats the terminal state as an additional variable; it is attractive when accurate interior costates are important. In practice the four methods give comparable results on smooth problems, and the choice is often dictated by whether automatic refinement is used (the Radau and Gauss methods have the most developed *hp* theory) and not by accuracy alone.

5.3 Choosing among the local methods

The trapezoidal method is second order and very robust; it is a good choice for an initial or coarse solve, for strongly nonsmooth problems, or as the starting order of an automatic local refinement. The Hermite–Simpson method is fourth order and gives markedly better accuracy for a similar node count when the solution is reasonably smooth. The `switch_order` option lets an automatic local refinement begin with the trapezoidal method and switch to Hermite–Simpson once the mesh is adequate (Section 2.4).

5.4 When to use mesh refinement

If the solution is globally smooth, a single pseudospectral solve on a sensible mesh may already meet the required accuracy, and manual refinement (trying a short increasing sequence of node counts) is often enough. Automatic refinement is worthwhile when the solution has localized features such as boundary layers, fast transients or near-discontinuities, or when a target accuracy is wanted without hand-tuning the mesh. The *hp*-adaptive driver (Section 2.3) raises the polynomial degree where the solution is smooth and subdivides the mesh where it is not, and is the recommended automatic strategy for the pseudospectral methods; the Betts refinement (Section 2.4) plays the same role for the local methods.

5.5 When to use the integrated-residual transcription

Standard collocation enforces the dynamics only at the collocation nodes, leaving the residual between nodes uncontrolled. The integrated-residual transcription of Chapter 3 instead measures, and optionally bounds, the dynamic residual *between* the nodes. This provides a certifiable measure of dynamic feasibility and guards against the inter-node constraint violation that ordinary collocation can hide. It is the preferred choice when a guaranteed bound on the dynamic infeasibility is required, when the dynamics are stiff or oscillatory so that the inter-node error matters, or when robustness is valued over raw speed. The DAIR algorithm (Section 3.6) automates the residual-box tolerance schedule, the Nie–Kerrigan flexible-order representation (Section 3.8) raises the

local approximation order, the flexible mesh (Section 3.9) lets the optimisation isolate a switching time whose location is not known in advance, and costates can be recovered by adjoint post-processing. The transcription is more expensive than plain collocation, so it is best reserved for the situations above instead of used by default.

5.6 When to use multiple shooting

Multiple shooting (Chapter 4) satisfies the dynamics by integrating them instead of by constraining a discretised trajectory, so its answer contains no inter-node dynamic error at all: what error there is belongs to the segment integrator and falls like the order of the scheme chosen for it, which is as high as eight (Section 4.2). That makes it the natural choice for a problem whose dynamics must be propagated faithfully (a long horizon, a trajectory whose sensitivity to the initial state is what is being studied, a model that is only defined along its own solution) and for a user who already trusts a particular integrator and wants the transcription to use it.

Two things argue against it. Its control parameterisation is piecewise constant, linear or quadratic, which is a lower order than collocation carries, so on a smooth problem collocation reaches a given accuracy with fewer variables (Section 4.3). And its path constraints hold only where they are sampled, the state between samples not being a decision variable (Section 4.5), which is a real restriction and not merely a tuning matter, since a boundary-only inequality returns a cost below the true optimum. What is *not* an argument against it any longer is the machinery: the segment count and the step count are both chosen automatically (Sections 4.9 and 4.10), semi-explicit index-1 DAEs are solved in the form in which they are written (Section 4.6), and stiff dynamics have two implicit schemes (Section 4.2). What remains outside it is higher-index DAEs (Section 4.11).

The comparison worth making before choosing is with the integrated-residual transcription instead of with collocation. Both address the inter-node error that collocation leaves uncontrolled, and they address it from opposite directions: multiple shooting removes it by integrating, and the integrated residual measures and bounds it. The first gives the dynamics exactly and the constraints approximately; the second gives the constraints exactly and the dynamics to a stated tolerance.

5.7 Summary guidance

Table 5.1 summarises which approaches tend to suit which problem features. The entries are guidelines, not guarantees.

In summary, begin with a pseudospectral method when the solution is expected to be smooth and high accuracy is the priority; switch to, or combine with, *hp*-adaptive refinement when the solution has localized or nonsmooth features; prefer a local method for large or strongly nonsmooth problems; and use the integrated-residual transcription when a certified measure of dynamic feasibility, or control of the residual between nodes, is required.

Problem feature	Global PS	<i>hp</i> -PS	Local	Integ. res.
Smooth solution, high accuracy wanted	✓	✓	◦	◦
Nonsmooth control (bang-bang, kinks)	–	✓	✓	✓
Localized features (boundary/internal layers)	–	✓	◦	✓
Many active path/state-constraint arcs	◦	✓	✓	✓
Large-scale (very many nodes)	–	◦	✓	◦
Costate / adjoint estimates needed	✓	✓	◦	✓
Certified inter-node feasibility needed	–	–	–	✓
Solution structure unknown, mesh hard to choose	–	✓	◦	✓

Table 5.1: Guidance on discretisation choice by problem feature. “Global PS” denotes a pseudospectral method (Legendre, Chebyshev, Radau or Gauss) on a single fixed mesh; “*hp*-PS” the same methods with *hp*-adaptive refinement; “Local” the trapezoidal or Hermite–Simpson methods; “Integ. res.” the integrated-residual transcription. The symbol ✓ marks an approach that is usually well suited, ◦ one that is usable but with caveats (for example, a local method can reach high accuracy on a smooth problem but only with many nodes), and – one that is normally a poor fit.

6. Solution diagnostics

Setting the option `algorithm.diagnostic_level` to a positive value causes PSOPT to print, after the solve has finished, a report that helps to interpret and validate the computed solution. The report is purely diagnostic: at level 1 it reads only quantities that PSOPT has already computed for the converged solution (the per-interval relative ODE errors, the states, the costates and the stored Hamiltonian), and at level 2 it performs in addition a small number of user-function evaluations. With `diagnostic_level = 0`, the default, the diagnostics are switched off entirely and the solve is unaffected.

At `diagnostic_level` ≥ 1 the report contains, for each phase:

- *discretisation-error localisation.* The intervals carrying the largest relative ODE error are listed with their midpoint times, identifying where the discretisation is least accurate.
- *Spectral smoothness.* For the global pseudospectral methods a per-state smoothness indicator σ is reported, computed as the exponential decay rate of the Legendre-coefficient envelope of the state over the phase, in the spirit of the non-smoothness detection of Liu, Hager and Rao [30]. The decay rate is fitted only over the region of genuine decay, so that the round-off plateau of a well-resolved state is not mistaken for slow decay; a small σ flags a state that is not smooth. Read together with the error localisation, this distinguishes the two reasons an interval may be inaccurate: a large error with high σ indicates a smooth but under-resolved region, for which the order should be raised, whereas a large error with low σ indicates genuine non-smoothness, for which the interval should be subdivided or the integrated-residual transcription of Chapter 3 used. The indicator is reported for the global pseudospectral methods only.
- *Costate structure.* For each state the peak and root-mean-square magnitude and the sign behaviour of the costate are summarised, and a costate that is negligible against the dominant one is flagged as near-null, indicating a state whose adjoint is structurally inactive. The peak magnitudes of any path-constraint multipliers are reported alongside.

At `diagnostic_level` ≥ 2 the report additionally contains two first-order optimality residuals associated with the necessary conditions of optimality [7]:

- *Hamiltonian constancy.* The normalised spread $(\max - \min)/(|\text{mean}| + \epsilon)$ of the stored Hamiltonian $H = L + \lambda^\top f$ along the trajectory. For an autonomous problem

H is constant along the optimal trajectory, so a small spread is an optimality indicator; for an explicitly time-varying problem H need not be constant, and a large spread is then not by itself a defect.

- *Control stationarity.* The stationarity condition $\partial H/\partial u = 0$, evaluated by central finite differences of H formed from the user `dae` and `integrand_cost` functions. The condition holds only at a free interior control with no active path constraint, so the residual is summarised as a maximum taken over exactly those nodes; control nodes lying at an active bound or at an active-path node are excluded and counted separately, because there $\partial H/\partial u$ is balanced by the corresponding multiplier and is expected to be nonzero.

The error localisation and the smoothness indicator are most useful for guiding manual mesh refinement (Section 2.3), while the costate and optimality diagnostics are most useful for confirming that a converged solution satisfies the necessary conditions of optimality.

6.1 Defect rows that hold no dynamics

Every transcription in PSOPT reuses the collocation layout, whose defect block holds $n_s(N + 1)$ rows for a phase of $N + 1$ stored nodes, and only the global Lobatto schemes fill all of it. Multiple shooting has N matching conditions; the trapezoidal and Hermite–Simpson methods have N intervals; the Radau method does not collocate its terminal node; the Gauss method collocates no breakpoint. Each of those writes zero into the rows it cannot fill, which is what the rank diagnostic above reports as rows identically zero over the free columns.

A row of zeros whose bounds are $[0, 0]$ is an equality constraint on nothing, and IPOPT counts the equality constraints against the variables and refuses a problem outright when there are more of the former, reporting

Too few degrees of freedom (n_x = ..., n_c = ...).

The count is n_s per phase, which on an ordinary problem is a handful of rows out of thousands and the refusal never arrives. It becomes decisive when the state is *replicated*. The clearest case is a scenario-augmented robust design, in which M copies of the state are carried against one copy of the control so that the design is non-anticipative: there the phantom deficit is $n_s M$ and it grows with the scenario count. Measured on a two-link arm with four states, two controls and twenty-five nodes, IPOPT reports $n_x = 1551$ and $n_c = 1560$ at fifteen scenarios and stops. Of those 1560 equalities, sixty are the padded rows and sixty are the pinned initial conditions, so 1500 hold any dynamics at all: the problem has 51 degrees of freedom and is refused for having -9 .

```
algorithm.free_padded_defect_rows = true;    // default false
```

frees those rows, giving them infinite bounds exactly as the integrated-residual transcription has always done for its whole defect block. The same arm then carries *eighty*

scenarios under multiple shooting and under trapezoidal collocation, where before it was refused above twelve.

The global Lobatto schemes are the exception, and for them the deficit is real and not phantom: they hold $N + 1$ collocation conditions per state against $N + 1$ stored values, one of which the initial condition pins, so a replicated state costs them n_s per copy whatever the mesh. The same arm is refused above twelve scenarios under the Legendre method with this option set or unset, and `validate` says so, and does not leave the user to wonder why nothing changed.

The default is `false`, and the reason is measured. The rows constrain nothing, so freeing them changes no feasible set. It does change how many equality rows the solver sees, hence its regularisation and inertia correction, hence the sequence of iterates, and on a problem that is already rank deficient that decides which local minimum is reached. Fourteen of the shipped examples move when it is set: most by less than the NLP tolerance, `twoburn` in its fourth significant figure, `mpec` by a factor of thirty on a residual that is heading to zero, that one being a complementarity problem and degenerate by construction, and `vega_ascent` from finishing to exceeding a five-minute budget. An option rather than a change, therefore, with the environment variable `PSOPT_FREE_PADDED_DEFECT_ROWS` provided so that the sweep can be repeated on any machine rather than taken on trust.

7. Defining optimal control and estimation problems for PSOPT

Defining an optimal control or parameter estimation problem involves specifying all the necessary values and functions that are needed to solve the problem. With PSOPT, this is done by implementing C++ functions (e.g. the cost function), and assigning values to data structures which are described below. Once a PSOPT has obtained a solution, the relevant variables can be obtained by interrogating a data structure.

7.1 Interface data structures

The role of each structure used in the PSOPT interface is summarised below.

- *Problem data structure:* This structure is used to specify problem information, including the number of phases and pointers to the relevant functions, as well as phase related information such as number of states, controls, parameters, number of grid points, bounds on variables (e.g. state bounds), and functions (e.g. path function bounds).
- *Algorithm data structure:* This is used to control the solution algorithm and to pass parameters to the NLP solver.
- *Solution data structure:* This is used to store the resulting variables of a PSOPT run.

7.2 Required functions

Table 7.1 lists and describes the parameters used by the interface functions.

7.2.1 endpoint_cost function

The purpose of this function is to specify the terminal costs $\phi_i[\cdot]$, $i = 1, \dots, N$. The function prototype is as follows:

```
adouble endpoint_cost(adouble* initial_states,  
                     adouble* final_states,
```

Parameter	Type	Role	Description
controls	adouble*	input	Array of instantaneous controls
derivatives	adouble*	output	Array of instantaneous state derivatives
e	adouble*	output	Array of event constraints
final_states	adouble*	input	Array of final states within a phase
initial_states	adouble*	input	Array of initial states within a phase
iphase	int	input	Phase index (starting from 1)
linkages	adouble*	output	Array of linkage constraints
parameters	adouble*	input	Array of static parameters within a phase
states	adouble*	input	Array of instantaneous states within a phase
time	adouble	input	Instant of time within a phase
t0	adouble	input	Initial phase time
tf	adouble	input	final phase time
xad	adouble*	input	vector of scaled decision variables
workspace	Workspace*	input	Pointer to workspace structure

Table 7.1: Description of parameters used by the PSOPT interface functions

```

adouble* parameters,
adouble& t0, adouble& tf,
adouble* xad, int iphase,
Workspace* workspace)

```

The function should return the value of the end point cost, depending on the value of phase index `iphase`, which takes on values between 1 and `problem.nphases`.

Example of writing the endpoint cost function for a single phase problem with the following endpoint cost:

$$\varphi(x(t_f)) = x_1(t_f)^2 + x_2(t_f)^2 \quad (7.1)$$

```

adouble endpoint_cost(adouble* initial_states,
                     adouble* final_states,
                     adouble* parameters,
                     adouble& t0, adouble& tf,
                     adouble* xad, int iphase,
                     Workspace* workspace)
{
    adouble x1f = final_states[ 0 ];
    adouble x2f = final_states[ 1 ];

    return ( x1f*x1f + x2f*x2f);
}

```

7.2.2 integrand_cost function

The purpose of this function is to specify the integrand costs $L_i[\cdot]$ for each phase as a function of the states, controls, static parameters and time. The function prototype is as follows:

```
adouble integrand_cost(adouble* states,
                      adouble* controls,
                      adouble* parameters,
                      adouble& time,
                      adouble* xad,
                      int iphase,
                      Workspace* workspace)
```

The function should return the value of the integrand cost, depending on the phase index `iphase`, which takes on values between 1 and `problem.nphases`.

Example of writing the integrand cost for a single phase problem with L given by:

$$L(x(t), u(t), t) = x_1(t)^2 + x_2(t)^2 + 0.01u(t)^2 \quad (7.2)$$

```
adouble integrand_cost(adouble* states,
                      adouble* controls,
                      adouble* parameters,
                      adouble& time,
                      adouble* xad,
                      int iphase,
                      Workspace* workspace)
{
    adouble x1 = states[ 0 ];
    adouble x2 = states[ 1 ];
    adouble u = controls[ 0 ];

    return ( x1*x1 + x2*x2 + 0.01*u*u );
}
```

If the problem does not involve any cost integrand, the user may simply not register any `cost_integrand` function, or register it as follows: `problem.cost_integrand=NULL`.

7.2.3 dae function

This function is used to specify the time derivatives of the states $\dot{x}^{(i)} = f_i[\cdot]$ for each phase as a function of the states themselves, controls, static parameters, and time, as well as the algebraic functions related to the path constraints. Its prototype is as follows:

```

void dae(adouble* derivatives,
        adouble* path,
        adouble* states,
        adouble* controls,
        adouble* parameters,
        adouble& time,
        adouble* xad,
        int iphase,
        Workspace* workspace)

```

This is an example of writing the `dae` function for a single phase problem with the following state equations and path constraints:

$$\begin{aligned}
 \dot{x}_1 &= x_2 \\
 \dot{x}_2 &= -3 \exp(x_1) - 4x_2 + u \\
 0 &\leq x_1^2 + x_2^2 \leq 1
 \end{aligned} \tag{7.3}$$

```

void dae(adouble* derivatives,
        adouble* path,
        adouble* states,
        adouble* controls,
        adouble* parameters,
        adouble& time,
        adouble* xad,
        int iphase,
        Workspace* workspace)
{
    adouble x1 = states[ 0 ];
    adouble x2 = states[ 1 ];
    adouble u = controls[ 0 ];

    derivatives[ 0 ] = x2;
    derivatives[ 1 ] = -3*exp(x1)-4*x2+u;
    path[ 0 ] = x1*x1 + x2*x2
}

```

Note that the bounds on the path constraints are specified separately, possibly in the `main()` function, as follows:

```

problem.phases(1).bounds.upper.path(0) = 0.0;
problem.phases(1).bounds.lower.path(0) = 1.0;

```

7.2.4 events function

This function is used to specify the values of the event constraint functions $e_i[\cdot]$ for each phase. Its prototype is as follows:

```
void events(adouble* e,
            adouble* initial_states,
            adouble* final_states,
            adouble* parameters,
            adouble& t0,
            adouble& tf,
            int iphase,
            Workspace* workspace)
```

Please note, if the initial or final values of the control variables within a phase need to be accessed within the `events()` function, this can be done through the functions `get_initial_controls()` and `get_final_controls()`. See sections [7.11.12](#) and [7.11.13](#).

The following is an example of writing the `events` function for a single phase problem with the event constraints:

$$\begin{aligned} 1 &= e_1(t_0) = x_1(t_0) \\ 2 &= e_2(t_0) = x_2(t_0) \\ -0.1 &\leq e_3(t_0) = x_1(t_f)x_2(t_f) \leq 0.1 \end{aligned} \tag{7.4}$$

```
void events(adouble* e,
            adouble* initial_states,
            adouble* final_states,
            adouble* parameters,
            adouble& t0,
            adouble& tf,
            int iphase,
            Workspace* workspace)
{
    adouble x1i = initial_states[ 0 ];
    adouble x2i = initial_states[ 1 ];
    adouble x1f = final_states[ 0 ];
    adouble x2f = final_states[ 1 ];

    e[ 0 ] = x1i;
    e[ 1 ] = x2i;
    e[ 2 ] = x1f*x2f;
```

```
}

```

Note that the bounds on the event constraints are specified separately, possibly in the `main()` function:

```
problem.phases(1).bounds.lower.events(0) = 1.0;
problem.phases(1).bounds.upper.events(0)= 1.0;
problem.phases(1).bounds.lower.events(1)= 2.0;
problem.phases(1).bounds.upper.events(1)= 2.0;
problem.phases(1).bounds.lower.events(2)=-0.1;
problem.phases(1).bounds.upper.events(2)=0.1;
```

7.2.5 linkages function

This function is used to specify the values of the phase linkage constraint functions $\Psi[\cdot]$. Its prototype is as follows:

```
void linkages( adouble* linkages,
               adouble* xad,
               Workspace* workspace)
```

The following is an example of writing the `linkages` function for a two phase problem with two states in each phase and with the linkage constraints:

$$\begin{aligned} 0 &= \Psi_1 = x_1^{(0)}(t_f^{(1)}) - x_1^{(2)}(t_i^{(2)}) \\ 0 &= \Psi_2 = x_2^{(1)}(t_f^{(1)}) - x_2^{(2)}(t_i^{(2)}) \\ 0 &= \Psi_3 = t_f^{(1)} - t_i^{(2)} \end{aligned} \tag{7.5}$$

These type of state and time continuity constraint can be entered automatically by using the `auto_link` function as shown below. Note that phase linkage bounds are set to zero by default. If they are different from zero, they need to be specified explicitly as shown in the second example below.

```
void linkages( adouble* linkages, adouble* xad, Workspace* workspace)
{
    int index = 0;
    // Link phases 1 and 2
    auto_link( linkages, &index, xad, 1, 2 );
}
```

It is of course also possible to implement more general linkage constraints, as illustrated through the following example.

Consider a two phase problem with two states in each phase and with the nonlinear linkage constraints:

$$\begin{aligned} 0 &= \Psi_1 = x_1^{(1)}(t_f^{(1)}) - \sin[x_1^{(2)}(t_i^{(2)})] \\ 0 &= \Psi_2 = x_2^{(1)}(t_f^{(1)}) - \cos[x_2^{(2)}(t_i^{(2)})] \\ 10 &\leq \Psi_3 = t_f^{(1)} - t_i^{(2)} \leq 100 \end{aligned} \tag{7.6}$$

```
void linkages( adouble* linkages, adouble* xad, Workspace* workspace)
{
    adouble xf_p0[ 2 ];
    adouble xi_p1[ 2 ];
    adouble tf_p0;
    adouble ti_p1;

    get_final_states( xf_p0, xad, 1 );
    get_initial_states( xi_p1, xad, 2 );
    tf_p0 = get_final_time( xad, 1 );
    ti_p1 = get_initial_time( xad, 2 );

    linkages[0]=xf_p0[0]-sin(xi_p1[0]);
    linkages[1]=xf_p0[1]-cos(xi_p1[1]);
    linkages[2]=tf_p0 - ti_p1;
}
```

Note that the bounds are specified separately, possibly in the `main()` function:

```
problem.bounds.lower.linkage(0)= 0.0;
problem.bounds.upper.linkage(0)= 0.0;
problem.bounds.lower.linkage(1)= 0.0;
problem.bounds.upper.linkage(1)= 0.0;
problem.bounds.lower.linkage(2)= 10.0;
problem.bounds.upper.linkage(2)= 100.0;
```

7.2.6 Using the power of Eigen from within the user functions

Eigen is a C++ template library for linear algebra, and thus it is possible to use this to create Eigen matrices with `adouble` elements, and to make operations with them from within the user functions. Recall that `adouble` is a type defined for automatic differentiation purposes. This has the potential of simplifying the implementation of user functions.

As an example, consider the `dae()` function. The list of arguments of this function includes arrays of type `adouble` named `states`, `controls`, `path` and `derivatives`. It is possible, for example, in the case of a problem with 3 states, 2 controls and 2 path constraints, to map these arrays into Eigen objects as follows:

```
Eigen::Map<Eigen::Matrix<adouble, 3, 1>> x(states, 3);
Eigen::Map<Eigen::Matrix<adouble, 2, 1>> u(controls, 2);
Eigen::Map<Eigen::Matrix<adouble, 2, 1>> p(path, 2);
Eigen::Map<Eigen::Matrix<adouble, 2, 1>> dx(derivatives, 3);
```

The user can then use these objects with the syntax and compatible functions that are available through Eigen. For example, assume that the dynamics are given by the linear equation $\dot{x} = Ax + Bu$, with A and B being matrices of the appropriate dimension. One could write the following code inside the `dae()` function:

```
// Define the model matrices A and B
Eigen::Matrix<adouble, 3, 3> A;
Eigen::Matrix<adouble, 3, 2> B;
// Assume that values are assigned to the elements of A and B.
// Assume that x, u and dx are defined as above.
// The following code performs the necessary matrix operations
// and assigns the result to the derivatives array.
dx = A*x + B*u;
```

7.2.7 Main function

Declaration of data structures

The `main()` function is used to declare and initialise the `problem`, `solution`, and `algorithm` data structures, to call the PSOPT algorithm, and to post-process the results. To declare the data structures the user may wish to use the following commands:

```
Alg  algorithm;
Sol  solution;
Prob problem;
```

Problem level information

The user should then define the problem name as follows:

```
problem.name = "My problem";
```

The number of phases and the number of linkage constraints should be declared afterwards. For example, for a single phase problem:

```
problem.nphases=1;
problem.nlinkages=0;
```

This declaration should be followed by the following function call, which initialises problem level structures.

```
psopt_level1_setup(problem);
```

Please note, PSOPT is able to solve problems with multiple phases, and some parameters need to be passed for each particular phase. In the PSOPT interface, the numbering of phases starts from 1, so that the first phase of the problem is phase 1, and problems with only a single phase have their phase number identified as 1.

Phase level information

After this, the user needs to specify phase level parameters using the following syntax:

```
problem.phases(iphase).nstates          = INTEGER;
problem.phases(iphase).ncontrols        = INTEGER;
problem.phases(iphase).nevents          = INTEGER;
problem.phases(iphase).npath            = INTEGER;
problem.phases(iphase).nalgebraic       = INTEGER;
```

where `iphase` represents the number of the phase for which the parameters are being entered.

`nalgebraic` declares how many of the phase's controls are the algebraic variables of a semi-explicit index-1 differential-algebraic system, and defaults to zero, which is every problem that is not one. It is read only by the multiple-shooting transcription, whose segment integrator then solves the algebraic equations at each of its own stages; under collocation or the integrated residual the trajectory is made of decision variables and an algebraic relation is an ordinary equality path constraint at every node, so the declaration is neither needed nor accepted there. The convention is that the *last* `nalgebraic` controls are the algebraic variables and the *first* `nalgebraic` path constraints are their equations, which must be equalities. See Section 4.6.

To specify the number of nodes (time instants) in the solution grid, there are different possibilities. If a single solution is required with a fixed number of nodes, this can be specified as follows:

```
problem.phases(iphase).nodes          << INTEGER
```

If a sequence of nodes needs to be tried, so that manual mesh refinement is performed with an increasing number of nodes from solution to solution, and using the previous solution as a guess, then this can be specified in the example below:

```
problem.phases(iphase).nodes=(RowVectorXi(2)<<50, 60).finished();
```

Note, the number 2 in `RowVectorXi(2)` indicates the number of elements in the sequence, and the sequence of nodes itself is specified by the values 50, 60. It is also possible to define the number of nodes using a previously defined variable of type `RowVectorXd`, as in the followign example:

```
RowVectorXi my_vector << 50, 60;
problem.phases(iphase).nodes = my_vector;
```

Once the phase level dimensions have been specified it is necessary to call the following function:

```
psopt_level2_setup(problem, algorithm);
```

Phase bounds information

The syntax to enter state bounds is as follows:

```
problem.phases(iphase).bounds.lower.states(j) = REAL;
problem.phases(iphase).bounds.upper.states(j) = REAL;
```

where `iphase` is a phase index between 1 and `problem.nphases`, and `j` is an index between 0 and `problem.phases(iphase).nstates-1`.

The syntax to enter control bounds is as follows:

```
problem.phases(iphase).bounds.lower.controls(j) = REAL;
problem.phases(iphase).bounds.upper.controls(j) = REAL;
```

where `j` is an index between 0 and `problem.phases(iphase).ncontrols-1`.

The syntax to enter event bounds is as follows:

```
problem.phases(iphase).bounds.lower.events(j) = REAL;
problem.phases(iphase).bounds.upper.events(j) = REAL;
```

where `j` is an index between 0 and `problem.phases(iphase).nevents`.

The bounds on the start time for each phase are entered using the following syntax:

```
problem.phases(iphase).bounds.lower.StartTime = REAL;
problem.phases(iphase).bounds.upper.StartTime = REAL;
```

The bounds on the end time for each phase are entered using the following syntax:

```
problem.phases(iphase).bounds.lower.EndTime = REAL;
problem.phases(iphase).bounds.upper.EndTime = REAL;
```

Linkage bounds information

The syntax to enter linkage bounds values is as follows:

```
problem.bounds.lower.linkage(j) = REAL;  
problem.bounds.upper.linkage(j) = REAL;
```

`j` is an index between 0 and `problem.nlinkages`. The default value of the linkage bounds is zero.

Specifying the initial guess for each phase

The user may wish to specify an initial guess for the solution, instead of allowing PSOPT to determine the initial guess automatically. The user may specify any of the following for each phase: the control vector history, the state vector history and the time vector corresponding to the control and state histories, as well as a guess for the static parameter vector.

To specify the initial guesses for each phase, the user needs to create `DMatrix` objects to hold the initial guesses, and then assign the addresses of these objects to relevant pointers within the `Guess` structure.

The syntax to specify the initial guess in a particular phase is as follows:

```
problem.phases(iphase).guess.controls      = uGuess;  
problem.phases(iphase).guess.states       = xGuess;  
problem.phases(iphase).guess.parameters   = pGuess;  
problem.phases(iphase).guess.time         = tGuess;
```

where `uGuess`, `xGuess`, `pGuess` and `tGuess` are `MatrixXd` objects that contain the relevant guesses. Users may find it useful to employ the functions `zeros`, `ones`, and `linspace` to specify the initial guesses.

For example, the following code creates an object to store an initial control guess with 20 grid points, and assigns zeros to it.

```
MatrixXd uGuess = zeros(1, 20);
```

The following code defines a linear history in the interval `[10,15]` for the first state, and a constant history for the second state, for a system with two states assuming 20 grid points:

```
MatrixXd xGuess(2,20);  
xGuess.row(0) = linspace( 10, 15, 20); // First row  
xGuess.row(1) = 10*ones( 1, 20); // Second row
```

The following code defines a time vector with equally spaced values in the interval `[0,10]` assuming 20 grid points:

```
MatrixXd tGuess = linspace( 0, 10, 20);
```

Scaling information

The user may wish to supply the scaling factors instead of allowing PSOPT to compute them automatically.

Scaling factors for controls, states, event constraints, derivatives, path constraints, and time for each phase can be entered as follows:

```
problem.phases(iphase).scale.controls(j)      = REAL;
problem.phases(iphase).scale.states(j)        = REAL;
problem.phases(iphase).scale.events(j)        = REAL;
problem.phases(iphase).scale.defects(j)       = REAL;
problem.phases(iphase).scale.time             = REAL;
```

The scaling factor for the objective function is entered as follows:

```
problem.scale.objective                       = REAL;
```

Scaling factors for the linkage constraints are entered as follows:

```
problem.scale.linkage(j)                     = REAL;
```

Passing user data to the interface functions

It is possible to pass user data to the different interface functions by using a void pointer that is a member of the `problem` data structure. This is good programming practice, as it helps avoid the use of global or static variables.

For example, the user could define a data structure to encapsulate some key data values, such as:

```
typedef struct{

double c1;
double c2;
double c3;

} Mydata;
```

In the `main` function, an instance of this data structure can be allocated:

```
Mydata* md = (Mydata*) malloc(sizeof(Mydata));
```

and its elements given values:

```
md->c1 = 1.0;
md->c2 = 100.0;
md->c3 = 1000.0;
```

In the `main()` function, after the problem data structure has been declared, then the pointer to the instance of the `Mydata` data structure can be stored in the `user_data` member of the `problem` data structure, as follows:

```
problem.user_data = (void*) md;
```

Finally, the user can access this data from within the interface functions. For example, to access the data from within the `integrand_cost` function, the following can be done:

```
adouble integrand_cost(adouble* states,
adouble* controls,
adouble* parameters,
adouble& time,
adouble* xad,
int iphase,
Workspace* workspace)
{
    adouble x1 = states[ 0 ];
    adouble x2 = states[ 1 ];
    adouble u = controls[ 0 ];

    Mydata* md = (Mydata*) workspace->problem->user_data;

    double c1 = md->c1;
    double c2 = md->c2;
    double c3 = md->c3;

    return ( c1*x1*x1 + c2*x2*x2 + c3*u*u );
}
```

Specifying algorithm options

Algorithm options and parameters can be specified as follows:

<code>algorithm.nlp_method</code>	<code>= STRING;</code>
<code>algorithm.qp_solver</code>	<code>= STRING;</code>
<code>algorithm.sqp_strategy</code>	<code>= STRING;</code>
<code>algorithm.qp_restoration</code>	<code>= STRING;</code>
<code>algorithm.elastic_penalty</code>	<code>= STRING;</code>
<code>algorithm.qp_iter_max</code>	<code>= INTEGER;</code>
<code>algorithm.trust_region</code>	<code>= STRING;</code>
<code>algorithm.trust_region_radius</code>	<code>= REAL;</code>
<code>algorithm.scaling</code>	<code>= STRING;</code>
<code>algorithm.defect_scaling</code>	<code>= STRING;</code>
<code>algorithm.constraint_scaling</code>	<code>= STRING;</code>
<code>algorithm.derivatives</code>	<code>= STRING;</code>
<code>algorithm.collocation_method</code>	<code>= STRING;</code>
<code>algorithm.transcription_method</code>	<code>= STRING;</code>
<code>algorithm.objective_form</code>	<code>= STRING;</code>
<code>algorithm.ms_integrator</code>	<code>= STRING;</code>
<code>algorithm.ms_steps_per_segment</code>	<code>= INTEGER;</code>
<code>algorithm.ms_control_parameterisation</code>	<code>= STRING;</code>
<code>algorithm.ms_path_samples</code>	<code>= INTEGER;</code>
<code>algorithm.ms_flexible_segments</code>	<code>= BOOLEAN;</code>
<code>algorithm.ms_min_segment_fraction</code>	<code>= REAL;</code>
<code>algorithm.ms_refine_tolerance</code>	<code>= REAL;</code>
<code>algorithm.ms_adaptive_steps</code>	<code>= BOOLEAN;</code>
<code>algorithm.ms_max_steps_per_segment</code>	<code>= INTEGER;</code>
<code>algorithm.ms_algebraic_iterations</code>	<code>= INTEGER;</code>
<code>algorithm.ms_implicit_iterations</code>	<code>= INTEGER;</code>
<code>algorithm.ir_objective</code>	<code>= STRING;</code>
<code>algorithm.ir_residual_nodes</code>	<code>= INTEGER;</code>
<code>algorithm.ir_residual_bound</code>	<code>= REAL;</code>
<code>algorithm.ir_regularization</code>	<code>= REAL;</code>
<code>algorithm.ir_dair</code>	<code>= BOOLEAN;</code>
<code>algorithm.ir_dair_delta_factor</code>	<code>= REAL;</code>
<code>algorithm.ir_local_order</code>	<code>= INTEGER;</code>
<code>algorithm.ir_element_local_controls</code>	<code>= BOOLEAN;</code>
<code>algorithm.ir_flexible_mesh</code>	<code>= BOOLEAN;</code>
<code>algorithm.ir_min_element_fraction</code>	<code>= REAL;</code>
<code>algorithm.ir_include_path</code>	<code>= STRING;</code>
<code>algorithm.ir_path_weight</code>	<code>= REAL;</code>
<code>algorithm.ir_residual_scaling</code>	<code>= STRING;</code>
<code>algorithm.nlp_iter_max</code>	<code>= INTEGER;</code>

<code>algorithm.nlp_tolerance</code>	<code>= REAL;</code>
<code>algorithm.print_level</code>	<code>= INTEGER;</code>
<code>algorithm.jac_sparsity_ratio</code>	<code>= REAL;</code>
<code>algorithm.hess_sparsity_ratio</code>	<code>= REAL;</code>
<code>algorithm.hessian</code>	<code>= STRING;</code>
<code>algorithm.hessian_verify</code>	<code>= BOOLEAN;</code>
<code>algorithm.diagnostic_level</code>	<code>= INTEGER;</code>
<code>algorithm.on_error</code>	<code>= STRING;</code>
<code>algorithm.parameter_statistics</code>	<code>= STRING;</code>
<code>algorithm.mesh_refinement</code>	<code>= STRING;</code>
<code>algorithm.ode_tolerance</code>	<code>= REAL;</code>
<code>algorithm.mr_max_iterations</code>	<code>= INTEGER;</code>
<code>algorithm.mr_min_order</code>	<code>= INTEGER;</code>
<code>algorithm.mr_max_order</code>	<code>= INTEGER;</code>
<code>algorithm.mr_max_growth_factor</code>	<code>= REAL;</code>
<code>algorithm.mr_kappa</code>	<code>= REAL;</code>
<code>algorithm.mr_M1</code>	<code>= INTEGER;</code>
<code>algorithm.mr_switch_detection</code>	<code>= INTEGER;</code>
<code>algorithm.switch_order</code>	<code>= INTEGER;</code>
<code>algorithm.parameter_estimation_norm</code>	<code>= INTEGER;</code>
<code>algorithm.save_sparsity_pattern</code>	<code>= INTEGER;</code>
<code>algorithm.ipopt_linear_solver</code>	<code>= STRING;</code>
<code>algorithm.ipopt_max_cpu_time</code>	<code>= REAL;</code>

Note that:

- `algorithm.nlp_method` takes the options “IPOPT” (default) or “SQP”. The latter selects PSOPT’s own sequential quadratic programming solver, which must have been built into the library; see Section 7.3 for what it is, how to build it and when it is worth using. IPOPT remains the default and is the better choice for most problems.
- `algorithm.scaling` takes the options “automatic” (default) or “user”.
- `algorithm.defect_scaling` takes the options “state-based” (default) or “jacobian-based”.
- `algorithm.constraint_scaling` selects how the constraint scale factors are computed when `algorithm.scaling` is “automatic”. Every constraint row other than a differential defect is given the factor that makes the corresponding row of the scaled Jacobian have unit Euclidean norm, and this option chooses how that Jacobian is obtained: “automatic” (the default) takes it from the automatic-differentiation tape in one sparse evaluation, and any other value forces the finite-difference route, which costs one Jacobian column per variable. There is normally no reason to change it when automatic differentiation is in use; it exists so that

the two routes can be compared. A row whose gradient vanishes at the initial guess carries no information about that constraint’s magnitude and is given a neutral factor of 1 instead of the largest the formula can produce; every other factor is clamped to $[10^{-7}, 10^7]$. Differential defects are scaled separately, by `algorithm.defect_scaling`.

- `algorithm.derivatives` takes the options “automatic” (default) or “numerical”.
- `algorithm.collocation_method` takes the options “Legendre” (default), “Chebyshev”, “trapezoidal”, “Hermite-Simpson”, “Radau”, or “Gauss”. The “Radau” and “Gauss” options select the Legendre–Gauss–Radau and Legendre–Gauss pseudospectral discretisations of Section 1.9.
- `algorithm.transcription_method` takes the options “collocation” (default), “integrated-residual” or “multiple-shooting”. The second selects the transcription of Chapter 3 and the options prefixed `ir` below apply only in that case; the third selects the transcription of Chapter 4 and the options prefixed `ms` apply only in that one.
- `algorithm.ms_integrator` chooses the scheme the segment integrator uses, and takes four options (Section 4.2). Two of them are explicit: “RK4” is the default, the classical fourth-order method, and “RK8” is Cooper and Verner’s eleven-stage eighth-order formula. RK8 costs 2.75 times the stages per step and converges four orders faster, so it buys a given integrator accuracy in far fewer right-hand-side evaluations, a saving in tape and not merely in arithmetic, though it does not improve an answer whose accuracy is capped by the control parameterisation. The other two, “TRBDF2” and “ESDIRK3”, are stiffly accurate, L-stable ESDIRKs of classical order two and three, for *stiff* dynamics, where an explicit scheme is not inaccurate but unbounded; see Section 4.2. On a non-stiff problem they are the wrong choice, being dearer per step than RK4 and of lower order than RK8.
- `algorithm.ms_steps_per_segment` is a positive integer (default 10) giving the number of fixed steps the segment integrator takes across one segment (Section 4.2). The tape grows linearly in it, and the error of the scheme falls like the p th power of the step for a scheme of order p .
- `algorithm.ms_control_parameterisation` sets the shape of the control across a segment (Section 4.3) and takes the options “constant” (default), “linear” or “quadratic”. The three are first, second and third order in the control and therefore second, fourth and sixth order in the cost. The quadratic form adds one control variable at the midpoint of each segment; the constant form is the only one of the three that is discontinuous across a boundary, which is what makes a bang-bang answer come out as a bang-bang answer, and the only one whose interpolant is guaranteed to respect the control bounds.
- `algorithm.ms_path_samples` is a non-negative integer (default 0) giving the number of interior points per segment at which the path constraints are also enforced

(Section 4.5). Zero enforces them at the segment boundaries only, which is correct but weak. The samples are placed at integrator step boundaries, so this must be at most `ms_steps_per_segment - 1`.

- `algorithm.ms_flexible_segments` is a boolean (default `false`). When true the segment boundaries become decision variables, so that the optimisation can put one at a switching time instead of leaving the switch inside a segment (Section 4.4). It is worth six orders of magnitude on a problem whose optimal control switches, and it should not be used on one whose optimal control is smooth.
- `algorithm.ms_min_segment_fraction` is the floor on a segment's width as a fraction of the uniform width (default 0.8), used only when `ms_flexible_segments` is true. It is not a safety device: a moving partition is a free-knot approximation problem and the floor is what makes it well posed. See Section 4.4.
- `algorithm.ms_refine_tolerance` is the tolerance automatic segment refinement compares its indicator against (default 10^{-3}), used only when `mesh_refinement` is "automatic" and the transcription is multiple shooting (Section 4.9). It is a separate option from `ode_tolerance` because it bounds a different quantity: the segment count controls the control parameterisation and the path-constraint coverage, and the reported discretisation error is the segment integrator's and is fixed by `ms_steps_per_segment` or `ms_integrator`. The indicator is dimensionless.
- `algorithm.ms_adaptive_steps` is a boolean (default `false`). When true, the step count is chosen per *segment* between solves, from the reported discretisation error and the value of `ode_tolerance`; see Section 4.10. It requires automatic mesh refinement, since a new step count needs a further solve to be used in, and `ms_steps_per_segment` then gives the starting point instead of the answer.
- `algorithm.ms_max_steps_per_segment` is the ceiling on any one segment's step count under `ms_adaptive_steps` (default 200). Reaching it is a diagnosis instead of a limit: an explicit scheme that cannot resolve a segment in that many steps is usually meeting stiffness, for which the implicit schemes of Section 4.2 exist.
- `algorithm.ms_algebraic_iterations` is the fixed, unrolled number of Broyden iterations the half-explicit scheme spends on a phase's algebraic equations at each stage (default 4), used only when a phase declares `nalgebraic` and the integrator is explicit (Section 4.6). The default is derived from the secant recurrence and is enough for both explicit schemes; it does not have to grow with the number of algebraic components.
- `algorithm.ms_implicit_iterations` is the fixed, unrolled number of modified-Newton iterations an implicit scheme spends on each stage system (default 4). It is read only when `ms_integrator` names one of the two implicit schemes; see Section 4.2.

- `algorithm.objective_form` takes the options “as-posed” (default), “mayer” or “auto”. It changes how the objective is presented to the NLP solver, not what it is. “as-posed” leaves it as the user wrote it, so a Lagrange term arrives as a dense weighted sum over every node. “mayer” carries the integral of each phase as one extra NLP variable J_i , tied to the quadrature by a single equality, so the objective the solver sees is a sum of individual variables and its gradient is sparse. “auto” applies the transformation only where it is known to pay: the built-in SQP solver, a Lobatto pseudospectral scheme (Legendre or Chebyshev), and a phase that actually has an integrand. The answer is unchanged in every case; what changes is the sparsity of the objective gradient, which matters to a solver that factorises it.
- `algorithm.ir_objective` takes the options “residual” (minimise the integrated residual, equation (3.7)) or “cost” (minimise the user cost subject to the residual box, equation (3.9)).
- `algorithm.ir_residual_nodes` is a positive integer giving the number of Gauss–Legendre residual-quadrature points (equation (3.3)), counted per mesh interval under the default cubic-Hermite representation and per element under the flexible-order one. When the flexible-order representation is used it must be at least `ir_local_order + 2`; smaller values are rejected, for the reason given in Section 3.2.
- `algorithm.ir_residual_bound` is the non-negative residual-box tolerance δ used when `ir_objective` is “cost” (equation (3.9)).
- `algorithm.ir_regularization` is the non-negative penalty weight ρ on the integrated residual added to the user cost (equation (3.8)); available with the Hermite–Simpson method.
- `algorithm.ir_dair` is a Boolean (default `false`); if `true`, PSOPT runs the Direct Alternating Integrated Residual (DAIR) algorithm of Section 3.6.
- `algorithm.ir_dair_delta_factor` is the positive factor K in the DAIR box-tolerance schedule $\delta = K(h/T)^p$ (equation (3.10), default 1).
- `algorithm.ir_local_order` is a non-negative integer selecting the local representation: 0 (default) gives the cubic-Hermite representation, and any $d \geq 2$ gives the degree- d Nie–Kerrigan flexible-order representation of Section 3.8 (the number of intervals per phase must then be a multiple of d).
- `algorithm.ir_element_local_controls` is a Boolean (default `true`) and applies to the flexible-order representation only. Under `true` each element carries its own control value at its left-hand end, so the control may jump between elements; under `false` neighbouring elements share that value and the control is continuous. The default is not a matter of taste: sharing makes the solve converge to the wrong

answer at $d = 2$, for the reason given in Section 3.8. Set it `false` only when what PSOPT is carrying as a control is genuinely continuous, such as the algebraic variable of a DAE.

- `algorithm.ir_flexible_mesh` is a Boolean (default `false`) which makes the element widths decision variables, so that the optimisation can place an element boundary at a switching time (Section 3.9). It applies to both local representations: the Nie–Kerrigan element of degree d , and the cubic-Hermite element, which is one mesh interval.
- `algorithm.ir_min_element_fraction` is a real number strictly between 0 and 1 (default 0.05) giving the smallest element width the flexible mesh may produce, as a fraction of the uniform width $2/M$. It is a bound and not a penalty, because a width free to reach zero gives a singular local problem.
- `algorithm.ir_include_path` takes the options “auto” (default) or “none”, and selects which path constraints are folded into the integrated residual (Section 3.3). Under “auto” every path constraint declared as an equality (equal lower and upper bounds) is folded in and freed from the pointwise constraint set; under “none” the path output is discarded by the residual and the algebraic equations are imposed pointwise only. Inequality path constraints are never folded under either setting.
- `algorithm.ir_path_weight` is the non-negative weight ϖ in (3.5) applied to each folded algebraic residual component (default 1).
- `algorithm.ir_residual_scaling` takes the options “state” (default) or “none”. Under “state” each differential residual component is non-dimensionalised by the ratio of its state scale to the time scale, so that a single box tolerance, and the DAIR schedule, are model independent; “none” bounds the raw residual.
- `algorithm.diff_matrix` takes the options “standard” (default), “reduced-roundoff”, or “central-differences”.
- `algorithm.print_level` takes the values 1 (default), which causes PSOPT and the NLP solver to print information on the screen, or 0 to suppress all output.
- `algorithm.nlp_tolerance` is a real positive number that is used as a tolerance to check convergence of the NLP solver (default 10^{-6}).
- `algorithm.jac_sparsity_ratio` is a real number in the interval $(0,1]$ which indicates the maximum Jacobian density, which is ratio of nonzero elements to the total number of elements of the NLP constraint Jacobian matrix (default 0.5).
- `algorithm.hess_sparsity_ratio` is a real number in the interval $(0,1]$ which indicates the maximum Hessian density, this is the ratio of nonzero elements to the total number of elements of the NLP Hessian matrix (default 0.2).

- `algorithm.hessian` takes the options “limited-memory” (default), “exact”, or “numerical”. The “limited-memory” option uses IPOPT’s internal limited-memory BFGS approximation and needs no second derivatives. The “exact” option supplies the exact Hessian of the Lagrangian by automatic differentiation, and so requires `algorithm.derivatives` to be “automatic”. The “numerical” option supplies a sparse finite-difference approximation of the same Hessian and is available with either automatic or numerical first derivatives. The “numerical” option is only used together with the IPOPT NLP solver; the “exact” option is used by both IPOPT and the SQP solver of Section 7.3, and is strongly recommended when the SQP is selected, since its alternative is a dense quasi-Newton model whose storage grows with the square of the number of variables. See Section 7.7.2.
- `algorithm.hessian_verify` is a Boolean (default `false`). When `true` and `algorithm.hessian` is “numerical”, PSOPT performs a one-off cross-check of the finite-difference Hessian (Section 7.7.2) and prints the result; it is a diagnostic only and has no effect at its default value.
- `algorithm.diagnostic_level` is a non-negative integer (default 0) selecting the solution diagnostics of Chapter 6: 0 disables them (the solve is unaffected), 1 reports the per-interval discretisation error, the per-state spectral smoothness and the costate structure, and 2 additionally reports the Hamiltonian-constancy and control-stationarity optimality residuals.
- `algorithm.free_padded_defect_rows` is a Boolean (default `false`). Every transcription reuses the collocation layout, and the schemes that cannot fill its whole defect block write zeros into the rest; with this option `true` those rows are given infinite bounds instead of $[0, 0]$, so that IPOPT does not count them as equality constraints against the variables. It matters where the state is replicated, a scenario-augmented robust design above all, and it is off by default because it moves the sequence of iterates on a problem that is already rank deficient. See Section 6.1.
- `algorithm.on_error` takes the values “fail-fast” (default) or “fail-soft”. It selects how PSOPT behaves if a solution is accessed after a solution that did not succeed; see Section 7.4.
- `algorithm.nsteps_error_integration` is an integer number that gives the number of integration steps to be taken within each interval when calculating the relative ODE error. The default value is 10.
- `algorithm.parameter_estimation_norm` is a positive integer selecting the norm in which the observation residuals of a parameter estimation problem are measured (default 2). The value 2 gives the sum of squares, 1 the sum of absolute values, and any value $p > 2$ the sum of p th powers. The covariance and confidence-interval machinery of Section 1.11.3 is derived for least squares and should be read with that in mind if a different norm is chosen.

- `algorithm.save_sparsity_pattern` takes the value 0 (default) or 1. With 1, and under IPOPT, PSOPT writes the sparsity pattern of the constraint Jacobian to the file `jacobian_pattern.dat` in triplet form with one-based indices, which reads directly into MATLAB, Octave or a sparse-matrix constructor for inspecting the structure a formulation produces. The pattern is written once, at the first evaluation, and the option is reset to 0 afterwards; it is a diagnostic and has no effect on the solve.
- `algorithm.ipopt_max_cpu_time` is the wall-clock budget in seconds passed to IPOPT's own `max_cpu_time` option (default 3600). A solution that reaches it stops and reports that it did; raising it is the right response to a large problem that was still making progress, and lowering it is useful when a sweep must not be held up by one point.
- `algorithm.parameter_statistics` takes the options "no" (default) or "yes". When "yes", and the problem carries an observation function, PSOPT computes the covariance matrix and the confidence intervals of the estimated parameters (Section 1.11.3) and records them on the solution object; they are also written to the solution summary file when `print_level` is above zero. The calculation costs two Jacobians and a QR decomposition of a matrix the size of the discretised problem, which is why it is not done unless asked for.
- `algorithm.mesh_refinement` takes the values "manual" (default) or "automatic". With a global pseudospectral method (Legendre, Chebyshev, Radau or Gauss), the "automatic" option invokes the *hp*-adaptive refinement of Section 2.3; with a local method (trapezoidal or Hermite–Simpson) it invokes the Betts refinement of Section 2.4. With the integrated-residual transcription using either the flexible-order representation or the flexible mesh it invokes the element refinement of Section 3.10 instead, which splits elements; the Betts refinement cannot serve there, because the element basis is built for the reference LGL abscissae and a node inserted at an interval midpoint belongs to no element. It is not supported at all with `transcription_method = "multiple-shooting"`, where how many segments to use is a question about the control parameterisation as opposed to the accuracy of the dynamics; manual refinement, a sequence of counts in `problem.phases(iphase).nodes`, is the way to refine such a problem.
- `algorithm.ode_tolerance` is a small real value that is used as one of the stopping criteria for mesh refinement. If the maximum relative ODE error falls below this value, the mesh refinement iterations are terminated. The default value is 10^{-3} .
- `algorithm.mr_max_iterations` is a positive integer with the maximum number of mesh refinement iterations (default 7).
- `algorithm.mr_min_order` is a positive integer (≥ 2) giving the minimum per-segment polynomial degree $N_{\min}$ used by the *hp*-adaptive refinement, i.e. the

degree at which a freshly subdivided segment is seeded (default 3). Only used with a global pseudospectral method.

- `algorithm.mr_max_order` is a positive integer ($\geq \text{mr_min_order}$) giving the maximum per-segment polynomial degree $N_{\max}$ used by the *hp*-adaptive refinement; a segment whose degree would exceed $N_{\max}$ is subdivided (*h*-refinement) instead of raised in order (default 12). Only used with a global pseudospectral method.
- `algorithm.mr_max_growth_factor` is a positive real number in the range $(0, 1]$ that bounds the per-iteration growth of the mesh (default 0.4). With a global pseudospectral method it limits the growth of the total number of collocation points N_{eff} ; with a local method it limits the number of nodes added per iteration.
- `algorithm.mr_kappa` is a positive real number used by the local mesh refinement algorithm (default 0.1).
- `algorithm.mr_M1` is a positive integer used by the local mesh refinement algorithm (default 5).
- `algorithm.mr_switch_detection` takes the value 0 (default) or 1, and applies to the *hp*-adaptive refinement of Section 2.3 only. With 1, the refinement locates the switches of the control inside an interval and cuts the mesh at them, so that each arc gets its own sub-interval. It is worth turning on for a problem whose Hamiltonian is linear in the control, so that the solution is bang-bang and the switches are what the mesh has to resolve, and it is off by default because most problems have no switches for it to find. The switch positions are estimated from the crossings of the level midway between the two plateaus instead of from the largest node-to-node jump, because a bang-bang control on a polynomial basis rings about the plateaus it is joining and the largest jump is then often part of the ringing; and an interval is considered to hold a switch only if the control varies over a good fraction of its range across the whole phase, which keeps the detector off intervals where the control is merely curved.
- `algorithm.switch_order` is a positive integer indicating the local mesh refinement iteration after which the order is switched from 2 (trapezoidal) to 4 (Hermite-Simpson). If the entered value is zero, then the order is not switched and the collocation method specified through the option `algorithm.collocation_method` is used in all mesh refinement iterations. This option only applies if a local collocation method is specified (default 2).
- The following seven options apply only when `algorithm.nlp_method` is “SQP”, and are ignored otherwise. Section 7.3 describes the solver they configure.
 - `algorithm.qp_solver` selects the backend that solves the quadratic programming subproblem: “GALAHAD” (default), “ProxQP”, “QPALM”, “OSQP”, “PIQP” or “Clarabel”. Whichever is named must have been built into the

library. All six factorise the KKT system sparsely. GALAHAD's QPA is the one the solver has been tuned and measured against, and is the one to use unless there is a reason to do otherwise; the choice is not cosmetic, and the reasons for changing it are given in Section 7.3.3.

- `algorithm.sqp_strategy` takes the options “FM” (default), “M” or “F”, which are the algorithm strategies of [5], Section 2.6.2. “FM” first finds a point that is feasible with respect to the constraints, ignoring the objective entirely, and then minimises from there; “M” minimises from the initial guess, taking the constraints and the objective together throughout; “F” stops once a feasible point has been found, which is useful for diagnosing whether the constraints alone are the source of a difficulty. “FM” is the default here for the same reason it is Betts's: difficulties caused by the constraints are more easily separated from difficulties caused by the objective than diagnosed together.
- `algorithm.qp_restoration` takes the options “elastic” (default) or “relaxation”. Both provide a relaxed subproblem to fall back on when the linearised constraints are inconsistent, which is the ordinary situation when the initial guess violates an equality constraint. “elastic” gives each constraint its own pair of non-negative slacks, charged in the objective; “relaxation” is the single-variable form of [5], Section 2.7.
- `algorithm.elastic_penalty` takes the options “weights” (default) or “multipliers”, and sets what a unit of constraint violation costs in the relaxed subproblem.
- `algorithm.qp_iter_max` gives the number of iterations the backend is allowed for a single subproblem. It is a budget instead of a promise: a backend that finishes sooner finishes sooner. Zero, the default, asks for a budget scaled to the subproblem (twice the number of variables and constraints, and never less than a thousand) and a positive value is used exactly as given. The scaling matters because an active-set method moves one constraint into or out of its working set per iteration, so what it needs grows with the problem: a constant budget is a tighter constraint on a fine mesh than on a coarse one, which is the wrong way round. A subproblem that stops at its budget returns a point on the way to a step, and a direction built from an interrupted search can be bad enough to undo the progress made so far.
- `algorithm.trust_region` takes the options “box” (default) or “l2”, and sets the shape of the region the step is restricted to. “box” restricts every component, $|d_j| \leq \Delta$, which costs nothing because it is expressed in the subproblem's variable bounds and every backend takes it. Its weakness is that it bounds the widest component of the step: it permits $\|d\|_2$ as large as $\Delta\sqrt{n}$, so the same Δ is a looser restriction the finer the mesh, which is the opposite of what refinement wants, and it is not the region the convergence theory of trust-region methods is stated for. “l2” restricts the step itself, $\|d\|_2 \leq \Delta$. A Euclidean ball is a second-order cone, so the subproblem is no longer a

quadratic programme and only a backend that solves conic programmes can accept it; of those built here that is Clarabel. Every other backend refuses such a subproblem, so the choice is reported and not absorbed.

- `algorithm.trust_region_radius` is the initial radius Δ_0 . Zero, the default, asks for the built-in value of 0.3, which was measured for the box, where it bounds a component of the step. Under “l2” the same number bounds the whole step, and since the ball of radius Δ sits inside the box of the same radius by a factor growing like $\sqrt{n}$, 0.3 is a far tighter start on a fine mesh than on a coarse one. It is exposed so that the question can be asked, not assumed.
- `algorithm.ipopt_linear_solver` is a string indicating what linear solver should be used by IPOPT. The default value is “mumps”, but other solvers are possible (see the [IPOPT documentation](#)). Any specified solver should be linked to the executable program as a dynamic or static library.
 - `ma27`: use the Harwell routine MA27
 - `ma57`: use the Harwell routine MA57
 - `ma77`: use the Harwell routine MA77
 - `ma86`: use the Harwell routine MA86
 - `ma97`: use the Harwell routine MA97
 - `pardiso`: use the Pardiso package
 - `wsmp`: use WSMP package
 - `mumps`: use MUMPS package (default)

PSOPT checks these options before it builds anything, and refuses a combination it cannot honour instead of proceeding and producing an answer to a different problem. A quadratic programming backend that this build does not carry, a residual-node count below `ir_local_order` + 2, automatic mesh refinement with the flexible-order representation, and a minimum element fraction outside (0, 1) are all rejected at set-up, with a message naming what was asked for and what the alternatives are.

Overriding options from the environment

A number of algorithm options can be overridden from the environment, so that one binary can be run many ways without editing and rebuilding it. This is for benchmarking and for measurement: asking whether a whole set of examples behaves differently under an exact Hessian, under the built-in SQP, or with mesh refinement switched off is then a command instead of an edit to every source file.

The facility is a build option and is **off by default**. The library must be configured with

```
cmake -DPSOPT_ALLOW_ENV_OVERRIDES=ON ...
```

for any of the variables below to be read at all; in an ordinary build they are ignored entirely, so a stray variable in a user's environment cannot change what a program does.

Each variable sets the algorithm option of the same name, in lower case:

PSOPT_NLP_METHOD	PSOPT_QP_SOLVER
PSOPT_DERIVATIVES	PSOPT_QP_RESTORATION
PSOPT_HESSIAN	PSOPT_ELASTIC_PENALTY
PSOPT_SCALING	PSOPT_SQP_STRATEGY
PSOPT_MESH_REFINEMENT	PSOPT_TRUST_REGION
PSOPT_MR_SWITCH_DETECTION	PSOPT_TRUST_REGION_RADIUS
PSOPT_DIAGNOSTIC_LEVEL	PSOPT_QP_ITER_MAX
PSOPT_NLP_TOLERANCE	
PSOPT_NLP_ITER_MAX	

The seven in the right-hand column configure the built-in SQP solver and have no effect under IPOPT.

Those that size the workspace (the Hessian, the scaling, the mesh refinement and the diagnostic level among them) are read before it is built, and the rest once per NLP solve. Every override that actually changes something announces itself on the standard output, naming the variable, the value the source asked for and the value used, so that a log records the options a run really had instead of the options its source names. An override takes precedence over the program's own setting: a program tuned for one configuration does not silently ignore the request. They are for experiments; a setting that a problem genuinely needs belongs in its source, where the next reader will find it.

Calling PSOPT

Once everything is ready, then the `psopt` algorithm can be called as follows:

```
psopt(solution, problem, algorithm);
```

Error checking

The `psopt()` function is *total*: it always returns and never propagates an exception to the caller. It sets `solution.error_flag` to a non-zero value if a run-time error is caught (for example, an inconsistency among the user-supplied parameters, many of which PSOPT checks automatically) and to zero on success. A diagnostic message is printed to the screen and is also available from `solution.error_msg`. The value returned by `psopt()` equals `solution.error_flag`, and the return value carries the `[[nodiscard]]` attribute, so a compiler will warn if it is ignored. The recommended pattern is to check it immediately after the call, as in the following example (here the program exits with code 1 when the solve did not succeed).

```
if (psopt(solution, problem, algorithm))
{
```

```

        exit(1);
    }

```

The behaviour when a failed solution is subsequently accessed, and how it can be configured for power users, are described in Section 7.4.

Postprocessing the results

The `psopt()` function returns the results of the optimisation within the `solution` data structure. The results may then be post-processed.

For example, to save the time, control and state vectors of the first phase, the user may use the following commands:

```

MatrixXd x = solution.get_states_in_phase(1);
MatrixXd u = solution.get_controls_in_phase(1);
MatrixXd t = solution.get_time_in_phase(1);

Save(x, "x.dat");
Save(u, "u.dat");
Save(t, "t.dat");

```

The complete control history under Hermite-Simpson

The Hermite-Simpson discretisation carries a control variable of its own at the midpoint of every interval, as well as at every node. A phase of M nodes therefore has $2M - 1$ control variables, and `get_controls_in_phase()` returns only the M at the nodes, which is the array that has the same number of columns as the states and the one a caller usually wants.

That array is not, however, the control trajectory. The Simpson defect constrains a weighted combination of the node and the midpoint value on each interval, not either of them alone. Where the transcription leaves their difference free, on a singular arc or across a bang-bang switch, the two branches separate by an amount that does not shrink with the mesh, and neither of them is the answer. Plotting the node branch there shows a control the solver never used.

Two accessors return the whole of it:

```

MatrixXd u_hs = solution.get_hs_controls_in_phase(1);
MatrixXd t_hs = solution.get_hs_time_in_phase(1);

```

`get_hs_controls_in_phase()` returns an $n_{\text{controls}} \times (2M-1)$ matrix and `get_hs_time_in_phase()` a $1 \times (2M-1)$ matrix, with the node and midpoint values interleaved into a single strictly increasing sequence $t_0, \bar{t}_0, t_1, \bar{t}_1, \dots, t_{M-1}$, so that the one can be plotted against the

other without further arrangement. The even columns 0, 2, 4, ... are the nodes and reproduce `get_controls_in_phase()` and `get_time_in_phase()` exactly; the odd columns are the midpoints.

Both return an *empty* matrix when the phase carries no midpoint control, which is every pseudospectral method, the trapezoidal method, and the first iteration of an automatic mesh refinement that begins on the trapezoidal method. They are filled for Hermite–Simpson, and for multiple shooting under the "quadratic" control parameterisation, whose control on an interval is the same parabola through node, midpoint and node (Section 4.3). A caller should test for that:

```
MatrixXd u_hs = solution.get_hs_controls_in_phase(1);
if (u_hs.cols() > 0) {
    // the phase carries a control at the midpoint of every interval
}
```

Plotting with the GNUplot interface

If the software GNUplot is available in the system where PSOPT is being run, then the user may employ the `plot()`, `multiplot()`, `surf()`, `plot3()` and `polar()` functions, which constitute a simple interface to GNUplot implemented within the PSOPT library.

The prototype of the `plot()` function is as follows:

```
void plot(MatrixXd& x,
          MatrixXd& y,
          const string& title,
          char* xlabel,
          char* ylabel,
          char* legend=NULL,
          char* terminal=NULL,
          char* output=NULL);
```

where x is a column or row vector with n elements and y is a matrix with one either row or column dimension equal to n . `xlabel` is a string with the label for the x-axis, `ylabel` is a string with the label for y-axis, `legend` is a string with the legends for each curve that is plotted, separated by commas. `terminal` is a string with the GNUplot terminal to be used (see Table 7.2), and `output` is a string with the filename to be used for the output, if any.

The function is overloaded, such that the user may plot together curves generated from different x, y pairs, up to three pairs. The additional prototypes are as follows:

```
void plot(MatrixXd& x1, MatrixXd& y1,
          MatrixXd& x2, MatrixXd& y2,
          const string& title,
          char* xlabel,
```

```

char* ylabel,
char* legend=NULL,
char* terminal=NULL,
char* output=NULL);

void plot(MatrixXd& x1, MatrixXd& y1,
MatrixXd& x2, MatrixXd& y2,
MatrixXd& x3, MatrixXd& y3,
const string& title,
char* xlabel,
char* ylabel,
char* legend=NULL,
char* terminal=NULL,
char* output=NULL);

```

For example, if the user wishes to display a plot of the control trajectories of a system with two control variables which have been stored in MatrixXd object “u”, and assuming that the corresponding time vector has been stored in MatrixXd object “t”, then an example of the syntax to call the `plot()` function is:

```
plot(t,u,"Control variable","time (s)", "u", "u1 u2");
```

It is also possible to save plots to graphical files supported by GNUplot. For example, to save the above plot to an encapsulated postscript file (instead of displaying it), the command is as follows:

```
plot(t,u,"Control variable", "time (s)", "u", "u1 u2",
"postscript eps", "filename.eps");
```

The function `spplot()` allows to plot one or more curves together with one or more sets of isolated points (without joining the dots). This can be useful, for example, to compare how an estimated continuous variable compares with experimental data points. The prototype is as follows:

```

void spplot(MatrixXd& x1, MatrixXd& y1,
MatrixXd& x2, MatrixXd& y2,
const string& title,
char* xlabel,
char* ylabel,
char* legend=NULL,
char* terminal=NULL,
char* output=NULL);

```

where $\mathbf{x1}$ is a column or row vector with n_1 elements and $\mathbf{y1}$ is a matrix with one either row or column dimension equal to n_1 . The pair $(\mathbf{x1}, \mathbf{y1})$ is used to generate curve(s). $\mathbf{x2}$ is a column or row vector with n_2 elements and $\mathbf{y2}$ is a matrix with one either row or column dimension equal to n_2 . The pair $(\mathbf{x2}, \mathbf{y2})$ is used to plot data points.

For example, if the user wishes to display a curve on the basis of the pair $(\mathbf{t}, \mathbf{y1})$ and on the same plot compare with experimental points stored in the pair $(\mathbf{te}, \mathbf{ye})$, then an example of the syntax to call the `splot()` function is:

```
plot(t,y,te, ye, "Data fit for y","time (s)", "u", "y ye");
```

The `multiplot()` function allows the user to plot on a single window an array of sub-plots, having one curve per subplot. The function prototype is as follows:

```
void multiplot(MatrixXd& x,
               MatrixXd& y,
               const string& title,
               char* xlabel,
               char* ylabel,
               char* legend,
               int nrows=0,
               int ncols=0,
               char* terminal=NULL,
               char* output=NULL ) ;
```

where x is a column or row vector with n elements and y is a matrix with one either row or column dimension equal to n , `xlabel` should be a string with the common label for the x-axis of all subplots, `ylabel` should be a string with the labels for all y-axes of all subplots, separated by spaces, `nrows` is the number of rows of the array of subplots, `ncols` is the number of columns of the array of subplots. If `nrows` and `ncols` are not provided, then the array of subplots has a single column. Note that the product `nrows*ncols` should be equal to n , which is the number of curves to be plotted.

For example, if the user wishes to display an array of subplots of the state trajectories of a system with four state variables which have been stored in MatrixXd object “ y ”, and assuming that the corresponding time vector has been stored in MatrixXd object “ t ”, then an example of the syntax to call the `multiplot()` function is:

```
multiplot(t,y,"State variables","time (s)",
          "y1 y2 y3 y4", "y1 y2 y3 y4");
```

In the above case, a 4×1 array of sub-plots is produced. If a 2×2 array of sub-plots is required, then the following command can be used:

```

multiplot(t,y,"State variables","time (s)",
          "y1 y2 y3 y4", "y1 y2 y3 y4", 2, 2);

```

The function `surf()` plots the colored parametric surface defined by three matrix arguments. The prototype of the `surf()` function is as follows:

```

void surf(MatrixXd& x,
          MatrixXd& y,
          MatrixXd& z,
          const string& title,
          char* xlabel,
          char* ylabel,
          char* zlabel,
          char* terminal=NULL,
          char* output=NULL,
          char* view=NULL);

```

Here `view` is a character string with two constants `<rot_x>`, `<rot_y>` (e.g. "50,60"), where `rot_x` is an angle in the interval $[0, 180]$ degrees, and `rot_y` is an angle in the interval $[0, 360]$ degrees. This is used to set the viewing angle of the surface plot.

For example, if the user wishes to display a surface plot of a $N \times M$ matrix Z with respect to the $1 \times N$ vector X and the $1 \times M$ vector Y , stored, respectively, in `MatrixXd` objects "z", "x" and "y", then an example of the syntax to call the `surf()` function is:

```

surf(x, y, z, "Title", "x-label", "y-label", "z-label");

```

The function `plot3()` plots a 3D parametric curve defined by three vector arguments. The prototype of the `plot3()` function is as follows:

```

void plot3(MatrixXd& x,
           MatrixXd& y,
           MatrixXd& z,
           const string& title,
           char* xlabel,
           char* ylabel,
           char* zlabel,
           char* terminal=NULL,
           char* output=NULL,
           char* view = NULL);

```

Here `view` is a character string with two constants `<rot_x>`, `<rot_y>` (e.g. "50,60"), where `rot_x` is an angle in the interval $[0, 180]$ degrees, and `rot_y` is an angle in the interval $[0, 360]$ degrees. This is used to set the viewing angle of the 3D plot.

For example, if the user wishes to display a 3D parametric curve of a $1 \times N$ vector Z with respect to the $1 \times N$ vector X and the $1 \times M$ vector Y , stored, respectively, in `MatrixXd` objects "z", "x" and "y", then an example of the syntax to call the `plot3()` function is:

```
plot3(x, y, z, "Title", "x-label", "y-label", "z-label");
```

The function `polar()` plots a polar curve defined by two vector arguments. The prototype of the `polar()` function is as follows:

```
void polar(MatrixXd& theta,
           MatrixXd& r,
           const string& title,
           char* legend=NULL,
           char* terminal=NULL,
           char* output=NULL);
```

For example, if the user wishes to display a polar plot using a of a $1 \times N$ vector θ (the angle values in radians), and a $1 \times N$ vector r (the corresponding values of the radius), stored, respectively, in `MatrixXd` objects "theta" and "r", then an example of the syntax to call the `polar()` function is:

```
polar(theta, r, "Title");
```

The `polar()` function is overloaded so that the user may plot together up to three different polar curves. The additional prototypes are given below. For two polar curves:

```
void polar(MatrixXd& theta,
           MatrixXd& r,
           MatrixXd& theta2,
           MatrixXd& r2,
           const string& title,
           char* legend=NULL,
           char* terminal=NULL,
           char* output=NULL);
```

For three polar curves.

Terminal	Description
postscript eps	Encapsulated postscript
pdf	Adobe portable document format (pdf)
Jpeg	jpg graphical format
Png	png graphical format
latex	LaTeX graphical code

Table 7.2: Some of the available GNUplot output graphical formats

```
void polar(MatrixXd& theta,
          MatrixXd& r,
          MatrixXd& theta2,
          MatrixXd& r2,
          MatrixXd& theta3,
          MatrixXd& r3,
          const string& title,
          char* legend=NULL,
          char* terminal=NULL,
          char* output=NULL);
```

Some common GNUplot terminals (graphical formats) are given in Table 7.2. See the GNUplot documentation for further details on the keywords needed to specify different graphical formats.

<http://www.gnuplot.info/documentation.html>

7.3 The built-in sparse SQP solver

In addition to IPOPT, PSOPT contains a sparse sequential quadratic programming (SQP) solver of its own, selected by setting `algorithm.nlp_method` to “SQP”. It is optional at build time and it is not the default. This section describes what it is, when to use it, how to build it, and, just as importantly, when it is not the right choice.

7.3.1 What it is

The algorithm is broadly based on the sparse SQP method described in Chapter 2 of [5]. Instead of reproducing that description here, it is more useful to state what has been taken from it and what has not.

Adopted from [5]: the augmented Lagrangian merit function of Section 2.4 with its constraint slacks and its least-norm penalty weights; the exact sparse Hessian of

the Lagrangian, obtained here by automatic differentiation; the modification of that Hessian by a multiple of the identity, with the multiple governed by the inertia of the KKT matrix as in Section 2.5, the Gerschgorin bound on the most negative eigenvalue setting its ceiling; the device of solving two quadratic programs at the first iteration, the first with a unit Hessian, so that the multipliers entering the merit function are not simply zero; the algorithm strategies M, FM and F of Section 2.6.2, with FM the default as it is his; the segregation of constraint difficulties from objective difficulties that FM exists to achieve (Section 2.7); and the feasible-point phase of Section 2.8, including its least-distance program, its relaxation, its constraint-violation merit function and the rule for switching between the two.

Not adopted: the Schur-complement active-set method of Section 2.3, which is left to whichever quadratic programming backend is in use; the warm start of the subproblem from the previous active set, which was measured and found not to pay for itself with the backends available; the bound slacks, bound multipliers and bound weights that appear in the merit function of Section 2.4, so that the merit function here charges the constraints but not the simple bounds; the trust-region rule of Section 2.5 for choosing the Hessian modification, which is replaced by a simpler carried-forward parameter; two of the three rungs of the inertia-control ladder of Section 2.6.1; the Wolfe condition in the line search; and the FME strategy.

The quadratic programming subproblem is not solved by PSOPT itself but handed to one of several sparse backends, chosen with `algorithm.qp_solver`; GALAHAD's QPA is the default.

7.3.2 When to use it, and when not to

IPOPT is the default and performs better across the board. It solves more of the examples distributed with PSOPT than the SQP does. On the examples both solve the difference in speed is not uniform: the median problem takes about as long either way, and the mean is carried by a tail of problems on which the SQP takes tens or hundreds of times longer. A user with no particular reason to do otherwise should use IPOPT, which is why it remains the default.

The SQP is nonetheless worth having for three reasons. It provides a second opinion from a genuinely different algorithm: where the two agree, on the examples they both solve, they agree to about four significant figures, which is a stronger statement about a solution than either produces alone. Every part of it lives in the PSOPT repository, so its behaviour can be inspected and changed. And there is at least one example distributed with PSOPT that the SQP solves and IPOPT does not.

The caveats are:

- **It does not solve every example.** Of the examples distributed with PSOPT, the SQP fails outright on one and does not reach a solution within a practical time budget on roughly a quarter of the remainder, where IPOPT succeeds. The proportion moves as the solver changes, so it should be read as indicative: a problem the SQP will not take is one to give to IPOPT.

- **It is slower.** The cost is dominated by the quadratic programming subproblems, of which there is at least one per iteration.
- **It needs the exact Hessian to be useful on any sizeable problem.** Set `algorithm.hessian` to “exact”. The alternative is a dense quasi-Newton model requiring storage quadratic in the number of variables, which on a collocation mesh of any size is not practical.
- **It is newer than the rest of PSOPT**, and has had correspondingly less exposure to problems its author did not write.

7.3.3 Choosing a quadratic programming backend

The backend named by `algorithm.qp_solver` is not an implementation detail that can be left alone. Measured over the sixty-one single-solve examples distributed with PSOPT, with the exact Hessian, each example on the mesh it prescribes, and nothing varying but the backend, the number solved runs from forty-four to fifty-five, and the time taken over the examples all of them solve varies by a factor of nearly twelve. It is worth a minute’s thought when a problem is not going well.

	GALAHAD (active set)	PIQP (interior pt.)	Clarabel (interior pt.)	OSQP (splitting)
solved (of 61)	55	52	46	44
time over the 33 all solve	385 s	82 s	177 s	965 s

GALAHAD remains the default, because it solves the most and because it is ahead on the kind of problem an optimal control user is most likely to bring. **PIQP is the one to reach for when it disappoints.** PIQP takes very nearly the same number of SQP iterations as GALAHAD, 1461 against 1395 over the examples all four backends solve, in a fifth of the wall clock, so the difference between them lies in what a subproblem costs and not in how many the method needs. It also solves three examples GALAHAD does not. Between them, GALAHAD and PIQP solve fifty-nine: every example any of the four backends solves. Clarabel and OSQP solve nothing that those two do not, which is why neither is worth trying first; they remain available, and are built the same way, for a problem that defeats both.

The division between the active-set and the other methods is structural. QPA is an active-set method built to accept an indefinite Hessian, and the Hessian modification described above is governed by the inertia of the KKT matrix, which makes the model positive definite on the null space of the Jacobian. That is the condition an SQP subproblem needs, and it is all QPA asks for. The interior-point and operator-splitting backends factorise a KKT matrix of their own once, at set-up, and need the model to be convex on the whole space, which a model convex only on that null space need not be. PSOPT answers a subproblem declined on those grounds by raising the shift and asking again, so the situation is handled, but a larger shift is a poorer model of the curvature and is paid for in iterations.

A backend can be confidently wrong, and this is the reason to be able to change one. Three configurations in the sweep returned an answer, reported it as optimal, and were not close. On `examples/low_thrust` both interior-point backends return an objective of about -0.089 where the correct value is -0.220 . On `examples/mpec` OSQP returns 57.2 against 1.87, and is wrong on the very first mesh instead of drifting there. A backend that fails visibly costs a run; one that succeeds visibly and is wrong costs more than that. On an unfamiliar problem, solving it a second time through a different backend is a cheap and worthwhile check.

Two symptoms are worth acting on. If the solver labours, or reports that its subproblems repeatedly reached `algorithm.qp_iter_max`, try “PIQP”. If it reports subproblems declined as non-convex, try “GALAHAD”. Changing backend costs one line and one run, and on this evidence it is among the first things to try.

An earlier edition of this manual advised trying “OSQP” on a problem with integer controls, on the reasoning that the SOS1 equalities PSOPT appends when convexifying an integer control make the constraint set degenerate at a vertex solution, that an active-set method must then choose among linearly dependent constraints and can stall, and that a splitting method maintains no working set at all. The reasoning is sound and the observation that prompted it was real, but it no longer describes the solver: once the subproblem iteration budget was made to scale with the size of the subproblem instead of being a fixed constant, GALAHAD solved every integer-control example in the set. What had looked like a difference between the methods was a budget set without reference to problem size.

The figures above were measured at the time of writing and will move as the solver changes.

7.3.4 Building and using it

The SQP is not built by default. It requires at least one sparse quadratic programming backend, and configuring `WITH_SQP=ON` without one is an error, since the solver has no quadratic programming method of its own. GALAHAD is the recommended backend:

```
cmake -B build -DCMAKE_BUILD_TYPE=Release -DBUILD_EXAMPLES=ON \
      -DWITH_SQP=ON -DWITH_GALAHAD=ON -DGALAHAD_DIR=<galahad prefix>
cmake --build build -j
```

The script `scripts/build_galahad.sh` in the PSOPT repository will install GALAHAD with the options PSOPT requires, on Linux or macOS.

GALAHAD’s QPA uses OpenMP cancellation, and the OpenMP runtime reads the relevant environment variables once, when it first initialises. They therefore cannot be set from inside a PSOPT program and must be present in the environment before it starts:

```
export OMP_CANCELLATION=TRUE
export OMP_PROC_BIND=TRUE
```

Without them the quadratic programming subproblems fail and the solver makes no progress, which is easily mistaken for a difficulty with the problem being solved.

A typical configuration in the user's main function is then:

```
algorithm.nlp_method    = "SQP";
algorithm.hessian       = "exact";
algorithm.derivatives   = "automatic";
algorithm.qp_solver     = "GALAHAD";
```

A note on SNOPT. Earlier releases of PSOPT offered SNOPT as a third NLP solver. SNOPT is commercial, so it was only ever available to a minority of users, and the SQP described here now occupies the same place, that of a sequential quadratic programming alternative to an interior point method, with no licence to obtain. The SNOPT interface has been removed, and `algorithm.nlp_method` no longer accepts “SNOPT”.

7.4 Error handling

PSOPT is designed so that no library call can crash the host program or leave it in an undefined state, whatever the outcome of a solution. This section describes the default behaviour, the reasoning behind it, and how it can be changed by the user.

7.4.1 The `psopt()` contract

The `psopt()` function is *total*: it always returns, and it never allows an exception to propagate to the caller. Internally PSOPT uses exceptions to report problems, such as an inconsistent option detected during the validation of the user-supplied data, but these are caught at the boundary of `psopt()` and converted into a status. On return, `solution.error_flag` is zero if the solve succeeded and non-zero otherwise, `solution.error_msg` holds the diagnostic message (which is also printed to the screen), and the integer returned by `psopt()` is equal to `solution.error_flag`. The return value carries the `[[nodiscard]]` attribute, so a compiler will warn if it is discarded; the recommended pattern is to check it immediately, as shown in the “Error checking” part of Section 7.2.7.

7.4.2 Accessing a solution after a failed solve

If a solution does not succeed, the per-phase result arrays are empty. Reaching into them, through the `get*_in_phase()` accessors or by passing the empty results to an output utility such as `plot()`, would read past the end of empty data and crash. To prevent this, every solution accessor first checks `solution.error_flag`; if it is set, the accessor reports the situation and PSOPT acts according to the `algorithm.on_error` policy below.

7.4.3 Default behaviour: fail-fast

By default (`algorithm.on_error` is “fail-fast”) the first access to a failed solution prints a diagnostic naming the accessor and the underlying error, and then stops the program cleanly with a non-zero exit code. No exception is thrown, so nothing escapes the user’s `main()`, and no undefined behaviour occurs. This is the safe default: a clear, immediate stop is preferable to silently continuing a post-processing pipeline with empty or meaningless results, which previously could crash deep inside an output routine.

7.4.4 Rationale

The design follows three principles: exceptions are used only internally and are always caught at the `psopt()` boundary; every failure is reported through `solution.error_flag` and `solution.error_msg`; and no public call (the solve, a solution accessor, or an output utility) may produce undefined behaviour, regardless of the state of the solution. The fail-fast default exists because the most common situation is a user who does not check `solution.error_flag` and proceeds straight to extracting and plotting results; that case should fail visibly and immediately instead of crashing.

7.4.5 Configuring the policy: `algorithm.on_error`

A power user who would rather a failed solve not halt the program can set

```
algorithm.on_error = "fail-soft";
```

Under “fail-soft”, a solution accessor on a failed solve prints a warning and returns an empty matrix instead of stopping the program; the output utilities (below) then skip that empty data, and execution continues. This is useful, for instance, in a batch that solves many problems and should carry on past an individual failure, or when the caller checks `solution.error_flag` itself and wants full control of the program flow. The two permitted values are “fail-fast” (the default) and “fail-soft”.

7.4.6 Output utilities and file output

The output utilities are protected in two complementary ways. First, `plot()` (and its variants `multiplot`, `spplot`, `polar`, `surf` and `plot3`), together with `Save()` and `Save_to_json_file()`, skip with a warning when handed empty data: an empty matrix is never plotted or written. Consequently, under “fail-soft” a failed solve cannot crash a subsequent plotting or saving call. Second, if one of these utilities cannot open its output file, because for example the path does not exist or is not writable, it prints a warning and skips that operation; it does not throw and does not stop the program. The computation itself is unaffected; only the individual output is skipped. The same applies to `load_data()`, which returns an empty matrix if its input file cannot be opened or is malformed.

7.5 Specifying a parameter estimation problem

To use the parameter estimation facilities implemented in PSOPT for problems where the observation function is defined, and where there is a set of observed data at given sampling points (see section 1.11). The user needs to specify, for each phase, the number of observed variables and the number of sampling points:

```
problem.phases(iphase).nobserved      = INTEGER;
problem.phases(iphase).nsamples       = INTEGER;
```

where `nobserved` is the number of simultaneous measurements taking place at each sampling node, and `nsamples` is the total number of sampling nodes. The above parameters should be entered before calling the function `psopt_level2_setup(problem, algorithm)`. After this, additional information may be entered:

```
problem.phases(iphase).observation_nodes = MatrixXd OBJECT;
problem.phases(iphase).observations      = MatrixXd OBJECT;
problem.phases(iphase).residual_weights  = MatrixXd OBJECT
```

where `observation_nodes` is a $1 \times \text{nsamples}$ matrix, `observations` is a $\text{nobserved} \times \text{nsamples}$ matrix, `residual_weights` is a $1 \times \text{nobserved} \times \text{nsamples}$ matrix. The `residual_weights` matrix is by default full of ones.

If parameter estimation data for a particular phase is saved in a text file with the column format specified below, then an auxiliary function, which is described below, can be used to load the data:

```
< Time > < Obs. # 1> < Weight # 1> ... <Obs. # n> < Weight # n>
```

where each column is separated by either tabs or spaces, the first column contains the time stamps of the samples, the second column contains the observations of the first variable, the third column contains the weights for each observation of the first variable, and so on. It is then possible to load observation nodes, observations, and residual weights and assign them to the appropriate fields of the `problem` structure by using the function `load_parameter_estimation_data`, whose prototype is given below.

```
void load_parameter_estimation_data() (
    Prob& problem,
    int iphase,
    char* filename
);
```

Note that the user should not register the `problem.end_point_cost` or the `problem.integrand_cost` functions, but the user needs to register `problem.observation_function`. The prototype of this function is as follows:

```

void observation_function(
    adouble* observations,
    adouble* states,
    adouble* controls,
    adouble* parameters,
    adouble& time_k,
    int k,
    adouble* xad,
    int iphase );

```

where on output the function should return the array of observed variables corresponding to sampling index k at sampling instant `time_k`. The rest of the interface is the same as for general optimal control problems.

7.6 Automatic scaling

If the user specifies the option `algorithm.scaling` as “automatic”, then PSOPT will calculate scaling factors as follows.

1. Scaling factors for controls, states, static parameters, and time, are computed based on the user supplied bounds for these variables. For finite bounds, the variables are scaled such that their original value multiplied by the scaling factor results in a number within the range $[-1, 1]$. If any of the bounds is greater or equal than the constant `inf`, then the variable is scaled to lie within the intervals, $[-\text{inf}, 1]$, $[1, \text{inf}]$ or $[-\text{inf}, \text{inf}]$. The constant `inf` is defined in the include file `psopt.h` as 1×10^{19} .
2. Scaling factors for all constraints (except for the differential defect constraints) are computed as follows. The scaling factor for the i -th constraint is the reciprocal of the norm of the i -th row of the Jacobian of the constraints [5]. If the computed norm is zero, then the scaling factor is set to 1.
3. The scaling factors of each differential defect constraint is by default equal to the scaling factor of the corresponding state by default [5]. However, if `algorithm.defect_scaling` is set to “jacobian-based”, then the scaling factors of the differential defect constraints are computed as is done for the other constraints.
4. The scaling factor for the objective function is the reciprocal of the norm of the gradient of the objective function evaluated at the initial guess. If the norm of the objective function at the initial guess is zero, then the scaling factor of the objective function is set to one.

7.7 Differentiation

Users are encouraged to use, whenever possible, the automatic differentiation facilities provided by the CppAD library. The use of automatic derivatives is the default

behaviour, but it may be specified explicitly by setting the `derivatives` option to “automatic”. PSOPT uses the CppAD drivers for sparsity determination, Jacobian and gradient evaluation. Automatic derivatives are more accurate than numerical derivatives as they are free of truncation errors. Moreover, PSOPT works faster when using automatic derivatives.

There may be cases, however, where it is preferable or necessary to use numerical derivatives. If the user specifies the option `algorithm.derivatives` as “numerical”, then the derivatives required by the nonlinear programming algorithm as follows.

7.7.1 First derivatives: numerical gradient and Jacobian

If IPOPT is being used for optimization, then the Jacobian of the constraints is computed by using sparse finite differences, such that groups of variables are perturbed simultaneously [15]. The gradient of the objective function is computed by perturbing one variable at a time. Normally the central difference formula is used, but if the perturbed variable is at (or very close to) one of its bounds, then the forward or backward difference formulas are employed. It is assumed that the Jacobian of the constraint function $G(y)$ is divided into constant and variable terms as follows:

$$\frac{\partial G(y)}{\partial y} = A + \frac{\partial g(y)}{\partial y} \quad (7.7)$$

where matrices A and $\partial g(y)/\partial y$ do not have non-zero elements with the same indices. The constant part A of the constraint Jacobian is estimated first, and only the variable part of the jacobian $\partial g(y)/\partial y$ is estimated by sparse finite differences.

7.7.2 Second derivatives: the numerical Hessian

When IPOPT is used as the NLP solver it can make use of the Hessian of the Lagrangian

$$H(y, \lambda, \sigma_f) = \sigma_f \nabla^2 F(y) + \sum_k \lambda_k \nabla^2 G_k(y), \quad (7.8)$$

where F is the objective, the G_k are the constraints, σ_f is the objective factor and the λ_k are the constraint multipliers. How this Hessian is supplied is selected by the `algorithm.hessian` option (Section 7.2.7). The “exact” option supplies (7.8) by automatic differentiation and is therefore available only with automatic first derivatives. The “numerical” option, described here, supplies a sparse finite-difference approximation of the same Hessian, and is available whether the first derivatives are automatic or numerical. It is intended for problems whose user functions are not amenable to second-order automatic differentiation, while still giving the faster and more robust convergence of a genuine second-order method in place of the limited-memory approximation.

The numerical Hessian is built in two stages. First its sparsity pattern is determined. A structural superset is taken from the non-constant part $\partial g(y)/\partial y$ of the constraint Jacobian, and this superset is then refined by probing the Lagrangian at several non-degenerate points and discarding entries that are structurally absent or negligible, using

a threshold that combines a relative bar with a finite-difference round-off floor. Probing at several points guards against an entry that happens to vanish at a particular point. Second, the values on the detected pattern are computed by central finite differences: the diagonal entries from $2n$ single-variable second differences of the Lagrangian, and the off-diagonal entries either by direct mixed differences or by the index-set recovery described next.

Computing every off-diagonal entry by a separate mixed difference costs four constraint evaluations per nonzero. Following the sparse-finite-difference approach of Betts [5], PSOPT reuses the Curtis–Powell–Reid colouring [15] that it already forms for the constraint Jacobian: two variables in the same colour group never share a constraint row, so perturbing a whole group at once and taking the four-corner difference of the constraint vector isolates a single Hessian entry per constraint. The constraint part of the off-diagonal entries is then recovered in a number of constraint-vector evaluations proportional to the number of *interacting* colour-group pairs, which is typically far smaller than the number of nonzeros. The objective contribution, being a single global function whose group differences would mix entries, is added separately over its own detected pattern.

The colouring and the choice of assembly method are refreshed at every mesh. The cheaper of the index-set recovery and direct mixed differences is selected automatically by comparing their predicted cost. The index-set recovery wins when the same colour-group pairs recur across many nonzeros, as they do for the local and *hp* collocation methods, where the coupling is local and the saving grows as the mesh is refined. For free-final-time global pseudospectral discretisations the time-scaled differentiation matrix makes the defect rows nearly dense, so almost all group pairs interact; the automatic selection then falls back to direct mixed differences. Dedicated symmetric (star or acyclic) colourings of the Hessian itself [38, 13] could reduce the count further, but PSOPT currently reuses the constraint-Jacobian colouring.

Setting `algorithm.hessian_verify` to `true` (the default is `false`) performs a one-off diagnostic at the first well-conditioned iterate: it cross-checks the two finite-difference assembly methods against each other and, when the user functions are automatic-differentiable, against the automatic-differentiation Hessian, reporting the largest discrepancies. It is intended for validation and adds no cost at its default value.

7.8 Generation of initial guesses

If no guesses are supplied by the user, then PSOPT computes the initial guess for the unspecified decision variables as follows. Each variable is assumed to be constant and equal to the mean value of its bounds, provided none of the bounds is defined as `inf` or `-inf`. If only one of the bounds is `inf` or `-inf`, then the variable is initialized with the value of the other bound. If the upper and lower bounds are `inf` and `-inf`, respectively, then the variable is initialized at zero.

The variables that are initialized automatically for each phase include: the control variables, the state variables, the static parameters, the initial time, and the final time.

The user may also compute initial guesses for the state variables by propagating the differential equations associated with the problem. Two auxiliary functions are provided for this purpose. See section 7.11 for more details.

7.8.1 L^AT_EX code generation

L^AT_EX code is generated automatically producing a table with a summary of information about the mesh refinement process. It may be useful to include this summary in publications that incorporate results generated with PSOPT. A file named `mesh_statistics_$$$tex` is automatically created, unless `algorithm.print_level` is set to zero, where `$$$` represents the characters of `problem.outfilename` which occur to the left of the file extension point “.”.

To include the generated table in a L^AT_EX document, simply use the command:

```
\input{mesh_statistics_$$$tex}
```

The generated table includes a caption associated with the problem name as set through `problem.name`, as well as a label which is generated by concatenating the string “`mesh_stats_`” with the characters of `problem.outfilename` which occur to the left of the file extension point “.”. The caption and label can easily be changed to suit the user requirements by editing or renaming the generated file. A key to the abbreviations used in the file is also printed. The abbreviations for the discretisation methods used are described in Table 7.3.

7.9 Using the integrated residual transcription

The integrated residual transcription described in Chapter 3 is selected and controlled through the algorithm options listed in Section 7.2.7. The transcription is enabled with

```
algorithm.transcription_method = "integrated-residual";
```

Its behaviour is then determined by `algorithm.ir_objective` and the options that follow. The four typical use cases are illustrated below.

Least-squares feasibility. To compute the most dynamically feasible trajectory for the mesh:

```
algorithm.transcription_method = "integrated-residual";
algorithm.ir_objective         = "residual";
algorithm.ir_residual_nodes    = 4;          // residual-quadrature points per interval
```

Residual box. To minimise the cost subject to a fixed residual tolerance δ :

```
algorithm.transcription_method = "integrated-residual";
algorithm.ir_objective         = "cost";
algorithm.ir_residual_bound    = 1.0e-3;    // delta
algorithm.ir_residual_nodes    = 4;
```

Abbreviation	Description
LGL-ST	Legendre-Gauss-Lobatto (LGL) collocation with standard differentiation matrix given by equation (1.12)
LGL-RR	Legendre-Gauss-Lobatto (LGL) collocation with reduced round-off differentiation matrix given by equation (1.22)
CGL-ST	Chebyshev-Gauss-Lobatto (CGL) collocation with standard differentiation matrix given by equation (1.19)
CGL-RR	Chebyshev-Gauss-Lobatto (CGL) collocation with reduced round-off differentiation matrix given by equation (1.22)
LGR	Legendre-Gauss-Radau (LGR) collocation, see Section 1.9
LG	Legendre-Gauss (LG) collocation, see Section 1.9
TRP	Trapezoidal discretisation, see equation (1.49)
H-S	Hermit-Simpson discretisation, see equation (1.50)
IR	Integrated residual transcription, see Chapter 3

Table 7.3: Description of the abbreviations used for the discretisation methods which are shown in the automatically generated L^AT_EX table

DAIR. To let PSOPT set the tolerance automatically and alternate the feasibility and optimality steps over the mesh-refinement sequence:

```
algorithm.transcription_method = "integrated-residual";
algorithm.ir_dair               = true;
algorithm.ir_dair_delta_factor = 1.0;      // K in delta = K (h/T)^p
algorithm.ir_residual_nodes     = 4;
```

Because DAIR composes with mesh refinement, supplying a sequence of node counts through `problem.phases(iphase).nodes` drives the tolerance down from one mesh to the next and the cost to the optimum.

Flexible-order representation. Any of the above may be combined with the Nie-Kerrigan flexible-order representation by selecting a local order $d \geq 2$:

```
algorithm.ir_local_order   = 4;    // degree-d local representation
algorithm.ir_residual_nodes = 6;    // must be >= ir_local_order + 2
```

The number of intervals per phase must be a multiple of the local order, and the residual-node count is per element here. For smooth problems this typically delivers substantially higher accuracy for the same number of nodes.

Flexible mesh. If the solution has a switch whose time is not known in advance, the element boundaries can be made decision variables as well, so that the optimisation isolates the switch instead of ringing around it:

```
algorithm.ir_local_order       = 4;
algorithm.ir_residual_nodes    = 6;
algorithm.ir_flexible_mesh     = true;
// floor on an element width, as a fraction of the uniform width:
algorithm.ir_min_element_fraction = 0.05;
```

This requires a local order of at least two, and cannot be combined with automatic mesh refinement. Section 3.9 describes what it does and what it does not do, and the directory `examples/flexmesh` holds the worked case.

Differential-algebraic problems. No additional option is required: any path constraint declared with equal lower and upper bounds is folded into the residual automatically (Section 3.3) and freed from the pointwise constraint set, so a semi-explicit DAE is solved in the form in which it was written:

```
// Equal bounds: this path constraint is folded into the residual
problem.phases(1).bounds.lower.path(0) = 0.0;
problem.phases(1).bounds.upper.path(0) = 0.0;

algorithm.transcription_method = "integrated-residual";
```

```

algorithm.ir_objective      = "cost";
algorithm.ir_residual_bound = 1.0e-4;    // delta
algorithm.ir_local_order   = 5;
algorithm.ir_residual_nodes = 10;

```

To keep the equality out of the residual and impose it pointwise as before, set `algorithm.ir_include_path = "none"`; to rebalance a badly scaled algebraic equation against the differential ones, set `algorithm.ir_path_weight`.

For all of these formulations the costates are recovered as described in Section 3.7, and can be read back in the usual way, for example

```
MatrixXd lambda = solution.get_dual_costates_in_phase(1);
```

7.10 Implementing multi-segment problems

Sometimes, it is useful for computational or other reasons to define a multi-segment problem. A multi-segment problem is an optimal control problem with multiple sequential phases that has the same dynamics and path constraints in each phase. The multi-segment facilities implemented in PSOPT allow the user to specify multi-segment problems in an easier way than defining a multi-phase problem. Special functions are called automatically to patch consecutive segments and ensure state and time continuity across the segment boundaries.

To specify a multi-segment problem it is necessary to create a data structure of the type `MSdata` (in addition to the `problem`, `algorithm` and `solution` structures) and assign values to its elements as follows:

```
MSdata msdata;
```

```

msdata.nsegments      = INTEGER;
msdata.nstates        = INTEGER;
msdata.ncontrols      = INTEGER;
msdata.nparameters    = INTEGER;
msdata.npath          = INTEGER;
msdata.n_initial_events = INTEGER;
msdata.n_final_events  = INTEGER;
msdata.nodes          = RowVectorXi OBJECT
msdata.continuous_controls = BOOLEAN

```

If it is desired to enforce control continuity across the segment boundaries, then set `msdata.continuous_controls` to `true`. By default the controls are allowed to be discontinuous across the segment boundaries.

The number of nodes per segment can be specified as follows (note that it is possible to create grids with segments that have different number of nodes):

- as a single value (e.g. 30), such that the same number of nodes is employed in each segment.
- If `algorithm.mesh_refinement` is set to "manual", a character string can be entered with the node sequence to be tried per segment as part of a manual mesh refinement strategy (e.g. "[30, 50, 60]"). Here the number of values corresponds to the number of mesh refinement iterations to be performed. It is assumed that the same node sequence is tried for each segment. If `algorithm.mesh_refinement` is set to "automatic", then only the first value of the specified sequence is used to start the automatic mesh refinement iterations.
- If `algorithm.mesh_refinement` is set to "manual", a matrix can be entered, such that each row of the matrix corresponds to the node sequence to be tried in the corresponding segment (a character string can be used to enter the matrix, e.g. "[30, 50, 60; 10, 15, 20; 5, 10, 15]", noting the semicolons that separate the rows). Here the number of rows corresponds to the number of segments, and the number of columns corresponds to the number of manual mesh refinement iterations to be performed. If `algorithm.mesh_refinement` is set to "automatic", then only the first value of the specified sequence for each segment is used to start the automatic mesh refinement iterations.

After this, the following function should be called:

```
multi_segment_setup(problem, algorithm, msdata);
```

The upper and lower bounds on the relevant event times of the problem (start time for each segment, and end time for the last segment) can be entered as follows:

```
problem.bounds.lower.times = MatrixXd object;
problem.bounds.upper.times = MatrixXd object;
```

where entered value specifies the time bounds in the following order:

$$[t_0^{(1)}, t_0^{(2)}, \dots, t_0^{(N_p)}, t_f^{(N_p)}]$$

For example, consider the following code:

```
MatrixXd tlower << 10.0, 20.0, 30.0;
MatrixXd tupper << 15.0, 25.0, 35.0;
problem.bounds.lower.times = tlower;
problem.bounds.upper.times = tupper;
```

At this point, the bound information for segment 1 (phase 1) can be entered (bounds for states, controls, event constraints, path constraints, and parameters), as described in section 7.2.7. This should be followed by the bound information for the event constraints of the last phase or segment. After entering the bound information, the auxiliary function `auto_phase_bounds` should be called as follows:

```
auto_phase_bounds(problem);
```

The initial guess for the solution can be specified by a call to the function `auto_phase_guess`. See section 7.11.

See the PSOPT Application Examples Document for an example on the use of the multi-segment facilities available within PSOPT.

7.11 Other auxiliary functions available to the user

PSOPT implements a number of auxiliary functions to help the user define optimal control problems. Most (but not all) of these functions are suitable for use with automatic differentiation. All the functions can also be used with numerical differentiation. See the examples section for further details on the use of these functions.

7.11.1 cross function

This function takes two arrays of adoubles x and y , each of dimension 3, and returns in array z (also of dimension 3) the result of the vector cross product of x and y . The prototype of the function is as follows:

```
void cross(adouble* x, adouble* y, adouble* z);
```

7.11.2 dot function

This function takes two arrays of adoubles x and y , each of dimension n , and returns the dot product of x and y . The prototype of the function is as follows:

```
adouble dot(adouble* x, adouble* y, int n);
```

7.11.3 get_delayed_state function

This function allows the user to implement DAE's with delayed states. Use only in single-phase problems. Its prototype is as follows:

```
void get_delayed_state(adouble* delayed_state,
                      int state_index,
                      int iphase,
                      adouble& time,
                      double delay,
                      adouble* xad,
                      Workspace* workspace);
```

The function parameters are as follows:

- `delayed_state`: on output, the variable pointed by this pointer contains the value of the delayed state.

- **state_index**: is the index of the state vector whose delayed value is to be found, **counting from zero**, as the **states** array of the **dae** function does. Releases before 2026 described this index as counting from one; that description was wrong, and code written to it reads the wrong state whenever the mistaken index happens to fall within range. PSOPT now reports an error when it does not.
- **iphase**: is the phase index (starting from 1).
- **time**: is the value of the current instant of time within the phase.
- **delay**: is the value of the delay.
- **xad**: is the vector of scaled decision variables.
- **workspace**: is the pointer to the workspace structure that the calling user function received.

7.11.4 `get_delayed_control` function

This function allows the user to implement DAE's with delayed controls. Use only in single-phase problems. Its prototype is as follows:

```
void get_delayed_control(adouble* delayed_control,
                        int control_index,
                        int iphase,
                        adouble& time,
                        double delay,
                        adouble* xad,
                        Workspace* workspace);
```

- **delayed_control**: on output, the variable pointed by this pointer constrains the value of the delayed state.
- **control_index**: is the index of the control vector whose delayed value is to be found, counting from zero (see the note under `get_delayed_state`).
- **iphase**: is the phase index (starting from 1).
- **time**: is the value of the current instant of time within the phase.
- **delay**: is the value of the delay.
- **xad**: is the vector of scaled decision variables.
- **workspace**: is the pointer to the workspace structure that the calling user function received.

7.11.5 The initial history of a problem with delays

A delay differential equation is not well posed without an *initial history*. For $t \in [t_0 - d, t_0)$ the right hand side asks for states or controls at times before the phase begins, where no trajectory has been defined, so the history

$$x(t) = \phi(t), \quad u(t) = \psi(t), \quad t < t_0, \quad (7.9)$$

is data of the problem in the same sense as the initial condition is.

By default PSOPT uses the constant history

$$\phi(t) = x(t_0), \quad \psi(t) = u(t_0), \quad (7.10)$$

obtained by clamping the delayed time to t_0 . It keeps the state continuous at t_0 , and it is the behaviour of every release before 2026, in which it was applied silently. A problem whose history is something else, a steady state different from the initial condition or a recorded input, supplies it through the optional user function

```
void initial_history(adouble* states, adouble* controls,
                   adouble* parameters, adouble& time,
                   adouble* xad, int iphase,
                   Workspace* workspace);

problem.initial_history = &initial_history;
```

which is called only with a time strictly before t_0 , and must fill every entry of **states** and of **controls**. Like the other user functions it is evaluated on the derivative tape, so it must be written in **adouble** arithmetic. Leaving **problem.initial_history** unset selects (7.10).

Two practical points. A history that does not meet the trajectory at t_0 , that is, one with $\phi(t_0^-) \neq x(t_0)$ or $\psi(t_0^-) \neq u(t_0)$, is admissible, and PSOPT will use it, but it makes the right hand side discontinuous at $t_0 + d$, and no collocation scheme resolves a discontinuity to better than the interval that contains it. And the solution of a delay equation is in general only continuous at t_0 , continuously differentiable at $t_0 + d$, twice differentiable at $t_0 + 2d$, and so on, so the mesh should place nodes at the multiples of the delay if the nominal order of the discretisation is to be realised.

The example **examples/delay_history** checks both routes against closed-form answers obtained by the method of steps, and **examples/delay1** is a delayed optimal control problem.

A restriction. The test that decides whether a delayed time falls before t_0 is a comparison on a taped quantity. With a fixed initial time, as in every delayed example distributed with PSOPT, it is decided once, when the tape is recorded, and remains correct. With a *free* initial time it also stays as first decided, and would be wrong if t_0 moved across a delay boundary during the solve; the derivatives would then be silently incorrect. PSOPT issues a warning when a phase uses delayed variables and has a free initial time. Fix t_0 , or select **algorithm.derivatives = "numerical"**, in which case the functions are re-evaluated at perturbed points and the comparison is taken afresh.

7.11.6 `get_interpolated_state` function

This function allows the user to obtain interpolated values of the state at arbitrary values of time within a phase. Its prototype is as follows:

```
void get_interpolated_state(adouble* interp_state,
                           int state_index,
                           int iphase,
                           adouble& time,
                           adouble* xad,
                           Workspace* workspace);
```

- **interp_state**: on output, the variable pointed by this pointer contains the value of the interpolated state.
- **state_index**: is the index of the state vector whose interpolated value is to be found, counting from zero.
- **iphase**: is the phase index (starting from 1).
- **time**: is the value of the current instant of time within the phase.
- **xad**: is the vector of scaled decision variables.
- **workspace**: is the pointer to the workspace structure that the calling user function received.

7.11.7 `get_interpolated_control` function

This function allows the user to obtain interpolated values of the control at arbitrary values of time within a phase. Its prototype is as follows:

```
void get_interpolated_control(adouble* interp_control,
                             int control_index,
                             int iphase,
                             adouble& time,
                             adouble* xad,
                             Workspace* workspace);
```

- **interp_control**: on output, the variable pointed by this pointer contains the value of the interpolated control.
- **control_index**: is the index of the control vector whose interpolated value is to be found, counting from zero.
- **iphase**: is the phase index (starting from 1).
- **time**: is the value of the current instant of time within the phase.

- **xad**: is the vector of scaled decision variables.
- **workspace**: is the pointer to the workspace structure that the calling user function received.

7.11.8 `get_control_derivative` function

This function allows the user to obtain the value of the derivative of a specified control variable at arbitrary values of time within a phase. Its prototype is as follows:

```
void get_control_derivative(adouble* control_derivative,
                           int control_index,
                           int iphase,
                           adouble& time,
                           adouble* xad,
                           Workspace* workspace);
```

- **control_derivative**: on output, the variable pointed by this pointer contains the value of the control derivative.
- **control_index**: is the index of the control vector whose interpolated value is to be found, counting from zero.
- **iphase**: is the phase index (starting from 1).
- **time**: is the value of the current instant of time within the phase.
- **xad**: is the vector of scaled decision variables.
- **workspace**: is the pointer to the workspace structure that the calling user function received.

7.11.9 `get_state_derivative` function

This function allows the user to obtain the value of the derivative of a specified state variable at arbitrary values of time within a phase. Its prototype is as follows:

```
void get_state_derivative(adouble* control_derivative,
                          int state_index,
                          int iphase,
                          adouble& time,
                          adouble* xad,
                          Workspace* workspace);
```

- **state_derivative**: on output, the variable pointed by this pointer contains the value of the state derivative.

- **state_index:** is the index of the state vector whose interpolated value is to be found, counting from zero.
- **iphase:** is the phase index (starting from 1).
- **time:** is the value of the current instant of time within the phase.
- **xad:** is the vector of scaled decision variables.
- **workspace:** is the pointer to the workspace structure that the calling user function received.

7.11.10 `get_initial_states` function

This function allows the user to obtain the values of the states at the initial time of a phase. Its prototype is as follows:

```
void get_initial_states(adouble* states, adouble* xad, int iphase);
```

- **states:** on output, this array contains the values of the initial states within the specified phase.
- **iphase:** is the phase index (starting from 1).
- **xad:** is the vector of scaled decision variables.
- **workspace:** is the pointer to the workspace structure that the calling user function received.

7.11.11 `get_final_states` function

This function allows the user to obtain the values of the states at the final time of a given phase. Its prototype is as follows:

```
void get_initial_states(adouble* states, adouble* xad, int iphase);
```

- **states:** on output, this array contains the values of the final states within the specified phase.
- **iphase:** is the phase index (starting from 1).
- **xad:** is the vector of scaled decision variables.
- **workspace:** is the pointer to the workspace structure that the calling user function received.

7.11.12 `get_initial_controls` function

This function allows the user to obtain the values of the controls at the initial time of a phase. Its prototype is as follows:

```
void get_initial_controls(adouble* controls, adouble* xad, int iphase);
```

- **controls:** on output, this array contains the values of the initial controls within the specified phase.
- **iphase:** is the phase index (starting from 1).
- **xad:** is the vector of scaled decision variables.
- **workspace:** is the pointer to the workspace structure that the calling user function received.

7.11.13 `get_final_controls` function

This function allows the user to obtain the values of the controls at the final time of a given phase. Its prototype is as follows:

```
void get_initial_controls(adouble* controls, adouble* xad, int iphase);
```

- **controls:** on output, this array contains the values of the final controls within the specified phase.
- **iphase:** is the phase index (starting from 1).
- **xad:** is the vector of scaled decision variables.
- **workspace:** is the pointer to the workspace structure that the calling user function received.

7.11.14 `get_initial_time` function

This function allows the user to obtain the value of the initial time of a given phase. Its prototype is as follows:

```
adouble get_initial_time(adouble* xad, int iphase);
```

- **iphase:** is the phase index (starting from 1).
- **xad:** is the vector of scaled decision variables.
- The function returns the value of the initial time within the specified phase as an `adouble` type.

7.11.15 `get_final_time` function

This function allows the user to obtain the value of the final time of a given phase. Its prototype is as follows:

```
adouble get_final_time(adouble* xad, int iphase);
```

- **iphase**: is the phase index (starting from 1).
- **xad**: is the vector of scaled decision variables.
- The function returns the value of the final time within a phase as an **adouble** type.

7.11.16 `auto_link` function

This function allows the user to automatically link two phases by generating suitable state and time continuity constraints. It is assumed that the number of states in the two phases being linked is the same. The function is intended to be called from within the user supplied **linkages** function. Each call to **auto_link** generates an additional number of linkage constraints given by the number of states being linked plus one.

The function prototype is as follows:

```
void auto_link(adouble* linkages,  
               int* index,  
               adouble* xad,  
               int iphase_a,  
               int iphase_b);
```

- **linkages**: on output, this is the updated array of linkage constraint values.
- **index**: on input, the variable pointed to by this pointer contains the next value of the linkages array to be updated. On output, this value is updated to be used in the next call to the **auto_link** function. The first time the function is called, the value should be 0.
- **xad**: is the vector of scaled decision variables.
- **iphase_a**: is the phase index (starting from 1) of one phase to be linked.
- **iphase_b**: is the phase index (starting from 1) of the other phase to be linked.

7.11.17 `auto_link_2` function

This function works in a similar way as the **auto_link** function, but it also forces the control variables to be continuous at the boundaries. It requires a match in the number of states and in the number of controls between the phases being linked. Each call to **auto_link_2** generates an additional number of linkage constraints given by the number of states plus the number of controls, plus one.

The function prototype is as follows:

```
void auto_link_2(adouble* linkages,
                int* index,
                adouble* xad,
                int iphase_a,
                int iphase_b);
```

- **linkages**: on output, this is the updated array of linkage constraint values.
- **index**: on input, the variable pointed to by this pointer contains the next value of the linkages array to be updated. On output, this value is updated to be used in the next call to the `auto_link_2` function. The first time the function is called, the value should be 0.
- **xad**: is the vector of scaled decision variables.
- **iphase_a**: is the phase index (starting from 1) of one phase to be linked.
- **iphase_b**: is the phase index (starting from 1) of the other phase to be linked.

7.11.18 auto_phase_guess function

This function allows the user to automatically specify the initial guess in a multi-segment problem. The function prototype is as follows:

```
void auto_phase_guess(Prob& problem,
                     MatrixXd& controls,
                     MatrixXd& states,
                     MatrixXd& param,
                     MatrixXd& time);
```

so that the controls, states, time and static parameters are specified as if the problem was single-phase.

7.11.19 set_guess_from_solution function

This function copies a solution into the initial guess of a problem, phase by phase, so that the next solve starts where the last one finished. The prototype is

```
void set_guess_from_solution(Prob& problem, Sol& solution);
```

where `solution` was returned by a previous call to `psopt()`. The guess is interpolated onto the next mesh, so the node counts need not agree, and a `Sol` from a solution that did not succeed leaves the existing guess alone instead of replacing it with an empty matrix.

This is continuation across a sequence of related problems (a parameter scan, a profile likelihood, a homotopy in a bound or in a penalty weight) and precision is needed about

what it prevents, because it is not a convergence failure. Solving every point of such a sequence from the same cold guess can leave isolated points at a different local solution. Every point converges; every point reports success; a few of them converge somewhere else. On a plot of the scan those points are indistinguishable from real structure in whatever is being scanned, and nothing the software prints tells the reader that two of the features are artefacts. The profile likelihood of `examples/cracking` is the worked case: traced from cold, two of seventeen points came back with objective ratios of 1.10 and 2.15 on a profile otherwise flat to two parts in 100 000; traced by continuation, they did not.

7.11.20 us76 function

The 1976 US Standard Atmosphere, from the ground to 1000 km, in a form that can be differentiated automatically. It is declared in the header `us76.h`, which is installed with the library:

```
#include "us76.h"
```

```
template <class T> void us76(T h, T& rho, T& pres, T& temp);
```

where `h` is the geometric altitude in metres and the results are density in kg m^{-3} , pressure in Pa and temperature in K. The template is instantiated at `adouble` inside a `dae` function and at `double` for an independent check.

The reason it is written as a library function is the one given in Section 7.11.21. The standard is defined layer by layer, and evaluating it directly means selecting a layer by comparison, which is exactly the construction that cannot be taped, because the branch is frozen when the tape is recorded and every later evaluation uses whichever layer happened to be active then. The implementation therefore has no branches at all: below 86 km the temperature is the sum of the lapse rates times smooth positive parts, which reproduces the piecewise-linear profile with its corners rounded over about a hundred metres, and the pressure is the closed-form solution of hydrostatic equilibrium through those same layers; above 86 km, where the standard has no closed form, pressure and density come from published band fits to the tabulated standard and the temperature from the standard's own upper-branch equations, the bands joined by hyperbolic-tangent steps of 350 m width. Against the layer equations over 0 to 86 km, a kilometre clear of any layer boundary, the temperature is within 0.003 K and pressure and density within two parts in 100 000; including the rounded corners, within 0.09 K and 0.12 per cent. Above 86 km the blended form is within 0.04 per cent in pressure and density and 0.07 K in temperature. Beyond 1000 km the altitude is clamped and an exponential of 233 km scale height continues the profile, which keeps it positive and monotone for any iterate the solver may produce without pretending to be the standard.

Two cautions. Above 86 km the mean molecular weight falls, from 28.96 to 21.3 kg/kmol at 200 km, so pressure, density and temperature up there are *not* related by the gas law with a fixed gas constant; each is given by its own fit instead of two of them being

derived from the third. For the same reason $\sqrt{\gamma RT}$ is wrong as a speed of sound up there, whereas $\sqrt{\gamma p/\rho}$, in which the molecular weight cancels, is exact at every altitude. The Vega ascent and HORUS entry examples both use this header.

7.11.21 Branching on optimisation variables, and the conditional helpers

The user functions of a PSOPT problem are evaluated with the active type `adouble`. When `algorithm.derivatives` is “automatic”, these evaluations are recorded on a tape, and the tape, and not the source of the user function, is what is differentiated and what is replayed at every subsequent point. A tape records the operations that were executed; it does not record the decisions that led to them. An ordinary C++ comparison involving an `adouble`, such as

```
if ( x > x_table[k] ) { ... } else { ... }
```

is therefore evaluated once, while the tape is being written, and the branch taken at that moment is the branch replayed for every later value of `x`. The value and the derivative returned are those of one branch, extended over the whole range of the variable. The NLP solver receives a function and a gradient that are consistent with each other and converges normally, so the failure is silent: the solution returned is the solution of a different problem. This is the most common way in which a model that runs correctly under “numerical” derivatives gives a wrong answer under “automatic” ones.

The remedy is to write the decision as an operation, so that it is recorded. PSOPT provides four helpers for this purpose, declared in `psopt.h` and available in every user function:

```
adouble psopt_cond_lt(const adouble& x, const adouble& y,
                     const adouble& a, const adouble& b);    // x < y ? a : b
adouble psopt_cond_gt(const adouble& x, const adouble& y,
                     const adouble& a, const adouble& b);    // x > y ? a : b
adouble psopt_max(const adouble& x, const adouble& y);
adouble psopt_min(const adouble& x, const adouble& y);
```

With a taped backend these compile to the backend’s conditional expression, which puts the comparison itself on the tape; with a forward-mode backend, where nothing is taped, they compile to the corresponding C++ expression. They may be used whether or not automatic differentiation is selected.

Two properties of the conditional expression matter in practice. Both arguments `a` and `b` are evaluated, so both must be finite even where they are not selected: a NaN or an infinity produced in the branch that is *not* taken will still enter the tape and propagate. A power law or a logarithm whose argument goes negative outside the range in which it is meant to be used therefore needs to be guarded, for example by clamping its argument with `psopt_max`. And the derivative recorded is that of the selected branch, so a function assembled this way is differentiated correctly but is no smoother than the

function it represents: selecting between two pieces that do not meet with matching slopes gives a kink, exactly as the underlying function has one.

A model built entirely from arithmetic, the elementary functions and these helpers can be taped, and the derivatives obtained from the tape are the derivatives of the model at every point. The interpolation routines described next are written this way. The minimum time to climb example, in `examples/climb` of the PSOPT distribution, is a worked case: its standard atmosphere selects a layer of the 1976 atmosphere with `psopt_cond_lt` and clamps the layer temperature with `psopt_max`, so that the pressure power law of an unselected layer cannot produce a NaN.

7.11.22 linear_interpolation function

This function interpolates a point defined function using classical linear interpolation. This is useful when the problem involves tabular data. The interval containing `x` is selected with the conditional helpers of Section 7.11.21, so the function may be used with automatic differentiation; note, however, that a piecewise linear interpolant is not differentiable at the junction points, and that a discontinuous derivative in the model may still give the NLP solver difficulty. `smooth_linear_interpolation` and `spline_interpolation` avoid that. Values outside the range of the table are extrapolated with the first or last segment. The function prototype is as follows:

```
void linear_interpolation(adouble* y,
                        adouble x,
                        MatrixXd& pointx,
                        MatrixXd& pointy,
                        int npoints);
```

An overload taking the table as arrays of `adouble` is also provided:

```
void linear_interpolation(adouble* y,
                        adouble& x,
                        adouble* pointx,
                        adouble* pointy,
                        int npoints);
```

- `y`: on output, the variable pointed to by this pointer contains the interpolated function value.
- `x`: is the value of the independent variable for which the interpolated function value is sought.
- `pointx`: is the `MatrixXd` object of independent data points.
- `pointy`: is a `MatrixXd` object of dependent data points.
- `npoints`: is the number of points in the data objects `pointx` and `pointy`.

7.11.23 `smooth_linear_interpolation` function

This function interpolates a point defined function using a smoothed linear interpolation. The method used avoids joining sharp corners between adjacent linear segments. Instead, smoothed pulse functions are used to join the segments. The function is suitable for automatic differentiation. This is useful when the problem involves tabular data. Extrapolation is not allowed. The function prototype is as follows:

```
void smooth_linear_interpolation(adouble* y,
                                adouble& x,
                                MatrixXd& pointx,
                                MatrixXd& pointy,
                                int npoints);
```

- `y`: on output, the variable pointed to by this pointer contains the interpolated function value.
- `x`: is the value of the independent variable for which the interpolated function value is sought.
- `pointx`: is the `MatrixXd` object of independent data points.
- `pointy`: is a `MatrixXd` object of dependent data points.
- `npoints`: is the number of points in the data objects `pointx` and `pointy`.

7.11.24 `spline_interpolation` function

This function interpolates a point defined function using natural cubic spline interpolation, which is continuous with continuous first and second derivatives. This is useful when the problem involves tabular data. The interval containing `x` is selected with the conditional helpers of Section 7.11.21, so the function may be used with automatic differentiation. Values outside the range of the table are extrapolated with the first or last cubic. The function prototype is as follows:

```
void spline_interpolation(adouble* y,
                          adouble& x,
                          MatrixXd& pointx,
                          MatrixXd& pointy,
                          int npoints);
```

- `y`: on output, the variable pointed to by this pointer contains the interpolated function value.
- `x`: is the value of the independent variable for which the interpolated function value is sought.

- **pointx**: is the MatrixXd object of independent data points.
- **pointy**: is a MatrixXd object of dependent data points.
- **npoints**: is the number of points in the data objects **pointx** and **pointy**.

7.11.25 bilinear_interpolation function

The function interpolates functions of two variables on a regular grid using the classical bilinear interpolation method. This is useful when the problem involves tabular data. The function prototype is as follows.

```
void bilinear_interpolation(adouble* z,
                           adouble& x,
                           adouble& y,
                           MatrixXd& X,
                           MatrixXd& Y,
                           MatrixXd& Z)
```

- **z**: on output the **adouble** variable pointed to by this pointer contains the interpolated function value.
- The **adouble** pair of variables (**x**, **y**) represents the point at which the interpolated value of the function is returned.
- **X**: is a vector (MatrixXd object) of dimension **nxpoints** $\times$ 1.
- **Y**: is a vector (MatrixXd object) of dimension **nypoints** $\times$ 1.
- **Z**: is a matrix (MatrixXd object) of dimensions **nxpoints** $\times$ **nypoints**. Each element **Z(i,j)** corresponds to the pair (**X(i)**, **Y(j)**)

The function does not deal with sparse data. The cell containing the pair (**x**,**y**) is selected with the conditional helpers of Section 7.11.21, so the function may be used with automatic differentiation; as with **linear_interpolation**, the interpolant itself has discontinuous derivatives at the cell boundaries. Points outside the range of the tables are extrapolated with the nearest cell.

7.11.26 smooth_bilinear_interpolation function

The function interpolates functions of two variables on a regular grid using the a smoothed bilinear interpolation method which allows the use of automatic differentiation. This is useful when the problem involves tabular data. The function prototype is as follows.

```
void smooth_bilinear_interpolation(adouble* z,
                                   adouble& x,
                                   adouble& y,
```

```

MatrixXd& X,
MatrixXd& Y,
MatrixXd& Z)

```

- **z**: on output the **adouble** variable pointed to by this pointer contains the interpolated function value.
- The **adouble** pair of variables (**x**, **y**) represents the point at which the interpolated value of the function is returned.
- **X**: is a vector (MatrixXd object) of dimension **nxpoints** $\times$ 1.
- **Y**: is a vector (MatrixXd object) of dimension **nypoints** $\times$ 1.
- **Z**: is a matrix (MatrixXd object) of dimensions **nxpoints** $\times$ **nypoints**. Each element **Z(i,j)** corresponds to the pair (**X(i)**, **Y(j)**)

The function does not deal with sparse data.

7.11.27 spline_2d_interpolation function

The function interpolates functions of two variables on a regular grid using a cubic spline interpolation method, applied first along one direction and then along the other. This is useful when the problem involves tabular data. It is built from **spline_interpolation** and inherits its properties, so it may be used with automatic differentiation. Note the trailing **workspace** argument, which the user function receives among its own arguments and passes through. The function prototype is as follows.

```

void spline_2d_interpolation(adouble* z,
                             adouble& x,
                             adouble& y,
                             MatrixXd& X,
                             MatrixXd& Y,
                             MatrixXd& Z,
                             Workspace* workspace)

```

- **z**: on output the **adouble** variable pointed to by this pointer contains the interpolated function value.
- The **adouble** pair of variables (**x**, **y**) represents the point at which the interpolated value of the function is returned.
- **X**: is a vector (MatrixXd object) of dimension **nxpoints** $\times$ 1.
- **Y**: is a vector (MatrixXd object) of dimension **nypoints** $\times$ 1.
- **Z**: is a matrix (MatrixXd object) of dimensions **nxpoints** $\times$ **nypoints**. Each element **Z(i,j)** corresponds to the pair (**X(i)**, **Y(j)**)

The function does not deal with sparse data.

7.11.28 `zoh_interpolation` function

The function interpolates a point defined function by a zero order hold: the value returned is the tabulated value at the left end of the interval containing `x`. This is useful for data that are piecewise constant by nature, such as a tariff or a duty cycle. The interval is selected with the conditional helpers of Section 7.11.21, so the function may be used with automatic differentiation, and the derivative it returns is zero, which is the derivative of the function it represents. A piecewise constant term in the right hand side of the dynamics is discontinuous, however: the discretisation cannot resolve a discontinuity that falls between two nodes, and a smooth approximation or a multi-phase formulation is usually preferable. The function prototype is as follows.

```
void zoh_interpolation(adouble* y,
                      adouble x,
                      MatrixXd& pointx,
                      MatrixXd& pointy,
                      int npoints);
```

- `y`: on output, the variable pointed to by this pointer contains the interpolated function value.
- `x`: is the value of the independent variable for which the interpolated function value is sought.
- `pointx`: is the `MatrixXd` object of independent data points.
- `pointy`: is a `MatrixXd` object of dependent data points.
- `npoints`: is the number of points in the data objects `pointx` and `pointy`.

7.11.29 `smooth_heaviside` function

This function implements a smooth version of the Heaviside function $H(x)$, defined as $H(x) = 1, x > 0$, $H(x) = 0$ otherwise. The approximation is implemented as follows:

$$H(x) \approx 0.5(1 + \tanh(x/a)) \quad (7.11)$$

where $a > 0$ is a small real number. The function prototype is as follows:

```
adouble smooth_heaviside(adouble x, double a);
```

7.11.30 `smooth_sign` function

This function implements a smooth version of the function $\text{sign}(x)$, defined as $\text{sign}(x) = 1, x > 0$, $\text{sign}(x) = -1, x < 0$, and $\text{sign}(0) = 0$. The approximation is implemented as follows:

$$\text{sign}(x) \approx \tanh(x/a) \quad (7.12)$$

where $a > 0$ is a small real number. The function prototype is as follows:

```
adouble smooth_sign(adouble x, double a);
```

See the examples section for further details on usage of this function.

7.11.31 smooth_fabs function

This function implements a smooth version of the absolute value function $|x|$. The approximation is implemented as follows:

$$|x| \approx \sqrt{x^2 + a^2} \quad (7.13)$$

where $a > 0$ is a small real number. The function prototype is as follows:

```
adouble smooth_fabs(adouble x, double a);
```

7.11.32 integrate function

The `integrate` function computes the numerical quadrature Q of a scalar function g over the a single phase as a function of states, controls, static parameters and time.

$$Q = \int_{t_0^{(i)}}^{t_f^{(i)}} g[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t] dt \quad (7.14)$$

The integration is done using the Gauss-Lobatto method. This is useful, for example, to incorporate constraints involving integrals over a phase, which can be included as additional event constraints:

$$Q_l \leq \int_{t_0^{(i)}}^{t_f^{(i)}} g[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t] dt \leq Q_u \quad (7.15)$$

Function `integrate` has the following prototype:

```
adouble integrate(
    adouble (*integrand)(adouble*,
                        adouble*,
                        adouble*,
                        adouble&,
                        adouble*,int),
    adouble* xad,
    int iphase )
```

- `integrand`: this is a pointer to the function to be integrated.
- `xad`: is the vector of scaled decision variables.
- `iphase`: is the phase index (starting from 1).

- The function returns the value of the integral as an `adouble` type.

The user needs to implement separately the `integrand` function, which must have the prototype:

```
adouble integrand( adouble* states,
                  adouble* controls,
                  adouble* parameters,
                  adouble& time,
                  adouble* xad,
                  int iphase)
```

- `states`: this is an array of instantaneous states.
- `controls`: is an array of instantaneous controls.
- `parameters`: is an array of static parameter values.
- `time`: is the value of the current instant of time within the phase.
- `xad`: is the vector of scaled decision variables.
- `iphase`: is the phase index (starting from 1).
- the function must return the value of the integrand function given the supplied parameters as an `adouble` type.

7.11.33 `product_ad` functions

There are two versions of this function. The first version has the prototype:

```
void product_ad(const MatrixXd& A,
               const adouble* x,
               int nx,
               adouble* y);
```

This function multiplies a constant matrix stored in `MatrixXd` object `A` by `adouble` vector stored in array `x`, which has length `nx`, and returns the result in `adouble` array `y`.

The second version has the prototype:

```
void product_ad(adouble* Apr,
               adouble* Bpr,
               int na,
               int ma,
               int nb,
               int mb,
               adouble* ABpr);
```

This function multiplies the ($\mathbf{na} \times \mathbf{nb}$) matrix stored in column-major format column in `adouble` array `Apr`, by the ($\mathbf{nb} \times \mathbf{mb}$) matrix stored in column-major format in `adouble` array `Bpr`. The result is stored in column-major format in `adouble` array `ABpr`.

7.11.34 `sum_ad` function

This function adds a matrix or vector stored columnwise in `adouble` array `a`, to a matrix or vector of the same dimensions stored columnwise in `adouble` array `b`. Both arrays are assumed to have a total of `n` elements. The result is returned in `adouble` array `c`. The function prototype is as follows.

```
void sum_ad(const adouble* a,
            const adouble*b,
            int n,
            adouble* c);
```

7.11.35 `subtract_ad` function

This function subtracts a matrix or vector stored columnwise in `adouble` array `a`, to a matrix or vector of the same dimensions stored columnwise in `adouble` array `b`. Both arrays are assumed to have a total of `n` elements. The result is returned in `adouble` array `c`. The function prototype is as follows.

```
void subtract_ad(const adouble* a,
                 const adouble*b,
                 int n,
                 adouble* c);
```

7.11.36 `inverse_ad` function

This function computes the inverse of an $\mathbf{n} \times \mathbf{n}$ square matrix stored columnwise in `adouble` array `a`. The result is returned in `adouble` array `ainv`, also using columnwise storage. The function prototype is as follows.

```
void inverse_ad(adouble* a,
                int n,
                adouble* ainv)
```

7.11.37 `resample_trajectory` function

This function resamples a trajectory given new values of the time vector using natural cubic spline interpolation.

```
void resample_trajectory(MatrixXd& Y,
                          MatrixXd& t,
                          MatrixXd& Ydata,
                          MatrixXd& tdata )
```

- **Y** is, on output, a MatrixXd object with dimensions $n_y \times N$ with the interpolated values of the dependent variable.
- **t** is a MatrixXd object of dimensions $1 \times N$ with the values of the independent variable at which the interpolated values are required. The elements of this vector should be monotonically increasing, i.e. $t(j+1) > t(j)$. The following restrictions should be satisfied: $t(1) \geq tdata(1)$, and $t(N) \leq tdata(M)$.
- **Ydata** is a MatrixXd object of dimensions $n_y \times M$ with the data values of the dependent variable.
- **tdata** is a MatrixXd object of dimensions $1 \times M$ with the data values of the independent variable. The elements of this vector should be monotonically increasing, i.e. $t(j+1) > t(j)$.

7.11.38 linspace function

This function generate a sequence of real values and returns it in a MatrixXd object of dimensions $1 \times N$.

`MatrixXd linspace(double X1, double X2, long N);`

- **X1** is the initial value of the sequence
- **X1** is the final value of the sequence
- **N** is the number of elements of the sequence.

7.11.39 zeros function

This function returns a MatrixXd object of dimensions **nrows** $\times$ **ncols**, having a zero value in all its elements.

`MatrixXd zeros(long nrows, long ncols)`

- **nrows** is the number of rows of the matrix
- **ncols** is the number of columns of the matrix

7.11.40 ones function

This function returns a MatrixXd object of dimensions **nrows** $\times$ **ncols**, having a value of 1 in all its elements.

`MatrixXd ones(long nrows, long ncols)`

- **nrows** is the number of rows of the matrix
- **ncols** is the number of columns of the matrix

7.11.41 eye function

This function returns a MatrixXd object of dimensions $n \times n$, having a value of 1 in all its diagonal elements and zeros elsewhere, this is, an identity matrix.

`MatrixXd eye(long n)`

- `n` is the number of rows and columns of the matrix

7.11.42 GaussianRandom function

This function returns a MatrixXd object of dimensions `nrows` $\times$ `ncols` and each of its elements has a random value obeying a Gaussian distribution with a mean of 0 and a standard deviation of 1.

`MatrixXd RandomGaussian(long nrows, long ncols)`

- `nrows` is the number of rows of the matrix
- `ncols` is the number of columns of the matrix

7.11.43 Elementwise mathematical functions on MatrixXd objects

A number of functions have been implemented returning each a MatrixXd object of dimensions `nrows` $\times$ `ncols` with an element-wise evaluation of a number of mathematical functions. The generic function prototype is as follows:

`MatrixXd <Function_Name>(MatrixXd& m)`

- `m` is the input matrix, which is the argument of the mathematical function being used. Note that the output matrix has the same dimensions of the input matrix.

The following functions are implemented:

- `sin`: trigonometric sine function
- `cos`: trigonometric cosine function
- `tan`: trigonometric tangent function
- `asin`: arc sine function
- `acos`: arc cosine function
- `atan`: arc tangent function
- `sinh`: hyperbolic sine function
- `cosh`: hyperbolic cosine function

- `tanh`: hyperbolic tangent function
- `exp`: exponential function
- `log`: natural logarithm function
- `log10`: base-10 logarithm function
- `sqrt`: square root function

7.11.44 `Save_to_json_file` function

This function exports the complete solution of a problem to a single file in JSON format, which is convenient for post-processing the results with external tools such as Python, MATLAB, or JavaScript. It is normally called after the call to `psopt()`.

```
void Save_to_json_file(const string& filename,
Prob& problem,
Sol& solution,
Alg& algorithm)
```

- `filename` is a string with the name of the JSON file to be created, for example "solution.json". Any existing file with the same name is overwritten.
- `problem` is the `Prob` object describing the problem.
- `solution` is the `Sol` object, returned by `PSOPT`, that contains the solution.
- `algorithm` is the `Alg` object containing the algorithm options.

The file contains a "summary" object and a "phases" array. The "summary" object holds the problem name, the NLP solver used, the `PSOPT` release and build date, the run date and time, the CPU time, the NLP return code, a Boolean success flag, the solver message, the optimal value of the objective function, and the number of phases. The "phases" array contains one object per phase, each reporting the number of nodes, states, controls and parameters, the endpoint and integrated costs, the initial and final times, the maximum relative local error, and the trajectories themselves: the time vector together with the `states`, `controls`, `costates`, `relative_errors`, and `parameters` arrays.

An example of the syntax to call the function is:

```
Save_to_json_file("solution.json", problem, solution, algorithm);
```

The resulting file can be read by any JSON-aware environment. Note that the `states`, `controls`, `costates`, and `relative_errors` arrays are stored row-wise, so they are read back as matrices of dimension (number of variables) $\times$ (number of nodes), while `time` and `parameters` are read as vectors. The following snippets load "solution.json"

and extract the optimal cost together with the time vector and state trajectories of the first phase.

In Python (using the `json` and `NumPy` libraries):

```
import json, numpy as np
with open("solution.json") as f:
    data = json.load(f)

print(data["summary"]["optimal_cost"])
phase    = data["phases"][0]           # first phase
t        = np.array(phase["time"])     # shape (n_nodes,)
states   = np.array(phase["states"])   # shape (n_states, n_nodes)
controls = np.array(phase["controls"]) # shape (n_controls, n_nodes)
```

In MATLAB:

```
data = jsondecode(fileread("solution.json"));

disp(data.summary.optimal_cost)
phase    = data.phases(1);             % first phase
t        = phase.time;                 % n_nodes x 1
states   = phase.states;               % n_states x n_nodes
controls = phase.controls;             % n_controls x n_nodes
```

In R (using the `jsonlite` package):

```
library(jsonlite)
data <- fromJSON("solution.json", simplifyDataFrame = FALSE)

print(data$summary$optimal_cost)
phase    <- data$phases[[1]]           # first phase
t        <- phase$time                 # length n_nodes
states   <- phase$states               # n_states x n_nodes matrix
controls <- phase$controls             # n_controls x n_nodes matrix
```

7.12 Pre-defined constants

The following constants are defined within the header file `psopt.h`:

- `pi`: defined as 3.141592653589793;
- `inf`: defined as 1×10^{19} .

The second of these is the convention for “no bound”, and it needs stating explicitly because it is a number and not a symbol. A bound written -1×10^{19} or $+1 \times 10^{19}$ (or anything beyond it, an IEEE infinity included) means that the variable, path constraint or event is unbounded on that side. The number is IPOPT’s: its `nlp_lower_bound_inf` and `nlp_upper_bound_inf` default to $\mp 10^{19}$ and it treats anything at or past them as infinite, so PSOPT adopts the same number in order that the same model means the same problem whichever solver is asked for it. PSOPT recognises such a bound as absent throughout: an absent bound is left absent by the automatic scaling instead of being multiplied like any other number, and it contributes no scale factor of its own. Writing a very large finite number instead, 10^{12} say, is a different statement, and one the solver will take literally.

7.13 Standard output

PSOPT will by default produce output information on the screen as it runs. PSOPT will produce a short file with a summary of information named with the string provided in `algorithm.outfilename`. This file contains the problem name, the total CPU time spent, the NLP solver used, the optimal value of the objective function, the values of the endpoint cost function and cost integrals, the initial and final time, the maximum discretisation error, and the output string from the NLP solver.

Additionally, every time a PSOPT executable is run, it will produce a file named `psopt_solution_$$$ .txt` (`$$$` represents the characters of `problem.outfilename` which occur to the left of the file extension point “.”). This file contains the problem name, time stamps, a summary of the algorithm options used, and results obtained, the final grid points, the final control variables, the final state variables, the final static parameter values. The file also contains a summary of all constraints functions associated with the NLP problem, including their final scaled value, bounds, and scaling factor used; a summary of the final NLP decision variables, including their final unscaled values, bounds and scaling factors used; and a summary of the mesh refinement process. An indication is given at the end of a constraint line, or decision variable line, if a scaled constraint function or scaled decision variable is within `algorithm.nlp_tolerance` of one of its bounds, or if a scaled constraint function or scaled decision variable has violated one of its bounds by more than `algorithm.nlp_tolerance`. For parameter estimation problems this file also contains the covariance and correlation matrices of the parameter vector, the estimated residual standard deviation with the number of scalar observations and the degrees of freedom behind it, and the 95% confidence interval for each estimated parameter (Section 1.11.3).

`LATEX` code to produce a table with a summary of the mesh refinement process is also automatically generated as described in section 7.8.1.

If `algorithm.print_level` is set to zero, then no output is produced.

7.13.1 The constraint-coverage check

Once per mesh iteration, before the NLP solver is given the problem, PSOPT fills its constraint buffer with a poison value, evaluates the constraints once, and checks that every row the layout promised was written by something. It is not conditional on an option, and it costs one constraint evaluation per mesh.

The check exists because a row that is counted but never written does not announce itself. The buffer is ordinarily zero-filled, and zero is a perfectly plausible value for a constraint: an unwritten equality row is enforced as $0 = 0$, an unwritten inequality row is satisfied by any bounds that admit zero, and the solve converges, reports success, and answers a problem that is not the one posed. Three defects of exactly this kind have been found in PSOPT, and none of the three was caught by a solution failing: two were found months later by a discrepancy in a printed number, and the third by a reader.

A user should never see this check fire. If it does, the message names how many rows were unwritten, which phase and offset the first of them falls in, and asks for the problem to be reported: it is a defect in PSOPT and not in the problem being solved. A separate warning, which is not fatal, is printed if any constraint row evaluates to `NaN` at the initial point of a mesh, since that can come from the user's model at a poor initial guess instead of coming from a disagreement inside PSOPT.

7.14 Implementing your own problem

A template C++ file named `user.cxx` is provided in the directory:

`psopt/examples/user`

This file can be modified by the user to implement their own problem. and an executable can then be built easily.

7.14.1 Building the user code

After modifying the `user.cxx` code, open a command prompt and `cd` the user example directory under the build tree and execute the `make` command:

```
$ cd psopt/build/examples/user
$ make
```

If no compilation errors occur, an executable named `user` should be created in the directory `psopt/PSOPT/examples/user`.

8. Integer (discrete-valued) controls

The problems considered so far assume that every control is continuous. PSOPT also supports optimal control problems in which one or more controls per phase are restricted to a finite set of admissible values, for example an on/off actuator, a gear selection or a mode switch. Such mixed-integer optimal control problems are handled by an ergonomic *declaration* interface: the dynamics are written once, with the discrete control accessed as an ordinary control, and PSOPT performs the underlying reformulation internally. This chapter describes the method, the interface and its conventions, the recovery of the integer controls from the solution, and the current limitations. It builds directly on the problem-definition machinery of Chapter 7. The same declaration interface is available from the Python binding, as described in Section 11.8.

8.1 Method

PSOPT solves integer-control problems in two stages, following the partial outer convexification and sum-up rounding approach of Sager and co-workers [41, 43].

8.1.1 Relaxation by outer convexification

Suppose that a single control $w(t)$ in a phase is restricted to the set of M admissible values $\{v_0, v_1, \dots, v_{M-1}\}$, while the remaining controls $u(t)$ are continuous. Outer convexification replaces w by M continuous *mode weights* $\omega_0(t), \dots, \omega_{M-1}(t)$, each in $[0, 1]$, subject to the special ordered set condition

$$\sum_{i=0}^{M-1} \omega_i(t) = 1, \quad (8.1)$$

an SOS1 equality. The dynamics are evaluated at each admissible value and combined convexly,

$$\dot{x} = \sum_{i=0}^{M-1} \omega_i(t) f[x(t), u(t), v_i, p, t], \quad (8.2)$$

and likewise for any running cost that depends on the discrete control. The result is a smooth nonlinear program in the weights and the continuous variables, whose optimum is a *lower bound* on the integer optimum.

8.1.2 Several integer controls: the product relaxation

A phase may declare K integer controls $w_1, \dots, w_K$, whose admissible sets have sizes $M_1, \dots, M_K$. The convexification is then carried out over the *Cartesian product* of those sets. The phase carries

$$P = \prod_{k=1}^K M_k \quad (8.3)$$

weight controls $\omega_0, \dots, \omega_{P-1}$, one per admissible *combination*, a single SOS1 equality (8.1) over all of them, and the convex combination

$$\dot{x} = \sum_{j=0}^{P-1} \omega_j(t) f[x(t), u(t), v^{(j)}, p, t], \quad (8.4)$$

where $v^{(j)} = (v_1^{(j)}, \dots, v_K^{(j)})$ is the j th combination of admissible values. Combinations are indexed in mixed radix with the first declared control as the leading digit, so combination 0 sets every integer control to its first admissible value. For $K = 1$ the product reduces to (8.2).

The product is what keeps the relaxation exact. Giving each control its own weight vector would be cheaper ($\sum_k M_k$ weights instead of $\prod_k M_k$), but those weights would enter the dynamics multiplicatively wherever two integer controls interact, the relaxed problem would no longer be convex in the weights, and the equality between the relaxed optimum and the integer infimum, on which the method rests, would be lost. The price is that the weight count grows as the product: two binary controls cost four weights per node, three cost eight. Keeping the admissible sets small, and the number of declarations per phase modest, is therefore part of using the method well.

8.1.3 Sum-up rounding

The relaxed weights are generally fractional. They are converted to a control that is active in exactly one mode on each mesh interval by *sum-up rounding*: sweeping across the mesh, the algorithm accumulates, for each mode, the integral of the relaxed weight minus the integral of the rounding already committed, and activates on the current interval the mode with the largest accumulated value. If ω_i denotes the relaxed weight of mode i and $p_i \in \{0, 1\}$ the rounded indicator, this keeps the control integrals close,

$$\left| \int_{t_0}^t (\omega_i(\tau) - p_i(\tau)) d\tau \right| \leq \bar{h}, \quad i = 0, \dots, M-1, \quad (8.5)$$

where $\bar{h}$ is the largest mesh interval width. Because the state and objective errors induced by the rounding are governed by this integral gap, they are $O(\bar{h})$ and vanish as the mesh is refined [43]. The objective obtained by simulating the rounded control is therefore an *upper bound* on the integer optimum that approaches it as $\bar{h} \rightarrow 0$.

With several integer controls the rounding is applied once, to the P product weights: the combination active on each interval is chosen by sum-up rounding and then decoded

into one value per declared control. The bound (8.5) therefore holds for the combination indicator, and the individual controls inherit it.

For a bang-bang problem the relaxed and integer optima coincide. For a singular problem no optimal integer control exists, but the relaxed infimum can be approached arbitrarily closely by a control that switches sufficiently often; refining the mesh then reduces the gap while increasing the number of switches.

8.2 Declaring an integer control

The interface is provided by a single header, which should be included in addition to `psopt.h`:

```
#include "integer_controls.h"
```

The phase dynamics are written *once*, with the discrete control accessed as an ordinary entry of the `controls` array, exactly as if it were continuous:

```
void dae(adouble* derivatives, adouble* path, adouble* states,
         adouble* controls, adouble* parameters, adouble& time,
         adouble* xad, int iphase, Workspace* workspace)
{
    adouble w = controls[0]; // the integer control: a single value
    // ... use w like any other control ...
}
```

The control is then declared as integer-valued, after `psopt_level2_setup`, by

```
RowVectorXd values(2); values << -0.05236, 0.05236;
declare_integer_control(problem, /*iphase=*/1, /*control_index=*/0, values);
```

The function

```
void declare_integer_control(Prob& problem, int iphase,
                           int control_index, const RowVectorXd& values);
```

records, on the given phase, that the control at `control_index` is restricted to the admissible set `values`. The declaration is consumed by `psopt` at solve time.

The function may be called once for each integer control of the phase. Declarations accumulate, and re-declaring the same control index replaces the earlier entry instead of adding a second one. The order of declaration fixes the mixed-radix ordering of the product modes (8.3) and the order in which the controls are reported after the solve:

```
RowVectorXd onoff(2); onoff << 0.0, 1.0;
declare_integer_control(problem, 1, 0, onoff); // first integer control
declare_integer_control(problem, 1, 1, onoff); // second integer control
```

Two auxiliary functions expose the product layout where it is needed:

```

int integer_control_product_size(const std::vector<IntegerControl>& v);
void integer_control_decode(const std::vector<IntegerControl>& v, int j,
                           std::vector<int>& mode_of_control);

```

returning P of (8.3), and the decoding of a product index j into the mode index of each declared control. The declarations themselves are held in the phase field `integer_controls`.

8.2.1 Conventions

- **The integer control is counted in `ncontrols`.** Set `ncontrols` to the total number of controls, including the integer one, exactly as the `dae` indexes them. The bounds and guess arrays are auto-sized to `ncontrols` as usual (Chapter 7); no manual sizing is required.
- **Bounds for an integer control are ignored.** Its admissible values come from `declare_integer_control`, not from the control bounds; PSOPT owns the weight bounds $[0, 1]$. The bounds entry for the integer control may be left at any value (writing the value range there reads naturally). A one-line notice is printed at solve time to make this explicit.
- **Several integer controls per phase are allowed.** A phase may declare as many as it needs, one call each. The convexification is over the Cartesian product of the admissible sets (8.3), so each further declaration multiplies the number of weight controls; different phases carry their own declarations.
- **Do not add the SOS1 constraint yourself.** Set `npath` to the number of your own path constraints only; PSOPT appends the SOS1 equality (8.1) internally.

8.2.2 Initial guess

Provide the integer control's guess row in *value terms* (each entry close to one of the admissible values). PSOPT maps each entry to the nearest admissible value and expands the row into one-hot mode weights; continuous control guesses and the state guess are passed through unchanged. If no control guess is supplied, the weights take their default.

8.3 What PSOPT does internally

The reformulation is performed automatically at the start of `psopt` and undone when it returns. For each phase that declares an integer control:

- the K integer control slots are removed and P weight controls are appended, so the internal control count becomes $(\text{ncontrols} - K) + P$, with P given by (8.3);
- one SOS1 equality path constraint (8.1) is appended, so `npath` becomes `npath + 1`;

- the control and path bounds, scale and guess are rewritten to match; and
- the user `dae` and running cost are evaluated at each admissible combination and combined as in (8.4).

When `psopt` returns, the problem object is restored to its original layout, but *the solution controls are in the weights layout*: the trailing P rows of `get_controls_in_phase(iphase)` are the mode weights $\omega_0, \dots, \omega_{P-1}$. The integer controls are recovered with the helpers of the next section. This whole process is inactive when no integer control is declared, so problems without integer controls are unaffected.

8.4 Reconstructing the integer controls

After a solution, the rounded integer controls of a phase are obtained in a single call, which returns one entry per declared control, in declaration order:

```
std::vector<IntegerControlReconstruction> rec =
    reconstruct_integer_controls(solution, problem, iphase);
```

Each entry is

```
struct IntegerControlReconstruction {
    RowVectorXd control;           // 1 x P : rounded integer control per
    ↪ interval
    RowVectorXi mode_index;        // 1 x P : active admissible-value index per
    ↪ interval
    RowVectorXd interval_widths;   // 1 x P : mesh interval widths
    double      integral_gap;       // accumulated sum-up-rounding gap (<= max
    ↪ interval width)
    int         n_switches;         // number of mode switches
};
```

The helper extracts the trailing P weight rows from the solution, forms the interval widths from the phase time, applies sum-up rounding over the product modes, and decodes each chosen combination into a value for each declared control using the admissible values recorded at declaration. Because the rounding is performed once on the combinations, the integral gap and the switch count are properties of the combination sequence and are therefore the same in every entry; `n_switches` counts the instants at which *any* declared control changes.

For the common case of a single integer control, and for picking one control out of several, a convenience overload returns a single structure:

```
IntegerControlReconstruction rec =
    reconstruct_integer_control(solution, problem, iphase, /*which=*/0);
```

where `which` is the declaration ordinal; an out-of-range ordinal, or a phase with no declaration, yields an empty structure. The relaxed objective read from the solved problem is a lower bound; the objective obtained by simulating the rounded controls is

an upper bound that tends to the relaxed value as the mesh is refined, in accordance with (8.5).

8.5 Worked example: the F-8 aircraft

The F-8 aircraft time-optimal control problem [42] steers the aircraft to the origin in minimum time using a tail deflection restricted to $w \in \{-0.05236, +0.05236\}$ rad. The dynamics are written once with the deflection as a single control:

```
void dae(adouble* d, adouble* path, adouble* states, adouble* controls,
        adouble* parameters, adouble& time, adouble* xad, int i, Workspace*
        ↪ w)
{
    adouble x0 = states[0], x1 = states[1], x2 = states[2];
    adouble u = controls[0];           // the integer tail deflection
    f8_rhs<adouble>(x0, x1, x2, u, d);  // cubic-in-control dynamics
}

int main() {
    Alg algorithm; Sol solution; Prob problem;
    problem.nphases = 1; problem.nlinkages = 0;
    psopt_level1_setup(problem);

    problem.phases(1).nstates = 3;
    problem.phases(1).ncontrols = 1;           // one control: the deflection
    problem.phases(1).nevents = 6;
    problem.phases(1).npath = 0;               // the SOS1 constraint is added
    ↪ internally
    problem.phases(1).nodes << 60;
    psopt_level2_setup(problem, algorithm);

    RowVectorXd values(2); values << -0.05236, 0.05236;
    declare_integer_control(problem, 1, 0, values);

    // ... state and event bounds, times, guess, function pointers, algorithm
    ↪ ...

    psopt(solution, problem, algorithm);

    IntegerControlReconstruction rec = reconstruct_integer_control(solution,
    ↪ problem, 1);
    // rec.control is the rounded bang-bang deflection; rec.n_switches its
    ↪ switches.
}
```

The relaxed solve reproduces the best-known minimum time $T = 3.78086$ with the expected three-switch bang-bang structure. Complete, runnable examples are provided under `examples/f8` (minimum-time, bang-bang), `examples/lotka` (the singular Lotka–

Volterra fishing problem [42], with a mesh sweep showing the relaxed objective converging to 1.34408 and the integer gap closing at $O(\bar{h})$, `examples/mixed_ic` (a continuous and an integer control in the same phase), and `examples/lotka2` (two independent integer controls, described next).

8.6 Worked example: two independent integer controls

The example `examples/lotka2` takes the same Lotka–Volterra fishing problem and gives the two species their own binary fishing control, w_1 acting on the prey and w_2 on the predator:

$$\dot{x}_1 = x_1 - x_1x_2 - c_0x_1w_1, \quad \dot{x}_2 = -x_2 + x_1x_2 - c_1x_2w_2, \quad w_1, w_2 \in \{0, 1\}, \quad (8.6)$$

with $\dot{x}_3 = (x_1 - 1)^2 + (x_2 - 1)^2$ accumulating the cost. The dynamics are written once in terms of `controls[0]` and `controls[1]`, and both are declared:

```
RowVectorXd onoff(2); onoff << 0.0, 1.0;
declare_integer_control(problem, 1, 0, onoff);
declare_integer_control(problem, 1, 1, onoff);
// ... solve ...
std::vector<IntegerControlReconstruction> rec =
    reconstruct_integer_controls(solution, problem, 1);
// rec[0].control is the rounded w1, rec[1].control the rounded w2.
```

The product relaxation (8.4) therefore carries $P = 2 \times 2 = 4$ weight controls and one SOS1 constraint. The single-control problem of `examples/lotka` is the special case $w_1 \equiv w_2$, so its relaxed optimum $\Phi^* = 1.34408$ is an *upper* bound here. A mesh sweep gives

nodes	relaxed Φ	integer Φ	gap to Φ^*	switches	$w_1 \neq w_2$
40	1.091351	1.236421	-1.077×10^{-1}	4	8
80	1.089091	1.153548	-1.905×10^{-1}	8	16
160	1.088291	1.092633	-2.514×10^{-1}	16	33
320	1.088069	1.089384	-2.547×10^{-1}	34	67
640	1.088051	1.088423	-2.557×10^{-1}	68	135

The relaxed objective converges monotonically to about 1.08805 and the integer objective closes on it at the $O(\bar{h})$ rate that (8.5) predicts, finishing some 19 per cent below the single-control optimum. The last column counts the intervals on which the two controls differ: those are exactly the ones a single joint control could not have produced, and they are the reason the two-control optimum lies below Φ^* .

8.7 Limitations

A phase may declare any number of integer controls, but the convexification is over the Cartesian product of their admissible sets (8.3), so the number of weight controls, and

with it the size of the nonlinear program at each node, grows as the product of the set sizes. Three binary controls already cost eight weights per node, and a pair of four-valued controls sixteen. The method is therefore intended for a small number of declarations over small admissible sets; a phase needing many discrete decisions is better posed with a coarser discrete parameterisation or a decomposition. The running cost may depend on the integer controls, as it is convex-combined in the same way as the dynamics; the endpoint cost does not receive the controls and so must not depend on the integer controls directly. User path constraints are assumed independent of the integer controls, being evaluated at a representative mode. Finally, for a minimum-time problem the rounded control does not, in general, reach the target exactly in the relaxed final time; the terminal miss is the integer approximation error and shrinks under mesh refinement, while an exactly feasible integer final time would require switching-time optimisation.

9. Integer (discrete-valued) static parameters

The previous chapter restricted a control to a finite set of values. PSOPT also supports optimal control problems in which one or more *static parameters* of a phase are restricted to a finite set of admissible values: a discrete design choice such as a component count, a gear ratio selected from a catalogue, or a material index. These integer static parameters are handled by a dedicated driver, `psopt_solve_integer`, which solves the problem *exactly* by enumeration. The dynamics, cost and events are written once, with the discrete parameter accessed as an ordinary static parameter, and a single declaration marks it integer-valued. This chapter describes the method, the interface and its conventions, the driver, the recovery of the selected value, and the current limitations. It builds on the problem-definition machinery of Chapter 7 and is a companion to the integer-control interface of Chapter 8. The declaration is also available from the Python binding (Section 11.8), which routes the solve to the dedicated driver automatically.

9.1 Method

Unlike a control, a static parameter takes a single value over the whole phase; it has no time profile. PSOPT therefore solves an integer-static-parameter problem by *exact enumeration* instead of employing the convex relaxation used for integer controls.

Suppose a problem declares static parameters $p_{j_1}, \dots, p_{j_D}$ as integer, the a -th restricted to an admissible set of M_a values $\{v_0^{(a)}, \dots, v_{M_a-1}^{(a)}\}$. The driver forms the Cartesian product of these sets and, for each combination, fixes every declared parameter to its chosen admissible value, by setting its lower and upper bounds equal, and solves the resulting fixed-parameter optimal control problem with the ordinary `psopt` solver. The combination that yields the smallest objective among the successful solves is returned. Because every admissible combination is tried, the result is the global integer optimum over the declared sets; no rounding or relaxation gap is incurred.

The number of solves is the product of the set sizes, taken across all phases,

$$N_{\text{comb}} = \prod_{a=1}^D M_a. \quad (9.1)$$

This grows multiplicatively, so exact enumeration is intended for small admissible spaces. To prevent an accidentally huge sweep, the driver refuses to start if N_{comb} exceeds `algorithm.max_integer_combinations` (default 4096).

9.1.1 Relation to integer controls

The outer-convexification and sum-up-rounding method of Chapter 8 realises a fractional *blend* of admissible values as a control that switches over time, with an integrality gap that vanishes as the mesh is refined [43]. A static parameter has no time dimension over which to switch, so a convex blend of admissible values is not realisable and no mesh refinement closes the gap. Exact enumeration is therefore the appropriate method for integer static parameters, in the small-space regime for which they are intended.

9.2 Declaring an integer parameter

The interface is provided by a single header, which should be included in addition to `psopt.h`:

```
#include "integer_parameters.h"
```

The discrete parameter is used *once*, as an ordinary entry of the `parameters` array, in the dynamics, cost or events, exactly as if it were continuous:

```
void dae(adouble* derivatives, adouble* path, adouble* states,
         adouble* controls, adouble* parameters, adouble& time,
         adouble* xad, int iphase, Workspace* workspace)
{
    adouble p = parameters[0]; // the integer static parameter
    // ... use p like any other static parameter ...
}
```

It is then declared integer-valued, after `psopt_level2_setup`, by

```
RowVectorXd values(4); values << 0.0, 1.0, 2.0, 3.0;
declare_integer_parameter(problem, /*iphase=*/1, /*parameter_index=*/0,
    ↪ values);
```

The function

```
void declare_integer_parameter(Prob& problem, int iphase,
                              int parameter_index, const RowVectorXd& values
    ↪ );
```

records, on the given phase, that the static parameter at `parameter_index` is restricted to the admissible set `values`. The record is dormant under an ordinary `psopt` call, which ignores it, and is consumed by `psopt_solve_integer` (Section 9.3).

9.2.1 Conventions

- **The parameter is counted in `nparameters`.** Declare it as an ordinary static parameter: set `nparameters` to include it, and access it at the same index in `dae`, `events` and the cost. Its bounds and guess arrays are auto-sized as usual (Chapter 7); no manual sizing is required.

- **Set the bounds to the admissible hull.** Give the parameter a continuous bound range spanning its admissible set, for example from its smallest to its largest value. `psopt_solve_integer` overwrites these bounds to pin the parameter to each admissible value in turn, and restores them when it returns; the range you set is used for the initial guess and automatic scaling.
- **Several parameters may be declared.** A phase may declare more than one integer parameter, and different phases may each declare their own. The driver enumerates the full Cartesian product across all phases, bounded by `algorithm.max_integer_combinations` as in (9.1).
- **Continuous parameters are unaffected.** Any static parameter that is not declared integer remains a free continuous variable and is optimised normally within each fixed-combination solve.

9.3 Solving with the dedicated driver

An integer-static-parameter problem is solved by the dedicated driver in place of `psopt`:

```
int rc = psopt_solve_integer(solution, problem, algorithm);
```

whose signature is

```
[[nodiscard]] int psopt_solve_integer(Sol& solution, Prob& problem, Alg&
    ↪ algorithm);
```

The driver enumerates the Cartesian product of the declared admissible sets; for each combination it fixes the declared parameters, solves the fixed-parameter problem with the ordinary `psopt`, and keeps the combination of smallest objective among the successful solves. On return, `solution` holds that best solve (states, controls, parameters, cost and diagnostics) exactly as an ordinary `psopt` call would leave it, and the return value `rc` is that solve's `psopt` return code. Combinations whose solve fails are skipped; if none succeeds, the last attempt is returned so that its return code remains visible to the caller.

If no integer parameter is declared, `psopt_solve_integer` performs a single ordinary solve and is equivalent to calling `psopt` directly. This makes it safe to route all solves of a given problem through the driver.

Before enumerating, the driver checks the combination count (9.1) against `algorithm.max_integer_combinations` and, if it is exceeded, aborts through the usual PSOPT error handler (Section 7.4) instead of launching an unintended long sweep. The count may be inspected beforehand with

```
long n = integer_parameter_combination_count(problem);
```

9.4 Reconstructing the selected value

After a solution, the selected admissible value of a declared parameter is read back in a single call:

```
IntegerParameterReconstruction sel =
    reconstruct_integer_parameter(solution, problem, /*iphase=*/1, /*
    ↪ parameter_index=*/0);
```

which returns

```
struct IntegerParameterReconstruction {
    double value;    // selected admissible value (NaN if not declared)
    int    index;    // index into the admissible set (-1 if not declared)
};
```

Because the winning combination was solved with the parameter fixed, the solved solution already carries the integer value exactly; the helper reads it from `get_parameters_in_phase` and snaps it to the nearest admissible value as a safety net. The optimal objective is available as usual in `solution.cost`.

9.5 Worked example: a discrete acceleration

The example under `examples/int_static_linear` is a minimal, control-free double integrator whose constant acceleration is an integer static parameter. With states s (position) and v (velocity), no controls, and one parameter p ,

$$\dot{s} = v, \quad \dot{v} = p, \quad s(0) = v(0) = 0, \quad t \in [0, 1], \quad (9.2)$$

so that $v(1) = p$. The parameter is restricted to $p \in \{0, 1, 2, 3\}$ and the objective is $J = (v(1) - 2.3)^2$, chosen so that the discrete optimum $p = 2$ (giving $J = 0.09$) differs from the continuous optimum $p = 2.3$. The dynamics read the parameter directly:

```
void dae(adouble* d, adouble* path, adouble* states, adouble* controls,
    adouble* parameters, adouble& time, adouble* xad, int i, Workspace*
    ↪ w)
{
    d[0] = states[1];    // s_dot = v
    d[1] = parameters[0]; // v_dot = p
}

int main() {
    // ... psopt_level1_setup / level2_setup, states, events, times, guess
    ↪ ...

    RowVectorXd values(4); values << 0.0, 1.0, 2.0, 3.0;
    declare_integer_parameter(problem, 1, 0, values);
```

```

    int rc = psopt_solve_integer(solution, problem, algorithm);

    IntegerParameterReconstruction sel =
        reconstruct_integer_parameter(solution, problem, 1, 0);
    // sel.value == 2.0 and solution.cost == 0.09
}

```

Enumeration solves the four fixed-parameter problems and selects $p = 2$ with $J = 0.09$, matching the closed-form optimum. Because the endpoint cost and the events function both receive the parameter array, the discrete parameter may enter the cost and the boundary conditions directly, in contrast to the integer-control interface, where the endpoint cost does not receive the control. A two-parameter co-design, each parameter drawn from a four-value set, is written in the same way and enumerates the full $4 \times 4 = 16$ combinations.

9.6 Limitations

The method is exact but combinatorial: the number of solves is the product (9.1) of the admissible-set sizes, so it is intended for small discrete spaces: a handful of parameters over modest sets. Wide ranges, or many jointly-declared parameters, quickly exceed a practical budget and are guarded by `max_integer_combinations`.

Each combination is solved independently, from the supplied initial guess; this version does not warm-start one combination from another and does no relaxation-based pruning. A failed fixed-parameter solve is skipped, so a poor guess that defeats every combination yields no usable solution; a guess that is valid across the admissible hull is advisable.

Finally, the worked example (9.2) is deliberately degenerate, with one non-trivial state and no controls, to isolate the mechanism. A realistic application will have the discrete parameter enter non-trivial dynamics, cost or constraints alongside continuous states and controls.

10. Robust optimal control

Every problem considered so far has been given a plant. This chapter is about the situation in which it has not: one or more parameters of the dynamics, the path constraints or the boundary conditions are known only through a set or a distribution, and one control history must be committed *before* the parameter is revealed. That is the ordinary situation in practice, and PSOPT solves it as it stands. No part of the core solver is modified. What the library adds is the construction that turns a robust problem into a deterministic one, two drivers that run the outer loop around it, and the arithmetic that says how large the construction may grow.

The chapter states the transcription and the formulation it discretises (Section 10.1), the three separate roles played by an uncertainty description (Section 10.2), the risk measures available (Section 10.3), ancillary feedback and the tube design (Section 10.4), the scenario budget (Section 10.5), what verification means here and what it establishes (Section 10.7), the C++ driver `psopt_solve_robust` (Section 10.8), the Python driver `psopt.robust` (Section 10.9), and the current limitations (Section 10.11). The worked derivation of a complete example, a two-link arm carrying a payload of uncertain mass, is in the PSOPT Application Examples Document, which should be read alongside this chapter. The division between the two is that the formulation and the interfaces are stated here, while that document carries one problem through from end to end: how the payload enters the dynamics, why a quadrature rule fails on it, and what the accuracy of its verification turned out to rest on.

10.1 The scenario transcription

Write the uncertainty as a parameter vector θ ranging over a set Θ , and let the state depend on it while the control does not:

$$\begin{aligned}
 & \min_{u(\cdot), p, t_f} && J[\{x(\cdot; \theta)\}_{\theta \in \Theta}, u(\cdot), p, t_f] \\
 \text{subject to} &&& \dot{x}(t; \theta) = f(x(t; \theta), u(t), p, \theta, t), && \theta \in \Theta, \\
 &&& x(t_0; \theta) = x_0(\theta), && \theta \in \Theta, \\
 &&& g(x(t; \theta), u(t), p, \theta, t) \leq 0, && \theta \in \Theta, \\
 &&& e_L \leq e(x(t_0; \theta), x(t_f; \theta), p, t_0, t_f, \theta) \leq e_U, && \theta \in \Theta, \\
 &&& u_{\min} \leq u(t) \leq u_{\max}.
 \end{aligned} \tag{10.1}$$

The objective is a functional of the whole family of state trajectories as well as of the control, the parameters and the final time, and which functional it is forms a separate decision, taken up in Section 10.3. The constraints are required to hold for *every* θ in Θ , so (10.1) has finitely many decision variables and infinitely many constraints. It is a semi-infinite problem, and everything in this chapter is about how that infinity is discretised.

10.1.1 What a robust control is, and what it is not

What is being computed needs a precise statement, because there is a tempting procedure that does not compute it. The requirement that gives (10.1) its character is that $u(\cdot)$ carries no dependence on θ . A control that did depend on θ would be using information that is not available when it has to be committed. That property is non-anticipativity, and it is what makes the family of trajectories one problem and not many.

Solving the problem once per sampled parameter and averaging the answers therefore computes something else. It computes the *wait-and-see* solution, in which each realisation is optimised with foreknowledge of its own uncertainty. Because $\mathbb{E}[\min] \leq \min \mathbb{E}[\cdot]$, its value is a lower bound attained by no implementable control, and the average of the control histories solves nothing at all, being the mean of solutions to different problems [45]. Both drivers can compute that bound, as a diagnostic, and neither offers it as a design. The bound is worth having for what it separates: the gap between it and a robust design is the price of not knowing θ , and no formulation closes that gap.

10.1.2 The scenario replacement

Replacing Θ by a finite list of *scenarios* $\{\theta_1, \dots, \theta_M\}$ turns (10.1) into one ordinary deterministic optimal control problem carrying M copies of the state against one copy of the control. If the original phase has n_s states, n_c controls, n_e events and n_g path constraints, the augmented phase has Mn_s states, n_c controls, Mn_e events and Mn_g path constraints, with the shared control and the shared final time coupling the copies. That is the whole of the transcription, and the augmented problem is handed to `psopt()` in the usual way. The approach is sample-average approximation where the objective is an expectation, and the scenario approach of Calafiore and Campi [9] where the constraints are what matter.

A scenario set does two jobs and they have to be kept apart. As a *constraint set* every member counts equally, there being no such thing as a constraint that holds with weight one sixth. As a *quadrature rule* for an expected cost the weights are the rule's and they matter. Both drivers therefore give a generated scenario weight zero in the objective while imposing its constraints in full: generated scenarios are the places at which the design was found to be failing, which is a biased sample of the uncertainty by construction, and admitting them to the expectation would silently replace the stated risk measure by a worst-case-weighted one.

10.1.3 Generating the scenarios

A fixed scenario set gives no information about the parameters it does not contain. The alternative, which both drivers implement, is to let the design choose them. Solve with the scenarios in hand; find the parameter the resulting design serves *worst* over the whole of Θ ; if that is worse than an allowance, add it as a scenario and solve again. Each iteration costs one call to `psopt()` and one scan of an independent integrator, and the scenario set grows only where the design is actually failing. This is a cutting-plane method on the semi-infinite constraint of (10.1), and it is what allows the loop to finish with a certificate and not with a statistic.

Two details decide whether it terminates, and neither is optional.

- **The design is given a tightened constraint and verified against the declared one.** The scenarios are satisfied to the stated tolerance *exactly*, an active constraint being active, so the violation at a parameter between two neighbouring scenarios is necessarily a little larger. A loop that demands the declared tolerance over the whole set, from a design that was only ever asked for it at finitely many points, cannot converge. The tightening is `tighten` in both drivers, a fraction of each two-sided half-width with default 0.9, applied to the events and the path constraints of every phase.

A fraction of a half-width is necessarily *symmetric*, and a bound's two ends often do not mean the same thing. A heating rate declared on $[0, 10^6]$ W/m² has an upper end that is a requirement and a lower end that is a statement of sign: the quantity cannot be negative, and the design is not being asked to hold it away from zero. With `tighten = 0.9` the derivation gives that lower end 5×10^4 , which an entry vehicle at its interface misses by a factor of four and can do nothing about, the initial state being pinned by the problem's own events. On a minimum heat load entry whose corridor is bounded by the heating rate, the normal load factor and the dynamic pressure, measured at 40 nodes, the three constraints acquire lower bounds of 5×10^4 W/m², 0.125 g and 500 N/m², against values at the interface of 1.2288×10^4 , 1.3333×10^{-4} and 0.4885. The problem is then infeasible at $t = 0$, before any dynamics, for a reason that has nothing to do with robustness, and the solver reports it as a locally infeasible problem and not as a tightening that overreached.

Each end therefore takes a margin of its own:

```
// C++: four fields of RobustPhase, so per phase. Each is empty by
    ↪ default, in which
// case every margin is derived from tighten; a single entry is
    ↪ broadcast; otherwise
// one entry per event or per path constraint of that phase.
RowVectorXd events_margin_lower, events_margin_upper;
RowVectorXd path_margin_lower,   path_margin_upper;
```

```
# Python: six keys of margin. The first two set both ends of their
    ↪ family at once.
margin = dict(events=..., path=...,
              events_lower=..., events_upper=...,
              path_lower=..., path_upper=...)
```

A key left out of the Python dictionary is derived from `tighten`, and a key present with value `None` gets no margin at all. Giving one family both ways is refused instead of resolved, either order of precedence being a guess at what was meant.

An explicitly given margin is also the only way a *one-sided* bound can have one, there being no half-width there to take a fraction of, and a design left sitting on a one-sided bound is exactly where the between-scenario overshoot appears. Both drivers say so when they find such a bound without a margin of its own. A pinned bound is an equality and takes no margin from either source, and a pair of margins that would close a bound's box is refused before a solve starts, an empty box being the silent infeasibility these exist to remove.

- **Each subproblem is warm-started from the previous design, with a dynamically consistent state guess.** Each augmented problem is nonconvex. A state guess that is merely the right shape leaves it with M copies of an infeasible arc, and the solver free to wander to a distant local minimum. The guess is built by integrating each scenario's own plant through the previous control, read the way the transcription means it (Table 10.4), which is the same reading the verifier uses.

That the two readings are the same matters more than it looks. Reading the nodal table as a chord under Hermite–Simpson discards the midpoint control and a third of the control variables, and the consequence is not a blunter guess but no guess at all: on the same entry problem the integration of the design's own nominal scenario left the trajectory well before the final time, the velocity went negative where the heat-rate term $v^{3.15}$ is undefined, and the driver fell back to the nominal guess at every iteration, leaving the loop with no warm chain.

The objective is not monotone down the iteration column, and should not be expected to be: each subproblem is nonconvex and a warm start does not guarantee the same local minimum. What the scheme controls is the worst case over the set, not the objective.

10.2 Uncertainty description

An uncertainty description is asked to do three different things, and conflating them is how a design ends up being scored against its own truncation. It supplies a *set* to be robust over and in which the worst case is sought; a *distribution* to draw from when a design is scored out of sample; and a *rule* for placing scenarios. Both drivers keep the three separate. A Gaussian's set is the ellipsoid at `truncate` standard deviations, a uniform's set is its box, and an explicit uncertainty is its own list of points.

How much of a Gaussian a truncated set holds falls away with dimension in a way that catches people out, so the figure is one to have at hand. The fraction inside the ellipsoid at k standard deviations is the chi-square distribution on n degrees of freedom evaluated at k^2 , and at $k = 3$ it is 99.73% on a line, 98.89% in the plane and 97.07% in three dimensions. A habit formed on scalar uncertainty truncates a multi-parameter set more tightly than it means to. The figure is available as `robust_set_coverage(k, n)` in C++ and is printed by the Python driver's description of its uncertainty.

Three rules for placing scenarios are provided.

- **The unscented, or sigma-point, set.** $2n + 1$ points reproducing the mean and the covariance exactly, with $\kappa = 3 - n$ [25]. For one Gaussian parameter these are $\{\mu, \mu \pm \sqrt{3}\sigma\}$ with weights $\{2/3, 1/6, 1/6\}$, which are the three points and weights of the Gauss–Hermite rule; the sigma points of the estimation literature and the Gauss–Hermite nodes of the quadrature literature are the same three numbers here. The rule is refused for $n > 3$, where the central weight would turn negative. A negative weight is admissible in a quadrature and not in a scenario set, every member of which is a constraint the design must satisfy.
- **A low-discrepancy sequence over the set**, by a Halton sequence [23]. For a Gaussian the radii are stratified over $(0, 1]$ with the value 1 attained, because the worst parameter very often sits on the boundary of the set and a sequence that reaches it only by accident would miss exactly the point a certificate is about.
- **An explicit list**, which is also how a design is made robust over a discrete family of plants instead of over a continuum.

Extreme points of the set are available separately, as `robust_boundary_points` in C++ and `Uncertainty.boundary_points` in Python: the $2n$ ends of a Gaussian's principal axes, the corners of a box up to eight dimensions and its face centres beyond, or an explicit uncertainty's own list. They are added by hand to every worst-case search, for the reason just given.

10.3 Risk measures

What is minimised is a separate decision from what is constrained, and both drivers offer the same four choices: `risk=` in Python, `spec.risk` in C++. In each case the constraints of every scenario are imposed in full; only the objective changes.

- **"nominal"** minimises the cost of the central scenario. The other scenarios enter as constraints alone. This is the right choice when the cost is shared, as it is in a minimum-time problem where $J = t_f$ and every scenario has the same final time.
- **"expectation"** minimises the scenario rule's estimate of $\mathbb{E}[J]$, which is a weighted sum of the per-scenario costs and goes straight into the integrand and the endpoint cost.

- "mean-variance" minimises $\mathbb{E}[J] + \lambda \text{Var}[J]$, with λ set by `mv_lambda`. The variance is the scenario rule's, so a generated scenario, which weighs nothing, does not enter it.
- "cvar" minimises the conditional value at risk at level `cvar_alpha`, the mean of the worst $1 - \alpha$ of the cost distribution, by the device of Rockafellar and Uryasev [39]. It is written with one slack static parameter per scenario and not with a smoothed hinge, so the constraints are exact. PSOPT has static parameters and this is what they are for.

In C++ the same four are selected, and their two parameters set, by

```
spec.risk      = ROBUST_EXPECTATION;    // ROBUST_NOMINAL is the default; the
    ↪ others                                     // are ROBUST_MEAN_VARIANCE and
    ↪ ROBUST_CVAR                                // the CVaR level
spec.cvar_alpha = 0.9;                  // the weight on the variance
spec.mv_lambda  = 1.0;                  // the scenario rule's own weights
spec.weights    = w;
```

The two jobs a scenario set does need separating here. As a constraint set every member counts equally, and every member is imposed in full. As a quadrature rule the weights are the rule's, and a scenario the loop generated belongs to no rule, so it weighs zero in the objective and still constrains the design completely.

The first two are weighted sums of per-scenario costs, which a Lagrange cost delivers without further machinery. The last two need each scenario's cost as a quantity in its own right, because the variance of a running cost across scenarios is not the integral of anything, so the driver carries one extra state per scenario to accumulate it. A state needs bounds, and those bounds are asked for, through `.cost_bounds` in Python and `model.cost_lower` and `model.cost_upper` in C++, and not guessed: a bound that turned out to be active would silently change the risk measure into something else. Both drivers refuse these two measures until the range is given.

10.3.1 About the tail measure

The sample-average approximation of a risk measure can only see the distribution that the scenario set describes, and for a tail measure that is a sharper restriction than it looks. At $\alpha = 0.95$ a set of sixteen points has eight tenths of a point in its tail. Minimising CVaR over it is not minimising a tail measure; it is minimising a quantity whose value is decided by which single scenario happens to be worst.

Table 10.1 is the measurement, taken on the van der Pol oscillator with two uncertain coefficients, forty-one nodes, trapezoidal collocation, and each design scored on 2889 plants that took no part in any design. At sixteen scenarios the CVaR design is worse in the real tail than the design that simply minimised the mean. At forty-eight the tail holds 2.4 of them and the measure earns its name, buying 9.2% off the tail for 19.2% on the mean. Whether that trade is worth making is the modelling decision the measure

exists to express; that it is available at all depends on the scenario budget of Section 10.5, because under the earlier counting this problem carried twenty-two scenarios and was refused at twenty-four.

Scenarios	$\mathbb{E}[J]$ design	CVaR design	
16	4.983	5.238	CVaR is 5.1% worse in the tail
48	5.103	4.635	CVaR is 9.2% better, paying 19.2% on the mean

Table 10.1: Out-of-sample CVaR at $\alpha = 0.95$ of two designs, one minimising the expected cost and one minimising CVaR, at two scenario densities. Van der Pol, forty-one nodes, trapezoidal collocation, scored on 2889 plants inside the set that no design saw. The example is `python/examples/robust_driver_cvar.py`.

The reported value of a risk measure is optimistic, as a sample average over the very scenarios that were optimised should be: 4.410 in sample against 4.635 realised in the forty-eight-scenario row, a gap of 4.8%. The reported value of a risk measure is never the certificate.

10.4 Ancillary feedback

A design of the kind described so far is open loop: one control history, committed before the uncertainty is revealed, serving every plant in the set unaided. That is the conservative choice, and over a long horizon it is an expensive one. On the two-link arm it costs a factor of about three in final time.

Setting a gain K turns the design into a tube,

$$u_k(t) = \bar{u}(t) + K(x_k(t) - x_{\text{ref}}(t)), \quad (10.2)$$

in which $\bar{u}$ remains the only control decision variable and the reference is scenario 0 of the rule, which is the centre of the set and runs open loop by construction. The design stays a here-and-now design: both $\bar{u}$ and K are fixed before θ is revealed, and nothing adapts to it. What changes is that one history no longer has to serve every plant by itself. Table 10.2 is the measurement on the arm, and both drivers reproduce it to the digits printed there.

Three consequences come with the gain and none of them is optional.

- **The realised control differs from scenario to scenario, so its bounds become path constraints.** A correction added to a control already sitting on its bound saturates, and a saturated correction is not the control that was designed, so the nominal trajectory has to back away from its bounds. Both drivers add those rows, n_c of them per corrected scenario. They are inequalities, so they cost no degrees of freedom, and scenario 0 needs none of them, being the reference.
- **Those rows have to hold between the nodes as well as at them.** Multiple shooting imposes path constraints where the trajectory is sampled (Section 4.5), and the first working version of this feature breached the actuator limits

Design	t_f	Against deterministic	Scenarios to certify
deterministic	3.1432		1
open loop	8.9670	+185.3%	12
tube	3.5583	+13.2%	5

Table 10.2: The price of robustness on the two-link arm with an uncertain payload, open loop and with an ancillary gain. The tube returns 93% of what the open-loop design paid and needs fewer than half as many scenarios to certify, which also relieves the budget of Section 10.5. The example is `python/examples/robust_driver_tube.py`, and the open-loop row of this table is the same design, to the printed precision, as the one Section 10.10 obtains through each driver in turn.

by 1.83×10^{-2} between the nodes while reporting a perfect certificate on the states. Setting `ms_path_samples` to four samples per segment reduced the breach to 8.13×10^{-4} without moving the final time. The quantity is reported either way, as `certificate["control_excess"]` in Python and `spec.control_excess` in C++, and it is reported *separately* from the violation and not added to it, a design that saturates its actuator and a design that misses its target being different faults that want different remedies. A design whose corrections quietly ask for more actuator than exists is one that no plant can execute, whatever its certificate says about the states.

- **The verification has to integrate the closed loop**, carrying the reference as an extra trajectory of the same sweep. This assumes that the controller regenerates the reference by integrating the nominal model, and not that it stores the reference at the design’s node spacing. An open-loop verification of a closed-loop design checks a controller nobody is going to build. The parameter of that reference is scenario 0 of the starting rule, fixed once and never moved as scenarios are added, and it is reported as `spec.reference_parameter`: a reference that moved with the loop would make each iteration’s certificate a statement about a different controller.

A fourth consequence is less visible and matters as much. The *warm start* has to integrate the closed loop too. The loop follows one chain of warm starts through a nonconvex problem and the chain decides which local minimum is reached (Section 10.10.2), so a guess built from an open-loop integration would start each solve from a trajectory the design’s own controller does not produce. Both drivers march the reference beside each plant in the same sweep, which is why the two agree on the tube rows of Table 10.2.

10.4.1 About the gain

Four kinds of gain are accepted, and the open loop is a fifth. A given constant gain is an $n_c \times n_s$ array. A given schedule is $K(t)$. A co-designed constant gain has its entries optimised alongside the trajectory as static parameters. A co-designed schedule

is optimised as extra *controls*, which is the natural representation because the transcription already gives a control a time profile at the resolution the trajectory has. In Python they are the value given to `.feedback`: an array, a callable of t , "co-design" or "co-design-schedule". In C++ they are the value given to `model.feedback_kind`:

ROBUST_FEEDBACK_NONE	// open loop, the default
ROBUST_FEEDBACK_CONSTANT	// the gain in <code>model.feedback_gain</code>
ROBUST_FEEDBACK_SCHEDULED	// $K(t)$ from <code>model.feedback_schedule</code>
ROBUST_FEEDBACK_CODESIGN	// its entries become static parameters
ROBUST_FEEDBACK_CODESIGN_SCHEDULE	// its entries become controls

The two co-designed forms require bounds on the gain's entries and accept a starting gain, and both drivers refuse a co-design that has no bounds: the gain is then a decision variable that nothing confines, and an unconfined one runs to saturation, where the realised control is not the control that was designed.

A given schedule is where the two interfaces differ, and C++ has the advantage. The Python front end emits the maths as C++ for the tape, and its emitter refuses a branch on a symbolic value, so every form of interpolation is out and a Python schedule has to be a polynomial in t . On the arm the Riccati gain needs degree nine to fit to 3%, degree three misses by a factor of three, and at degree nine the first solve does not converge. A C++ schedule is an ordinary `adouble` function of time, written as

<code>void schedule(adouble& time, adouble* gain, void* user_data);</code>
--

so an interpolant through the conditional helpers of Section 7.11.21, a table lookup, or anything else that tapes may be used, and the polynomial restriction does not apply. The Python remedy for it is "co-design-schedule", a gain carried as extra controls needing no basis and no branch.

Co-design, and why it cannot be relied on. The measurements point both ways and the distinction matters, so both are given. Most co-designed variants tried on the arm, constant and scheduled, with bounds from wide to a box around a gain known to certify, on scenario sets of three, nine and sixteen points, came out cheaper on the objective than the given LQR gain and *failed* their certificate, by between one and five orders of magnitude against a slack of 10^{-3} . Against that, one run of the C++ driver on the same problem, in a box of ± 5 started from that LQR gain, produced a co-designed constant gain that certifies: $t_f = 3.4210$ against 3.5583 for the given gain, no violation anywhere in the set, and a realised control 6.4×10^{-4} outside its bounds. Three integrators written independently of one another agree on that verdict, one of them the Python driver's, which passes the C++ design when it is handed it; the Python solve on the same data reaches a different local minimum, at almost the same objective, which misses by 6.1×10^{-2} . The two drivers therefore disagree about the *solve* and not about the verification.

So a co-designed gain sometimes certifies, and whether it does is a property of the local minimum a solution happens to reach and not of the formulation. What makes

it unreliable is structural and not numerical. A scenario set enters the design as a constraint set, so a co-designed gain is rewarded for making those M plants cheap and is charged nothing for what it does to the rest of the family. A gain multiplies a deviation that is itself a function of the uncertain parameter, so its leverage on an unsampled plant is bounded by nothing the design can see. Neither a denser scenario set nor a tighter bound repairs that. A gain chosen by a criterion that quantifies over the whole family, which is what solving a Riccati equation does, carries no such hazard, and that is the gain to reach for first.

Both drivers therefore offer co-design and then measure what it did. The number of gain entries that finished on their bound is reported, as `gain_on_bound` in the Python certificate and `spec.gain_on_bound` in C++, together with the largest entry of the gain: an entry held at its bound is being chosen by the bound and not by the problem, and the C++ driver prints the largest entry whatever the count, because the count's tolerance is a millionth of the box and the gain quoted above sits at 99.995% of its own. Python also reports `integrator_drift`, which comes out as large as the violation itself on a co-designed design, because an aggressive gain makes the closed loop stiff and a step count adequate for the nominal problem is not adequate for that. The helper `riccati_gain(out, Q, R)` is provided as a diagnostic and not as a design tool: it sweeps the Riccati equation backwards along a design and reports how much the ideal gain varies, together with the error of fitting it as a polynomial in t .

The practical reading is short. Co-design is worth trying, its cheaper objective means nothing until it has been verified, and the verification is where the decision is made.

10.5 How many scenarios will fit

The augmented problem grows its state with M and does not grow its control, so there is a scenario count at which the transcription runs out of freedom. The count is

$$\begin{aligned} \text{dof} = & \text{free initial states} + n_c N + \text{free time endpoints} \\ & + \text{free static parameters} - \text{equality events}, \end{aligned} \tag{10.3}$$

where N is the number of stored nodes, and the important feature of it is a cancellation. A scenario brings n_s state values at t_0 that the defect equations do not determine, and n_s pinned initial conditions that determine them. Those cancel, so a scenario is free of itself. What costs a degree of freedom is an equality event *beyond* a scenario's initial condition, and a pinned terminal state above all, which one open-loop control cannot meet for several different plants in any case. Relaxing such an event to a tolerance costs nothing at all, an inequality event taking no degree of freedom.

Equation (10.3) is a lower bound, because an equality event can be linearly dependent on the defect equations and a dependent constraint removes no freedom. A non-positive count therefore does not prove that a problem is over-determined. Both drivers use it to *explain* a failure and never to refuse in advance. Left to IPOPT the failure arrives as `Not_Enough_Degrees_Of_Freedom`, return code -10 , after the whole problem has been assembled and taped, and the message names no remedy.

There is one exception and it is real. A transcription that collocates *every* stored node holds one defect condition per stored state value, and the initial condition is one more, so a replicated state costs it n_s per copy whatever the mesh. The global Lobatto schemes, **Legendre** and **Chebyshev**, are in that position. Every other transcription in PSOPT fills fewer defect rows than the collocation layout allocates and writes zeros into the rest, and those zero rows, whose bounds are $[0, 0]$, are counted by IPOPT as equality constraints against the variables. On a scenario-augmented problem the phantom deficit is $n_s M$ and it grows with the scenario count. This is the subject of Section 6.1, which should be read with this one; the option `algorithm.free_padded_defect_rows` frees those rows, it is off by default in PSOPT, and both robust drivers turn it on, the phantom wall being the whole of what limited the scenario count. Table 10.3 is the measurement.

Transcription	Rows counted	Rows freed
multiple shooting, RK4 $\times$ 12, linear	refused above 12	80 scenarios, 305 s
trapezoidal	refused above 12	80 scenarios, 125 s
Hermite-Simpson	refused above 20	80 scenarios, 97 s
Legendre	refused above 12	refused above 12

Table 10.3: Scenarios carried on the two-link arm at twenty-five nodes, one cold solve per cell, with the padded defect rows counted and freed. Hermite-Simpson reached twenty before because its midpoint controls pay for the phantom rows. The **Legendre** row is the genuine deficit of a scheme that collocates every stored node, and no node count rescues it. Trapezoidal collocation is the cheap choice for a large set.

Neither driver refuses a global Lobatto scheme, since a small scenario set on one is perfectly sound. The Python driver warns when it is given one, naming the deficit and the two transcriptions that do not have it, and when a solution does fail it prints the arithmetic of (10.3) with the count per scenario and the remedy that fits. In C++, `validate` notes that `free_padded_defect_rows` has no effect under one of those schemes, so that a user who set the option to escape a refusal is told why nothing changed, and `robust_dof_message` supplies the same explanation as the Python one.

10.6 More than one phase

A robust problem may have any number of phases in either driver. What a phase boundary means when the plant is uncertain is a question about the formulation before it is a question about an interface, and the answer imposes one restriction that has to be stated first, because nothing in either implementation can lift it.

A phase boundary time is shared by every scenario. PSOPT gives a phase one t_0 , one t_f and one control, so letting scenario k switch at its own time would need M separate chains of phases, and those cannot share a control grid. The shared control grid is what makes the design non-anticipative, in the sense of Section 10.1.1, so per-scenario switching and the property the whole formulation rests on cannot both be had. A switching time is therefore a here-and-now decision of exactly the kind the control

is: committed before θ is revealed, optimised over the whole scenario set, and the same number for every plant. That is consistent with (10.1), where t_f is already such a decision, and it excludes a boundary triggered by a state event, which is a different problem and not this one.

What the augmentation does at a boundary follows from the replacement of Section 10.1.2. A linkage between two phases is a statement about one trajectory, so the augmented problem needs one copy of it per scenario, and the time is carried across once, being shared. A boundary therefore contributes $Mn_s + 1$ equality rows, plus one per static parameter that is tied across it. Those rows are equalities of the whole problem and belong to no phase, so the scenario budget of Section 10.5 is a sum over the phases with them subtracted:

$$\text{dof} = \sum_{p=1}^P \text{dof}_p - \sum_{\text{boundaries}} (Mn_s + 1 + n_p), \quad (10.4)$$

where dof_p is (10.3) evaluated on phase p of the augmented problem and n_p is the number of static parameters. A count that had not subtracted the linkage rows would report freedom the problem does not have, which is the direction that lets a starved solve through, and it is the reason the arithmetic is written down rather than taken as the first phase's.

The verification has to cross the boundary too, and that needs one thing from the user on one route. Both drivers integrate each scenario from its initial state through the designed control; at a boundary the next phase's starting state is fixed by the linkage, which the library cannot in general solve. Continuity, $x(t_0^+) = x(t_f^-)$, covers most multi-phase problems and is what both drivers assume when nothing says otherwise. Where the linkage is not continuity, a staged launch vehicle dropping a mass being the usual example, the same statement has to be supplied in numbers as well as in tape maths, and the C++ driver refuses a declared linkage without it: carrying the state across unchanged would verify a trajectory the design does not have, and it would do so silently. A verifier that stopped at the first boundary would be certifying part of a design, which is the same failure in a more obvious form.

What each driver does not yet reach at a boundary is listed in Section 10.11. The two interfaces are Sections 10.8.3 and 10.9, and the worked measurement is in Section 10.10.1: on a two-phase problem the two drivers return the same design, the same certificate and the same worst parameter, which is the strongest cross-check this chapter has.

10.7 Verification

A robust design is only as good as the check on it, and the check has to be independent of the transcription that produced the design. Checking a design against the integrator that produced it checks nothing.

Both drivers build a verifier out of the user's own equations and integrate them separately, and in both the user may replace it with one of their own. What that buys

needs stating precisely, because the word independent is doing two jobs. A verifier built by the library is independent of the *transcription*: its own integrator, its own step control, its own code path, sharing with the solve only the equations the user wrote. It is not independent of the *library*. Supplying `spec.violation` in C++, or writing the whole loop out by hand as `python/examples/robust_arm.py` does, verifying with NumPy code that shares nothing with the driver, is the stronger claim, and each driver reports which of the two produced its certificate.

The Python verification works at two levels, because the two requirements differ. The search wants the violation at thousands of parameters, and uses a fixed-step RK4 evaluated at every candidate simultaneously through CasADi's `map`, so that a sweep over five hundred parameters costs about what a sweep over one costs. The reported numbers want a handful of values to be right, and use SciPy's adaptive DOP853 at tolerances far tighter than the design's: a different method, a different step control and a different code path. The disagreement between the two is measured at the certificate point and at every design scenario and reported as `certificate["integrator_drift"]`. If that approached the slack, the certificate would be about the wrong trajectory.

Both levels cross a phase boundary themselves, by the rule of Section 10.6, so the drift still measures what it is for on a problem of several phases. It has to. A drift measured over the first phase of three would say nothing about the step count over the other two, and a boundary is where a fixed step is most likely to be wrong, the node spacing there being whatever the two meshes happen to leave. On the two-phase problem of Section 10.10.1 the two integrators agree to 5.3×10^{-12} against a slack of 10^{-3} .

10.7.1 What control the verifier integrates

A transcription decides where the control is a decision variable, and it also decides what the control *is* between those points. A verification integrator has to use the same reading or it is measuring a different controller. Table 10.4 states the reading each transcription means.

Transcription	Reading	Exact?
multiple shooting, "constant"	held across the segment	by construction
multiple shooting, "linear"	the ramp the integrator used	by construction
multiple shooting, "quadratic"	the parabola through node, midpoint, node	by construction
Hermite-Simpson	the same parabola	by construction
trapezoidal	straight lines	by convention
Legendre, Chebyshev	straight lines	no

Table 10.4: How each driver's verifier reads a design's control between the nodes. The trapezoidal entry is exact by convention rather than by construction, straight lines being that scheme's own quadrature assumption and the best-defined reading available for it. The global Lobatto entry reads a polynomial of degree N as a sequence of chords, and the driver says so.

The midpoint values come from `solution.controls_full`, the complete control history, which PSOPT fills for exactly the two discretisations that carry a midpoint control (Section 7.2.7). The driver checks that its even columns are the nodal table before trusting them, and falls back to straight lines with a warning if they are not.

What the reading is worth is in Table 10.5: one design per transcription on the arm, each verified twice, once with the matched reading and once with the blanket straight line.

Transcription	Matched	As a line	Ratio
multiple shooting, "constant"	1.511	3.475	2.3
multiple shooting, "linear"	1.380	1.380	1.0
multiple shooting, "quadratic"	2.947	1.843	0.6
Hermite-Simpson	8.301×10^{-2}	2.631	31.7
trapezoidal	2.827	2.827	1.0

Table 10.5: Worst terminal miss over 601 payloads for one design per transcription, the arm at twenty-five nodes with five sigma points, verified with the matched reading and with a straight line. The two rows at 1.0 are the check that the matched reading is a no-op wherever the straight line was already right.

Two entries repay a closer look. Hermite-Simpson designs read as chords are charged about thirty-two times the violation they have. More importantly, the quadratic row is below one: reading a parabola as a chord *flatters* a design as readily as it damns one, the parabola lying outside the chord on one side, so the blanket straight line was not conservatively pessimistic. A verifier that can flatter a design is worse than one that penalises it.

10.7.2 What a certificate is

The worst-case search seeds with the boundary points and a low-discrepancy sequence and then refines with a shrinking cloud around the best few candidates. It uses no derivative and must not want one, because the violation is a maximum over constraints and over time and is not differentiable where the active constraint changes, which is exactly where the worst case tends to sit.

For one uncertain parameter the seeding alone is effectively exhaustive. In more than one it is not, and nothing in either driver claims otherwise. What is reported is the worst violation that was *found*, over a number of evaluations that can be budgeted in advance with `robust_worst_case_evaluations`, and the inner problem is not concave. The reports say “nothing worse was found” rather than “nothing worse exists”. On the one-parameter arm the search agrees with a dense scan of 4001 payloads to 1.4×10^{-10} , at the same payload; for more than one parameter no such check exists, which is the reason for the wording.

10.8 The C++ driver

The C++ interface is provided by one header, included in addition to `psopt.h`:

```
#include "robust.h"
```

`psopt_solve_robust` sits in the same architectural slot as `psopt_solve_integer` of Section 9.3, namely an outer loop around an unmodified solver. It offers two routes, and the choice is about how much of the augmentation the caller wants to write.

The first route is to write it. The caller supplies `spec.setup`, which builds the augmented problem for a given scenario list, and `spec.violation`, which says how badly a design serves one parameter vector. It leaves every part of the construction visible and changeable, and it is the route to take for a problem the second route cannot reach, which is one whose nominal equations cannot be evaluated numerically: a `dae` reading `get_delayed_state` or `get_interpolated_state`, for instance. `python/examples/robust_arm.py` is that route written out in full, in Python, and reads as the C++ would.

The second route is to state the *nominal* problem once, as a `RobustModel`, and let the driver build the augmented one. Section 10.8.3 describes it. On the arm it is the difference between 317 and 163 lines of code, of which 87 are the problem itself and the rest is the example's own printed comparison.

10.8.1 The uncertainty and the building blocks

An uncertainty is a `RobustUncertainty`, built by one of three functions:

```
RobustUncertainty robust_gaussian(const RowVectorXd& mean, const MatrixXd&
    ↪ cov,
                                double truncate);
RobustUncertainty robust_uniform(const RowVectorXd& lo, const RowVectorXd& hi
    ↪ );
RobustUncertainty robust_explicit(const std::vector<RowVectorXd>& points);
```

The remaining building blocks are pure functions, separately testable, and usable on their own:

```
int    robust_dimension(const RobustUncertainty& U);
double robust_set_coverage(double k, int n);      // chi-square at k^2 on n d
    ↪ .o.f.
double robust_norm_ppf(double p);                // inverse standard normal
    ↪ CDF
bool    robust_sigma_points(const RobustUncertainty& U,
    ↪ std::vector<RowVectorXd>& points, RowVectorXd&
    ↪ weights);
void    robust_low_discrepancy(const RobustUncertainty& U, int m, int skip,
    ↪ std::vector<RowVectorXd>& points);
void    robust_boundary_points(const RobustUncertainty& U,
    ↪ std::vector<RowVectorXd>& points);
```

```

bool      robust_contains(const RobustUncertainty& U, const RowVectorXd&
    ↪ theta);
RowVectorXd robust_clip(const RobustUncertainty& U, const RowVectorXd& theta)
    ↪ ;
double      robust_mahalanobis(const RobustUncertainty& U, const RowVectorXd&
    ↪ theta);

```

`robust_sigma_points` returns `false`, leaving its outputs untouched, when $n > 3$ would make the central weight negative. The scenario budget of Section 10.5 is available directly, as an integer and as a message:

```

bool robust_prefer_cold(double cold_objective, double cold_worst,
    double warm_objective, double warm_worst, double
    ↪ slack);
int  robust_degrees_of_freedom(Prob& problem, int iphase, Alg& algorithm);
void robust_dof_message(Prob& problem, int iphase, int nscenarios, Alg&
    ↪ algorithm,
    char* buffer, size_t buffer_size);
bool robust_guess_is_finite(Prob& problem, int iphase, const char** offender)
    ↪ ;

```

Those two count one phase, which a diagnostic should, because a failure belongs to a phase. Three more answer the question of (10.4) for the whole problem:

```

int robust_problem_degrees_of_freedom(Prob& problem, Alg& algorithm);
int robust_linkage_equalities(Prob& problem);
int robust_tightest_phase(Prob& problem, Alg& algorithm);

```

`robust_linkage_equalities` reads the linkage bounds when the caller has set them and takes all `problem.nlinkages` of them as equalities otherwise, which is what `auto_link` builds. `robust_tightest_phase` names the phase with the least freedom, which is the one whose arithmetic a failure should print.

`robust_guess_is_finite` earns its own check. A guess built by integrating a previous design can diverge, because the horizon it integrates over is a decision variable and a robust final time grows as scenarios are added, so a step size that was ample at the start need not be by the end. The infinities then reach the constraint evaluation as NaN, some rows are never written, and PSOPT's constraint-coverage check (Section 7.13.1) reports a defect in the library. Checking first turns that into one sentence naming the offending array.

10.8.2 The design, the oracle and the driver

A solution is kept in a `RobustDesign`, which carries the parts of a `Sol` that the outer loop and the user need. It exists because `Sol` owns raw arrays and defines no copy constructor, so it cannot be held from one iteration to the next:

```

struct RobustPhaseTrajectory {
    MatrixXd time;          // 1 x N

```

```

MatrixXd controls;      // ncontrols x N, the nodal table
MatrixXd states;        // nstates x N, all scenario copies as solved
MatrixXd controls_full; // the complete history, midpoints included, or
↪ empty
MatrixXd time_full;     // its time, or empty
};

struct RobustDesign {
    // The FIRST PHASE, and the whole design when there is only one phase:
    MatrixXd time;        // 1 x N
    MatrixXd controls;    // ncontrols x N, the nodal table
    MatrixXd states;      // nstates x N, all scenario copies as solved
    MatrixXd controls_full; // the complete history, midpoints included, or
↪ empty
    MatrixXd time_full;   // its time, or empty

    MatrixXd parameters;  // static parameters, shared by every phase
    double   objective;
    bool     valid;

    std::vector<RobustPhaseTrajectory> phase; // every phase, in order
    int nphases() const;
};

```

The five spelled without a phase index are the first phase and nothing else, so a single-phase problem, which is most of them, reads exactly as it did before the driver learned about phases, and a `violation` function written for one never has to know that `phase` exists. A violation function written for a problem of several phases reads `phase` and should check its size: a design that carried only the first would otherwise be verified as though the rest of the trajectory were not there.

The worst-case search is available on its own, against any violation function of the signature `double (*)(const RowVectorXd&, void*)`:

```

double robust_worst_case(const RobustUncertainty& U,
                        RobustViolationFn violation, void* user_data,
                        int n_seed, int n_refine, unsigned seed,
                        RowVectorXd& worst_at);
long   robust_worst_case_evaluations(const RobustUncertainty& U, int n_seed,
                                    int n_refine);

```

The driver is configured through a `RobustSpec`, whose input fields are the uncertainty, the allowance `slack`, the loop limits `max_iterations` (default 12), `n_seed` (default 128) and `n_refine` (default 3), the `seed` for reproducibility, `verbose`, the risk measure `risk` with its `cvar_alpha` and `mv_lambda` (Section 10.3), the scenario `weights`, and two function pointers:

```

void   (*setup)(Prob& problem, Alg& algorithm,
                const std::vector<RowVectorXd>& scenarios,
                const RobustDesign& previous, void* user_data);

```

```
double (*violation)(const RowVectorXd& theta, const RobustDesign& design,
                    void* user_data);
void* user_data;
```

`setup` is called before every solve, with the previous solution when there is one. It does the level 1 and level 2 setup, sizes the phase for `scenarios.size()` copies of the state, sets the bounds including the inward tightening, builds the warm start, and points `problem.user_data` at whatever the augmented `dae` and `events` need in order to read the scenario list. `violation` returns the violation of the current design at one parameter vector, by the user's own independent integrator: zero when every declared constraint holds, and otherwise the amount by which the worst of them is exceeded. It is required on this route and optional on the model route of Section 10.8.3, where leaving it null selects the library's own.

`slack` is the violation *of the bounds the user declared* that a design may leave. It is not the constraint. A terminal ball of radius 0.02 belongs in the event bounds, and `slack` is how far outside that ball the design may still stray somewhere in the set; passing the ball's own radius here as well would quietly double it.

Two further fields control the polish step of Section 10.10.2 and the driver's single change to the caller's Alg:

```
bool polish; // default true
```

With `polish` set, the driver solves the final scenario set once more after the loop stops, calling `setup` with a default-constructed `RobustDesign` as previous so that it builds its cold guess, and keeps that design if `robust_prefer_cold` prefers it. The second field is

```
bool keep_padded_defect_rows; // default false
```

`psopt_solve_robust` sets `algorithm.free_padded_defect_rows` because the rows it frees are n_s per scenario of counted equalities holding no dynamics, and they are the whole of what caps the scenario count (Section 10.5). Setting this field keeps them, with the consequence stated there.

The driver is then called in place of `psopt`:

```
[[nodiscard]] int psopt_solve_robust(Sol& solution, RobustSpec& spec,
                                    Prob& problem, Alg& algorithm);
```

It starts from the unscented set, or from `spec.scenarios` if the caller has filled it, and then solves, searches, adds and re-solves as described in Section 10.1.3. The return value is the `psopt` return code of the design left in `solution`. On return, `spec` also carries its report: the final `scenarios`, the `design` itself, the `certificate` and `certificate_at`, the number of `evaluations` that went into it, `n_solves`, `converged`, `own_verifier`, and `budget_exhausted`, which distinguishes a loop that stopped because the design was good enough from one that stopped because the transcription could not carry another scenario. Under an ancillary gain it carries four more: `control_excess`, what the corrections asked of the actuator (Section 10.4); `reference_parameter`, the parameter

of the reference trajectory; and `gain_on_bound`, `gain_entries` and `gain_norm` for a gain the driver designed and not one the caller gave.

10.8.3 The nominal model, and what the driver builds from it

Writing the augmentation by hand is most of the work, and not where it is expected to be. Of the 317 code lines the arm example took before this interface existed, 173 were scenario boilerplate, and they divided as eight for the `dae` loop, thirteen for the `events` loop, fifty-one to size the phase and replicate the bounds, forty-nine for the warm start, seven of plumbing, and forty-five for the verification integrator.

A `RobustModel` is the nominal problem stated once, with the uncertain parameter as one extra argument:

```
void nominal_dae(adouble* derivatives, adouble* path, adouble* states,
                adouble* controls, adouble* parameters, adouble& time,
                const double* theta, int ntheta,
                adouble* xad, int iphase, Workspace* workspace);
void nominal_events(adouble* e, adouble* initial_states, adouble*
    ↪ final_states,
                adouble* parameters, adouble& t0, adouble& tf,
                const double* theta, int ntheta,
                adouble* xad, int iphase, Workspace* workspace);
```

The sizes, those two functions, the shared costs, the nominal bounds, the guess, the per-constraint scales and the per-end inward margins of Section 10.1.3 belong to a *phase* and are held in a `RobustPhase`. A `RobustModel` *is* one of those, plus the phases after it and the fields that belong to the whole problem:

```
struct RobustModel : public RobustPhase {
    std::vector<RobustPhase> later_phases;    // empty for a single-phase
    ↪ problem
    RobustLinkFn link;  int nlink;  void* link_data;
    RobustJumpFn jump;  void* jump_data;
    int          nparameters;
    RowVectorXd parameters_lower, parameters_upper, guess_parameters;
    RowVectorXd initial_state;
    RobustInitialStateFn initial_state_fn;  void* initial_state_data;
    double tighten;
    int    verify_substeps, warm_substeps;
    double cost_lower, cost_upper;
    // the six feedback fields below, and configure
};
```

Inheritance is the reason every model written before the driver learned about phases still compiles and still runs: a model is one phase and optionally more, the model itself being the first, and `later_phases` empty is the single-phase problem exactly as it was. The state the verification starts from, the static parameters, the tightening and the substep

counts are the whole problem's, because a trajectory starts once and a scenario is one trajectory through all of the phases.

Then `psopt_solve_robust` takes the model directly:

```
[[nodiscard]] int psopt_solve_robust(Sol& solution, RobustSpec& spec,
                                   RobustModel& model,
                                   Prob& problem, Alg& algorithm);
```

From that the driver installs, for each phase, a `dae` and an `events` that loop that phase's nominal pair over the scenario list, sizes the phase for M copies of the state, replicates its bounds with the tightening applied, replicates the linkages once per scenario, builds the warm start by integrating each scenario's own plant through the previous control and chaining it across every boundary, and verifies the result. `spec.setup` is not called and may be left null; `spec.violation` becomes optional.

The scenario is data. `theta` arrives as `const double*` and not as an `adouble`, because a scenario is not a decision variable and has no business on the derivative tape. The consequence matters more than the tidiness. A function of that shape can be evaluated numerically, outside any taping context, so the library can build the warm start and the verifier out of the equations the user has already written instead of asking for a second copy of the physics by hand. PSOPT already did this internally, in the stationarity residual of Chapter 6, and the model interface exposes it:

```
void robust_dae_value(const RobustModel& model, const double* theta, int
    ↪ ntheta,
                    const double* states, const double* controls,
                    const double* parameters, double time,
                    double* derivatives, double* path);
void robust_events_value(const RobustModel& model, const double* theta, int
    ↪ ntheta,
                    const double* initial_states, const double*
    ↪ final_states,
                    const double* parameters, double t0, double tf,
    ↪ double* e);
double robust_integrand_value(const RobustModel& model, const double* states,
    ↪ const double* controls, const double*
    ↪ parameters,
                    double time);
void robust_phase_dae_value(const RobustPhase& phase, const RobustModel&
    ↪ model,
                    const double* theta, int ntheta,
                    const double* states, const double* controls,
                    const double* parameters, double time,
                    double* derivatives, double* path);
```

The first two read the model's own first phase; the last takes any phase of it, the equations of a later phase being that phase's. All are available whether or not the driver is used. One consequence needs stating, which is that the nominal functions

must be pure arithmetic on their arguments: a `dae` that reaches into the tape through `get_delayed_state` or `get_interpolated_state` cannot be evaluated this way, and such a problem writes its own `spec.setup`. The Python driver has the same restriction for the same reason.

How the phases are joined. Leaving `model.link` null joins consecutive phases by continuity, which is what most multi-phase problems want and what Section 10.6 describes: every state of the scenario carried across unchanged, and the time carried across once. Set, it states the linkage for *one* scenario and the driver replicates it:

```
void link(adouble* link, adouble* final_states_prev, adouble& tf_prev,
         adouble* initial_states_next, adouble& t0_next, adouble* parameters
    ↪ ,
         const double* theta, int ntheta, void* user_data);
void jump(const double* theta, int ntheta, int from_phase,
         const double* final_states_prev, double t,
         double* initial_states_next, void* user_data);
```

`link` writes `model.nlink` values the driver holds at zero, once per scenario per boundary. `jump` says the same thing in numbers, for the verifier, which integrates one scenario across the boundary and cannot solve an implicit linkage. It *writes* the next phase's initial state, so there is no sign to get wrong here; the Python `jumps` dictionary of Section 10.9.1 states an offset instead and follows that interface's own sign. Declaring `link` without `jump` is refused, for the reason given in Section 10.6, and the two have to agree: the library has no way of checking that they do, and a disagreement between them is a certificate about a trajectory the design does not have.

An initial state that depends on the parameter. The uncertainty is sometimes in where the plant starts and not in how it behaves. A caller writes that as an ordinary event, $e = x(t_0) - x_0(\theta)$, the nominal `events` receiving θ like any other datum, so the *design* already knows about it. What the library cannot infer from an event is the *value*, and the warm start and the verifier both need it, so there is one function that supplies it:

```
void initial_state_fn(const double* theta, int ntheta, double* x0, void*
    ↪ user_data);
void robust_initial_state(const RobustModel& model, const double* theta, int
    ↪ ntheta,
                        double* x0);
```

Setting it means `model.initial_state` is not read. `robust_initial_state` answers the question in one place, for a caller writing their own verifier, which has to start where the library's would.

Without it the failure is silent, and the direction the error takes deserves notice. On $\dot{x} = -x + u$ with $x(0) = \theta$ over a 3σ set of 0.3, brought to $x = 1$ within 0.02, the design is right either way, at $t_f = 2.8134$, which is the closed form $\ln(0.3/(0.9 \times 0.02))$

this problem happens to have. What differs is the picture the library forms of it. With the function set the loop stops at five scenarios with a certificate of zero, which an integrator written independently of the library confirms. Without it the loop runs to twelve scenarios chasing a violation of 3.0×10^{-1} that the same independent integrator puts at zero: the verifier was starting every scenario at the mean while the design held each one to its own starting point. The error is pessimistic there, because the initial condition is itself an event and the verifier charges the design for missing it, and it would be optimistic on a problem where the initial condition is not an event. What is common to the two directions is that the certificate is not about the design.

Where the algorithm options go. In `model.configure`, a callback the driver runs immediately after every `psopt_level2_setup`, and not in `main`. `psopt_level2_setup` resets every field of the `Alg` it is given to that field's default, which is why a hand-written `setup` assigns its options after calling it, and the generated one has to offer the same moment. Options set before `psopt_solve_robust` is called are overwritten and lost.

A risk measure over the model. Setting `spec.risk` to any measure but the nominal one makes the driver build the objective of Section 10.3 as well. For the two measures that carry each scenario's cost as a state, that means M extra states whose derivatives are the nominal integrand at their own scenario, M extra events pinning them to zero at t_0 , and, under CVaR, one η and one slack static parameter per scenario with the Rockafellar–Uryasev row that relates them. None of it is the caller's to write; what the caller supplies is the range the per-scenario cost lies in, as `model.cost_lower` and `model.cost_upper`. The sizes the augmented problem takes are available in advance, so that a scenario count can be costed before it is tried:

```
void robust_augmented_sizes(const RobustModel& model, int M, RobustRisk risk,
                           int& nstates, int& ncontrols, int& nevents,
                           int& npath, int& nparameters);
void robust_augmented_phase_sizes(const RobustModel& model, const RobustPhase
    ↪ & phase,
                                int M, RobustRisk risk,
                                int& nstates, int& ncontrols, int& nevents,
                                int& npath, int& nparameters);
```

The first is the second called on the model's own first phase, which is the whole of a single-phase problem.

An ancillary gain over the model. The gain of Section 10.4.1 is declared by six fields of the model:

```
RobustFeedbackKind feedback_kind;           // which of the five kinds
MatrixXd feedback_gain;                     // ncontrols x nstates, for
    ↪ CONSTANT
RobustGainFn feedback_schedule;              // for SCHEDULED
void* feedback_data;                         // passed to the schedule
```

MatrixXd	feedback_lower;	// a co-design's box, one number
↪ for the		
MatrixXd	feedback_upper;	// whole matrix or one entry per
↪ entry		
MatrixXd	feedback_guess;	// a gain to start a co-design from

Everything that follows from (10.2) is then the driver's: the correction inside the augmented `dae`, the realised control's bounds as path rows, a warm start that integrates the closed loop, and a verification that does the same.

The verifier the driver supplies. Leaving `spec.violation` null selects

double robust_model_violation(const RobustModel& model, Alg& algorithm,	const RowVectorXd& theta, const RobustDesign&
↪ design,	
	const RowVectorXd* theta_ref = 0,
	double* control_excess = 0);

which integrates the plant from the initial state of `robust_initial_state` with a fixed-step RK4 at `model.verify_substeps` steps per node interval, phase by phase and across every boundary by the rule of Section 10.6, reads the control the way the transcription means it (Table 10.4), samples the path constraints at every substep boundary, and returns the largest scaled amount by which the trajectory falls outside the bounds the user *declared*. The declared bounds are the point: the tightening of Section 10.1.3 is a margin for the design and not a change to what is being verified. `model.event_scale` and `model.path_scale` keep a metre and a radian from being added together.

The last two arguments are for a design with a gain and both default, so code written against the open-loop form keeps compiling. `theta_ref` is the parameter of the reference trajectory, which the verifier integrates beside the plant because the correction is a deviation from it; passing null with a gain in place makes the plant its own reference, which verifies the open loop and not the design. `control_excess` receives what the corrections asked of the actuator. The driver fills both from `spec` and needs no help.

What independence this gives is stated in Section 10.7 and is repeated here because it is the one thing to be clear-eyed about: the supplied verifier is independent of the transcription and not of the library. `spec.own_verifier` records which one produced the certificate, and the driver prints it.

10.8.4 Worked example

`examples/robust_driver` is the two-link arm stated as a `RobustModel` and handed to the driver, and it shows what is left for the user to write once the library owns the augmentation: 87 code lines are the problem itself, against the 317 the same example took when the augmentation, the warm start and the verification integrator were the user's. It is documented in the PSOPT Application Examples Document.

The arm itself is the one of Weinreb and Bryson [48], whose Section IV carries the tip mass symbolically as the ratio $\mu = M/m$, and the payload used here is their $\mu = 1 + m_p$.

The uncertain model is therefore theirs and not a reconstruction, which matters because it is the one thing in the example that everything else is built on.

With a payload $m_p \sim \mathcal{N}(0.5, 0.15^2)$ over the 3σ set, a terminal ball of 0.03, a slack of 10^{-3} and twenty-five nodes, it reaches twelve scenarios in eleven calls to `psopt` in under thirty seconds, taking t_f from 3.1432 for the nominal design to 7.6387 for the robust one, a factor of 2.43. The last of those eleven calls is the polish step of Section 10.10.2, and on this problem it is worth fifteen per cent. It then checks its certificate against a dense scan of 4001 payloads, which for one uncertain parameter is exhaustive to its resolution.

Two further measurements in that example are about the method and not about the interface, and are the reason it is a study as well as a demonstration. It solves the same problem on *fixed* scenario sets at the Gauss-Hermite nodes of the payload distribution, $M = 1, 3, 5, 7$ and 9, each given the full tolerance and no inward margin, and scores every design over the whole set: the in-sample miss is the tolerance by construction at every M , the worst miss over the set never once falls below it, and the fraction of a thousand sampled payloads that arrive within the tolerance runs 0, 10, 4, 51 and 36 per cent, which is no trend at all, because where the next quadrature node lands has nothing to do with where the previous design was failing. Quadrature discretises an expectation; a constraint that must hold everywhere needs its own worst case in the loop, which is what Section 10.1.3 is. And it solves the final scenario set at eight and at sixteen steps per segment, reporting the terminal miss as the transcription reports it beside the miss the verification integrator finds: at eight steps they differ by 8.1×10^{-4} on a tolerance of 0.03, and at sixteen by 8.5×10^{-8} . The lesson is the last item of Section 10.11.

10.9 The Python driver

The Python driver is `psopt.robust`, and it builds the augmented problem itself. The user states the *nominal* problem once, with the uncertain parameter vector as one extra argument to the dynamics, the path constraints and the events, and the driver replicates the equations, the bounds and the guess.

```
from psopt.robust import RobustProblem, Gaussian

rp = RobustProblem(name="arm")
ph = rp.add_phase(nstates=4, ncontrols=2, nevents=8)
ph.nodes        = [25]
ph.dynamics     = lambda x, u, p, t, th: ...      # th is the uncertain
               ↪ parameter
ph.events       = lambda xi, xf, p, t0, tf, th: ...
# ... bounds and guess, at the NOMINAL size ...

rp.uncertainty  = Gaussian(mean=[0.5], cov=[[0.15 ** 2]], truncate=3.0)
rp.initial_state = [0.0, 0.0, 0.5, 0.0]

out = rp.solve(alg, slack=1e-3, risk="expectation", generate=True)
out.report()
```

A `RobustPhase` is a `psopt.Phase` (Section 11.4) with two differences. Its `dynamics`, `path` and `events` take the uncertain parameter vector as a final argument, while `integrand` and `endpoint` do not, being shared across the scenarios. And its bounds and guess are stated at the nominal size, the driver replicating them. It carries three further attributes: `cost_bounds`, required by the risk measures that carry a cost state (Section 10.3); `feedback`, `feedback_bounds` and `feedback_guess` (Section 10.4); each of which may equivalently be set on the `RobustProblem`. On the problem itself, `uncertainty` and `initial_state` are required. The initial state is asked for and not inferred, because the driver cannot in general read $x(t_0)$ off the event bounds and a guess there would put a silent error in the one place the whole verification rests on; it may be an array or a callable of θ . The optional `event_scale` and `path_scale` give per-constraint scale factors for the violation measure, so that a metre and a radian are not added together; left unset the measure is the raw infinity norm, which is right only when the constraints share units.

The arguments of `solve` are listed in Table 10.6.

Argument	Default	Meaning
<code>algorithm</code>	<code>Algorithm()</code>	the solver options, as in Section 11.7
<code>slack</code>	0.0	violation of the declared bounds a design may leave
<code>risk</code>	"nominal"	the risk measure (Section 10.3)
<code>cvar_alpha</code>	0.9	the CVaR level
<code>mv_lambda</code>	1.0	the weight on the variance in mean-variance
<code>scenarios</code>	"sigma-points"	the rule: also "qmc" or "explicit"
<code>n_scenarios</code>	None	how many, for "qmc"
<code>generate</code>	True	add the worst parameter found and re-solve
<code>max_iterations</code>	12	generation iterations
<code>tighten</code>	0.9	inward tightening, as a fraction of each half-width
<code>margin</code>	None	inward margins, by family or by end; overrides <code>tighten</code>
<code>n_seed</code>	None	search seeds; 128 in one dimension, $64 \cdot 2^n$ beyond
<code>n_refine</code>	3	cloud centres kept during refinement
<code>out_of_sample</code>	1000	parameters drawn for scoring, taking no part in the design
<code>wait_and_see</code>	0	independent solves averaged for the lower bound
<code>seed</code>	20260927	reproducibility
<code>nsub</code>	16	fixed steps per node interval in the search integrator
<code>polish</code>	True	re-solve the final set from the user's own guess
<code>verbose</code>	True	print the iteration table

Table 10.6: The arguments of `RobustProblem.solve`.

What comes back is a `RobustSolution`. Its `design` is the `psopt.Solution` of the final augmented solve, with `time`, `controls` and `objective` lifted out of it for convenience, and `scenarios` is the set it was built on. `time` and `controls` are the first phase, which is the whole design when there is one phase; `phase_time` and `phase_controls` are lists holding every phase in order. Three dictionaries carry the evidence: `certificate`,

the worst violation found anywhere in the set and where, over how many evaluations, together with the integrator disagreement and, where they apply, `control_excess` and `gain_on_bound`; `out_of_sample`, the scoring on parameters that took no part in the design, inside the set and outside it, with the realised cost distribution; and `history`, one row per outer iteration. `wait_and_see` holds the lower bound when it was asked for, `gain_schedule` a co-designed schedule when there is one, `n_solves` and `n_verifications` the work done, `converged` whether nothing worse than the slack was found, and `budget_exhausted` whether the loop stopped for want of degrees of freedom. The method `out.report()` prints all of it in the order in which it should be read, each number with the sentence that says what it does and does not establish.

The driver inspects its `Algorithm` before starting. It sets `free_padded_defect_rows` unless the caller has set it explicitly, and warns if the caller has turned it off; and it warns when the transcription is a global Lobatto scheme, both because such a scheme cannot carry a large scenario set (Section 10.5) and because the verifier cannot reproduce its control exactly (Section 10.7.1).

10.9.1 Phases

`add_phase` may be called more than once, and the phases run in the order they were added. How they join is stated by

<code>rp.link_phases(1, 2)</code>	# continuity, and the default
<code>rp.link_phases(1, 2, jumps={0: 0.3})</code>	# state 0 DROPS by 0.3 at the
<code>↪ boundary</code>	

The indices in `jumps` are the *nominal* ones, the driver expanding each to every scenario's copy of that state, and the sign is the one `Problem.link_phases` uses throughout the interface: a positive δ is a drop, $x(t_0^b) = x(t_f^a) - \delta$, which is why mass jettison is written with a positive number. Both of the verifiers of Section 10.7 cross the boundary by that same rule. Consecutive phases are joined by continuity when no link is stated at all, so a two-phase problem whose boundary is an ordinary staging point needs no `link_phases` call; the example writes one anyway, so that a reader can see where a jump would go.

The driver then traces each phase's equations in its own scope, sizes each augmented phase for M copies of that phase's state, replicates the linkage once per scenario, chains the warm start across the boundaries, verifies a trajectory that crosses them, and counts the linkage equalities out of the freedom of (10.4). What it refuses, each with a message saying what the increment would be, is in Section 10.11.

10.9.2 Examples

Nine examples ship with the binding, listed in Table 10.7. Each carries a pass criterion that is a statement about the method and not a stored number, so that a change which broke the method would fail the example even if it left the objective looking plausible.

One of them should be singled out. A covariance that somebody chose is the weakest part of a robust design, and PSOPT's own parameter estimation returns one (Sec-

Example	What it establishes
<code>robust_arm.py</code>	the loop written out by hand; the nominal design must fail out of sample
<code>robust_driver_arm.py</code>	the same problem through <code>psopt.robust</code>
<code>robust_driver_vdp.py</code>	two uncertain parameters, a state constraint, an expected cost
<code>robust_driver_risk.py</code>	expectation, mean-variance and CVaR on one problem
<code>robust_driver_estimate.py</code>	estimate the plant, then design against its covariance
<code>robust_driver_tube.py</code>	an ancillary gain against open loop, checked independently
<code>robust_driver_gain.py</code>	who chooses the gain: a measured negative result on co-design
<code>robust_driver_cvar.py</code>	a tail measure on a set dense enough to resolve its tail
<code>robust_driver_twophase.py</code>	two phases joined by a linkage; the design and the certificate must agree with the C++ driver's on the same problem, and a declared jump must cross the boundary the way the solved linkage did

Table 10.7: The robust-design examples distributed with the Python binding. The robust examples are the slow ones, each calling `Problem.solve` once per outer iteration; `robust_driver_gain.py` is the outlier, co-designing a gain.

tion 1.11), reachable from Python as `sol.parameter_statistics.covariance` when `algorithm.parameter_statistics` asked for it. That posterior is the distribution a design ought to run against, and `robust_driver_estimate.py` closes the loop inside one model: estimate two arm parameters from noisy observations of a prescribed manoeuvre, hand the covariance to the driver, and check the result on the true plant, which nothing in the estimate or the design ever saw. The estimated posterior is strongly correlated, and the example shows that discarding the off-diagonal term is not a conservative simplification. It points the uncertainty set along the wrong axes and leaves the design exposed in the one direction the data did not determine.

10.10 The same problem through both drivers

The two-link arm is posed identically for the C++ driver, in `examples/robust_driver`, and for the Python driver, in `python/examples/robust_driver_arm.py`: twenty-five nodes, a terminal ball of $\delta = 0.03$, a slack of 10^{-3} , an inward tightening to 0.9δ , multiple shooting with RK4 at twelve steps a segment and a linear control, and the unscented three-point set to start from. What the two return, in Table 10.8, says something about the method that neither run says on its own.

The deterministic row agrees exactly on both sides, which is the check that the formulation, the discretisation and the verifier are the same: $t_f = 3.1432$, and a worst terminal excess of 1.575×10^{-1} at $m_p = 0.05$ when that design is scored over the whole set. Before the polish step the two loops agree to 0.05%, on the same number of scenarios, both certified well inside the slack. Their scenario sets agree on nine of twelve points

Design	Final M	t_f	Certificate	psopt calls
deterministic, either driver	1	3.1432	1.575×10^{-1}	1
psopt_solve_robust, warm chain	12	8.971289	6.669×10^{-5}	10
psopt_solve_robust, polished	12	7.638675	0	11
psopt.robust	12	8.967018	1.418×10^{-4}	13

Table 10.8: The arm at twenty-five nodes with a terminal ball of 0.03, through each driver in turn. The first two rows are the same run, before and after the polish step of Section 10.10.2. The deterministic row is the same design on both sides and is scored over the whole uncertainty set, which is why it carries a certificate at all.

to three decimals and differ in the other three, because the two worst-case searches seed their low-discrepancy sequences differently, and a scenario added at one iteration changes the warm start of the next. After the polish step they differ by seventeen per cent, and the reason is the subject of the next section.

10.10.1 A two-phase problem, where the two do agree

The arm is a single phase, and the seventeen per cent above is about the method’s sensitivity to which scenarios a search happens to add. A second comparison is worth having because it is the same check on a problem with a phase boundary in it, and because it comes out the other way.

The problem is $\dot{x} = -\theta x + u$ with $\theta \sim \mathcal{N}(1, 0.2^2)$ over the 3σ set, from $x(0) = 0$ to a ball of 0.05 about $x = 1$, in minimum time, transcribed as two phases of eleven nodes joined by continuity, with the boundary time itself a decision variable and therefore, by Section 10.6, one number shared by every scenario. It is deliberately a problem whose answer is known: nothing about the physics changes at the boundary, so the two-phase answer should be the single-phase answer up to the mesh, and two phases of eleven nodes carry twenty-two control values over the horizon where one phase carries eleven. What the second phase buys is the four things a driver has to get right at a boundary, none of which a single-phase problem exercises.

Both drivers return

Driver	Final M	t_f	Certificate	at θ
psopt_solve_robust	4	1.040263	9.352×10^{-4}	0.406
psopt.robust	4	1.040263	9.352×10^{-4}	0.406

to every figure either of them prints, with the boundary landing at $t = 0.1705$. The two share no code. One augments the problem in C++ and verifies with a hand-written RK4; the other emits CasADi expressions from Python, replicates the linkage symbolically once per scenario, and verifies twice, with a vectorised fixed-step sweep and with SciPy’s DOP853. Counted exactly: two independent augmentations, and four integrators sharing no code, the fourth being the NumPy one the example writes for itself, each of them crossing the boundary on its own account.

Agreement here does not repeal the caveat of the next section, and the two facts sit together without difficulty. The loop follows one chain of warm starts through a nonconvex problem, so two loops that generated different scenarios can reach different local minima; on this problem they generated the same ones. What is reproducible is the certificate, and here the objective happens to be reproducible as well.

The example then solves the same problem with a staging event at the boundary, the state dropping by 0.2 there, and the second measurement is about the method and not about the interface. Solving each version nominally as well as robustly gives

Boundary	nominal t_f	robust t_f
continuity	0.383215	1.040263
a drop of 0.2	0.454305	0.980470

so the drop costs the nominal design time, as losing 0.2 of a state that has to reach 1 should, and it *saves* the robust design time. The explanation is in where the robust design goes. Under continuity it takes the state to -0.3135 at the boundary, on the far side of zero from its target, and it does that to buy insensitivity: the uncertainty enters as $-\theta x$, so the sensitivity of the trajectory to θ is proportional to x and changes sign with it, and an excursion through negative x lets the sensitivity accumulated there cancel what is accumulated on the way up. The continuity design pays control effort for that excursion. The staging drop performs part of it for nothing.

The general point is that **the price of robustness is not additive with a physical penalty**, because the two can be paying for the same thing. A penalty has to be measured against the robust design and not against the nominal one, and a designer who costed this staging event on the nominal trajectory would have concluded it was expensive.

One practical note belongs with the measurement. The example asks for five thousand NLP iterations where the others ask for a thousand, and that number is measured. Both drivers set `free_padded_defect_rows`, without which the phantom wall of Section 10.5 refuses a scenario set of any size; on this problem, with two phases and the terminal ball tightened, freeing those rows makes the first solve end in a long flat tail. Feasibility reaches 5×10^{-13} within a few hundred iterations and the objective then creeps down by about 7×10^{-9} an iteration for two thousand more. It converges, to 0.849830918 against 0.849830910 with the rows kept, so the answer is not in question and the iteration count is. At a thousand the same solve stops two parts in a thousand short and the driver correctly calls it a failure. An option that lifts one limit can move where another one binds.

10.10.2 What the objective is, and is not

Anyone comparing those numbers should know what kind of quantity is being compared. What the method controls is the certificate, and every design in Table 10.8 meets it. The objective is not determined by the problem, and the second row shows that it is not determined by the scenario set either.

Solving the C++ run's own final twelve scenarios once more, cold, from the plain initial guess the user already supplies for the first iteration, returns $t_f = 7.638675$ with no violation anywhere in the set. The worst terminal miss over a scan of 4001 payloads, through an integrator that shares nothing with the transcription, is 0.0287 against a declared ball of 0.030, and DOP853 agrees at the worst payload. That design is fifteen per cent faster than the warm chain's and carries a cleaner certificate, for the price of one extra call to `psopt`.

The sequential warm start accounts for it. The loop needs one while the scenario set is small and growing, for the reason given in Section 10.1.3, and it also conditions the answer: the loop follows one homotopy through a nonconvex minimum-time problem and finishes at a local minimum that the final scenario set does not require. This is the same phenomenon the obstacle example of the Python interface reports, where a continuation converges to a genuine local minimum in the wrong homotopy class.

Both drivers now take that step. `spec.polish` in C++ and the `polish` argument in Python, each on by default, re-solve the final scenario set from the user's own guess and keep the cold design when it certifies and is cheaper. The rule is deliberately asymmetric, and in C++ it is the separate function `robust_prefer_cold` so that it can be tested on its own: a cold design that undercuts the warm one on the objective while violating the constraints somewhere in the set is not a design, and it is refused whatever the saving.

That is also why the two drivers end seventeen per cent apart on this problem rather than a fraction of a per cent. On the C++ driver's scenario set the cold solve certifies, and the polish step takes it. On the Python driver's set it does not: that cold solve comes out at 8.606331 with a worst violation of 7.7×10^{-3} , seven times the slack, so the polish step correctly refuses it and the loop returns its warm design at 8.967018. Both answers are certified designs for the same problem. The difference between them is not a difference between the two implementations of the method; it is the method's own sensitivity to which parameters its search happened to add.

Three conclusions follow, and they are the reason this section exists.

- **Read the certificate, not the objective.** A reported objective is an upper bound on what the method can deliver on that problem. It is not the value of the robust problem, and it is not comparable between two runs that generated different scenario sets.
- **A design that matters should be re-solved cold on its final scenario set.** Both drivers now do this. A user who has switched it off, or who is running the loop by hand through `spec.setup`, should do it explicitly.
- **Where the objective genuinely matters, run the loop more than once.** Changing `spec.seed`, or asking for a low-discrepancy starting set of a different size, generates a different scenario set and therefore a different certified design. Keeping the cheapest design that certifies is sound, because the certificate is checked against the whole set and not against the set that produced it.

10.11 Limitations

Not every problem can be made robust open loop. If the sensitivity of a constrained state to the uncertain parameter obeys a linear equation whose homogeneous part the control cannot shape, then that sensitivity is fixed by the boundary conditions and no control reduces it. A linear plant with an uncertain gain and a pinned terminal state is in that position, and so is a kinematic vehicle with an uncertain speed. The example `robust_driver_vdp.py` sets the argument out. It should be checked for before reaching for either driver, because the symptom is a loop that adds scenarios without improving anything.

A certificate is a claim about a search. Section 10.7.2 states this and the reports repeat it. For more than one uncertain parameter the seeding is not exhaustive and the inner problem is not concave.

Scenario-dependent static parameters are not supported. A static parameter is shared by every scenario, as the control is. A parameter that should be chosen after the uncertainty is revealed is a recourse decision, and this version has no recourse.

A phase boundary time cannot depend on the scenario, and no version of this will lift that: it is the non-anticipativity of Section 10.1.1 and not an unwritten increment. Section 10.6 sets out why, and the consequence to remember is that a boundary triggered by a state event is a different problem from the one either driver solves.

Three things do not yet reach a second phase, and each is refused with a message saying what it would take. A risk measure that carries a cost state, which is mean-variance and CVaR, because that state has to be carried across a boundary by the linkage and the Rockafellar–Uryasev rows moved to the final phase; the nominal measure and the expectation work over any number of phases, being sums the transcription already forms per phase. An ancillary gain, because a gain is $n_c \times n_s$ and those belong to a phase, so a multi-phase tube needs one gain per phase and a rule for what happens to the correction at a boundary. And, in Python alone, static parameters of any kind: a static parameter belongs to a phase in PSOPT and the Python front end’s linkages carry the states and the time and nothing else, so each phase would optimise its own copy while the verification integrated one vector across all of them. The C++ driver ties them with a linkage row per parameter per boundary and has no such restriction.

The global Lobatto schemes cannot carry a large scenario set, for the reason given in Section 10.5, and no node count alters that.

The risk measures and the ancillary feedback need the nominal model route in C++. Both are reached from either driver, but in C++ only through a `RobustModel` (Section 10.8.3): they are built into the augmented problem, and a caller who writes that problem through `spec.setup` writes the cost states, the Rockafellar–Uryasev rows or the correction of (10.2) along with it. A problem whose nominal equations cannot be evaluated numerically, one reaching into the tape through `get_delayed_state` for instance, is on that route by necessity and has neither for free.

The objective a loop returns is not reproducible across drivers or seeds. Section 10.10.2 measures seventeen per cent between two certified designs for the same

problem. The certificate is the reproducible part, and it is what a robust design should be judged on.

A co-designed gain cannot be relied on, as measured in Section 10.4.1. It sometimes certifies and usually does not, the difference being the local minimum a solution reaches, so its cheaper objective means nothing until it has been verified. Both drivers offer it and both report where the gain finished.

An accuracy setting validated on the nominal problem has to be validated again on the robust one. This has bitten three separate parts of this work: the segment integrator step count, the path-constraint sampling, and the step size of the warm start's own integration. A robust solution is a different trajectory, usually a longer and gentler one, so a different balance of errors applies to it. The two-integrator check of Section 10.7 exists because of it, and it costs nothing to read.

11. The Python interface

PSOPT provides a Python interface in which an entire optimal-control or estimation problem (its dynamics, cost, path constraints, boundary events, phase linkages and observation functions) is written symbolically in Python using CasADi [1], and solved by the native PSOPT/IPOPT core. The C++ engine is not modified. The Python layer traces the user’s symbolic maths, generates and just-in-time compiles the corresponding derivative code, and drives the existing solver through a compiled extension module. Every problem is therefore solved by exactly the same engine as a hand-written C++ example, and the numerical results match to solver tolerance.

11.1 Overview and when to use it

The Python interface is convenient for rapid prototyping, for users who prefer Python to C++, and for problems whose data are most naturally assembled with NumPy, for example an estimation problem driven by measured samples, or dynamics built from tabulated coefficients. Because each problem is compiled to native code and solved by the unchanged core, there is no interpreted inner loop and no loss of accuracy relative to the C++ route.

The cost of this convenience is a dependency, at run time, on CasADi and a C++ toolchain: the first solve of a given problem compiles its maths to a small shared library (subsequently cached, see Section 11.2). Where PSOPT must be embedded in a larger C++ application, where no compiler is available at run time, or where a feature not yet surfaced in the Python API is required, the C++ interface of Chapter 7 remains the primary route. The two interfaces describe the same problem structure, so a problem prototyped in Python maps directly onto its C++ counterpart.

11.2 How it works

The interface is built from two layers.

- A *driver* (`src/_psopt_driver.cpp`), a pybind11 extension module that constructs the PSOPT `Prob` and `Alg` objects in the required order, registers the user functions, calls the solver, and returns the trajectory as NumPy arrays.

- A *code generator* (`psopt/codegen.py` and `psopt/_emitter.py`). Each problem's CasADi maths is traced to an `adouble`-templated C++ source file, just-in-time compiled to a small mathematics shared library, and cached by a hash of that source. The driver `dlopens` the library and registers its `extern "C"` functions as the PSOPT user-function pointers.

The mathematics library performs `adouble` arithmetic only and links CppAD, so the automatic differentiation used by the solver is identical to that of a hand-written example. PSOPT's own symbols (such as `auto_link`) are resolved at load time: the extension module whole-archives `libPSOPT` and is imported with `RTLD_GLOBAL`, so those symbols are visible to each dynamically loaded mathematics library. This arrangement requires a position-independent `libPSOPT`, the extension to be linked with `--whole-archive` and `--export-dynamic` (`-rdynamic`), and default symbol visibility on the extension. These requirements are handled automatically by the supplied build (Section 11.3); they are noted here because a symbol-resolution error at import or first solve almost always indicates that one of them has been lost.

The just-in-time cache defaults to `$XDG_CACHE_HOME/psopt` and can be redirected with the environment variable `PSOPT_CACHE_DIR`. Because compiled problems are keyed by the hash of their generated source, an unchanged problem is compiled only once across runs.

11.3 Building and installing

The recommended build is in-tree, from the top-level PSOPT build, which makes `libPSOPT` position-independent and builds the binding against it:

```
cmake -S . -B build -DPSOPT_PYTHON=ON
cmake --build build
```

The binding may also be built standalone against a prebuilt position-independent `libPSOPT.a`:

```
cmake -S python -B build/python \
    -DPSOPT_ROOT=<psopt source root> \
    -DPSOPT_LIB=<.../libPSOPT.a> \
    -Dpybind11_DIR=$(python -m pybind11 --cmakedir)
cmake --build build/python
```

In either case CMake builds the extension `psopt/_psopt*.so` and *generates* `psopt/_toolchain.py` (the compiler, flags, include directories and CppAD library used for the just-in-time compilation) from the very configuration used to build PSOPT. There is no hand-maintained, machine-specific toolchain file to keep in step with the installation.

Installation with `pip` drives the same CMake build through `scikit-build-core`:

```
pip install ./python
```

For a standalone install the PSOPT location is passed through, for example
`--config-settings=cmake.define.PSOPT_ROOT=...` and the corresponding `PSOPT_LIB`.

11.4 Defining and solving a problem

A problem is a `psopt.Problem`. Phases are added to it with `add_phase`, which returns a phase object on which the user maths, bounds and initial guess are set. An `Algorithm` object carries the solver options, and `Problem.solve` returns a `Solution`.

The user maths are supplied as Python callables over CasADi symbols. The expected callables and their signatures are listed in Table 11.1; any that are not set default to the natural empty value (no path constraints, a zero cost term, and so on). In every signature `x`, `u` and `p` are the CasADi vectors of the states, controls and static parameters, `t` is the time, and `xi`, `xf` are the initial and final state vectors.

Attribute	Signature	Returns
<code>dynamics</code>	<code>f(x,u,p,t)</code>	state derivative $\dot{x}$ (length <code>nstates</code>)
<code>path</code>	<code>g(x,u,p,t)</code>	path constraint vector (length <code>npath</code>)
<code>integrand</code>	<code>L(x,u,p,t)</code>	Lagrange (running) cost term, scalar
<code>endpoint</code>	<code>phi(xi,xf,p,t0,tf)</code>	Mayer (terminal) cost term, scalar
<code>events</code>	<code>e(xi,xf,p,t0,tf)</code>	boundary-event vector (length <code>nevents</code>)
<code>observation</code>	<code>h(x,u,p,t)</code>	observed outputs (length <code>nobserved</code>)

Table 11.1: The user-maths callables set on a phase. Each is a plain Python function (often a `lambda`) written in CasADi. Only `dynamics` is normally required; the rest are optional and `observation` is used only for estimation (Section 11.6).

Bounds are set through `ph.bounds`: `lower` and `upper` each carry `states`, `controls`, `events`, `path` and `parameters` (Python lists), while `ph.bounds.t0` and `ph.bounds.tf` are (`lower`, `upper`) tuples for the initial and final times. The initial guess is set through `ph.guess` (`states`, `controls`, `time` and `parameters` as NumPy arrays, the state and control guesses shaped as `nstates×N` and `ncontrols×N`), and `ph.nodes` is the list of node counts that seeds the mesh.

The complete single-phase Bryson–Denham problem reads as follows.

```
import numpy as np, casadi as ca
import psopt

prob = psopt.Problem(name="bryson_denham")
ph = prob.add_phase(nstates=3, ncontrols=1, nevents=5)
ph.nodes = [50]

# user maths -- pure CasADi, traced and CppAD-taped under the hood
ph.dynamics = lambda x, u, p, t: ca.vertcat(x[1], u[0], 0.5*u[0]**2)
ph.endpoint = lambda xi, xf, p, t0, tf: xf[2] # minimise x3(
    ↪ tf)
```

```

ph.events = lambda xi, xf, p, t0, tf: ca.vertcat(xi[0], xi[1], xi[2], xf
    ↪ [0], xf[1])

ph.bounds.lower.states = [0.0, -10.0, -10.0]
ph.bounds.upper.states = [1.0/9.0, 10.0, 10.0]
ph.bounds.lower.controls = [-10.0]
ph.bounds.upper.controls = [10.0]
ph.bounds.lower.events = [0.0, 1.0, 0.0, 0.0, -1.0]
ph.bounds.upper.events = [0.0, 1.0, 0.0, 0.0, -1.0]
ph.bounds.t0 = (0.0, 0.0)
ph.bounds.tf = (0.0, 50.0)

x0 = np.zeros((3, 50))
x0[1, :] = np.linspace(1.0, -1.0, 50)
ph.guess.states = x0
ph.guess.controls = np.zeros((1, 50))
ph.guess.time = np.linspace(0.0, 0.5, 50).reshape(1, 50)

alg = psopt.Algorithm(collocation_method="Legendre", nlp_iter_max=1000,
    ↪ nlp_tolerance=1e-6)
sol = prob.solve(alg)

print(sol.objective)          # 3.9995386676223115

```

The returned `Solution` carries `objective`, the `states` and `controls` arrays (shaped `nstates×N` and `ncontrols×N`), the time vector, and, for estimation problems, the `parameters` estimates. Two discretisations carry a control at the midpoint of every interval: Hermite–Simpson, and multiple shooting under the "quadratic" control parameterisation. For those, `controls_full` and `time_full` hold the complete control history with node and midpoint values interleaved, the Python counterparts of `get_hs_controls_in_phase` and `get_hs_time_in_phase` (Section 7.2.7). They are `None` for every other discretisation, where the nodal table *is* the control history. Anything that has to reproduce the designed control, a verification integrator above all, should prefer them: reading the nodal values alone under those two parameterisations gives two thirds of the control variables and none of the curvature. A small number of guess helpers (`psopt.ramp`, `psopt.const` and `psopt.times`) are provided for building these arrays compactly.

11.5 Multiphase problems and linkages

A multiphase problem is built by adding several phases and linking consecutive ones. `link_phases(a, b)` enforces continuity of every state and of time across the boundary between phases `a` and `b`; an optional `jumps` dictionary subtracts a fixed offset from a chosen state's continuity residual, so that the state *drops* by that offset across the boundary, $x(t_0^b) = x(t_f^a) - \delta$, which expresses a discrete jump such as the mass jettisoned between launch stages:

```

prob = psopt.Problem(name="launch")
p1 = prob.add_phase(nstates=7, ncontrols=3, nevents=8, npath=1)
p2 = prob.add_phase(nstates=7, ncontrols=3, npath=1)
# ... set maths, bounds and guess on each phase ...
prob.link_phases(1, 2, jumps={6: m_dry}) # state index 6 (mass) drops by
    ↪ m_dry at the join
# ... further phases and links ...
sol = prob.solve(alg)
sol.states[0] # phase-1 state history; sol.time, sol.controls are per-
    ↪ phase lists

```

When more than one phase or any linkage is present, `solve` returns a `MultiSolution` whose `states`, `controls` and `time` are lists with one entry per phase. The four-phase Delta-III ascent in `examples/launch.py` is a complete worked example, including the orbital-element dynamics helper `examples/rv2oe_casadi.py`.

11.6 Parameter estimation

For a parameter-estimation problem a phase carries an **observation** function $h(x, u, p, t)$ returning `nobserved` outputs, together with the sample times `observation_nodes` (shaped $1 \times \text{nnsamples}$) and the measured data `observations` (shaped `nobserved` $\times$ `nnsamples`). PSOPT then minimises the least-squares mismatch between the observed outputs and the model. Static parameters are declared through `nparameters` and bounded through `ph.bounds.lower.parameters` and `ph.bounds.upper.parameters`; their estimates are returned in `sol.parameters`.

```

ph = prob.add_phase(nstates=2, ncontrols=0, nparameters=3, nobserved=2,
    ↪ nsamples=21)
ph.dynamics = lambda x, u, p, t: ca.vertcat(-(p[0]+p[2])*x[0]**2,
    p[0]*x[0]**2 - p[1]*x[1])
ph.observation = lambda x, u, p, t: ca.vertcat(x[0], x[1]) # observe both
    ↪ states
ph.observation_nodes = np.array(tmeas).reshape(1, 21)
ph.observations = np.vstack([y1meas, y2meas])
# ... bounds and guess ...
sol = prob.solve(alg)
sol.parameters # estimated kinetic coefficients

```

The full example is `examples/cracking.py`.

11.7 Algorithm options and transcriptions

The `Algorithm` constructor exposes the same options as the C++ `algorithm` structure of Section 7.2.7. The core options `collocation_method`, `nlp_method`, `derivatives`, `scaling`, `nlp_iter_max` and `nlp_tolerance` always take effect. The remaining options

default to `None`, meaning “leave the C++ default”, and are forwarded to the engine only when set. They cover:

- the *hp*-adaptive mesh refinement of Section 2.3 (`mesh_refinement`, `mr_max_iterations`, `ode_tolerance`, `mr_max_growth_factor`, `mr_min_order`, `mr_max_order`);
- the integrated residual transcription, the Nie–Kerrigan flexible-order representation and the flexible mesh of Chapter 3 (`transcription_method`, `ir_residual_nodes`, `ir_regularization`, `ir_objective`, `ir_residual_bound`, `ir_dair`, `ir_dair_delta_factor`, `ir_local_order`, `ir_element_local_controls`, `ir_flexible_mesh`, `ir_min_element_fraction`, `ir_include_path`, `ir_path_weight`, `ir_residual_scaling`);
- the multiple-shooting transcription of Chapter 4 (`ms_integrator`, `ms_steps_per_segment`, `ms_control_parameterisation`, `ms_path_samples`, `ms_flexible_segments`, `ms_min_segment_fraction`, `ms_algebraic_iterations`, `ms_implicit_iterations`, `ms_adaptive_steps`, `ms_max_steps_per_segment`);
- general settings (`print_level`, `hessian`, `ipopt_linear_solver`, `constraint_scaling`, `ipopt_max_cpu_time`, `diagnostic_level`, `free_padded_defect_rows`).

Each option has exactly the meaning documented for its C++ counterpart, so the Python interface reaches mesh refinement and the integrated residual transcription with no additional machinery; the examples `examples/bryson_mesh.py` and `examples/bryson_ir.py` exercise them directly.

11.8 Integer controls and integer parameters

The discrete-valued quantities of Chapters 8 and 9 are reached from Python through the same declaration idiom as in C++. The dynamics are written once, with the quantity treated as an ordinary control or static parameter, and one call per quantity makes it discrete:

```
# arguments: the index of the quantity, and its admissible values
ph.declare_integer_control(0, [0.0, 1.0])
ph.declare_integer_parameter(0, [0.0, 1.0, 2.0, 3.0])
```

Both accumulate, so several integer controls may be declared on the same phase; re-declaring an index replaces the earlier declaration. The convexification over the product of the admissible sets (8.3), the SOS1 constraint, the sum-up rounding and, for integer parameters, the enumeration of Section 9.3 are all performed by the engine

exactly as for a C++ problem. Declaring an integer parameter routes the solve to `psopt_solve_integer` automatically; no separate call is needed.

When any integer control is declared, `sol.controls` is in the weights layout. The rounded trajectories are returned separately, one entry per declared control, in declaration order:

```
ic = sol.integer_controls[k]
ic.control          # rounded value on each mesh interval
ic.mode_index       # index into the admissible set on each interval
ic.interval_widths  # mesh interval widths, handy for forward simulation
ic.integral_gap     # accumulated sum-up-rounding gap
ic.n_switches       # instants at which any declared control changes

sol.integer_parameters[j]  # {'index': ..., 'value': ...}
```

A `MultiSolution` carries the same two fields per phase, as lists.

The example `examples/lotka_integer.py` is the Lotka–Volterra fishing problem of Section 8.5 written in Python; it reproduces the native mesh sweep row for row (at 40 nodes, relaxed 1.348104 and integer 1.351850 with four switches, against 1.348103 and 1.351850 natively). The example `examples/integer_parameter.py` is the discrete-acceleration problem of Section 9.5, and returns the closed-form answer $p = 2$, $J = 0.09$; it also solves the same problem with the parameter left continuous, to show that the relaxed answer $p = 2.3$, $J = 0$ is a bound attained by no admissible design.

11.9 Robust optimal control

The robust-design driver `psopt.robust` is part of the Python interface and is documented, together with the transcription it implements and the C++ driver beside it, in Chapter 10. Only the points that belong to this chapter are repeated here.

The driver builds the scenario-augmented problem itself, which is what makes it a Python feature and not a library one. A `RobustPhase` is a phase of the kind described in Section 11.4, whose `dynamics`, `path` and `events` take the uncertain parameter vector as a final argument, and whose bounds and guess are stated at the nominal size; the driver traces those callables once per scenario and hands the result to `Problem.solve`. Because the augmented problem is an ordinary `psopt.Problem`, the whole of Section 11.7 applies to it: a robust design may be transcribed by any method the interface offers, and the choice matters more here than elsewhere, a replicated state being what the scenario budget of Section 10.5 is about.

Three parts of the interface exist mainly for this driver. The option `free_padded_defect_rows` is what lifts the scenario count from tens to hundreds, and the driver sets it. The arrays `controls_full` and `time_full` of a `Solution` are what let the verification integrator reproduce the control that was designed and not a chord approximation to it (Section 10.7.1). And `sol.parameter_statistics.covariance`, the posterior of a parameter estimation (Section 11.6), is the distribution a robust design

ought to run against instead of one that somebody chose; `robust_driver_estimate.py` closes that loop inside one model.

The risk measures and the ancillary feedback of Section 10.4 were once reached from Python alone, because each augments the user’s equations. They are now in both drivers, the C++ side building them from a nominal `RobustModel` (Section 10.8.3). What remains particular to each is narrower: a Python schedule $K(t)$ has to be a polynomial in t , where a C++ one is an ordinary `adouble` function, and the two-level verification of Section 10.7, with its reported drift between a fixed-step and an adaptive integrator, is Python’s alone.

Both drivers take any number of phases. On the Python side a second `add_phase` and a `link_phases` call are the whole of the interface (Section 10.9.1). One thing the front end does not yet reach is a static parameter shared across a boundary, the linkages of `Problem.link_phases` carrying the states and the time and nothing else. Section 10.10.1 is the measurement, and it is the sharpest agreement between the two drivers recorded anywhere in this manual: on a two-phase problem they return the same design and the same certificate to every printed figure, having nothing in common but the equations the user wrote.

11.10 Examples and validation

The examples distributed with the binding each reproduce the objective of their hand-written C++ counterpart, as summarised in Table 11.2. Taken together they exercise single- and multiphase problems, phase linkages, parameter estimation, the integrated residual transcription, *hp*-adaptive mesh refinement and both kinds of discrete-valued quantity. The nine robust-design examples are listed separately, in Table 10.7, because what each of them establishes is a statement about the method and not a published objective to reproduce. The integrated residual entry is a pass-through fidelity check instead of a published value: with `ir_objective` left at its default the reported objective is the integrated residual, a feasibility measure whose magnitude depends on the residual-scaling convention, and the Python result agrees with a native driver carrying the same options to twelve significant figures ($3.498289481767 \times 10^{-4}$).

Example	Exercises	Objective (C++ baseline)
<code>bryson_denham.py</code>	single phase	3.999539 (bit-identical)
<code>launch.py</code>	four phases, linkages (Delta-III)	−7529.66 (within NLP tolerance)
<code>cracking.py</code>	parameters + observation	4.3195×10^{-3} (bit-identical)
<code>bryson_ir.py</code>	integrated residual transcription	3.498289×10^{-4}
<code>bryson_mesh.py</code>	<i>hp</i> mesh refinement (10 → 29 nodes)	3.9999969175
<code>lotka_integer.py</code>	integer control, sum-up rounding	1.348104/1.351850, 4 switches
<code>integer_parameter.py</code>	integer static parameter (enumeration)	$p = 2$, $J = 0.09$
<code>rv2oe_casadi.py</code>	CasADi orbital-element helper	(used by <code>launch.py</code>)

Table 11.2: Python-interface examples and the native C++ objectives they reproduce.

Provenance. The Python interface was co-developed with the assistance of an AI system (Anthropic’s Claude). All of its results were validated against the corresponding native PSOPT baselines, as recorded in Table [11.2](#).

Acknowledgements

The author is very grateful to Philipp Waxweiler from DLR, Germany, for developing the CMake build process for PSOPT and for providing suggestions for improving the code, including the use of Eigen3.

The author is thankful to Martin Otter from the Institute for Robotics and System Dynamics, DLR, Germany, for kindly allowing the publication of a translated version of the DLR model 2 of the Manutec R3 robot (original Fortran subroutine R3M2SI) with the distribution of PSOPT.

The author is also grateful to Naz Bedrossian from the Draper Laboratory (USA) for facilitating the thesis by S. Bhatt, where the dynamic model of the International Space Station is described.

Finally, the author is indebted to all users who have provided feedback on PSOPT since its first release.

References

- [1] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi – a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.
- [2] R. E. Bank, W. M. Coughran, W. Fichtner, E. H. Grosse, D. J. Rose, and R. K. Smith. Transient Simulation of Silicon Devices and Circuits. *IEEE Transactions on Electron Devices*, 32(10):1992–2007, 1985.
- [3] D. A. Benson. *A Gauss Pseudospectral Transcription for Optimal Control*. PhD thesis, MIT, Department of Aeronautics and Astronautics, Cambridge, Mass., 2004.
- [4] D. A. Benson, G. T. Huntington, T. P. Thorvaldsen, and A. V. Rao. Direct Trajectory Optimization and Costate Estimation via an Orthogonal Collocation Method. *Journal of Guidance, Control, and Dynamics*, 29(6):1435–1440, 2006.
- [5] John T. Betts. *Practical Methods for Optimal Control Using Nonlinear Programming*, volume 36 of *Advances in Design and Control*. SIAM, Philadelphia, Pennsylvania, third edition, 2020.
- [6] H. G. Bock. Recent advances in parameter identification techniques for ordinary differential equations. *Progress in Scientific Computing*, 2:95–121, 1983.
- [7] A.E. Bryson and Yu-Chi Ho. *Applied Optimal Control*. Halsted Press, 1975.
- [8] R.L. Burden and J.D. Faires. *Numerical Analysis*. Thomson Brooks/Cole, 2005.
- [9] G. Calafiore and M. C. Campi. The scenario approach to robust control design. *IEEE Transactions on Automatic Control*, 51(5):742–753, 2006.
- [10] C. Canuto, M.Y. Hussaini, A. Quarteroni, and T.A. Zang. *Spectral Methods: Fundamentals in Single Domains*. Springer-Verlag, Berlin, 2006.
- [11] C. Canuto, M.Y. Hussaini, A. Quarteroni, and T.A. Zang. *Spectral Methods: Evolution to Complex Geometries and Applications to Fluid Dynamics*. Springer, Heidelberg, Germany, 2007.
- [12] C.G. Canuto, M.Y. Hussaini, A. Quarteroni A., and T.A. Zang. *Spectral Methods in Fluid Dynamics*. Springer-Verlag, 1988.

- [13] T. F. Coleman and J. J. Moré. Estimation of Sparse Hessian Matrices and Graph Coloring Problems. *Mathematical Programming*, 28(3):243–270, 1984.
- [14] G. J. Cooper and J. H. Verner. Some Explicit Runge–Kutta Methods of High Order. *SIAM Journal on Numerical Analysis*, 9:389–405, 1972.
- [15] A. R. Curtis, M. J. D. Powell, and J. K. Reid. On the Estimation of Sparse Jacobian Matrices. *Journal of the Institute of Mathematics and Applications*, 13:117–120, 1974.
- [16] C. L. Darby, W. W. Hager, and A. V. Rao. An hp-Adaptive Pseudospectral Method for Solving Optimal Control Problems. *Optimal Control Applications and Methods*, 32(4):476–502, 2011.
- [17] G. Elnagar, M. A. Kazemi, and M. Razzaghi. The Pseudospectral Legendre Method for Discretizing Optimal Control Problems. *IEEE Transactions on Automatic Control*, 40:1793–1796, 1995.
- [18] F. Fahroo and I.M. Ross. Costate Estimation by a Legendre Pseudospectral Method. *Journal of Guidance, Control, and Dynamics*, 24:270–277, 2001.
- [19] F. Fahroo and I.M. Ross. Costate Estimation by a Legendre Pseudospectral Method. *Journal of Guidance, Control, and Dynamics*, 24:270–277, 2002.
- [20] F. Fahroo and I.M. Ross. Direct Trajectory Optimization by a Chebyshev Pseudospectral Method. *Journal of Guidance Control and Dynamics*, 25, 2002.
- [21] D. Garg, M. Patterson, W. W. Hager, A. V. Rao, D. A. Benson, and G. T. Huntington. A Unified Framework for the Numerical Solution of Optimal Control Problems Using Pseudospectral Methods. *Automatica*, 46(11):1843–1851, 2010.
- [22] D. Garg, M. A. Patterson, C. Francolin, C. L. Darby, G. T. Huntington, W. W. Hager, and A. V. Rao. Direct Trajectory Optimization and Costate Estimation of Finite-Horizon and Infinite-Horizon Optimal Control Problems Using a Radau Pseudospectral Method. *Computational Optimization and Applications*, 49(2):335–358, 2011.
- [23] J. H. Halton. On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerische Mathematik*, 2:84–90, 1960.
- [24] J. Hesthaven, S. Gottlieb, and D. Gottlieb. *Spectral Methods for Time-Dependent Problems*. Cambridge University Press, Cambridge, 2007.
- [25] S. J. Julier and J. K. Uhlmann. Unscented filtering and nonlinear estimation. *Proceedings of the IEEE*, 92(3):401–422, 2004.
- [26] S. Kameswaran and L. T. Biegler. Convergence Rates for Direct Transcription of Optimal Control Problems Using Collocation at Radau Points. *Computational Optimization and Applications*, 41(1):81–126, 2008.

- [27] W. Kang and N. Bedrossian. Pseudospectral Optimal Control Theory Makes Debut Flight, Saves nasa \$1m in Under Three Hours. *SIAM News*, 40, 2007.
- [28] W. Kang, Q. Gong, I. M. Ross, and F. Fahroo. On the Convergence of Nonlinear Optimal Control Using Pseudospectral Methods for Feedback Linearizable Systems. *International Journal of Robust and Nonlinear Control*, 17:1251–1277, 2007.
- [29] E. Kostina, M.A. Saunders, and I. Schierle. Computation of covariance matrices for constrained parameter estimation problems using lsqr. Technical Report SOL-2009-1, Stanford University, System Optimization Laboratory, 2009.
- [30] F. Liu, W. W. Hager, and A. V. Rao. Adaptive Mesh Refinement Method for Optimal Control Using Nonsmoothness Detection and Mesh Size Reduction. *Journal of the Franklin Institute*, 352(10):4081–4106, 2015.
- [31] S. Marsili-Libelli, S. Guerrizio, and N. Checchi. Confidence regions of estimated parameters for ecological systems. *Ecological Modelling*, 165:127–146, 2003.
- [32] M. P. Neuenhofen and E. C. Kerrigan. An Integral Penalty-Barrier Direct Transcription Method for Optimal Control. In *2020 59th IEEE Conference on Decision and Control (CDC)*, pages 456–463, 2020.
- [33] Y. Nie, O. J. Faqir, and E. C. Kerrigan. ICLOCS2: Try this Optimal Control Problem Solver Before you Try the Rest. In *2018 UKACC 12th International Conference on Control (CONTROL)*, pages 336–336. IEEE, 2018.
- [34] Y. Nie and E. C. Kerrigan. Solving Dynamic Optimization Problems to a Specified Accuracy: An Alternating Approach Using Integrated Residuals. *IEEE Transactions on Automatic Control*, 68(1):548–555, 2023.
- [35] L. Nita, E. C. Kerrigan, E. M. G. Vila, and Y. Nie. Solving Optimal Control Problems with Non-smooth Solutions Using an Integrated Residual Method and Flexible Mesh. In *2022 IEEE 61st Conference on Decision and Control (CDC)*, pages 1211–1216, 2022.
- [36] M. A. Patterson, W. W. Hager, and A. V. Rao. A ph Mesh Refinement Method for Optimal Control. *Optimal Control Applications and Methods*, 36(4):398–421, 2015.
- [37] J.A. Pietz. Pseudospectral Collocation Methods for the Direct transcription of Optimal Control Problems. Master’s thesis, Rice University, Houston, Texas, 2003.
- [38] M. J. D. Powell and Ph. L. Toint. On the Estimation of Sparse Hessian Matrices. *SIAM Journal on Numerical Analysis*, 16(6):1060–1074, 1979.
- [39] R. T. Rockafellar and S. Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.

- [40] I.M. Ross and F. Fahroo. Pseudospectral Knotting Methods for Solving Nonsmooth Optimal Control Problems. *Journal of Guidance Control and Dynamics*, 27:397–405, 2004.
- [41] S. Sager. *Numerical Methods for Mixed-Integer Optimal Control Problems*. PhD thesis, University of Heidelberg, Heidelberg, Germany, 2005. Published by Der andere Verlag, Tönning, Lübeck, Marburg.
- [42] S. Sager. A Benchmark Library of Mixed-Integer Optimal Control Problems. In J. Lee and S. Leyffer, editors, *Mixed Integer Nonlinear Programming*, volume 154 of *The IMA Volumes in Mathematics and its Applications*, pages 631–670. Springer, 2012.
- [43] S. Sager, H. G. Bock, and M. Diehl. The Integer Approximation Error in Mixed-Integer Optimal Control. *Mathematical Programming*, 133(1–2):1–23, 2012.
- [44] K. Schittkowski. *Numerical Data Fitting in Dynamical Systems*. Kluwer Academic Publishers, Dordrecht, The Netherlands, 2002.
- [45] A. Shapiro, D. Dentcheva, and A. Ruszczyński. *Lectures on Stochastic Programming: Modeling and Theory*. SIAM, Philadelphia, PA, second edition, 2014.
- [46] Lloyd N. Trefethen. *Spectral Methods in MATLAB*. SIAM, Philadelphia, 2000.
- [47] J. Vlassenbroeck and R. Van Doreen. A Chebyshev Technique for Solving Nonlinear Optimal Control Problems. *IEEE Transactions on Automatic Control*, 33:333–340, 1988.
- [48] A. Weinreb and A.E. Bryson. Optimal Control of Systems with Hard Control Bounds. *IEEE Transactions on Automatic Control*, AC-30(11):1135–1138, 1985.

Licensing Agreement

The software package PSOPT is distributed under the GNU Lesser General Public License version 2.1. Users of the software must abide by the terms of the license.

GNU LESSER GENERAL PUBLIC LICENSE
Version 2.1, February 1999

Copyright (C) 1991, 1999 Free Software Foundation, Inc.
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.

[This is the first released version of the Lesser GPL. It also counts
as the successor of the GNU Library Public License, version 2, hence
the version number 2.1.]

Preamble

The licenses for most software are designed to take away your
freedom to share and change it. By contrast, the GNU General Public
Licenses are intended to guarantee your freedom to share and change
free software--to make sure the software is free for all its users.

This license, the Lesser General Public License, applies to some
specially designated software packages--typically libraries--of the
Free Software Foundation and other authors who decide to use it. You
can use it too, but we suggest you first think carefully about whether
this license or the ordinary General Public License is the better
strategy to use in any particular case, based on the explanations below.

When we speak of free software, we are referring to freedom of use,
not price. Our General Public Licenses are designed to make sure that
you have the freedom to distribute copies of free software (and charge
for this service if you wish); that you receive source code or can get
it if you want it; that you can change the software and use pieces of
it in new free programs; and that you are informed that you can do
these things.

To protect your rights, we need to make restrictions that forbid
distributors to deny you these rights or to ask you to surrender these
rights. These restrictions translate to certain responsibilities for
you if you distribute copies of the library or if you modify it.

For example, if you distribute copies of the library, whether gratis
or for a fee, you must give the recipients all the rights that we gave
you. You must make sure that they, too, receive or can get the source
code. If you link other code with the library, you must provide

complete object files to the recipients, so that they can relink them with the library after making changes to the library and recompiling it. And you must show them these terms so they know their rights.

We protect your rights with a two-step method: (1) we copyright the library, and (2) we offer you this license, which gives you legal permission to copy, distribute and/or modify the library.

To protect each distributor, we want to make it very clear that there is no warranty for the free library. Also, if the library is modified by someone else and passed on, the recipients should know that what they have is not the original version, so that the original author's reputation will not be affected by problems that might be introduced by others.

Finally, software patents pose a constant threat to the existence of any free program. We wish to make sure that a company cannot effectively restrict the users of a free program by obtaining a restrictive license from a patent holder. Therefore, we insist that any patent license obtained for a version of the library must be consistent with the full freedom of use specified in this license.

Most GNU software, including some libraries, is covered by the ordinary GNU General Public License. This license, the GNU Lesser General Public License, applies to certain designated libraries, and is quite different from the ordinary General Public License. We use this license for certain libraries in order to permit linking those libraries into non-free programs.

When a program is linked with a library, whether statically or using a shared library, the combination of the two is legally speaking a combined work, a derivative of the original library. The ordinary General Public License therefore permits such linking only if the entire combination fits its criteria of freedom. The Lesser General Public License permits more lax criteria for linking other code with the library.

We call this license the "Lesser" General Public License because it does Less to protect the user's freedom than the ordinary General Public License. It also provides other free software developers Less of an advantage over competing non-free programs. These disadvantages are the reason we use the ordinary General Public License for many libraries. However, the Lesser license provides advantages in certain special circumstances.

For example, on rare occasions, there may be a special need to encourage the widest possible use of a certain library, so that it becomes a de-facto standard. To achieve this, non-free programs must be allowed to use the library. A more frequent case is that a free library does the same job as widely used non-free libraries. In this case, there is little to gain by limiting the free library to free software only, so we use the Lesser General Public License.

In other cases, permission to use a particular library in non-free programs enables a greater number of people to use a large body of free software. For example, permission to use the GNU C Library in non-free programs enables many more people to use the whole GNU operating system, as well as its variant, the GNU/Linux operating system.

Although the Lesser General Public License is Less protective of the

users' freedom, it does ensure that the user of a program that is linked with the Library has the freedom and the wherewithal to run that program using a modified version of the Library.

The precise terms and conditions for copying, distribution and modification follow. Pay close attention to the difference between a "work based on the library" and a "work that uses the library". The former contains code derived from the library, whereas the latter must be combined with the library in order to run.

GNU LESSER GENERAL PUBLIC LICENSE
TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License Agreement applies to any software library or other program which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Lesser General Public License (also called "this License"). Each licensee is addressed as "you".

A "library" means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

The "Library", below, refers to any such software library or work which has been distributed under these terms. A "work based on the Library" means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term "modification".)

"Source code" for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You may copy and distribute verbatim copies of the Library's complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and distribute a copy of this License along with the Library.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

- a) The modified work must itself be a software library.
- b) You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.
- c) You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License.
- d) If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

(For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.

Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy.

This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a "work that uses the Library". Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this License.

However, linking a "work that uses the Library" with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a "work that uses the library". The executable is therefore covered by this License. Section 6 states terms for distribution of such executables.

When a "work that uses the Library" uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if the work can be linked without the Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law.

If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you may also combine or link a "work that uses the Library" with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit modification of the work for the customer's own use and reverse engineering for debugging such modifications.

You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

- a) Accompany the work with the complete corresponding

machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable "work that uses the Library", as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)

b) Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (1) uses at run time a copy of the library already present on the user's computer system, rather than copying library functions into the executable, and (2) will operate properly with a modified version of the library, if the user installs one, as long as the modified version is interface-compatible with the version that the work was made with.

c) Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.

d) If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.

e) Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the "work that uses the Library" must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the materials to be distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

7. You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:

a) Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.

b) Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

8. You may not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Library is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

9. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.

10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties with this License.

11. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this License may add an explicit geographical distribution limitation excluding those countries,

so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

13. The Free Software Foundation may publish revised and/or new versions of the Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you may choose any version ever published by the Free Software Foundation.

14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

15. BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS